Inteligență artificială

Versiune: 9 noiembrie 2026

Lucian M. Sasu, PhD

Cuprins

Listă de figuri **6**

Listă de algoritmi **9**

1 Introducere **10**
1.1 Rețele neurale artificiale . . . 12
1.1.1 Bazele biologice . . . 12
1.1.2 Diferențe între rețele neurale artificiale și naturale . . . 14
1.1.3 Aplicabilitate . . . 14
1.2 Calcul evoluționist . . . 15
1.2.1 Bazele biologice . . . 15
1.2.2 Cromozomi . . . 16
1.2.3 Diferențe între cromozomii biologici și cei artificiali . . . 16
1.2.4 Aplicabilitate . . . 16
1.3 Tipuri de învățare în învățarea automată . . . 16
1.3.1 Învățarea supervizată . . . 17
1.3.2 Învățarea prin consolidare . . . 17
1.3.3 Învățarea nesupervizată . . . 18
1.4 Autoorganizarea . . . 18

2 Regresia liniară **20**
2.1 Exemplu și notații . . . 20
2.2 Funcția de eroare . . . 24
2.3 Metoda gradientului descendent pentru regresia liniară univariată . . . 28
2.4 Regresia liniară multivariată . . . 32
2.5 Metoda ecuațiilor normale . . . 41

3 Regresia logistică **44**
3.1 Încadrare și exemple . . . 44
3.2 Regresia logistică binară . . . 45
3.2.1 Setul de antrenare . . . 45
3.2.2 Exemplu numeric: setul de date . . . 45
3.2.3 Reprezentarea modelului . . . 46

3.2.4 Funcția de cost . . . 48
3.2.5 Exemplu numeric: funcția de cost pentru parametri nuli 51
3.2.6 Algoritmul de antrenare . . . 52
3.2.7 Exemplu numeric: antrenarea prin metoda gradientului descendent . . . 55
3.2.8 Etapa de predicție . . . 55
3.2.9 Suprafața de decizie a regresiei logistice binare . . . 56
3.2.10 Exemplu numeric: predicția și pragul de decizie . . . 58
3.3 Regresia logistică multinomială . . . 59
3.3.1 Setul de antrenare . . . 59
3.3.2 Funcția softmax . . . 60
3.3.3 Exemplu numeric: funcția softmax . . . 60
3.3.4 Reprezentarea modelului . . . 61
3.3.5 Funcția de cost . . . 62
3.3.6 Gradientul funcției de cost . . . 64
3.3.7 Exemplu numeric: funcția de cost și gradientul pentru trei clase . . . 66
3.3.8 Algoritmul de antrenare pentru regresia logistică multinomială . . . 66
3.3.9 Predicția pentru regresia logistică multinomială . . . 67
3.4 Interpretarea coeficienților . . . 68
3.4.1 Șanse și logaritmul șanselor . . . 68
3.4.2 Interpretarea fiecărui coeficient . . . 68
3.4.3 Exemplu numeric: interpretarea modelului pentru apartamente . . . 69
3.4.4 Precauții la interpretare . . . 69
3.4.5 Cazul multinomial . . . 70
3.5 Precizări . . . 71
3.5.1 Redundanța parametrilor pentru regresia logistică multinomială . . . 71
3.5.2 Exemplu numeric: redundanța parametrilor . . . 71
3.5.3 Relația dintre cele două tipuri de regresii logistice . . . 72
3.5.4 Calculul numeric al funcției softmax . . . 72
3.5.5 Exemplu numeric: depășirea de reprezentare la calculul funcției softmax . . . 73
3.5.6 Trucul „log sum exp” . . . 73
3.5.7 Exemplu numeric: trucul „log sum exp” . . . 74

4 Regularizarea și selecția modelelor 75
4.1 Underfitting și overfitting pentru regresia liniară . . . 75
4.2 Regularizarea L_2 pentru modele de regresie . . . 77
4.3 Regularizarea modelelor de regresie logistică . . . 80
4.3.1 Regularizarea L_2 pentru regresia logistică binară . . . 80
4.3.2 Exemplu numeric: date liniar separabile . . . 81

4.3.3 Regularizarea L_2 pentru regresia logistică multinomială 85
4.4 Regularizarea cu coeficient variabil . . . 86
4.5 Seturi de antrenare, validare și testare . . . 87
4.5.1 Motivație . . . 87
4.5.2 Împărțirea datelor . . . 88
4.5.3 Rolul setului de validare . . . 89
4.5.4 Rolul setului de testare . . . 90
4.5.5 Depistarea underfitting-ului și a overfitting-ului . . . 90
4.5.6 Corectarea underfitting-ului și a overfitting-ului . . . 92

5 Perceptronul liniar 94
5.1 Încadrare și exemplu . . . 94
5.2 Definiții și notații . . . 96
5.3 Modelul perceptronului . . . 97
5.4 Algoritmul de instruire a perceptronului . . . 98
5.5 Exemplu numeric . . . 101
5.6 Modificarea ponderilor prin metoda gradientului descendent . 102
5.7 Convergența perceptronului . . . 105
5.8 Algoritmul lui Gallant . . . 107
5.9 Comentarii . . . 108

6 Perceptronii multistrat 111
6.1 Motivație pentru rețele neurale multistrat . . . 111
6.2 Notații . . . 112
6.3 Setul de antrenare . . . 112
6.4 Rețeaua neurală MLP . . . 112
6.4.1 Arhitectură . . . 112
6.4.2 Funcții de activare . . . 117
6.5 Pasul de propagare înainte . . . 122
6.6 Funcții de cost . . . 124
6.6.1 Funcția de cost pentru problemă de regresie . . . 125
6.6.2 Funcția de cost pentru discriminarea a două clase . . . 126
6.6.3 Funcția de cost pentru mai mult de două clase independente . . . 127
6.6.4 Funcția de cost pentru clasificare cu mai mult de două clase . . . 128
6.7 Inițializarea ponderilor rețelei . . . 128
6.8 Grafuri computaționale și calcul de derivate parțiale . . . 129
6.8.1 Derivarea funcțiilor compuse . . . 129
6.8.2 Exemplul 1: căi unice . . . 131
6.8.3 Exemplul 2: căi multiple . . . 133
6.9 Algoritmul backpropagation . . . 134
6.9.1 Scopul algoritmului . . . 134
6.9.2 Variante de calcul . . . 135

6.9.3 Calculul gradienților – justificare matematică 136
6.9.4 Algoritmul de instruire a rețelei MLP 142
6.10 Utilizarea rețelei pentru inferență 143
6.11 Discuții 143

7 Rețele neurale cu funcții de activare radială **144**
7.1 Motivația rețelei: teorema lui Cover 144
7.2 Funcții cu activare radială 147
7.3 Rețele cu funcții cu activare radială 148
7.4 Clustering folosind algoritmul K-means 150
7.5 Algoritmul de inițializare K-means++ 152
7.6 Determinarea ponderilor dintre stratul ascuns și cel de ieșire . 153
7.7 Algoritmul de instruire a rețelei RBF 154

8 Fuzzy ARTMAP **155**
8.1 Învățarea incrementală 155
8.2 Proprietăți dezirabile ale sistemelor instruibile 155
8.3 Dilema stabilitate–plasticitate 156
8.4 Fuzzy ARTMAP 157
8.4.1 Arhitectura rețelei FAM 158
8.4.2 Algoritmul de învățare pentru FAM 160

9 Rețele neurale convoluționale **167**
9.1 Convoluție 169
9.1.1 Exemplificare de convoluție 170
9.1.2 Bordarea intrării 174
9.1.3 Pasul kernelului 176
9.1.4 Numărul de ponderi și de operații pentru convoluție cu intrare pe un singur canal 178
9.1.5 Convoluții pentru intrări cu mai multe canale 179
9.2 Funcții de activare 182
9.3 Straturi de pooling 182
9.3.1 Max pooling 183
9.3.2 Average pooling 183
9.3.3 Global average pooling 183
9.4 Straturi complet conectate 184
9.5 Funcții de cost, regularizare 185
9.6 Rețelele neurale de convoluție: soluție end-to-end 186
9.7 Exemple de rețele neurale de convoluție 186
9.7.1 LeNet-5 189
9.7.2 VGG16 189

10 Calcul evoluționist **191**
10.1 Taxonomie 191
10.2 Algoritmi genetici 192
10.3 Fundamente teoretice 195
10.4 Problema reprezentării datelor în algoritmii genetici 198
10.4.1 Varianta cu penalizare 201
10.4.2 Varianta cu reparare 201
10.4.3 Codificarea adecvată a indivizilor 202
10.5 Exemplu: problema orarului 203

Bibliografie **205**

Listă de figuri

1.1 Neuron natural 13
1.2 Neuron de tip Purkinje din cortexul cerebelar 13
1.3 Schema de lucru pentru învățare supervizată 17
1.4 Schema de lucru pentru învățare prin consolidare 18
1.5 Schema de lucru pentru învățare nesupervizată 18

2.1 Reprezentarea grafică a datelor de vânzare a unor apartamente 21
2.2 Model liniar de predicție și aproximarea prețului unui apartament de 80 de metri pătrați. 22
2.3 Fluxul de lucru în antrenarea unui model. 23
2.4 Regresie liniară univariată 25
2.5 Funcția de eroare pătratică, cu $\theta_0 = 0$ și θ_1 variabil. 26
2.6 Funcția de eroare pentru model liniar univariat cu θ_0 și θ_1 variabili. 27
2.7 Curbe de contur pentru funcția de eroare a unui model liniar univariat 28
2.8 Graful de calcul pentru modelul de regresie liniară multivariată 33

3.1 Graficul sigmoidei logistice definite în ecuația (3.4) 47
3.2 Graf de calcul pentru regresia logistică binară 48
3.3 Minim global și minim local pentru funcție neconvexă 49
3.4 Cele două ramuri ale funcției Cost din ecuația (3.10) 50
3.5 Suprafață de decizie circulară obținută cu trăsături polinomiale 57
3.6 Suprafață de decizie eliptică obținută cu trăsături polinomiale 58
3.7 Probabilitățile estimate și pragurile de decizie pentru exemplul cu apartamente 59

4.1 Trei polinoame pentru aproximarea prețului pornind de la suprafață 76
4.2 Regresia logistică pe date liniar separabile, fără și cu regularizare 84
4.3 Depistarea underfitting-ului și a overfitting-ului 91

5.1 Mai multe drepte pentru mulțimi liniar separabile 95
5.2 Semiplanele pozitive și negative 95

5.3 Suprafața de decizie și vectorul de ponderi 97
5.4 Graful de calcul al unui perceptron liniar. 98
5.5 Exemplu numeric: dreapta inițială 102
5.6 Exemplu numeric: prima modificare a dreptei 103
5.7 Exemplu numeric: ultima modificare a dreptei 103
5.8 Problemă neseparabilă liniar XOR 109

6.1 Două clase de puncte ce nu sunt liniar separabile 112
6.2 Rețea MLP cu 3 straturi 115
6.3 Rețea MLP cu 4 straturi 115
6.4 Legăturile între noduri din straturile $l-1$ și l. 116
6.5 Graficul funcției de activare ReLU 119
6.6 Graficul funcției de activare PReLU pentru $\alpha = 0.1$ 121
6.7 Graficul funcției de activare ELU pentru $\alpha = 0.1$ 121
6.8 Graficul funcției de activare Swish pentru $\beta \in \{0.5, 2\}$ 122
6.9 Functiile mean squared error, mean absolute error, Huber ($\delta = 0.5$) pentru cazul unidimensional 127
6.10 Graf computațional pentru funcție multivariată compusă . . . 130
6.11 Graful computațional pentru calculul valorii funcției $f(x, y, z) = x \cdot (y + z)$ 131
6.12 Graful pentru calculul derivatelor parțiale 132
6.13 Graf computațional pentru $f(x, y, z) = (x + y) \cdot (y + z)$ . . . 133
6.14 Calculul gradienților pentru $f(x, y, z) = (x + y) \cdot (y + z)$ . . . 134
6.15 Curbe de eroare pentru antrenare și validare 136
6.16 Rețea neurală cu un strat ascuns, folosită ca schemă pentru determinarea fomulelor gradienților. 138
6.17 Graf computațional pentru ponderile și valorile de bias din ultimul strat. 138
6.18 Graf computațional printr-un neuron din stratul ascuns. . . . 140

7.1 Set de dete neseparabil liniar transformat în set liniar separabil 145
7.2 Transformarea problemei XOR, ce nu permite separare liniară (partea stângă) în problemă liniar separabilă 146
7.3 Structura unei rețele RBF, plecând de la funcția de interpolare din ecuația (7.4). 149
7.4 Structura unei rețele RBF cu centri selectați 150
7.5 Caz nefavorabil pentru K-means la alegerea centroizilor inițiali 152
7.6 Alegerea optimă a centroizilor inițiali 153

8.1 Arhitectura Fuzzy ARTMAP 158
8.2 Interpretarea geometrică a ponderilor $\mathbf{w}_j^a$ 165
8.3 Interpretarea geometrică a învățării de către o categorie în stil fast learning 165

9.1 Clasificare de imagini cu rețea neurală de convoluție 168

9.2 Detectare de obiecte cu rețea neurală de convoluție 168
9.3 Segmentare semantică și segmentare de instanțe 168
9.4 Exemplu de cifră 5 din setul de date MNIST 169
9.5 Imagine color, cu reprezentare numerică pe trei canale de culoare 169
9.6 Kernel răsturnat 171
9.7 Exemplificare intrare, kernel și ieșire 171
9.8 Prima convoluție (cu kernel răsturnat), aplicată peste intrările marcate cu linie punctată 171
9.9 A doua convoluție 171
9.10 A treia convoluție 171
9.11 Ultima convoluție 172
9.12 Exemplu numeric pentru convoluție de 3×3 peste o intrare de 5×5, fără bordare și cu pas 1. Preluare din [1] 172
9.13 O valoare centrală a intrării este folosită de mai multe ori într-o convoluție 175
9.14 Convoluție validă cu pas 2 177
9.15 Convoluție de 3×3 peste o intrare de 5×5, cu pas 2 si padding 1 178
9.16 Filtru de convoluție pentru un strat de intrare cu 4 canale . . 180
9.17 Convoluție cu două filtre pentru un strat de intrare cu 4 canale 181
9.18 Max pooling 183
9.19 Average pooling 184
9.20 Global average pooling pe un canal, respectiv pe 3 canale . . 184
9.21 Straturi complet conectate 185
9.22 Arhitectura generică a unei rețele neurale convoluționale . . . 186
9.23 Succesiune de trăsături învățate de o rețea neurală profundă . 187
9.24 Schiță a unei rețele de convoluție 188
9.25 Arhitectura rețelei convoluționale LeNet-5 188
9.26 Arhitectura rețelei convoluționale VGG16 190

10.1 Reprezentarea unui cromozom în algoritmii genetici 193

Listă de algoritmi

2.1 Metoda gradientului descendent pentru regresia liniară univariată 30
2.2 Metoda gradientului descendent pentru regresia liniară multivariată, în formă vectorizată 36
3.1 Metoda gradientului descendent pentru regresia logistică binară 52
3.2 Metoda gradientului descendent pentru regresia logistică multinomială 67
3.3 Metoda gradientului descendent pentru regresia logistică multinomială, în formă vectorizată 67
4.1 Metoda gradientului descendent pentru regresia liniară cu regularizare L_2 79
4.2 Metoda gradientului descendent pentru regresia logistică binară cu regularizare L_2 81
4.3 Metoda gradientului descendent pentru regresia logistică multinomială cu regularizare L_2 86
5.1 Algoritmul de instruire a perceptronului 101
5.2 Algoritmul buzunarului, varianta cu clichet 108

Capitolul 1

Introducere

Inteligența artificială (IA) este un domeniu care combină elemente de învățare automată, evoluție, modelare logică și probabilistă, metode de căutare etc. pentru a rezolva probleme care, abordate tradițional, sunt dificil sau imposibil de abordat. Câteva subdomenii ale inteligenței artificiale sunt:

- modele cu învățare automată (machine learning) – modele liniare, mixturi gaussiene, rețele neurale (sau „neuronale") artificiale, Support Vector Machines, modele de tip transformer etc.;
- calcul evoluționist;
- modelare probabilistă;
- imunitate artificială;
- inteligența mușuroiului.

O vedere amplă a domeniului este dată în lucrarea [2].

Fiecare subdomeniu a evoluat rapid și s-a impus ca potențială metodă de rezolvare efectivă a unor probleme complexe și de interes, pentru care abordările uzuale sunt nefructuoase. De regulă, prototipizarea unui sistem inspirat din inteligența artificială este rapidă, iar pentru o problemă se pot folosi mai multe abordări: optimizarea se poate face prin euristici bazate pe algoritmi genetici, prin algoritmi de căutare locală sau metode de tip gradient.

Metodele din inteligența artificială sunt frecvent inspirate din biologie: rețelele neurale au pornit de la modelul imaginat pentru neuronul biologic, calculul evoluționist este bazat pe teoria evoluției și pe genetică. Sistemele fuzzy sunt introduse pentru a permite manipularea impreciziei, altfel decât prin teoria probabilităților.

Este o mare diferență între abordarea clasică, algoritmică a unei probleme și cea dată de IA. În primul caz este pusă la bătaie toată abilitatea celui care

imaginează algoritmul pentru a rezolva problema; este un demers creativ, depinzând de imaginația, puterea de abstractizare și experiența persoanei în cauză; până nu demult propunerile de rezolvări erau făcute doar de către oameni. Tot aici, de cele mai multe ori rezultatele sunt exacte și se insistă permanent pe micșorarea complexității de calcul sau de memorie a algoritmilor; de multe ori însă o soluție exactă presupune resurse de timp și memorie prohibitive.

Abordarea pentru rețele neurale sau algoritmi genetici este total diferită: definitorie este capacitatea de *adaptare* și *căutare* automată sau *autoorganizare* la condițiile problemei. Este modelul inspirat din natură: un sistem biologic preia semnale din mediu și printr-un proces de învățare se adaptează, astfel încât să își îndeplinească scopul sau să obțină o mai bună integrare în mediu. Soluția la care se ajunge nu este întotdeauna optimă, dar este un răspuns „suficient de bun" pentru problema propusă. În implementarea unui sistem din cadrul IA accentul cade mai mult pe abilitatea sistemului rezultat de a se adapta, de a învăța, decât pe imaginația și experiența celui care îl concepe.

Sistemele studiate aici sunt cu un mare grad de aplicabilitate. De exemplu, algoritmii genetici pot fi folosiți pentru o clasă largă de funcții, nedepinzând prea mult – așa cum se întâmplă în cercetările operaționale – de ipoteze care în practică pot fi prea restrictive.

O definiție a „inteligenței" potrivită pentru contextul de IA este:

Definiția 1. *Inteligența este abilitatea unui sistem de a-și adapta comportamentul pentru a-și îndeplini scopurile în mediul său. Este o proprietate a tuturor entităților ce trebuie să ia decizii și al căror comportament este condus de scop.*

Definiția de mai sus a fost dată în 1995 de către David Fogel, scoțând în evidență elementul esențial al comportamentului inteligent și în particular al subdomeniului de machine learning: adaptarea.

Rețelele neurale artificiale reprezintă grupuri interconectate de neuroni artificiali care au abilitatea de a învăța din mediul lor și a se adapta la el, construind un model al lumii. Ele au apărut ca răspuns la modelarea activității creierului biologic, precum și ca modalitate propusă pentru a obține sisteme artificiale capabile să recunoască șabloane. Exemple de rețele neurale și algoritmi de instruire se găsesc în [3], [4].

Modelarea probabilistă este o metodă de reprezentare și analiză a situațiilor care prezintă incertitudine și caracter aleator. Sunt folosite modelele matematice bazate pe variabile aleatoare și distribuții de probabilitate pentru a aproxima și prezice comportamentul acestor fenomene. Modelarea probabilistă ia în considerare faptul că unele evenimente pot apărea sau nu în mod aleator, sau că pentru anumite trăsături și intrări valorile sunt necunoscute, dar la rândul lor modelate probabilist, iar rezultatele sunt exprimate tot sub formă de probabilități.

Calculul evoluționist se ocupă în special de optimizarea unor funcții de cost și de probleme de căutare; tehnicile sunt bazate pe concepte preluate din genetică și evoluționism. Se pleacă de la ideea evoluției unei populații de indivizi, fiecare dintre ei fiind o soluție potențială a problemei ce se vrea rezolvată. Domeniul include algoritmii genetici, programarea evoluționistă, programarea genetică și strategiile de evoluție.

Sistemele rezultate prin inteligență artificială pot reprezenta hibridizări ale celor de mai sus; de exemplu, există sisteme care îmbină modelele neurale cu cele probabiliste, iar ajustarea (hiper)parametrilor pentru o rețea neurală se poate face prin metode euristice bazate pe algoritmi genetici. Alegerea metodei potrivite pentru problema în cauză poate fi o provocare, deoarece de multe ori se pot folosi diverse abordări; nu se știe întotdeauna care e varianta cea mai potrivită de abordare.

1.1 Rețele neurale artificiale

1.1.1 Bazele biologice

Rețeaua neurală biologică a evoluat de-a lungul timpului, ajungând la performanțe greu de egalat de calculatoarele electronice: de exemplu, recunoașterea de imagini sau interpretarea ecoului reflectat de către obstacole sau insecte, în cazul liliecilor – chiar dacă au creierul foarte mic, procesarea în cazul lor se face mai rapid și cu consum energetic mult mai redus decât cu sistemele electronice actuale.

Studiile efectuate în ultimul secol au permis enunțarea unor principii asupra modului de funcționare a sistemelor neurale biologice; suntem însă departe de a cunoaște toate detaliile funcționale și structurale. Chiar și așa, prin implementarea modelelor obținute, rezultatele sunt mai mult decât notabile.

Figura 1.1 ([5]) reprezintă cel mai comun tip de neuron natural. În creierul uman există circa 86 de miliarde de neuroni interconectați, fiecare putând avea până la 10^4 conexiuni cu alți neuroni; modul de grupare a acestora și interdependențele nu sunt pe deplin cunoscute.

Un neuron artificial are o structură asemănătoare, fiind un element de procesare conectat cu alte elemente ce preia intrare de la niște neuroni și produce o ieșire ce devine intrare pentru alți neuroni; legăturile neurale sunt niște coeficienți numerici, iar prin algoritmi de învățare se obține adaptarea convenabilă a rețelei neurale. Adaptarea (sau învățarea) este aspectul esențial al rețelelor neurale: plecând de la seturi de date, se detectează automat șabloanele existente și se construiesc niște modele care pot fi folosite mai departe.

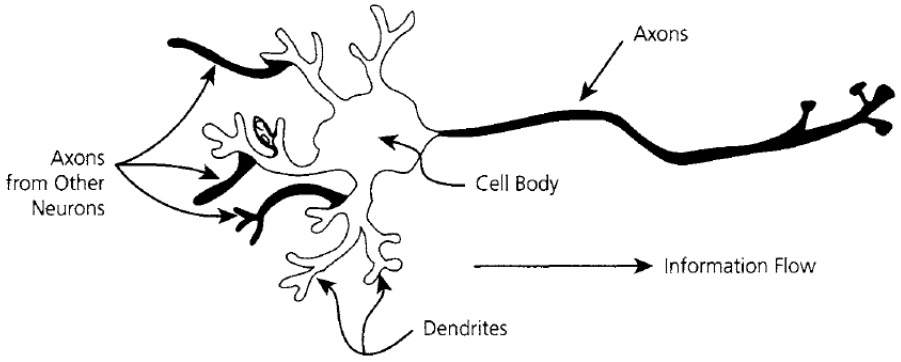


Figura 1.1: Neuron natural [5]

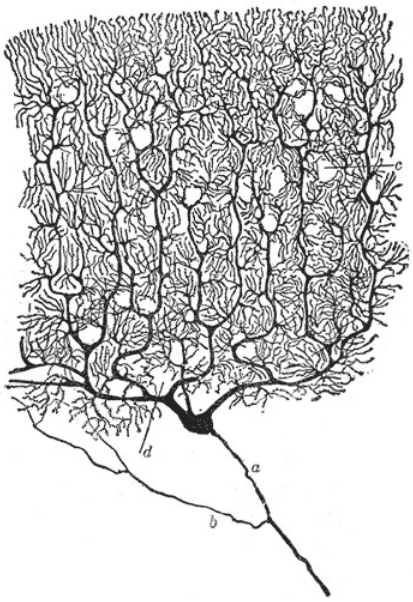


Figura 1.2: Neuron de tip Purkinje din cortexul cerebelar; sursa http://en.wikipedia.org/wiki/Neuron

1.1.2 Diferențe între rețele neurale artificiale și naturale

În mod cert, însă, există diferențe: nu sunt modelate toate tipurile cunoscute de neuroni; apoi, o lege biologică spune că un neuron poate să excite sau să inhibe un neuron cu care este conectat; în modelarea de rețele neurale artificiale, o pondere de legătură este fie excitatoare, fie inhibitoare, dar forma ei este fixată după ce s-a făcut învățarea.

O altă diferență (și punct de critică pentru rețelele neurale artificiale) este faptul că modelarea semnalului făcută sub formă de valori continue este de negăsit în rețelele biologice; în rețelele neurale biologice se folosesc de fapt trenuri de impulsuri care sunt transmise către neuroni, apărând variație în frecvența semnalului. Acest aspect a fost abordat relativ târziu, în cadrul rețelelor neurale cu pulsuri.

Viteza rețelelor neurale este iarăși un loc în care apar diferențe. Se estimează că neuronii naturali au cicli de timp între 10 și 100 de milisecunde; implementările de rețele neurale artificiale funcționează pe procesoare de câțiva gigahertzi, deci cu un ciclu de mai puțin de o nanosecundă. Chiar și așa, rețelele neurale biologice sunt cu mult mai performante decât cele artificiale, la un consum de energie mult mai redus.

Altă diferență este că neuronii naturali sunt grupați în cantități mari, uneori de sute de milioane de unități. Se ajunge astfel la un grad de paralelism masiv.

1.1.3 Aplicabilitate

- *Clasificarea* – pe baza unui set de date de forma (intrare – ieșire asociată) se construiește un sistem care detectează asocierile dintre datele de intrare și etichetele ce le sunt asociate; etichetele – sau clasele – sunt dintr-o mulțime discretă, finită. Clasificarea se folosește pentru recunoașterea automată a formelor, recunoașterea vorbirii, diagnoză medicală și altele.

- *Estimarea de probabilitate condiționată* – similar cu clasificarea, dar se produce un sistem care estimează probabilitatea ca un obiect să aparțină unei clase, date fiind trăsăturile de intrare; de exemplu, dat fiind conținutul unui mesaj de e-mail, care este probabilitatea ca acesta să fie mail legitim sau spam.

- *Regresie* – asemănător cu clasificarea, dar ieșirile nu sunt dintr-o mulțime discretă și finită, ci valori numerice continue.

- *Regăsirea de date pe baza conținutului*, folosind memorie asociativă – se poate regăsi o dată pe baza unei părți a ei. Este un mecanism diferit de modul în care calculatoarele regăsesc informația – pe baza adreselor sau a unei căutări – dar apropiat de modul în care se face regăsirea elementelor reținute de către o persoană.

- *Grupare automată (clustering)* – pe baza similarităților existente într-un set de date, se detectează grupările de date; elementele dintr-un grup sunt mai apropiate între ele decât de altele din alt grup.
- *Detectarea automată de trăsături* – a acelor elemente care fac ca procesul de recunoaștere a unui obiect să fie mai bun decât dacă se folosesc cunoștințe specifice domeniului.
- *Controlul sistemelor* – atunci când un proces trebuie să fie ghidat pentru a îndeplini o anumită sarcină, cu anumite constrângeri; utilitatea rețelelor neurale provine din faptul că nu se presupune că există dependențe liniare între acțiune și efect.

1.2 Calcul evoluționist

Principalele paradigme[1] ale calculului evoluționist sunt:

- algoritmii genetici – evoluția unei populații de indivizi (cromozomi), folosind selecția, încrucișarea și mutația;
- programarea evoluționistă – similară cu precedenta, dar fără a folosi încrucișarea; este văzută ca evoluția de specii diferite, între care nu există hibridizări;
- strategiile de evoluție – similare cu algoritmii genetici, dar se folosește recombinarea în loc de încrucișare și deseori alte metode de mutație;
- programarea genetică – metode evolutive aplicate programelor de calculator.

1.2.1 Bazele biologice

Domeniile de inspirație sunt genetica și teoria evoluționistă. Genetica tratează ereditatea, adică transmiterea trăsăturilor de la părinți la urmași. Astfel, adaptarea obținută în generațiile anterioare este preluată de către urmași și continuată. Codificarea caracteristicilor este dată de cromozomi. Noțiunile și mecanismele sunt preluate din teoria eredității întemeiată de Gregor Mendel și teoria evoluționistă a lui Charles Darwin.

[1] „Paradigma este o construcție mentală larg acceptată, care oferă unei comunități sau unei societăți pe perioadă îndelungată o bază pentru crearea unei identități de sine (a activității de cercetare de exemplu) și astfel pentru rezolvarea unor probleme sau sarcini.”, conform Wikipedia.

1.2.2 Cromozomi

Cromozomii sunt structuri din interiorul celulelor care mențin informația genetică. În cazul oamenilor, sunt 46 de cromozomi, jumătate moșteniți de la tată și jumătate de la mamă. Cromozomii sunt alcătuiți din gene, fiecare fiind identificată prin locația pe care o ocupă și prin funcția asociată.

1.2.3 Diferențe între cromozomii biologici și cei artificiali

Cromozomii artificiali sunt reprezentări simplificate ale celor biologici. În timp ce cromozomii biologici sunt secvențe de acizi nucleici, cromozomii artificiali sunt șiruri de cifre binare.

Cromozomii biologici care definesc organismele vii variază în lungime, chiar dacă de la un organism la altul din aceeași specie pentru un cromozom specific lungimea este constantă. În algoritmii genetici, lungimea este fixă.

La reproducerea indivizilor dintr-o populație naturală, jumătate din informația genetică este preluată de la tată și jumătate de la mamă. În algoritmii genetici gradul de combinare poate să difere.

1.2.4 Aplicabilitate

Principala arie de aplicare este optimizarea, pentru situațiile în care căutarea soluției cere un timp îndelungat. Algoritmii genetici sunt folosiți ca o metodă euristică; problemele abordate sunt dintre cele mai diverse – optimizarea unui plan de lucru sau circuit, balansarea încărcării, optimizarea ingredientelor, design automat, încărcarea containerelor, optimizarea structurilor moleculare, testarea mutațiilor, optimizarea sistemelor de compresie, selectarea modelelor optime, găsirea defectelor hardware etc.

1.3 Tipuri de învățare în învățarea automată

Învățarea permite unui sistem să se adapteze la mediul în care operează; pe baza semnalelor provenite din exterior, sistemul inteligent își modifică parametrii pentru o îndeplinire cât mai bună a sarcinii propuse. Trebuie făcută distincția între „învățare” și „memorare cu regăsire exactă” – această din urmă problemă este rezolvată de structuri și baze de date.

Există trei tipuri principale de învățare:

1. supervizată (eng: supervised learning)
2. nesupervizată (eng: unsupervised learning)
3. prin consolidare (eng: reinforcement learning)

Există și variante intermediare, de exemplu învățarea semi-supervizată și cea activă.

1.3.1 Învățarea supervizată

Se presupune că există un „profesor” care poate prezenta un set de date de antrenare având forma (intrare – ieșire asociată), relevant, care este preluat de către sistem și învățat. Se folosește o funcție de eroare, care măsoară cât de departe este răspunsul cerut față de cel furnizat de sistem; pe baza erorii se desfășoară un proces de ajustare a valorilor din sistemul computațional inteligent până când eroarea scade sub un anumit prag. Rezultatul final este obținerea unui sistem ce poate să furnizeze o valoare de ieșire adecvată pentru o anumită valoare de intrare ce nu este prezentă în setul de antrenare.

Exemple de sisteme ce folosesc instruirea supervizată: perceptronul, perceptronul multistrat, Fuzzy ARTMAP, rețelele cu activare radială, rețelele convoluționale.

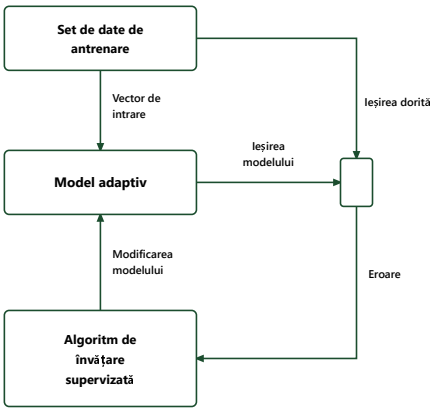


Figura 1.3: Schema de lucru pentru învățare supervizată

1.3.2 Învățarea prin consolidare

Învățarea prin consolidare (eng: reinforcement learning) este similară cu învățarea supervizată, numai că în loc de a se furniza ieșirea asociată unei intrări, se pune la dispoziție o indicație care arată cât de bine a acționat sistemul respectiv. Este un sistem bazat pe critică sau aprobare, fiind instruit în raport cu măsura în care ieșirea obținută de un sistem corespunde valorii dorite – dar fără ca această valoare dorită să fie precizată sistemului. Rolul profesorului este luat de un critic, care precizează în ce măsură ieșirea obținută se apropie de cea dorită. Pe termen lung, sistemul își va modifica propriul comportament astfel încât să se reducă criticile obținute.

Acest tip de învățare este plauzibil din punct de vedere biologic, deoarece o ființă sau un agent artificial inteligent va încerca să își minimizeze starea de disconfort prilejuită de un comportament neadecvat. Rolul criticului este dat aici de mediul înconjurător. Schema de lucru este dată în figura 1.4.

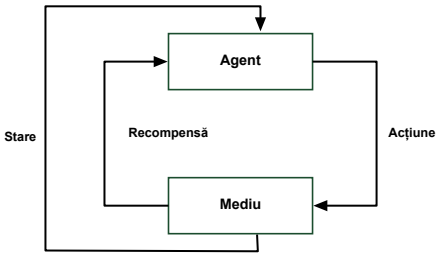


Figura 1.4: Schema de lucru pentru învățare prin consolidare

1.3.3 Învățarea nesupervizată

Spre deosebire de precedentele moduri de învățare, în acest caz nu se primește niciun semnal de tip ieșire sau critică asociată. Sistemului capabil de grupare i se dau doar valori de intrare. El face o grupare automată sau folosește o învățare de tip competitiv. Aplicațiile clasice sunt analiza asocierilor, gruparea pe bază de similaritate și estimarea de densitate de probabilitate.

Schema de lucru este dată în figura 1.5. Acest tip de adaptare este prezent în modele ce efectuează clustering, analiza de asocieri, analiza componentelor principale, detectarea de anomalii etc.

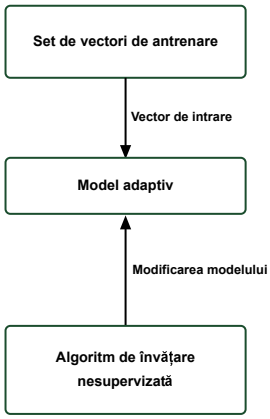


Figura 1.5: Schema de lucru pentru învățare nesupervizată

1.4 Autoorganizarea

Autoorganizarea, alături de învățare, este un alt atribut important al sistemelor cu inteligență artificială. Este prezentă în sistemele naturale, de exemplu în creierul nou-născuților, unde autoorganizarea se manifestă în principal prin distrugerea legăturilor nefuncționale. Autoorganizarea este definită astfel:

Definiția 2. *Spunem că un sistem se autoorganizează dacă, după ce se primesc intrarea și ieșirea unui fenomen necunoscut, sistemul se organizează singur astfel încât să simuleze fenomenul necunoscut [6].*

sau:

Definiția 3. *Sistemele cu autoorganizare se autoorganizează pentru a clasifica percepțiile din mediu în percepții ce pot fi recunoscute, sau șabloane [6].*

Modelele neurale care posedă proprietatea de autoorganizare includ: Self Organizing Maps[2], variantele de Neural Gas[3,4,5] sau variante de rețele neurale recurente[6].

[2]T. Kohonen, *Self-Organizing Maps*, Springer, 2001.

[3]T. Martinetz, S. Berkovich, K. Schulten, „Neural-gas network for vector quantization and its application to time-series prediction", IEEE Transactions on Neural Networks, vol. 4, no. 4, pp. 558–569, 1993.

[4]B. Fritzke, „A Growing Neural Gas Network Learns Topologies", Proceedings of the 7th International Conference on Neural Information Processing Systems, NIPS'94, (Cambridge, MA, USA), p. 625–632, MIT Press, 1994.

[5]Y. Prudent, A. Ennaji, „An incremental growing neural gas learns topologies", Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005., vol. 2, pp. 1211–1216 vol. 2, 2005.

[6]De exemplu, lucrarea lui D. Han, K. Doya, J. Tani, „Self-organization of action hierarchy and compositionality by reinforcement learning with recurrent neural networks", Neural Networks, vol. 129, pp. 149–162, 2020.

Capitolul 2

Regresia liniară

Regresia liniară este o metodă de învățare supervizată care construiește un model ce prezice o valoare numerică continuă, pe baza trăsăturilor de intrare. Modelul construit pleacă de la presupunerea că valoarea ce trebuie prezisă depinde în mod liniar de valorile trăsăturilor de intrare.

2.1 Exemplu și notații

Să presupunem că vrem să facem estimarea (predicția) prețului unui apartament, dată fiind suprafața sa. Se cunosc date anterioare despre vânzarea unor astfel de apartamente, precum în tabelul 2.1; datele sunt culese într-o perioadă scurtă de timp, în care prețul nu variază semnificativ cu timpul. Pentru simplitatea expunerii, să presupunem că singura trăsătură considerată pentru un apartament este suprafața lui, exprimată în metri pătrați.

Suprafața (m^2)	Prețul (€)
62	87900
30	62600
54	85400
...	...

Tabelul 2.1: Valorile de vânzare ale unor apartamente, pentru care se știe doar trăsătura „suprafața”.

Pe baza acestor date vom construi o funcție, numită model de predicție, care să ne permită aproximarea prețului (un număr real) pentru alte apartamente similare, în același interval de timp, în aceeași zonă din care provin datele inițiale. Reprezentarea grafică a datelor este în figura 2.1.

Să presupunem că se dorește estimarea valorii unui apartament de suprafață $s = 80$ de metri pătrați; întrucât nu avem în setul de antrenare (sau:

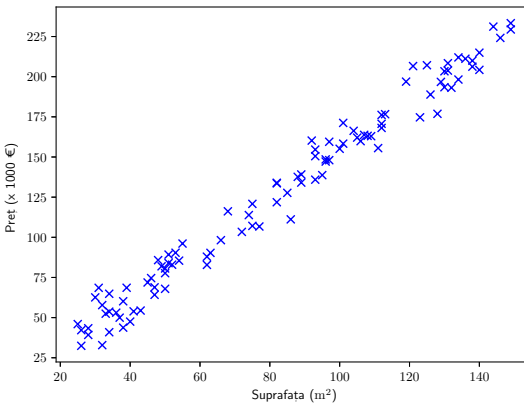


Figura 2.1: Reprezentarea grafică a datelor de vânzare a unor apartamente. Pe abscisă este măsurată suprafața (în metri pătrați), pe ordonată este prețul (în mii de euro).

de instruire) niciun apartament cu această suprafață, modelul va trebui să „ghicească" un preț. Se poate proceda în felul următor: se trasează o dreaptă care să aproximeze cât mai bine[1] norul de puncte reprezentat. Valoarea estimată de model pentru apartamentul cu o suprafață s se află simplu: se duce verticala prin punctul de coordonate $(s, 0)$ și se găsește punctul de intersecție cu dreapta dată de model; valoarea ordonatei $\hat{y}(s)$ a punctului de intersecție este prețul estimat de model – figura 2.2.

Eroarea estimării este dată de diferența dintre valoarea reală (engl: ground truth) și cea prezisă de model, pentru cazurile cunoscute. Această diferență se numește reziduu.

Modelul liniar de estimare a prețului unui apartament luând în considerare suprafața este:

$$\text{preț estimat} = a \cdot \text{suprafață} + b \tag{2.1}$$

unde a și b sunt coeficienți reali ce vor fi determinați; a se numește pantă (engl: slope), iar b termen liber (engl: intercept). Desigur, se pot folosi ca modele de predicție funcții polinomiale de grad mai mare decât 1, modele local liniare, rețele neurale artificiale etc. Alegerea celui mai bun model pentru un set de date cunoscut este o problemă în sine. Preferința pentru model liniar se motivează prin aceea că în practică e considerat un punct de plecare bun; în plus, se recomandă a fi încercate mai întâi modelele mai simple[2]. Nu în ultimul rând, *un model liniar este ușor de interpretat*: pentru problema curentă, coeficientul a este prețul pe metru pătrat (exprimat în €/m^2), b (măsurat în €) este prețul prezis de model pentru suprafață zero

[1] O formulă pentru a măsura cât de bună e aproximarea rezultată se dă în secțiunea 2.2.

[2] Așa-numitul „Brici al lui Occam": *Entia non sunt multiplicanda praeter necessitatem*, entitățile nu trebuie să fie multiplicate dincolo de necesar.

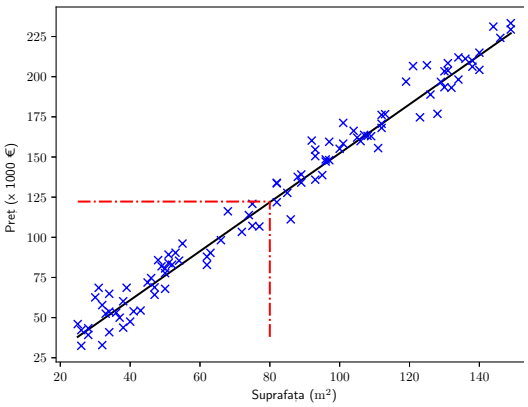


Figura 2.2: Model liniar de predicție și aproximarea prețului unui apartament de 80 de metri pătrați.

(nu are aici o interpretare fizică); creșterea valorii suprafață cu 10 m^2 duce la creșterea prețului total cu $10 \cdot a$ €.

Avem mai sus un caz de antrenare supervizată: se pornește de la un set de date cu perechi formate din valoare de intrare (*e.g.* suprafața) și valoare de ieșire asociată (*e.g.* prețul apartamentului cu acea suprafață). Se cere determinarea unui model care să fie folosit pentru prezicerea (aproximarea) unor valori de ieșire, date fiind valori de intrare furnizate; pentru exemplul considerat, vrem să estimăm prețul unor apartamente cu suprafețe date.

Formal, într-o problemă de regresie se dau:

- m, reprezentând numărul de perechi de valori (sau cazuri, sau înregistrări) din setul de antrenare; pentru desenul din figura 2.1 este numărul de puncte reprezentate, adică numărul de apartamente pentru care se știe prețul de vânzare;
- $\mathbf{x}^{(i)}$, $1 \leq i \leq m$, reprezentând cei m vectori de intrare; un vector (coloană) de intrare este compus de regulă din n valori numerice, numite atribute sau trăsături (engl: features). Vectorul de atribute $\mathbf{x}^{(i)} = \left(x_1^{(i)}, x_2^{(i)}, \ldots, x_n^{(i)}\right)^t$ conține valori numerice precum suprafața, numărul de camere, numărul de băi, gradul de poluare a zonei etc. Trăsăturile $x_j^{(i)}$ se mai numesc și variabile predictive sau independente[3]. Dacă nu se precizează altfel, astfel de vectori sunt considerați vectori coloană. Exponentul ^{t} reprezintă transpusa unui vector sau a unei matrice.

[3]A nu se confunda cu noțiunea de independență liniară din algebră, sau cu independența evenimentelor și a variabilelor aleatoare din teoria probabilităților.

- $y^{(i)}$, $1 \leq i \leq m$ – variabila de ieșire (sau dependentă; engl: ground truth) asociată vectorului $\mathbf{x}^{(i)}$; în cazul exemplificat este un număr real (prețul), dar în general poate fi un vector de valori reale. În acest caz, fiecare componentă se tratează independent, iar parametrii devin o matrice $\boldsymbol{\Theta}$.

Perechea i din setul de antrenare este $\left(\mathbf{x}^{(i)}, y^{(i)}\right)$, $1 \leq i \leq m$. Întregul set de antrenare se scrie ca:

$$\mathscr{S} = \left\{ \left(\mathbf{x}^{(i)}, y^{(i)}\right) \mid 1 \leq i \leq m \right\} \tag{2.2}$$

Setul de antrenare se specifică frecvent sub formă tabelară, precum în tabelul 2.1.

Fluxul de lucru în învățarea automată[4] este dat în figura 2.3: se pornește de la un set de antrenare (engl: training set), se aplică un algoritm de învățare și se produce un model. Din motive istorice modelul rezultat se mai numește „ipoteză” și se notează de regulă cu h. Algoritmul de antrenare are ca scop determinarea unei forme adecvate a modelului, în cazul de față a unor valori potrivite ale coeficienților funcției h.

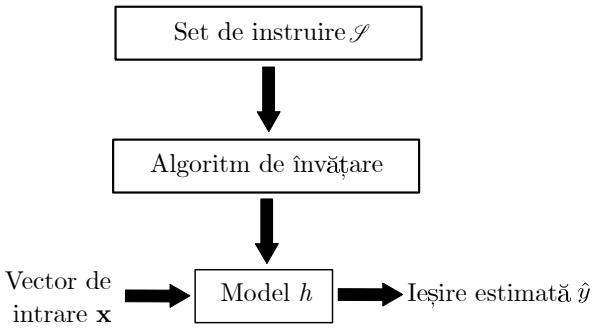


Figura 2.3: Fluxul de lucru în antrenarea unui model.

După ce antrenarea se termină, modelului rezultat i se furnizează o intrare – în exemplul nostru: suprafața apartamentului – și el va calcula o valoare de ieșire – prețul estimat de model. În notație formală avem ecuația (2.3):

$$\hat{y} = h(\mathbf{x}) \tag{2.3}$$

unde notația cu căciulă se folosește pentru valori estimate de model.

Una dintre întrebările esențiale este: ce formă are modelul h? Există mai multe variante. Mai sus am pornit de la presupunerea că prețul crește liniar cu suprafața vândută, deci:

$$h(x) = h_{\boldsymbol{\theta}}(x) = \theta_0 + \theta_1 \cdot x = \hat{y} \tag{2.4}$$

[4]În limba engleză: machine learning.

unde indicele lui h este vectorul coloană $\boldsymbol{\theta} = (\theta_0, \theta_1)^t$ de coeficienți (numiți și ponderi, parametri antrenabili, engl: weights). Comparând cu (2.1), $\theta_0 = b$ este termenul liber, iar $\theta_1 = a$ este panta.

Acest model (ipoteză) se numește regresie liniară cu o variabilă, sau regresie liniară univariată. Se poate ca pe lângă suprafață – singura valoare de intrare considerată până acum – să se mai considere și alte trăsături de intrare; în acest caz, modelul ar fi unul multivariat (mai multe valori de intrare considerate). Coeficienții θ_0 și θ_1 din ecuația (2.4) se mai numesc parametri ai modelului de predicție și se determină în etapa de învățare.

Facem următoarele precizări:

1. Permitem parametrului θ_0 să fie diferit de zero, deci dreapta de regresie nu trebuie să treacă neapărat prin origine. În cazul în care se impune ca dreapta să treacă prin origine, atunci $\theta_0 = 0$ și modelul devine $h_{\boldsymbol{\theta}}(x) = \theta_1 \cdot x$. Se lasă libertatea ca θ_0 să fie diferit de zero pentru că în practică, de regulă, nu se cunoaște dacă relația dintre trăsături și valoarea de ieșire trece sau nu prin origine. În plus, în cazul în care se consideră mai multe trăsături de intrare, este posibil ca niciuna dintre ele să nu fie corelată cu valoarea de ieșire, caz în care modelul ar trebui să prezică o constantă ce nu e obligatoriu să fie 0;

2. Modelul (2.4) este de fapt un model afin; e liniar dacă și numai dacă $\theta_0 = 0$. Totuși, folosim în continuare denumirea consacrată de model liniar;

3. Ca funcție de parametrii $\boldsymbol{\theta}$, modelul este liniar: $h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^t\mathbf{x}$ este o combinație liniară a parametrilor. Denumirea „regresie liniară” se referă la această proprietate. Trăsăturile pot fi transformate neliniar (x^2, $\log x$ ...) și modelul rămâne liniar în parametri.

Modelul (2.4) se mai poate scrie astfel:

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = \theta_0 \cdot 1 + \theta_1 \cdot x = \theta_0 \cdot x_0 + \theta_1 \cdot x_1 = \boldsymbol{\theta}^t\mathbf{x} = \hat{y} \tag{2.5}$$

unde: x_0 e mereu 1, $x_1 = x$, vectorul $\boldsymbol{\theta}$ este vectorul coloană $(\theta_0, \theta_1)^t$ ca mai sus, vectorul $\mathbf{x}$ este $(x_0, x_1)^t$. Valoarea $x_0 = 1$ permite existența unui termen liber θ_0 în modelul liniar, adică pentru cazul considerat, dreapta de regresie nu trebuie să treacă neapărat prin punctul de coordonate $(0, 0)$. Produsul scalar a doi vectori fiind comutativ, estimarea $\hat{y}$ se poate calcula și ca $\hat{y} = \mathbf{x}^t\boldsymbol{\theta}$.

Graful de calcul pentru modelul de regresie liniară (2.5) este reprezentat în figura 2.4.

2.2 Funcția de eroare

Există o infinitate de moduri în care se poate trasa dreapta din figura 2.2; altfel zis, există o infinitate de valori pentru coeficienții din modelul dat de

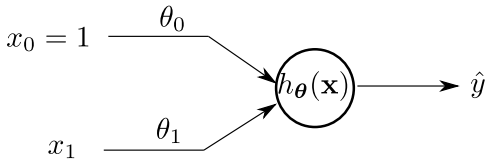


Figura 2.4: Graful de calcul pentru regresie liniară univariată

ecuația (2.4).

Cum alegem cât mai bine acești coeficienți? O variantă naturală este determinarea acestora astfel încât valorile *prezise* de model, $\hat{y}^{(i)} = h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$, să fie cât mai apropiate de valorile *cunoscute* $y^{(i)}$, pentru setul de antrenare $\mathscr{S}$ din (2.2). Pentru toate valorile din setul de antrenare, eroarea se poate măsura cu funcția de cost

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(y^{(i)} - \hat{y}^{(i)} \right)^2 = \frac{1}{2m} \sum_{i=1}^{m} \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right)^2 \tag{2.6}$$

Funcția de eroare $J : \mathbb{R}^2 \to \mathbb{R}_+$ – cu valori nenegative, fiind o medie de pătrate – se mai numește și funcție de cost a modelului (engl: error, loss, cost function; în unele lucrări se face distincție între loss function – eroare pentru un singur caz – și cost function – eroarea agregată peste mai multe valori). Se pot folosi și alte funcții de cost, de exemplu incluzând constrângeri impuse valorilor parametrilor $\boldsymbol{\theta}$ – a se vedea capitolul 4 privind regularizarea. Funcția de mai sus este o alegere populară pentru problemele de regresie, dar nu singura posibilă. Factorul m de la numitor arată că vizăm calcularea unei valori medii a erorii. Altfel, eroarea ar crește de fiecare dată când se adaugă în setul de antrenare o pereche $(\mathbf{x}^{(i)}, y^{(i)})$ pentru care $h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \neq y^{(i)}$, în timp ce media permite compararea erorilor modelului pentru mulțimi cu numere diferite de elemente; numitorul 2 poate fi omis, dar se utilizează din comoditate: la derivare, factorul 2 se simplifică. Eroarea J din formula (2.6) este jumătate din eroarea pătratică medie (engl: mean squared error):

$$\mathrm{MSE}(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^{m} \left(y^{(i)} - \hat{y}^{(i)} \right)^2 = 2 \cdot J(\theta_0, \theta_1) \tag{2.7}$$

Exemplu numeric: considerăm cele trei apartamente din tabelul 2.1, deci $m = 3$, și valorile parametrilor $\theta_0 = 40000$ € și $\theta_1 = 800$ €/m^2. Valorile sunt alese oarecum arbitrar și nu sunt cele optime; determinarea lor e prezentată în secțiunile următoare. Modelul (2.4) devine $h_{\boldsymbol{\theta}}(x) = 40000 + 800 \cdot x$, unde x este suprafața apartamentului. Valorile estimate $\hat{y}^{(i)}$, reziduurile și pătratele acestora sunt date în tabelul 2.2.

De exemplu, pentru primul apartament, $\hat{y}^{(1)} = 40000 + 800 \cdot 62 = 89600$ €. Reziduurile negative arată că modelul supraestimează prețul primelor două apartamente, iar cel pozitiv că subestimează prețul celui de-al treilea; prin

i	$x^{(i)}$ (m^2)	$y^{(i)}$ (€)	$\hat{y}^{(i)}$ (€)	$y^{(i)} - \hat{y}^{(i)}$ (€)	$\left(y^{(i)} - \hat{y}^{(i)}\right)^2$ (€^{2})
1	62	87900	89600	−1700	2890000
2	30	62600	64000	−1400	1960000
3	54	85400	83200	2200	4840000

Tabelul 2.2: Valorile estimate de modelul $h_{\boldsymbol{\theta}}(x) = 40000 + 800 \cdot x$ pentru apartamentele din tabelul 2.1, reziduurile și pătratele lor.

ridicarea la pătrat, ambele tipuri de abateri contribuie pozitiv la eroare. Valoarea funcției de eroare (2.6) este:

$$J(40000, 800) = \frac{1}{2 \cdot 3}\left((-1700)^2 + (-1400)^2 + 2200^2\right) = \\ = \frac{9690000}{6} = 1615000\,€^2$$

iar eroarea pătratică medie este $\text{MSE} = 2 \cdot J(40000, 800) = 3230000\ €^2$.

E util să discutăm comportamentul funcției J pentru cazuri particulare. De exemplu, dacă $\theta_0 = 0$, funcția de eroare $J(0, \theta_1)$ este o funcție de gradul doi depinzând de o singură variabilă (θ_1) și având minimul mai mare sau egal cu zero, a se vedea figura 2.5. Pentru θ_0 și θ_1 oarecare forma funcției de eroare este dată în figura 2.6.

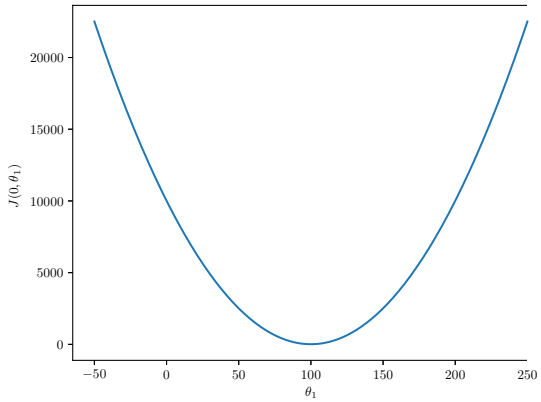


Figura 2.5: Funcția de eroare pătratică, cu $\theta_0 = 0$ și θ_1 variabil.

O altă variantă de reprezentare grafică a funcției de eroare este pe baza curbelor de nivel (de contur): reprezentarea este plană, pe axe se reprezintă θ_0 și θ_1. Pentru o valoare $v \geq 0$ se consideră mulțimea tuturor perechilor de parametri θ_0, θ_1 pentru care se obține aceeași valoare a erorii, adică $J(\theta_0, \theta_1) = v$. Rezultatul este dat de o mulțime de curbe, precum cele reprezentate în figura 2.7. Se poate arăta că pentru model de predicție liniar

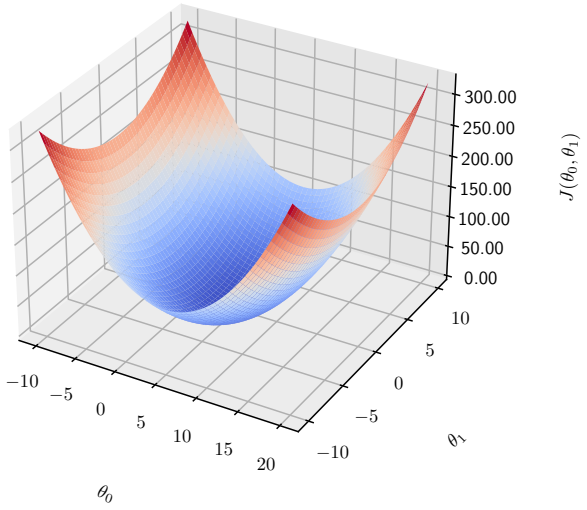


Figura 2.6: Funcția de eroare pentru model liniar univariat cu θ_0 și θ_1 variabili.

aceste contururi sunt elipse cu același centru; pentru valori v tot mai mari avem elipse tot mai mari.

Trebuie să găsim acele valori ale coeficienților θ_0^*, θ_1^* pentru care se atinge minimul funcției de eroare[5]:

$$\begin{aligned}(\theta_0^*, \theta_1^*)^t &= \arg \min_{(\theta_0,\theta_1)^t \in \mathbb{R}^2} J(\theta_0, \theta_1) = \\ &= \arg \min_{(\theta_0,\theta_1)^t \in \mathbb{R}^2} \frac{1}{2m} \sum_{i=1}^{m} \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right)^2 = \\ &= \arg \min_{(\theta_0,\theta_1)^t \in \mathbb{R}^2} \sum_{i=1}^{m} \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right)^2\end{aligned} \tag{2.8}$$

Mai sus, la trecerea la ultimul pas, am eliminat factorul $1/(2m)$ din expresia de minimizare, deoarece acesta nu afectează poziția minimului.

Vectorul de coeficienți $\boldsymbol{\theta}^* = (\theta_0^*, \theta_1^*)^t$ pentru care valoarea erorii J e cea mai mică ne dă modelul de regresie cel mai bun (cu eroarea J cea mai mică) pentru setul pe care s-a făcut antrenarea. Pentru o valoare de intrare x vom face predicția:

$$h_{\boldsymbol{\theta}^*}(\mathbf{x}) = \mathbf{x}^t \boldsymbol{\theta}^* = (1, x) \begin{pmatrix} \theta_0^* \\ \theta_1^* \end{pmatrix} = \theta_0^* + \theta_1^* \cdot x = \hat{y} \tag{2.9}$$

[5]Notația „$\arg\min f$” se referă la acele argumente pentru care se obține minimul pentru f. De exemplu, pentru un vector de valori reale $\mathbf{v} = (v_0, \ldots, v_{n-1})^t$, $\min \mathbf{v}$ este valoarea minimă, în timp ce $\arg\min_{0 \le i \le n-1} v_i$ este un indice pe care se află valoarea minimă a lui $\mathbf{v}$. Dacă minimul e atins în mai multe puncte, se ia unul dintre punctele respective.

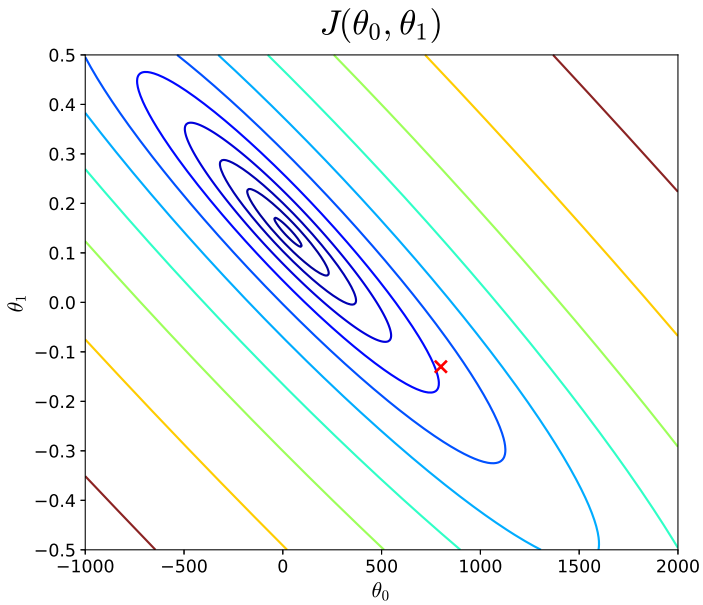


Figura 2.7: Curbe de contur pentru funcția de eroare a unui model liniar univariat, cu parametrii θ_0, θ_1. Punctul marcat cu roșu corespunde valorilor inițiale ale lui θ_0 și θ_1.

Întrebarea cea mai importantă acum este: cum determinăm valorile $\boldsymbol{\theta}$ pentru care se atinge minimul funcției de eroare J? Două astfel de metode sunt prezentate în cele ce urmează.

2.3 Metoda gradientului descendent pentru regresia liniară univariată

În această secțiune se va prezenta o metodă iterativă – metoda gradientului descendent (engl: gradient descent) – prin care se face minimizarea funcției de eroare J.

Ideea e simplă:

- se pornește cu valori θ_0, θ_1 inițiale, setate aleator sau chiar 0;
- se modifică în mod iterativ valorile curente ale parametrilor θ_0, θ_1 astfel încât J să scadă.

Pentru ultimul punct: valorile curente ale parametrilor θ_0, θ_1 se modifică conform

$$\theta_0 \leftarrow \theta_0 - \alpha \cdot \frac{\partial J}{\partial \theta_0}(\theta_0, \theta_1) \tag{2.10}$$

$$\theta_1 \leftarrow \theta_1 - \alpha \cdot \frac{\partial J}{\partial \theta_1}(\theta_0, \theta_1) \tag{2.11}$$

și (important!) atribuirile se efectuează simultan pentru θ_0, θ_1.

Această simultaneitate e cerută din cauză că la calculele (2.10), (2.11) trebuie să ne asigurăm că aceiași θ_0, θ_1 sunt folosiți pentru evaluarea ambelor derivate parțiale. Simultaneitatea se poate obține astfel: se calculează expresiile din membrii drepți ai ecuațiilor (2.10) și (2.11) și se memorează în variabile temporare $\theta_0^{(\text{temp})}$ și respectiv $\theta_1^{(\text{temp})}$; doar după ce ambele variabile temporare sunt calculate, valorile lor se atribuie corespunzător: $\theta_0 \leftarrow \theta_0^{(\text{temp})}$ și $\theta_1 \leftarrow \theta_1^{(\text{temp})}$. Alternativ, se poate folosi calcul vectorizat, în care se operează concomitent peste toate componentele vectorului $\boldsymbol{\theta}$.

Aceleași formule se scriu în mod vectorial ca:

$$\begin{pmatrix} \theta_0 \\ \theta_1 \end{pmatrix} \leftarrow \begin{pmatrix} \theta_0 \\ \theta_1 \end{pmatrix} - \alpha \cdot \begin{pmatrix} \frac{\partial J}{\partial \theta_0}(\theta_0, \theta_1) \\ \frac{\partial J}{\partial \theta_1}(\theta_0, \theta_1) \end{pmatrix} \tag{2.12}$$

Dacă folosim notația nabla pentru vectorul de derivate parțiale (gradientul)[6]:

$$\nabla_{\boldsymbol{\theta}} J = \begin{pmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \end{pmatrix} \tag{2.13}$$

atunci mai putem scrie formula de modificare a ponderilor ca:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \tag{2.14}$$

În bibliotecile care permit calcul matricial atribuirea simultană este asigurată de faptul că expresia vectorială folosește vechea valoare a lui $\boldsymbol{\theta}$ pentru calculul gradientului. O formă matricială pentru gradientul funcției de cost este dată în continuare.

Coeficientul $\alpha > 0$ se numește rată de învățare; poate fi o constantă sau o cantitate care variază de-a lungul iterațiilor. Alegerea lui α este crucială: dacă valoarea lui e prea mică, atunci algoritmul va face foarte multe iterații până se va opri, deci am avea un cost computațional mare. Dacă e prea mare, procesul poate să rateze minimul sau chiar să diveargă (valoarea lui J să crească mereu sau să alterneze perioade de scădere cu perioade de creștere). Dacă se constată acest al doilea fenomen, valoarea lui α trebuie micșorată. Odată ce o valoare potrivită pentru α este găsită, nu e neapărat nevoie ca aceasta să fie modificată de-a lungul iterațiilor: gradientul tinzând la 0, norma pasului efectiv $\alpha \cdot \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ scade și ea.

Metoda se poate folosi pentru reducerea valorilor unei funcții *diferențiabile* ce depinde de oricâte variabile. În general, se poate ajunge într-un minim

[6]Vectorul de derivate parțiale este un vector de funcții; acestea se vor evalua pentru perechea de valori θ_0, θ_1, obținând deci valori numerice. Vectorul de derivate parțiale este, totodată, transpusa matricei jacobiene a funcției J.

local, minim global sau alt punct staționar (de exemplu, punct șa) al funcției căreia i se aplică. În toate aceste situații, gradientul fiind 0, ponderile nu mai suferă modificări.

Algoritmul gradientului descendent are forma:

Algoritmul 2.1 Metoda gradientului descendent pentru regresia liniară univariată

Intrare: setul de antrenare $\mathscr{S}$, rata de învățare $\alpha > 0$, pragul $\varepsilon > 0$, numărul maxim de iterații K

Ieșire: parametrii θ_0, θ_1

1: inițializează θ_0, θ_1 cu valori aleatoare mici în jurul lui 0 sau cu 0; $k \leftarrow 0$
2: **repetă**
3: $\quad J \leftarrow J(\theta_0, \theta_1)$ ▷ valoarea funcției de cost, pentru monitorizare
4: $\quad g_j \leftarrow \dfrac{\partial J}{\partial \theta_j}(\theta_0, \theta_1)$, pentru $j = 0, 1$ (2.15) ▷ calculul gradientului
5: $\quad \theta_j \leftarrow \theta_j - \alpha\, g_j$, pentru $j = 0, 1$ (2.16)
6: $\quad k \leftarrow k + 1$
7: **până la** îndeplinirea condiției $\sqrt{g_0^2 + g_1^2} < \varepsilon$ sau $k = K$
8: **returnează** θ_0, θ_1

Mai sus, algoritmul folosește drept criteriu de oprire norma gradientului; alte variante sunt: scăderea nesemnificativă a lui J între două iterații succesive, sau o variație maximă a parametrilor sub un prag. În general, iterațiile nu ating exact gradientul nul, ci se apropie asimptotic de el; de aceea, în practică se impune uneori un număr maxim de iterații.

Putem calcula derivatele parțiale pentru forma funcției de eroare considerate, reamintind că derivata unei sume de termeni este suma derivatelor fiecărui termen:

$$\begin{aligned}
\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) &= \frac{\partial}{\partial \theta_j} \left\{ \frac{1}{2m} \sum_{i=1}^{m} \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right)^2 \right\} = \\
&= \frac{1}{2m} \sum_{i=1}^{m} \frac{\partial}{\partial \theta_j} \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right)^2 = \\
&= \frac{1}{2m} \sum_{i=1}^{m} 2 \cdot \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right) \cdot \frac{\partial}{\partial \theta_j} \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right) = \\
&= \frac{1}{m} \sum_{i=1}^{m} \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right) \cdot \left(-\frac{\partial}{\partial \theta_j} \sum_{k=0}^{1} \theta_k \cdot x_k^{(i)} \right) = \\
&= -\frac{1}{m} \sum_{i=1}^{m} \left(y^{(i)} - \hat{y}^{(i)} \right) \cdot x_j^{(i)} = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right) \cdot x_j^{(i)}
\end{aligned} \tag{2.17}$$

și derivatele parțiale g_j din ecuația (2.15) se calculează cu:

$$g_j = \frac{1}{m} \sum_{i=1}^{m} \left[\left(\hat{y}^{(i)} - y^{(i)} \right) \cdot x_j^{(i)} \right], \quad j = 0, 1 \tag{2.18}$$

În ce situație iterațiile din algoritmul de mai sus se opresc? Dacă $\boldsymbol{\theta}$ este un punct de extrem[7] al lui J, gradientul lui J în acel punct este zero (vectorul nul) și deci valoarea parametrilor $\boldsymbol{\theta}$ nu se va mai modifica, odată ce s-a atins o valoare de minim a lui J. Această observație explică toate cele trei criterii de oprire.

Se recomandă a se urmări valorile funcției de cost J; dacă ele nu au o tendință descrescătoare (funcția J crește sau are scăderi urmate de creșteri) atunci se va încerca o valoare mai mică pentru rata de învățare α; dacă valoarea funcției J scade foarte lent se poate mări valoarea lui α.

Rata de învățare α este un hiperparametru[8] care influențează succesul și viteza învățării. Valoarea optimă a lui α depinde de setul de date peste care se calculează funcția de eroare J.

Exemplu numeric: aplicăm algoritmul 2.1 pe cele trei apartamente din tabelul 2.1, pornind de la valorile $\theta_0 = 40000$ și $\theta_1 = 800$ din exemplul numeric de la pagina 26, pentru care $J(40000, 800) = 1615000$. Alegem rata de învățare $\alpha = 0.0005$.

La prima iterație, diferențele $\hat{y}^{(i)} - y^{(i)}$ sunt opusele reziduurilor din tabelul 2.2: 1700, 1400 și -2200. În formula (2.17), luând $x_0^{(i)} = 1$ și $x_1^{(i)}$ suprafața, obținem gradientul în punctul $(\theta_0, \theta_1) = (40000, 800)$:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{3} \left(1700 + 1400 + (-2200)\right) = \frac{900}{3} = 300$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{3} \left(1700 \cdot 62 + 1400 \cdot 30 + (-2200) \cdot 54\right) = \frac{28600}{3} \approx 9533.33$$

Ambele derivate parțiale sunt pozitive, deci, local, J scade dacă θ_0 și θ_1 se micșorează puțin. Derivata în raport cu θ_1 este mult mai mare decât cea în raport cu θ_0, deoarece suprafețele $x_1^{(i)}$ (zeci de metri pătrați) sunt mult mai mari decât $x_0^{(i)} = 1$. Actualizarea ponderilor duce la:

$$\theta_0 \leftarrow 40000 - 0.0005 \cdot 300 = 39999.85$$
$$\theta_1 \leftarrow 800 - 0.0005 \cdot 9533.33 \approx 795.23$$

iar valoarea funcției de eroare scade de la 1615000 la aproximativ 1598555.

[7] Sau alt punct staționar; pentru funcția de eroare considerată în acest capitol orice punct staționar este și punct de minim global.

[8] Trebuie să facem deosebire între parametri și hiperparametri. Parametrii sunt variabile ale căror valori se determină prin învățare; în cazul dat sunt ponderile $\theta_0, \ldots, \theta_n$. Hiperparametrii – aici α – sunt de regulă fixați înainte de procesul de învățare și au impact asupra modelului de predicție.

Tabelul 2.3 conține primele cinci iterații. Pe ultima coloană sunt valorile lui J obținute cu același punct de pornire, dar cu rata de învățare $\alpha = 0.001$.

	$\alpha = 0.0005$			$\alpha = 0.001$
k	θ_0	θ_1	J	J
0	40000.00	800.00	1615000	1615000
1	39999.85	795.23	1598555	1640194
2	39999.82	796.56	1597285	1701075
3	39999.75	796.19	1597181	1848168
4	39999.69	796.29	1597167	2203524
5	39999.63	796.27	1597159	3061987

Tabelul 2.3: Primele iterații ale metodei gradientului descendent pentru apartamentele din tabelul 2.1, pornind de la $\theta_0 = 40000$, $\theta_1 = 800$. Valorile sunt rotunjite.

Se observă că:

- Pentru $\alpha = 0.0005$, J scade la fiecare iterație, dar după primul pas scăderea devine foarte lentă. Panta θ_1 se stabilizează după câteva iterații, cu mici oscilații în jurul valorii 796.27, în timp ce termenul liber θ_0 se modifică doar cu câteva sutimi pe iterație. Valorile optime pentru aceste date, calculate cu metoda din secțiunea 2.5, sunt $\theta_0^* \approx 38366.35$ și $\theta_1^* \approx 827.40$, cu $J^* \approx 1500577$. Prin rularea codului, pentru $\alpha = 0.0005$ sunt necesare 25728 de iterații ca J să ajungă la cel mult 1% peste J^*.
- Pentru $\alpha = 0.001$, J crește de la o iterație la alta: rata de învățare este prea mare și algoritmul diverge, deci, conform recomandării de mai sus, valoarea lui α trebuie micșorată. Pentru aceste date se poate arăta că algoritmul diverge pentru orice α mai mare decât aproximativ 0.00078.
- Convergența lentă se datorează ordinelor de mărime foarte diferite ale trăsăturilor: $x_0 = 1$, iar x_1 ia valori între 30 și 62. Dacă suprafața este standardizată (a se vedea recomandarea de scalare a trăsăturilor din secțiunea următoare), algoritmul pornit din $\boldsymbol{\theta} = \mathbf{0}$ cu $\alpha = 0.5$ ajunge la cel mult 1% mai mult decât J^* după doar 9 iterații.

2.4 Regresia liniară multivariată

Extinderea la date de intrare cu mai multe trăsături (date multivariate), *i.e.* $\mathbf{x}^{(i)} = \left(x_1^{(i)}, x_2^{(i)}, \ldots, x_n^{(i)}\right)^t$ necesită adaptarea modelului de regresie liniară pentru a include toate trăsăturile; optimizarea parametrilor se face tot prin metoda gradientului descendent, cu modificări minime.

1. modelul de predicție se extinde la:

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = \theta_0 + \theta_1 \cdot x_1 + \cdots + \theta_n \cdot x_n = \boldsymbol{\theta}^t \mathbf{x} = \mathbf{x}^t \boldsymbol{\theta} \tag{2.19}$$

unde

$$\mathbf{x} = (x_0 = 1, x_1, \ldots, x_n)^t, \ \boldsymbol{\theta} = (\theta_0, \theta_1, \ldots, \theta_n)^t \tag{2.20}$$

Am făcut trecerea de la vectorul $\mathbf{x}^{(i)} = \left(x_1^{(i)}, x_2^{(i)}, \ldots, x_n^{(i)}\right)^t$ de n componente la unul de $n+1$ componente, prin adăugarea unei valori $x_0 = 1$, care se va înmulți cu termenul liber θ_0. Graful de calcul pentru regresia liniară multivariată (2.19) este reprezentat în figura 2.8.

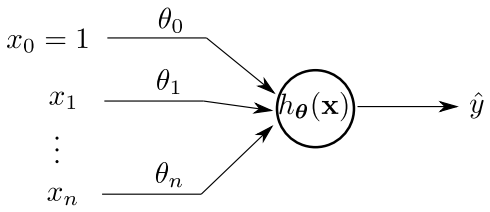


Figura 2.8: Graful de calcul pentru modelul de regresie liniară multivariată. Este o extindere a grafului reprezentat în figura 2.4.

2. funcția de eroare J se păstrează, dar argumentul ei devine vectorul $\boldsymbol{\theta}$, deci $J : \mathbb{R}^{n+1} \to \mathbb{R}_+$, iar pentru predicție se folosește modelul $h_{\boldsymbol{\theta}}$ din (2.19);

3. vrem să determinăm vectorul optim de parametri $\boldsymbol{\theta}^*$ pentru care J este minimă:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^{n+1}} J(\boldsymbol{\theta}) \tag{2.21}$$

scopul fiind același, dar numărul de parametri e $n + 1$, nu 2;

4. pentru gradientul funcției de cost avem:

$$\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^{m} \left[\left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)} \right], \quad j = 0, 1, \ldots, n \tag{2.22}$$

Pentru modelul multivariat se recomandă ca valorile trăsăturilor de intrare să aibă ordine de mărime comparabile. Acest lucru se obține făcând în prealabil o scalare a datelor la un interval convenabil ales, *e.g.* $[0, 1]$ (componenta x_0 nu se scalează). Alternativ, se poate face standardizarea datelor: datele sunt transformate astfel încât fiecare trăsătură are media zero și dispersia (varianța) 1. Pentru fiecare trăsătură $j = 1, \ldots, n$ se calculează, pe setul de antrenare, media μ_j și abaterea standard σ_j:

$$\mu_j = \frac{1}{m} \sum_{i=1}^{m} x_j^{(i)}, \qquad \sigma_j = \sqrt{\frac{1}{m} \sum_{i=1}^{m} \left(x_j^{(i)} - \mu_j \right)^2}$$

apoi fiecare valoare a trăsăturii j se înlocuiește cu

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

Ca și la scalarea la $[0, 1]$, componenta $x_0 = 1$ nu se modifică. La predicție, trăsăturile unui exemplu nou se standardizează cu aceleași valori μ_j și σ_j, calculate pe setul de antrenare. În ambele cazuri, beneficiul este un număr mult mai mic de iterații până la convergența algoritmului.

Rescriem în cele ce urmează funcția de cost (2.6) și gradientul (2.22) folosind calcul matricial. Acest lucru permite o implementare eficientă, vectorizată cu NumPy sau MATLAB.

Pentru început, notăm cu $\mathbf{X}$ matricea datelor de intrare din setul de antrenare $\mathscr{S}$:

$$\mathbf{X} = \begin{pmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_n^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_n^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(m)} & x_2^{(m)} & \cdots & x_n^{(m)} \end{pmatrix} \tag{2.23}$$

unde linia i conține valorile de intrare asociate vectorului $\mathbf{x}^{(i)}$, iar coloana j conține valorile trăsăturii j pentru toate cele m exemple. Valorile de ieșire asociate vectorilor de intrare sunt de asemenea stocate matricial, folosind un vector coloană:

$$\mathbf{y} = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{pmatrix} \in \mathbb{R}^m \tag{2.24}$$

Așa cum în (2.20) am adăugat o componentă $x_0 = 1$ pentru a permite un termen liber în modelul liniar, vom extinde matricea de date $\mathbf{X}$ din ecuația (2.23) cu o primă coloană plină cu 1; pentru simplitatea notațiilor, vom folosi și în continuare litera $\mathbf{X}$ pentru această matrice, numită matrice de design:

$$\mathbf{X} = \begin{pmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \cdots & x_n^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \cdots & x_n^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(m)} & x_2^{(m)} & \cdots & x_n^{(m)} \end{pmatrix} \tag{2.25}$$

Notăm cu $\hat{\mathbf{y}}$ vectorul de valori prezise de modelul $h_{\boldsymbol{\theta}}$ pentru matricea $\mathbf{X}$:

$$\hat{\mathbf{y}} = \begin{pmatrix} \hat{y}^{(1)} \\ \hat{y}^{(2)} \\ \vdots \\ \hat{y}^{(m)} \end{pmatrix} \in \mathbb{R}^m \tag{2.26}$$

Se verifică ușor că

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta} \tag{2.27}$$

Funcția de cost J se rescrie matricial astfel:

$$\begin{aligned} J(\boldsymbol{\theta}) &= \frac{1}{2m}\sum_{i=1}^{m}\left(y^{(i)} - \hat{y}^{(i)}\right)^2 = \\ &= \frac{1}{2m}(\mathbf{y} - \hat{\mathbf{y}})^t(\mathbf{y} - \hat{\mathbf{y}}) = \\ &= \frac{1}{2m}\left(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\right)^t\left(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\right) \end{aligned} \tag{2.28}$$

Pentru calculul vectorului gradient $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ vom utiliza următoarele relații cunoscute din algebra liniară și analiza matematică:

- transpusa unei sume de matrice este suma transpuselor:

$$(\mathbf{A} + \mathbf{B})^t = \mathbf{A}^t + \mathbf{B}^t \tag{2.29}$$

- transpusa unui produs de matrice este produsul transpuselor matricelor în ordine inversă:

$$(\mathbf{A}_1\mathbf{A}_2\cdots\mathbf{A}_p)^t = \mathbf{A}_p^t\mathbf{A}_{p-1}^t\cdots\mathbf{A}_1^t \tag{2.30}$$

- dacă c este un număr real, atunci el poate fi interpretat ca o matrice cu o linie și o coloană și din acest motiv

$$c^t = c \tag{2.31}$$

- conform excelentei lucrări de calcul matricial a lui Petersen și Pedersen[9], ecuația (69) pagina 10, avem

$$\nabla_{\boldsymbol{\theta}}\left(\boldsymbol{\theta}^t\mathbf{a}\right) = \mathbf{a} \tag{2.32}$$

  pentru $\mathbf{a}$ vector coloană;

- conform aceleiași lucrări secțiunea 2.4.2, ecuația (81):

$$\nabla_{\boldsymbol{\theta}}\left(\boldsymbol{\theta}^t\mathbf{A}\boldsymbol{\theta}\right) = \left(\mathbf{A} + \mathbf{A}^t\right)\boldsymbol{\theta} \tag{2.33}$$

  pentru $\mathbf{A}$ matrice pătratică.

[9] K. B. Petersen și M. S. Pedersen, „The matrix cookbook", 2012, disponibilă la https://www2.imm.dtu.dk/pubdb/edoc/imm3274.pdf.

Pentru calculul matricial al vectorului de derivate parțiale (a gradientului) $\left(\frac{\partial}{\partial\theta_j}J(\boldsymbol{\theta})\right)_{j=0,\ldots,n}$ dezvoltăm (2.28):

$$\begin{aligned} J(\boldsymbol{\theta}) &= \frac{1}{2m}\left(\mathbf{y}-\mathbf{X}\boldsymbol{\theta}\right)^t\left(\mathbf{y}-\mathbf{X}\boldsymbol{\theta}\right) = \\ &= \frac{1}{2m}\left\{\mathbf{y}^t\mathbf{y}-\mathbf{y}^t\left(\mathbf{X}\boldsymbol{\theta}\right)-\left(\mathbf{X}\boldsymbol{\theta}\right)^t\mathbf{y}+\left(\mathbf{X}\boldsymbol{\theta}\right)^t\mathbf{X}\boldsymbol{\theta}\right\} = \\ &= \frac{1}{2m}\left\{\boldsymbol{\theta}^t\mathbf{X}^t\mathbf{X}\boldsymbol{\theta}-2\boldsymbol{\theta}^t\mathbf{X}^t\mathbf{y}+\mathbf{y}^t\mathbf{y}\right\} \end{aligned} \tag{2.34}$$

Ținem cont de faptul că derivata parțială e operator liniar, adică derivata parțială a unei sume de funcții este suma derivatelor parțiale ale lor:

$$\nabla_{\boldsymbol{\theta}}J(\boldsymbol{\theta}) = \frac{1}{2m}\left\{\nabla_{\boldsymbol{\theta}}\left(\boldsymbol{\theta}^t\mathbf{X}^t\mathbf{X}\boldsymbol{\theta}\right)-2\nabla_{\boldsymbol{\theta}}\left(\boldsymbol{\theta}^t\mathbf{X}^t\mathbf{y}\right)+\nabla_{\boldsymbol{\theta}}\left(\mathbf{y}^t\mathbf{y}\right)\right\} \tag{2.35}$$

Considerând relațiile (2.29)–(2.33), apoi observând că scalarul $\mathbf{y}^t\mathbf{y}$ nu depinde de $\boldsymbol{\theta}$ (și deci derivarea în raport cu $\boldsymbol{\theta}$ produce valoarea 0) și că matricea $\mathbf{X}^t\mathbf{X}$ e simetrică, obținem vectorul gradient:

$$\nabla_{\boldsymbol{\theta}}J(\boldsymbol{\theta}) = \frac{1}{m}\left\{\mathbf{X}^t\mathbf{X}\boldsymbol{\theta}-\mathbf{X}^t\mathbf{y}\right\} = \frac{1}{m}\mathbf{X}^t\left(\mathbf{X}\boldsymbol{\theta}-\mathbf{y}\right) = \frac{1}{m}\mathbf{X}^t\left(\hat{\mathbf{y}}-\mathbf{y}\right) \tag{2.36}$$

Se observă, din nou, dispariția numitorului 2; se înțelege acum de ce am preferat ca funcția de eroare să fie jumătate din eroarea pătratică medie.

Algoritmul de antrenare prin metoda gradientului descendent pentru regresia liniară multivariată, în formă vectorizată, este dat în algoritmul 2.2.

Algoritmul 2.2 Metoda gradientului descendent pentru regresia liniară multivariată, în formă vectorizată

Intrare: matricea datelor de intrare $\mathbf{X}\in\mathbb{R}^{m\times n}$, vectorul valorilor de ieșire $\mathbf{y}\in\mathbb{R}^m$, rata de învățare $\alpha>0$, pragul $\varepsilon>0$, numărul maxim de iterații K

Ieșire: vectorul de parametri $\boldsymbol{\theta}\in\mathbb{R}^{n+1}$

1: adaugă la $\mathbf{X}$ o primă coloană plină cu 1, obținând matricea de design din (2.25)
2: inițializează $\boldsymbol{\theta}$ cu valori aleatoare mici sau cu vectorul nul; $k\leftarrow 0$
3: **repetă**
4: $\quad\hat{\mathbf{y}}\leftarrow\mathbf{X}\boldsymbol{\theta}$ ▷ predicțiile modelului
5: $\quad\mathbf{r}\leftarrow\mathbf{y}-\hat{\mathbf{y}}$ ▷ vectorul reziduurilor
6: $\quad J\leftarrow\frac{1}{2m}\mathbf{r}^t\mathbf{r}$ ▷ valoarea funcției de cost, pentru monitorizare
7: $\quad\mathbf{g}\leftarrow-\frac{1}{m}\mathbf{X}^t\mathbf{r}$ ▷ gradientul $\nabla_{\boldsymbol{\theta}}J(\boldsymbol{\theta})$, ec. (2.36)
8: $\quad\boldsymbol{\theta}\leftarrow\boldsymbol{\theta}-\alpha\,\mathbf{g}$ ▷ modificarea ponderilor
9: $\quad k\leftarrow k+1$
10: **până la** îndeplinirea condiției $\|\mathbf{g}\|<\varepsilon$ sau $k=K$
11: **returnează** $\boldsymbol{\theta}$

Exemplu numeric: extindem setul de antrenare din tabelul 2.1 cu încă o trăsătură, numărul de camere al fiecărui apartament, și cu încă trei apartamente. Avem acum $m = 6$ exemple și $n = 2$ trăsături de intrare, x_1 – suprafața și x_2 – numărul de camere; valoarea de ieșire y este prețul. Primele trei apartamente sunt cele din tabelul 2.1. Datele noi sunt în tabelul 2.4.

Suprafața (m^2)	Numărul de camere	Prețul (€)
62	3	87900
30	1	62600
54	2	85400
45	2	76500
75	4	104000
38	1	66000
...	...	...

Tabelul 2.4: Valorile de vânzare ale unor apartamente, pentru care se cunosc trăsăturile „suprafața” și „numărul de camere”.

Calculăm valoarea funcției de eroare pentru valorile parametrilor $\theta_0 = 39000$ €, $\theta_1 = 800$ €/m^2 și $\theta_2 = 500$ €/cameră. Modelul $h_{\boldsymbol{\theta}}(\mathbf{x}) = 39000 + 800 \cdot x_1 + 500 \cdot x_2$ păstrează panta din exemplul univariat și adaugă 500 € pentru fiecare cameră. Matricea de design (2.25), vectorul valorilor de ieșire (2.24) și vectorul de parametri sunt, respectiv:

$$\mathbf{X} = \begin{pmatrix} 1 & 62 & 3 \\ 1 & 30 & 1 \\ 1 & 54 & 2 \\ 1 & 45 & 2 \\ 1 & 75 & 4 \\ 1 & 38 & 1 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 87900 \\ 62600 \\ 85400 \\ 76500 \\ 104000 \\ 66000 \end{pmatrix}, \quad \boldsymbol{\theta} = \begin{pmatrix} 39000 \\ 800 \\ 500 \end{pmatrix}$$

Calculul urmează liniile 4–6 ale algoritmului 2.2. Predicțiile modelului pentru toate cele șase apartamente se obțin printr-un singur produs dintre o matrice și un vector:

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta} = \begin{pmatrix} 39000 + 800 \cdot 62 + 500 \cdot 3 \\ 39000 + 800 \cdot 30 + 500 \cdot 1 \\ 39000 + 800 \cdot 54 + 500 \cdot 2 \\ 39000 + 800 \cdot 45 + 500 \cdot 2 \\ 39000 + 800 \cdot 75 + 500 \cdot 4 \\ 39000 + 800 \cdot 38 + 500 \cdot 1 \end{pmatrix} = \begin{pmatrix} 90100 \\ 63500 \\ 83200 \\ 76000 \\ 101000 \\ 69900 \end{pmatrix}$$

iar vectorul reziduurilor este:

$$\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}} = \begin{pmatrix} 87900 \\ 62600 \\ 85400 \\ 76500 \\ 104000 \\ 66000 \end{pmatrix} - \begin{pmatrix} 90100 \\ 63500 \\ 83200 \\ 76000 \\ 101000 \\ 69900 \end{pmatrix} = \begin{pmatrix} -2200 \\ -900 \\ 2200 \\ 500 \\ 3000 \\ -3900 \end{pmatrix}$$

Modelul supraestimează prețul apartamentelor 1, 2 și 6 (reziduuri negative) și îl subestimează pentru celelalte trei. Produsul scalar $\mathbf{r}^t\mathbf{r}$ este suma pătratelor reziduurilor:

$$\mathbf{r}^t\mathbf{r} = (-2200)^2 + (-900)^2 + 2200^2 + 500^2 + 3000^2 + (-3900)^2 = 34950000$$

iar cu forma matricială (2.28), pentru $m = 6$, valoarea funcției de eroare este:

$$J(39000, 800, 500) = \frac{1}{2 \cdot 6}\,\mathbf{r}^t\mathbf{r} = \frac{34950000}{12} = 2912500\,€^2$$

Eroarea pătratică medie este $\text{MSE} = 2 \cdot J(39000, 800, 500) = 5825000\ €^2$.

Aplicăm acum un pas de ajustare a ponderilor prin gradient descendent, conform liniilor 7–8 ale algoritmului 2.2, cu rata de învățare $\alpha = 0.0005$, aceeași ca în exemplul univariat. Gradientul se obține din vectorul reziduurilor $\mathbf{r}$ calculat mai sus:

$$\mathbf{g} = -\frac{1}{m}\,\mathbf{X}^t\mathbf{r} = -\frac{1}{6}\begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 62 & 30 & 54 & 45 & 75 & 38 \\ 3 & 1 & 2 & 2 & 4 & 1 \end{pmatrix}\begin{pmatrix} -2200 \\ -900 \\ 2200 \\ 500 \\ 3000 \\ -3900 \end{pmatrix} =$$

$$= -\frac{1}{6}\begin{pmatrix} -1300 \\ 54700 \\ 6000 \end{pmatrix} \approx \begin{pmatrix} 216.67 \\ -9116.67 \\ -1000 \end{pmatrix}$$

Fiecare componentă a lui $\mathbf{X}^t\mathbf{r}$ este produsul scalar dintre o coloană a matricei $\mathbf{X}$ și vectorul reziduurilor; de exemplu, a treia componentă, corespunzătoare numărului de camere, este:

$$3 \cdot (-2200) + 1 \cdot (-900) + 2 \cdot 2200 + 2 \cdot 500 + 4 \cdot 3000 + 1 \cdot (-3900) = 6000$$

Deoarece $g_0 > 0$, termenul liber θ_0 va scădea, iar deoarece $g_1 < 0$ și $g_2 < 0$, coeficienții θ_1 și θ_2 vor crește. Ca în exemplul univariat, componenta corespunzătoare suprafeței este mult mai mare în valoare absolută decât

celelalte, deoarece suprafețele sunt de ordinul zecilor. Ponderile actualizate sunt:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha\,\mathbf{g} \approx \begin{pmatrix} 39000 \\ 800 \\ 500 \end{pmatrix} - 0.0005 \cdot \begin{pmatrix} 216.67 \\ -9116.67 \\ -1000 \end{pmatrix} \approx \begin{pmatrix} 38999.89 \\ 804.56 \\ 500.50 \end{pmatrix}$$

Refăcând calculul de mai sus (liniile 4–6 ale algoritmului) pentru noile valori ale parametrilor, funcția de eroare scade de la 2912500 la aproximativ 2899691.

Efectul scalării trăsăturilor este arătat în cele ce urmează. Reluăm pasul de ajustare de mai sus după ce trăsăturile de intrare sunt scalate la intervalul $[0, 1]$, conform recomandării de scalare de mai sus. Fiecare trăsătură se transformă prin

$$x'_j = \frac{x_j - \min_j}{\max_j - \min_j}$$

unde $\min_j$ și $\max_j$ sunt cea mai mică, respectiv cea mai mare valoare a trăsăturii j în setul de antrenare. Pentru suprafață, $\min_1 = 30$ și $\max_1 = 75$, deci $x'_1 = (x_1 - 30)/45$; pentru numărul de camere, $\min_2 = 1$ și $\max_2 = 4$, deci $x'_2 = (x_2 - 1)/3$. Componenta $x_0 = 1$ nu se scalează. Matricea de design devine:

$$\mathbf{X}' = \begin{pmatrix} 1 & 32/45 & 2/3 \\ 1 & 0 & 0 \\ 1 & 8/15 & 1/3 \\ 1 & 1/3 & 1/3 \\ 1 & 1 & 1 \\ 1 & 8/45 & 0 \end{pmatrix} \approx \begin{pmatrix} 1 & 0.711 & 0.667 \\ 1 & 0 & 0 \\ 1 & 0.533 & 0.333 \\ 1 & 0.333 & 0.333 \\ 1 & 1 & 1 \\ 1 & 0.178 & 0 \end{pmatrix}$$

Pentru o comparație corectă pornim de la același model. Înlocuind $x_1 = 30 + 45x'_1$ și $x_2 = 1 + 3x'_2$ în modelul de mai sus, obținem:

$$\begin{aligned} h_{\boldsymbol{\theta}}(\mathbf{x}) &= 39000 + 800 \cdot x_1 + 500 \cdot x_2 = \\ &= 39000 + 800 \cdot (30 + 45x'_1) + 500 \cdot (1 + 3x'_2) = \\ &= 63500 + 36000 \cdot x'_1 + 1500 \cdot x'_2 \end{aligned}$$

Notăm cu $\boldsymbol{\theta}' = (63500, 36000, 1500)^t$ parametrii corespunzători trăsăturilor scalate. Coeficientul $\theta'_1 = 45 \cdot 800 = 36000$ este diferența de preț estimată între apartamentul cel mai mic și cel mai mare (diferență de 45 m^2), la același număr de camere. Modelul este același, deci și predicțiile sunt aceleași:

$$\hat{\mathbf{y}} = \mathbf{X}'\boldsymbol{\theta}' = \begin{pmatrix} 63500 + 36000 \cdot 32/45 + 1500 \cdot 2/3 \\ 63500 + 36000 \cdot 0 + 1500 \cdot 0 \\ 63500 + 36000 \cdot 8/15 + 1500 \cdot 1/3 \\ 63500 + 36000 \cdot 1/3 + 1500 \cdot 1/3 \\ 63500 + 36000 \cdot 1 + 1500 \cdot 1 \\ 63500 + 36000 \cdot 8/45 + 1500 \cdot 0 \end{pmatrix} = \begin{pmatrix} 90100 \\ 63500 \\ 83200 \\ 76000 \\ 101000 \\ 69900 \end{pmatrix}$$

de unde rezultă același vector al reziduurilor $\mathbf{r}$ și aceeași valoare a funcției de eroare:

$$J(\boldsymbol{\theta}') = \frac{1}{2 \cdot 6}\,\mathbf{r}^t\mathbf{r} = \frac{34950000}{12} = 2912500\,€^2$$

Scalarea nu schimbă modelul, ci doar modul în care sunt exprimați parametrii. Diferența apare la calculul gradientului:

$$\mathbf{g}' = -\frac{1}{m}\,\mathbf{X}'^t\mathbf{r} = -\frac{1}{6}\begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 32/45 & 0 & 8/15 & 1/3 & 1 & 8/45 \\ 2/3 & 0 & 1/3 & 1/3 & 1 & 0 \end{pmatrix}\begin{pmatrix} -2200 \\ -900 \\ 2200 \\ 500 \\ 3000 \\ -3900 \end{pmatrix} =$$

$$= -\frac{1}{6}\begin{pmatrix} -1300 \\ 93700/45 \\ 7300/3 \end{pmatrix} \approx \begin{pmatrix} 216.67 \\ -347.04 \\ -405.56 \end{pmatrix}$$

De exemplu, a treia componentă a lui $\mathbf{X}'^t\mathbf{r}$ este:

$$\frac{2}{3}\cdot(-2200) + 0\cdot(-900) + \frac{1}{3}\cdot 2200 + \frac{1}{3}\cdot 500 + 1\cdot 3000 + 0\cdot(-3900) = \frac{7300}{3}$$

Fără scalare, gradientul era $\mathbf{g} \approx (216.67, -9116.67, -1000)^t$; acum componentele lui $\mathbf{g}'$ au ordine de mărime apropiate. Prima componentă a rămas aceeași, deoarece $x_0 = 1$ nu se scalează.

Deoarece trăsăturile au acum valori în $[0, 1]$, putem folosi o rată de învățare mult mai mare, $\alpha = 1$. Ponderile actualizate sunt:

$$\boldsymbol{\theta}' \leftarrow \boldsymbol{\theta}' - \alpha\,\mathbf{g}' \approx \begin{pmatrix} 63500 \\ 36000 \\ 1500 \end{pmatrix} - 1 \cdot \begin{pmatrix} 216.67 \\ -347.04 \\ -405.56 \end{pmatrix} \approx \begin{pmatrix} 63283.33 \\ 36347.04 \\ 1905.56 \end{pmatrix}$$

Cu noii parametri, predicțiile și reziduurile sunt:

$$\hat{\mathbf{y}} = \mathbf{X}'\boldsymbol{\theta}' \approx \begin{pmatrix} 90400.49 \\ 63283.33 \\ 83303.60 \\ 76034.20 \\ 101535.93 \\ 69745.03 \end{pmatrix}, \quad \mathbf{r} = \mathbf{y} - \hat{\mathbf{y}} \approx \begin{pmatrix} -2500.49 \\ -683.33 \\ 2096.40 \\ 465.80 \\ 2464.07 \\ -3745.03 \end{pmatrix}$$

iar noua valoare a funcției de eroare este:

$$J(\boldsymbol{\theta}') = \frac{1}{2 \cdot 6}\,\mathbf{r}^t\mathbf{r} \approx \frac{31428119}{12} \approx 2619010\,€^2$$

Comparăm cele două variante:

- fără scalare, un pas cu $\alpha = 0.0005$ scade J de la 2912500 la aproximativ 2899691, adică cu 0.44%; cu scalare, un pas cu $\alpha = 1$ scade J la aproximativ 2619010, adică cu 10.1%;
- exprimați în unitățile inițiale ($\theta_j = \theta'_j/(\max_j - \min_j)$ pentru $j = 1, 2$ și $\theta_0 = \theta'_0 - \theta_1 \min_1 - \theta_2 \min_2$), noii parametri sunt $\theta_0 \approx 38416.79$, $\theta_1 \approx 807.71$ și $\theta_2 \approx 635.19$. Fără scalare, θ_2 crescuse doar de la 500 la 500.50; acum crește la 635.19, apropiindu-se de valoarea optimă $\theta_2^* \approx 1764.56$, determinată cu metoda din secțiunea 2.5;
- pentru ca J să ajungă la cel mult 1% peste minimul $J^* \approx 1941034$ sunt necesare 36909 de iterații fără scalare, cu $\alpha = 0.0005$, și doar 13 iterații cu scalare, cu $\alpha = 1$.

La predicție, trăsăturile unui apartament nou se scalează cu aceleași valori $\min_j$ și $\max_j$, calculate pe setul de antrenare.

2.5 Metoda ecuațiilor normale

Există o metodă care dă valorile optime $\boldsymbol{\theta}^*$ pe baza unui calcul algebric.

Valorile căutate pentru $\boldsymbol{\theta}$ sunt cele care produc minimul valorii lui J. Conform teoremei lui Fermat, o condiție necesară pentru ca $\boldsymbol{\theta}^*$ să minimizeze pe J este ca vectorul derivatelor parțiale calculat în $\boldsymbol{\theta}^*$ să fie vectorul nul:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^*) = \mathbf{0} \tag{2.37}$$

unde $\mathbf{0}$ este vectorul coloană format din $n+1$ valori de zero.

Studiind matricea hessiană a lui J, se poate arăta ușor că J e convexă; rezultă că orice punct în care gradientul se anulează este totodată și punct de minim global[10] al lui J. Înlocuind formula gradientului $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ din ecuația (2.36) în (2.37) obținem:

$$\mathbf{X}^t \mathbf{X} \boldsymbol{\theta}^* - \mathbf{X}^t \mathbf{y} = \mathbf{0} \tag{2.38}$$

sau

$$\mathbf{X}^t \mathbf{X} \boldsymbol{\theta}^* = \mathbf{X}^t \mathbf{y} \tag{2.39}$$

Sistemul (2.39) este un sistem de ecuații liniare cu $n+1$ necunoscute, care se numește sistemul de ecuații normale. Mai departe, dacă matricea $\mathbf{X}^t\mathbf{X}$ este inversabilă (nesingulară), vectorul de parametri $\boldsymbol{\theta}^*$ se determină ca

$$\boldsymbol{\theta}^* = \left(\mathbf{X}^t \mathbf{X}\right)^{-1} \mathbf{X}^t \mathbf{y} \tag{2.40}$$

Precizări:

[10] Minimul este unic dacă și numai dacă matricea $\mathbf{X}$ are coloanele liniar independente, ceea ce face ca funcția J să fie strict convexă. Altfel, punctele de minim formează un subspațiu afin, conținând o infinitate de puncte, toate cu aceeași valoare minimă a lui J.

1. Dacă $\mathbf{X}^t\mathbf{X}$ e inversabilă, expresia $\left(\mathbf{X}^t\mathbf{X}\right)^{-1}\mathbf{X}^t$ coincide cu pseudoinversa Moore–Penrose $\mathbf{X}^+$; pentru o matrice inversabilă inversa și pseudoinversa ei coincid; pentru calculul pseudoinversei unei matrice se poate folosi în Octave și MATLAB funcția `pinv`, iar în Python funcția `numpy.linalg.pinv` apelată pentru matricea $\mathbf{X}$;

2. Pseudoinversa există pentru orice matrice, pătratică sau nu, inversabilă sau nu; se calculează, de exemplu, prin descompunerea valorilor singulare;

3. Când se folosește metoda ecuațiilor normale, nu este necesar să se facă scalarea trăsăturilor de intrare, precum se recomandă la metoda iterativă.

O problemă care trebuie discutată este: cum se procedează când matricea $\mathbf{X}^t\mathbf{X}$ este singulară? Acest lucru se datorează de regulă uneia dintre situațiile de mai jos:

- există trăsături de intrare redundante, de exemplu două coloane ale lui $\mathbf{X}$ sunt liniar dependente; în acest caz avem în mod clar o redundanță informațională și putem elimina oricare dintre aceste două coloane; mai general, una dintre coloane poate fi combinație liniară a altor coloane și dacă se știe care e, se poate elimina;

- se folosesc prea multe trăsături față de numărul de cazuri din setul de antrenare (mai precis, pentru $m \leq n$); în acest caz se poate renunța la câteva trăsături, adică se elimină coloane din $\mathbf{X}$, sau se folosește regularizarea – a se vedea capitolul 4.

Ordinea de mai sus este cea sugerată pentru acționare: se elimină coloanele redundante, apoi dacă încă e nevoie, se folosește regularizarea.

Dat fiind faptul că avem două metode de determinare a lui $\boldsymbol{\theta}^*$, se pune problema pe care dintre ele să o preferăm. Iată câteva comparații:

1. În timp ce pentru metoda gradientului descendent trebuie ca rata de învățare să fie aleasă cu grijă, pentru varianta algebrică așa ceva nu e necesar, neavând de fapt rată de învățare;

2. La metoda de calcul bazată pe gradientul descendent sunt necesare mai multe iterații, pe când metoda algebrică nu este iterativă; vectorul optim $\boldsymbol{\theta}^*$ se obține printr-un singur calcul;

3. Pentru n mare, calculul pseudoinversei e prohibitiv din punct de vedere al memoriei și timpului de calcul necesar. Metoda gradientului descendent funcționează bine pentru n mare și m nu foarte mare; pentru m mare se folosesc variantele stochastic gradientul descent stochastic (engl: stochastic gradient descent) și pe loturi de date (engl: mini-batch), prezentate în secțiunea 6.9.2;

4. Metoda gradientului descendent se aplică și funcțiilor de cost fără soluție analitică – de exemplu funcția de cost a regresiei logistice (capitolul 3); ecuațiile normale există doar pentru cost pătratic și model liniar în parametri.

Capitolul 3

Regresia logistică

3.1 Încadrare și exemple

Regresia logistică este folosită pentru estimarea probabilităților condiționate și pentru clasificare. Inițial dezvoltată pentru recunoașterea a două clase (categorii), a fost extinsă pentru a discrimina între oricâte categorii.

Ca mod de antrenare se folosește învățarea supervizată. Intrările sunt vectori numerici, iar clasele sunt fie două – pentru regresia logistică binară – fie mai multe – pentru regresia logistică multinomială.

Exemple de probleme de clasificare cu două clase, tratate de regresia logistică (binară), sunt:

- clasificarea unui e-mail ca fiind de tip spam sau legitim, dându-se atribute ale e-mailului: conținutul lui, subiectul e-mailului, faptul că expeditorul face sau nu parte din lista de contacte etc.;
- clasificarea unei tumori ca fiind benignă sau malignă, date fiind rezultatele de laborator;
- clasificarea unei imagini: conține un câine sau o pisică (aici presupunem că fiecare imagine are exact unul dintre aceste două animale); sau vrem ca modelul să învețe să recunoască dacă imaginea conține sau nu un anumit obiect.

Exemple de probleme pentru care există mai mult de două clase sunt:

- clasificarea unui e-mail ca fiind de tip: știri, muncă, socializare, anunțuri, spam etc.;
- clasificarea unei imagini ca reprezentând măr, pară, banană, cireașă etc.

E important să subliniem că numărul de clase trebuie cunoscut a priori.

Modelul dat de regresia logistică (fie ea binară sau multinomială) construiește o estimare a probabilității condiționate, dată fiind intrarea curentă (conținut e-mail, imagine etc.); mai precis, se determină care este probabilitatea ca obiectul descris de vectorul de intrare să fie dintr-o clasă anume:

$$P(\text{clasa}_1|\text{vector_de_intrare}), \ldots, P(\text{clasa}_K|\text{vector_de_intrare}) \tag{3.1}$$

de exemplu probabilitățile ca e-mailul să fie de tip știre, respectiv muncă, socializare etc. Faptul că se estimează probabilități, adică valori continue din $(0, 1)$, justifică cuvântul „regresie" din denumirile modelului.

Odată ce avem un model care produce probabilități condiționate, clasificarea se face găsind clasa_k pentru care probabilitatea ei din (3.1) este maximă.

3.2 Regresia logistică binară

3.2.1 Setul de antrenare

În cazul regresiei logistice binare se urmărește discriminarea între două clase. Clasele sunt convenabil date ca fiind „1" – clasa pozitivă – și respectiv „0" – clasa negativă[1]. Setul de antrenare este de forma:

$$\mathscr{S} = \left\{ \left(\mathbf{x}^{(i)}, y^{(i)} \right) \mid 1 \leq i \leq m \right\} \tag{3.2}$$

unde vectorul coloană $\mathbf{x}^{(i)} = \left(x_0^{(i)}, x_1^{(i)}, \ldots, x_n^{(i)} \right)^t \in \mathbb{R}^{n+1}$ conține valorile trăsăturilor obiectului i, iar $y^{(i)} \in \{0, 1\}$ este clasa căreia îi aparține obiectul i (engl: ground truth). Ca și până acum, notația $\mathbf{v}^t$ reprezintă transpunerea vectorului sau a matricei $\mathbf{v}$, iar m e numărul de date din setul de antrenare. Vom considera că $x_0^{(i)} = 1$ pentru orice i, pentru a permite un termen liber în discriminatorul implementat de regresia logistică.

3.2.2 Exemplu numeric: setul de date

În exemplele numerice pentru regresia logistică binară folosim apartamentele din capitolul 2, transformând problema de regresie într-una de clasificare: un apartament este de clasă pozitivă ($y = 1$) dacă prețul lui depășește 80000 € și de clasă negativă ($y = 0$) altfel. Singura trăsătură este suprafața $x_1 = s$, măsurată în m^2, deci $\mathbf{x} = (1, s)^t$. Datele sunt în tabelul 3.1: primele șase apartamente sunt cele din tabelul 2.4, la care am adăugat încă două. Fără ultimele două, datele ar fi liniar separabile: apartamentele de cel mult 45 m^2 sunt toate de clasă negativă, cele de cel puțin 54 m^2 sunt toate de clasă pozitivă, iar orice prag de suprafață dintre 45 și 54 m^2 desparte perfect

[1] De exemplu, clasa pozitivă poate fi „e-mail de tip spam" sau „poză cu pisică". Clasa negativă este „e-mail legitim" și respectiv „poză cu câine".

cele două clase. Pentru date liniar separabile, funcția de eroare definită în secțiunea 3.2.4 nu are punct de minim, iar coeficienții cresc nelimitat pe parcursul instruirii. Cu cele două apartamente adăugate, clasele se suprapun – apartamentul de 50 m^2 este de clasă pozitivă, iar cel de 58 m^2 de clasă negativă –, deci niciun prag de suprafață nu le mai desparte perfect.

Suprafața s (m^2)	Prețul (€)	Clasa y
62	87900	1
30	62600	0
54	85400	1
45	76500	0
75	104000	1
38	66000	0
50	81000	1
58	79000	0

Tabelul 3.1: Apartamentele folosite în exemplele numerice pentru regresia logistică binară; $y = 1$ dacă prețul depășește 80000 €. Primele șase sunt cele din tabelul 2.4; ultimele două sunt adăugate pentru ca datele să nu fie liniar separabile.

3.2.3 Reprezentarea modelului

Pentru regresia logistică modelul de predicție trebuie să producă o valoare reprezentând probabilitatea condiționată $P(\text{clasa} = 1|\text{vector_de_intrare})$. Vom folosi în acest scop o funcție $h_{\boldsymbol{\theta}}(\cdot)$:

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = P(y = 1|\mathbf{x}; \boldsymbol{\theta}) = \frac{1}{1 + \exp(-\boldsymbol{\theta}^t \mathbf{x})} = \hat{y} \tag{3.3}$$

unde $\boldsymbol{\theta} = (\theta_0, \theta_1, \ldots, \theta_n)^t \in \mathbb{R}^{n+1}$ este un vector coloană cu $n + 1$ parametri – sau coeficienți, sau ponderi (engl: weights) – ce vor fi determinați prin procesul de învățare. Se arată ușor că funcția $h_{\boldsymbol{\theta}}$ este cu valori în intervalul $(0, 1)$, deci putem să o folosim pe post de probabilitate. Probabilitatea din (3.3) este o probabilitate condiționată de intrarea curentă – adică vectorul $\mathbf{x}$ – și parametrizată de $\boldsymbol{\theta}$. $P(y = 1|\mathbf{x}; \boldsymbol{\theta})$ este gradul de încredere al modelului că obiectul descris de vectorul $\mathbf{x}$ face parte din clasa 1, iar încrederea este influențată și de ponderile din vectorul $\boldsymbol{\theta}$.

Funcția care stă la baza definirii modelului $h_{\boldsymbol{\theta}}$ este:

$$\sigma : \mathbb{R} \to (0, 1),\ \sigma(z) = \frac{1}{1 + \exp(-z)} \tag{3.4}$$

și numită sigmoida logistică – engl: logistic sigmoid, expit. Sigmoida logistică este inversa unei alte funcții larg folosite, logit : $(0, 1) \to \mathbb{R}$, definite ca

$$\text{logit}(p) = \ln \frac{p}{1-p} \tag{3.5}$$

Sigmoida logistică este reprezentată grafic în figura 3.1. Codomeniul funcției logistice este $(0, 1)$, compatibil cu valorile admisibile pentru funcție de probabilitate; extremele 0 și 1 care sunt permise pentru probabilități nu se ating însă de către sigmoida logistică. Funcția σ este derivabilă, strict crescătoare și $\lim_{z\to-\infty} \sigma(z) = 0$, $\lim_{z\to\infty} \sigma(z) = 1$. Denumirea de „sigmoidă" este dată de alura graficului, amintind de litera S.

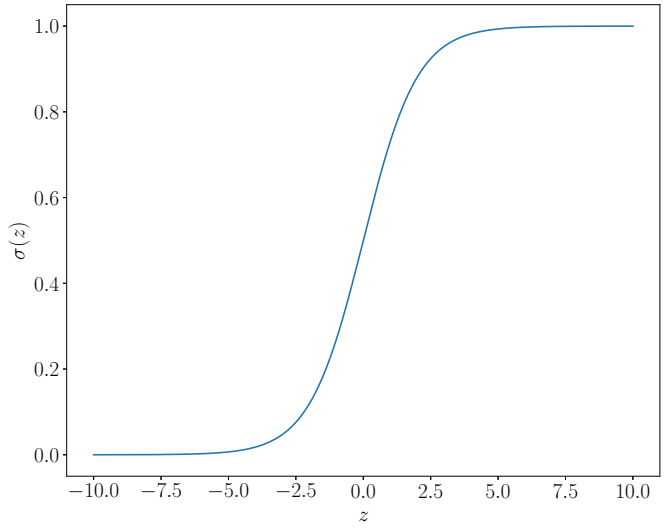


Figura 3.1: Graficul sigmoidei logistice definite în ecuația (3.4)

Graful de calcul implementat de regresia logistică binară este dat în figura 3.2. În context de rețele neurale artificiale, funcția σ se mai numește și funcție de activare, iar regresia logistică binară este un neuron cu o singură unitate de ieșire, cu funcția de activare σ și cu $n + 1$ intrări.

Pentru funcția σ următoarele proprietăți sunt notabile:

$$1 - \sigma(z) = \sigma(-z) \tag{3.6}$$

iar derivata lui σ se calculează ușor pe baza lui $\sigma(z)$:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \tag{3.7}$$

Probabilitatea ca obiectul $\mathbf{x}$ să fie de clasă 0, sau clasă negativă, este $P(y = 0|\mathbf{x}; \boldsymbol{\theta})$ și e determinată ca fiind complementul față de 1 al probabilității

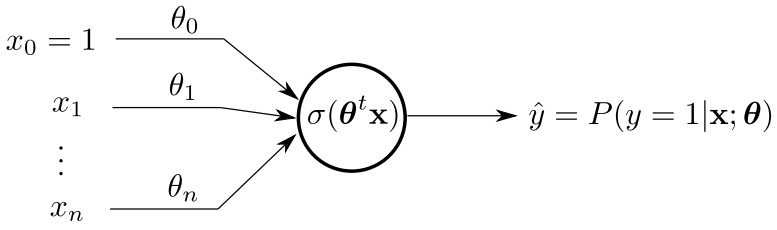


Figura 3.2: Graf de calcul pentru regresia logistică binară

evenimentului de a fi de clasă pozitivă[2]:

$$P(y = 0|\mathbf{x}; \boldsymbol{\theta}) = 1 - P(y = 1|\mathbf{x}; \boldsymbol{\theta}) = 1 - \sigma(\boldsymbol{\theta}^t \mathbf{x}) = \sigma(-\boldsymbol{\theta}^t \mathbf{x}) \tag{3.8}$$

Dorim să determinăm ponderile din vectorul $\boldsymbol{\theta}$ astfel încât:

1. pentru acei vectori $\mathbf{x}^{(i)}$ pentru care eticheta asociată $y^{(i)}$ este 1 să avem $P(y^{(i)} = 1|\mathbf{x}^{(i)}; \boldsymbol{\theta}) = h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$ cât mai aproape de 1, adică de $y^{(i)}$;

2. pentru datele de intrare $\mathbf{x}^{(i)}$ cu $y^{(i)} = 0$ să avem $P(y^{(i)} = 0|\mathbf{x}^{(i)}; \boldsymbol{\theta})$ cât mai apropiată de 1; echivalent, vrem aici ca $P(y^{(i)} = 1|\mathbf{x}^{(i)}; \boldsymbol{\theta}) = h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$ să fie cât mai aproape de 0, adică de $y^{(i)}$.

Evident, în ambele cazuri, dacă $P(y^{(i)} = 1|\mathbf{x}^{(i)}; \boldsymbol{\theta})$ este apropiat de $y^{(i)}$, atunci și $P(y^{(i)} = 0|\mathbf{x}^{(i)}; \boldsymbol{\theta})$ e apropiat de $1 - y^{(i)}$, deci estimările de probabilitate $\hat{y}^{(i)}$ sunt apropiate de etichetele $Y^{(i)}$ – ceea ce vrem de la un model bun[3].

Ponderile din vectorul $\boldsymbol{\theta}$ vor fi determinate prin învățare automată (engl: machine learning).

3.2.4 Funcția de cost

Funcția care ne permite să decidem cât de bun este un vector de ponderi $\boldsymbol{\theta}$ și care e totodată utilizată pentru ajustarea ponderilor în procesul de antrenare este notată tradițional cu $J(\cdot)$, $J : \mathbb{R}^{n+1} \to \mathbb{R}_+$. Argumentul ei este vectorul de ponderi $\boldsymbol{\theta} = (\theta_0, \theta_1, \ldots, \theta_n)^t \in \mathbb{R}^{n+1}$. Valoarea ei se calculează peste setul de antrenare $\mathscr{S}$ din (3.2), sau peste orice mulțime cu perechi formate din vectori de intrare și clasă de ieșire asociată.

Considerarea procentului de clasificări corecte – acuratețea – nu e o soluție: funcția de acuratețe este discretă, producând valori în mulțimea $\{0, \frac{1}{m}, \frac{2}{m}, \ldots, 1\}$, și deci nu permite calculul derivatelor parțiale.

[2]Avem două evenimente complementare: un obiect e fie de clasă pozitivă, fie negativă. Suma probabilităților acestor două evenimente este 1, conform axiomelor care fundamentează teoria probabilităților.

[3]Pentru moment nu ne preocupăm de posibilitatea de overfitting, chestiune discutată în capitolul următor.

O altă posibilitate este dată de utilizarea aceleiași funcții de eroare din capitolul de regresie liniară, ecuația (2.28) pagina 35. Se arată însă că pentru problema estimării de probabilitate condiționată, dată fiind forma funcției $h_{\boldsymbol{\theta}}$ din ecuația (3.3), funcția de cost nu mai este convexă și în acest caz o căutare bazată pe gradient se poate opri într-un minim local – situație exemplificată în figura 3.3.

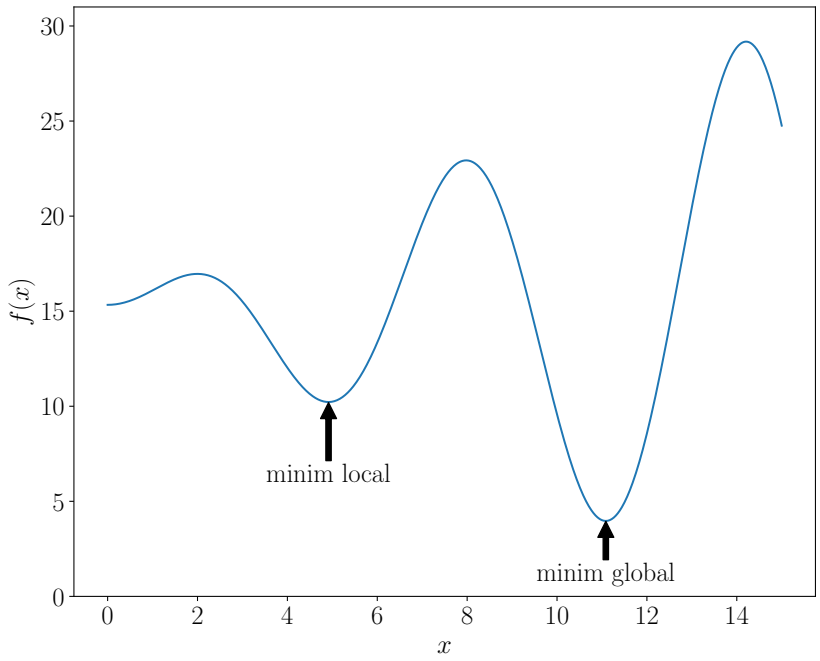


Figura 3.3: Minim global și minim local pentru funcția neconvexă $f : [0, 15] \to \mathbb{R}$, $f(x) = 15 + \sin(x) \cdot x + \frac{1}{x^2+3}$

Vom defini J astfel încât să fie convexă, deci cu proprietatea că orice punct de minim este și minim global. Pentru fiecare pereche de intrare-ieșire asociată $(\mathbf{x}^{(i)}, y^{(i)})$ vom considera un termen de eroare Cost $\left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}\right)$, iar funcția de cost va fi media acestor erori:

$$J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^{m} \text{Cost}\left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}\right) \tag{3.9}$$

și în plus funcția $\text{Cost}(\cdot, \cdot)$ să îndeplinească următoarele condiții de (di)similaritate[4]:

[4]În cele ce urmează, pentru simplificarea notațiilor vom renunța la indicele i.

1. condiția de apropiere: dacă $h_{\boldsymbol{\theta}}(\mathbf{x})$ și y sunt valori apropiate, atunci valoarea lui $\text{Cost}(h_{\boldsymbol{\theta}}(\mathbf{x}), y)$ trebuie să fie apropiată de 0, și reciproc;
2. condiția de depărtare: dacă $h_{\boldsymbol{\theta}}(\mathbf{x})$ și y sunt valori îndepărtate, atunci valoarea lui $\text{Cost}(h_{\boldsymbol{\theta}}(\mathbf{x}), y)$ trebuie să fie mare, și reciproc.

Definim convenabil funcția $\text{Cost}(\cdot, \cdot)$ astfel:

$$\text{Cost}(h_{\boldsymbol{\theta}}(\mathbf{x}), y) = \begin{cases} -\ln h_{\boldsymbol{\theta}}(\mathbf{x}) & \text{dacă } y = 1 \\ -\ln(1 - h_{\boldsymbol{\theta}}(\mathbf{x})) & \text{dacă } y = 0 \end{cases} \tag{3.10}$$

Cele două ramuri ale funcției Cost sunt reprezentate în figura 3.4. Dreptele $h_{\boldsymbol{\theta}} = 0$ și respectiv $h_{\boldsymbol{\theta}} = 1$ sunt asimptote verticale pentru cele două ramuri ale lui Cost.

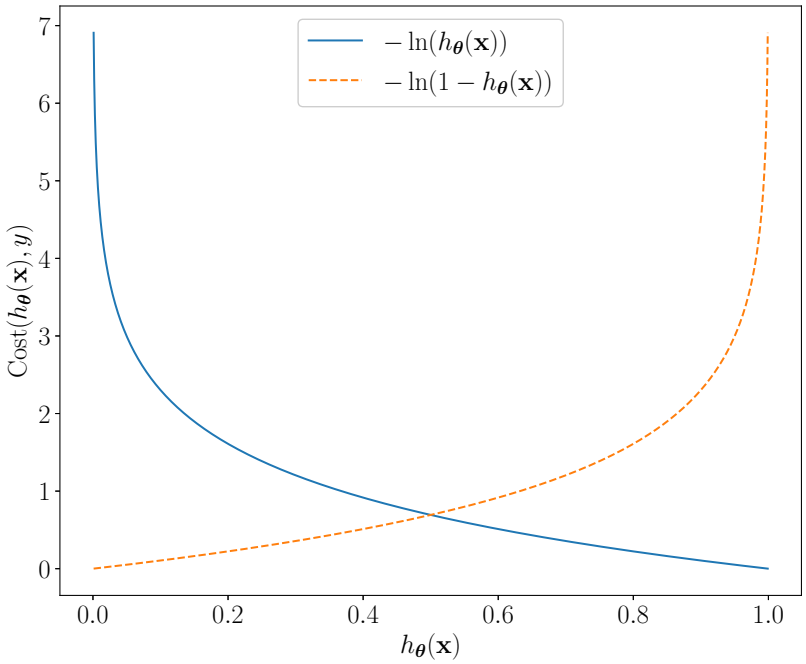


Figura 3.4: Cele două ramuri ale funcției Cost din ecuația (3.10)

Rescriem funcția Cost sub forma echivalentă și mai compactă:

$$\begin{aligned} \text{Cost}(h_{\boldsymbol{\theta}}(\mathbf{x}), y) &= -y \cdot \ln h_{\boldsymbol{\theta}}(\mathbf{x}) - (1 - y) \cdot \ln(1 - h_{\boldsymbol{\theta}}(\mathbf{x})) && (3.11) \\ &= -y \cdot \ln \hat{y} - (1 - y) \cdot \ln(1 - \hat{y}) && (3.12) \end{aligned}$$

Să verificăm că se îndeplinesc cele două condiții cerute mai sus. Pentru condiția de apropiere, vrem să verificăm că:

$$\text{Cost}(h_{\boldsymbol{\theta}}(\mathbf{x}), y) \approx 0 \Leftrightarrow h_{\boldsymbol{\theta}}(\mathbf{x}) \approx y \tag{3.13}$$

Pentru cazul $y = 1$ avem:

$$\text{Cost}(h_{\boldsymbol{\theta}}(\mathbf{x}), y) \approx 0 \Leftrightarrow -\ln h_{\boldsymbol{\theta}}(\mathbf{x}) \approx 0 \Leftrightarrow h_{\boldsymbol{\theta}}(\mathbf{x}) \approx 1 = y \Leftrightarrow h_{\boldsymbol{\theta}}(\mathbf{x}) \approx y$$

Pentru cazul $y = 0$, relația (3.13) devine:

$$\text{Cost}(h_{\boldsymbol{\theta}}(\mathbf{x}), y) \approx 0 \Leftrightarrow -\ln(1 - h_{\boldsymbol{\theta}}(\mathbf{x})) \approx 0 \Leftrightarrow h_{\boldsymbol{\theta}}(\mathbf{x}) \approx 0 = y \Leftrightarrow h_{\boldsymbol{\theta}}(\mathbf{x}) \approx y$$

deci prima condiție, legată de apropierea de 0 a lui $\text{Cost}(h_{\boldsymbol{\theta}}(\mathbf{x}), y)$, este îndeplinită de definiția din ecuația (3.12).

Pentru condiția de depărtare, dacă[5] $\text{Cost}(h_{\boldsymbol{\theta}}(\mathbf{x}), y) = M \gg 0$, atunci obținem echivalent $h_{\boldsymbol{\theta}}(\mathbf{x}) = \exp(-M)$ (pentru $y = 1$), respectiv $h_{\boldsymbol{\theta}}(\mathbf{x}) = 1 - \exp(-M)$ (pentru $y = 0$); y este unul dintre capetele intervalului $[0, 1]$, iar dacă M este o valoare din ce în ce mai mare, atunci $h_{\boldsymbol{\theta}}(\mathbf{x})$ se îndreaptă spre celălalt capăt al intervalului. Rezultă deci că și această a doua condiție este îndeplinită de definiția din formula (3.12).

Se poate arăta că funcțiile $-\ln \sigma(\boldsymbol{\theta}^t \mathbf{x})$ și $-\ln(1 - \sigma(\boldsymbol{\theta}^t \mathbf{x}))$ sunt convexe în $\boldsymbol{\theta}$. Funcția J, ca sumă a unui număr finit de funcții convexe, este ea însăși convexă. Ca atare, orice punct de minim al lui J este garantat și minim global.

Funcția de eroare J se rescrie astfel:

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \cdot \ln h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) + \left(1 - y^{(i)} \right) \cdot \ln \left(1 - h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) \right) \right] \tag{3.14}$$

$$= -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \cdot \ln \hat{y}^{(i)} + \left(1 - y^{(i)} \right) \cdot \ln \left(1 - \hat{y}^{(i)} \right) \right] \tag{3.15}$$

și această formă se numește binary cross-entropy.

Dacă considerăm vectorii coloană $\mathbf{y} = \left(y^{(1)}, \ldots, y^{(m)} \right)^t$ și respectiv $\hat{\mathbf{y}} = \left(\hat{y}^{(1)}, \ldots, \hat{y}^{(m)} \right)^t$, funcția de eroare binary cross-entropy se rescrie vectorizat ca:

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \left(\mathbf{y}^t \cdot \ln \hat{\mathbf{y}} + (\mathbb{1}_m - \mathbf{y})^t \cdot \ln(\mathbb{1}_m - \hat{\mathbf{y}}) \right) \tag{3.16}$$

unde $\mathbb{1}_m$ este vectorul coloană cu m componente, toate egale cu 1, iar funcția logaritm se aplică punctual, pe fiecare componentă a vectorului argument. Pentru calculul vectorizat al lui $\hat{\mathbf{y}}$ a se vedea formula (3.26) pagina 54.

Observăm că se folosesc produse scalare de vectori, de exemplu $\mathbf{y}^t \cdot \ln \hat{\mathbf{y}}$, deci $J(\boldsymbol{\theta})$ este într-adevăr un număr.

3.2.5 Exemplu numeric: funcția de cost pentru parametri nuli

Pentru datele din tabelul 3.1 și $\boldsymbol{\theta} = \mathbf{0}$, toate predicțiile sunt $\hat{y}^{(i)} = \sigma(0) = 0.5$. Fiecare termen din paranteza dreaptă a ecuației (3.15) este atunci

[5] Relația $\gg$ este „mult mai mare decât”.

egal cu $\ln 0.5 = -\ln 2$, indiferent de clasa $y^{(i)}$, deci $J(\mathbf{0}) = \ln 2 \approx 0.6931$. Aceasta este eroarea unui model care atribuie probabilitatea 0.5 oricărei intrări, adică nu favorizează niciuna dintre clase. Valoarea este un reper util pentru exemplele care urmează: deoarece $\boldsymbol{\theta} = \mathbf{0}$ este unul dintre vectorii de parametri posibili, valoarea minimă a lui J este cel mult $\ln 2$, oricare ar fi setul de antrenare.

3.2.6 Algoritmul de antrenare

Setul de antrenare $\mathscr{S}$ este utilizat pentru a deduce valori adecvate ale lui $\boldsymbol{\theta}$, astfel încât predicțiile $h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$ date de model pentru valorile de intrare $\mathbf{x}^{(i)}$ să fie cât mai apropiate de valorile reale ale etichetelor corespunzătoare $y^{(i)}$. Datorită proprietăților funcției Cost din ecuația (3.10) și a faptului că J este valoarea medie a funcției Cost peste setul de antrenare, deducem că minimizând valoarea lui J, obținem (în medie) valori $\hat{y}^{(i)} = h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$ apropiate de $y^{(i)}$.

Pentru determinarea lui $\boldsymbol{\theta}^*$ care minimizează funcția J:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^{n+1}} J(\boldsymbol{\theta}) \tag{3.17}$$

se folosește algoritmul gradientului descendent (engl: gradient descent): se pornește cu valori aleatoare setate pentru componentele vectorului $\boldsymbol{\theta}$ – sau chiar cu vectorul nul – și se modifică în mod iterativ, scăzând la fiecare pas valoarea gradientului înmulțită cu un coeficient pozitiv mic α, numit rată de învățare.

Metoda gradientului descendent pentru regresia logistică binară este dată în algoritmul 3.1.

Algoritmul 3.1 Metoda gradientului descendent pentru regresia logistică binară

Intrare: setul de antrenare $\mathscr{S}$, rata de învățare $\alpha > 0$, pragul $\varepsilon > 0$, numărul maxim de iterații T

Ieșire: vectorul de parametri $\boldsymbol{\theta} = (\theta_0, \theta_1, \ldots, \theta_n)^t$

1: inițializează $\boldsymbol{\theta}$ cu valori aleatoare mici în jurul lui 0 sau cu vectorul nul; $t \leftarrow 0$
2: **repetă**
3: $\quad J \leftarrow J(\boldsymbol{\theta})$ ▷ valoarea funcției de cost, pentru monitorizare
4: $\quad g_j \leftarrow \dfrac{\partial J}{\partial \theta_j}(\boldsymbol{\theta})$, pentru $j = 0, 1, \ldots, n$ ▷ gradientul, cu valorile curente
5: $\quad \theta_j \leftarrow \theta_j - \alpha\, g_j$, pentru $j = 0, 1, \ldots, n$ ▷ modificarea ponderilor
6: $\quad t \leftarrow t + 1$
7: **până la** îndeplinirea condiției $\|\mathbf{g}\| < \varepsilon$ sau $t = T$, unde $\mathbf{g} = (g_0, \ldots, g_n)^t$
8: **returnează** $\boldsymbol{\theta}$

Mai sus, algoritmul folosește drept criteriu de oprire norma gradientului, împreună cu un număr maxim de iterații; alte condiții de convergență ce sunt folosite în practică sunt:

- valorile succesive ale funcției de eroare scad și diferența dintre două valori succesive este sub un prag $\varepsilon > 0$ specificat a priori;
- norma L_2 a diferenței dintre două valori succesive ale vectorului $\boldsymbol{\theta}$ este sub un prag $\varepsilon > 0$; cu alte cuvinte, distanța dintre doi vectori succesivi de ponderi devine foarte mică.

Derivatele parțiale $\partial J/\partial\theta_j$ sunt:

$$\frac{\partial J}{\partial \theta_j}(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m}\left[\left(h_{\boldsymbol{\theta}}\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right)\cdot x_j^{(i)}\right] = \frac{1}{m}\sum_{i=1}^{m}\left[\left(\hat{y}^{(i)} - y^{(i)}\right)\cdot x_j^{(i)}\right] \tag{3.18}$$

formă justificată în cele ce urmează: funcția J se calculează ca o medie aritmetică a m termeni

$$J^{(i)} := -y^{(i)}\cdot\ln\hat{y}^{(i)} - \left(1-y^{(i)}\right)\cdot\ln\left(1-\hat{y}^{(i)}\right)$$

și deoarece derivata unei medii de funcții este media derivatelor funcțiilor componente, este suficient să calculăm $\frac{\partial J^{(i)}}{\partial\theta_j}$, apoi facem media lor.

$J^{(i)}$ depinde indirect de θ_j: $J^{(i)}$ depinde direct de $\hat{y}^{(i)}$, care la rândul lui se calculează ca $\hat{y}^{(i)} = \sigma(z^{(i)})$, unde $z^{(i)} = \boldsymbol{\theta}^t\mathbf{x}^{(i)}$. Folosind formula de derivare a funcțiilor compuse, avem:

$$\frac{\partial J^{(i)}}{\partial\theta_j} = \frac{\partial J^{(i)}}{\partial\hat{y}^{(i)}}\cdot\frac{\partial\hat{y}^{(i)}}{\partial z^{(i)}}\cdot\frac{\partial z^{(i)}}{\partial\theta_j} \tag{3.19}$$

Calculăm pe rând fiecare derivată parțială:

1. Derivata lui $J^{(i)}$ în raport cu $\hat{y}^{(i)}$:

$$\frac{\partial J^{(i)}}{\partial\hat{y}^{(i)}} = -\left(\frac{y^{(i)}}{\hat{y}^{(i)}} - \frac{1-y^{(i)}}{1-\hat{y}^{(i)}}\right) \tag{3.20}$$

2. Derivata sigmoidei $\hat{y}^{(i)}$ în raport cu $z^{(i)}$, folosind (3.7):

$$\frac{\partial\hat{y}^{(i)}}{\partial z^{(i)}} = \hat{y}^{(i)}(1-\hat{y}^{(i)}) \tag{3.21}$$

3. Derivata lui $z^{(i)} = \sum_{k=0}^{n}\theta_k x_k^{(i)}$ în raport cu θ_j:

$$\frac{\partial z^{(i)}}{\partial\theta_j} = x_j^{(i)} \tag{3.22}$$

Prin înmulțirea termenilor (3.20)–(3.22) și simplificările de rigoare, obținem:

$$\frac{\partial J^{(i)}}{\partial \theta_j} = \left(\hat{y}^{(i)} - y^{(i)}\right) x_j^{(i)} \tag{3.23}$$

ceea ce justifică ecuația (3.18).

Introducând forma (3.18) în linia 5 a algoritmului 3.1, obținem:

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right) x_j^{(i)} \tag{3.24}$$

Se observă că modificarea ponderilor $\theta_0, \ldots, \theta_n$ are aceeași formă ca la regresia liniară, ecuația (2.22) pagina 33. Singura diferență este forma funcției $h_{\boldsymbol{\theta}}$: formă liniară (afină) la regresia liniară, funcție neliniară (sigmoida logistică) la regresia logistică.

Atribuirile simultane cerute pentru $j = 0, 1, \ldots, n$ se fac fie prin utilizarea unor variabile temporare $\theta_j^{(\text{temp})}$, fie prin calcul vectorizat. Pentru aceasta, notăm cu $\mathbf{X}$ matricea care are drept linii vectorii $\mathbf{x}^{(i)t}$ – matricea de design întâlnită și la regresia liniară, cu prima coloană setată la valoarea 1:

$$\mathbf{X} = \begin{pmatrix} x_0^{(1)} & x_1^{(1)} & x_2^{(1)} & \cdots & x_n^{(1)} \\ x_0^{(2)} & x_1^{(2)} & x_2^{(2)} & \cdots & x_n^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_0^{(m)} & x_1^{(m)} & x_2^{(m)} & \cdots & x_n^{(m)} \end{pmatrix} \tag{3.25}$$

Cele m probabilități $\hat{y}^{(i)} = P(y^{(i)} = 1 | \mathbf{x}^{(i)}; \boldsymbol{\theta}) = h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$ sunt componentele vectorului coloană

$$\hat{\mathbf{y}} = \sigma(\mathbf{X} \cdot \boldsymbol{\theta}) \tag{3.26}$$

cu funcția σ aplicată pe fiecare componentă a vectorului coloană $\mathbf{X} \cdot \boldsymbol{\theta}$.

Mai mult, se arată ușor că gradientul lui J este:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \frac{1}{m} \mathbf{X}^t (\hat{\mathbf{y}} - \mathbf{y}) \tag{3.27}$$

și deci modificarea din interiorul iterațiilor de antrenare devine:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \frac{1}{m} \mathbf{X}^t (\hat{\mathbf{y}} - \mathbf{y}) \tag{3.28}$$

unde vectorul $\hat{\mathbf{y}}$ e calculat ca în (3.26). Remarcăm similaritatea dintre (3.28) și (2.36) pagina 36.

Se păstrează discuția legată de valorile ratei de învățare α (dinamica funcției de eroare, valori α prea mari sau prea mici) din secțiunea 2.3, pagina 29.

Spre deosebire de regresia liniară, unde prin metoda ecuațiilor normale avem valoarea optimă a vectorului $\boldsymbol{\theta}$, la regresia logistică nu există o formă analitică de determinare a coeficienților, adică nu avem o formă analitică pentru determinarea lui $\boldsymbol{\theta}^*$ din (3.17).

3.2.7 Exemplu numeric: antrenarea prin metoda gradientului descendent

Aplicăm algoritmul 3.1 pe datele din tabelul 3.1, pornind de la $\boldsymbol{\theta} = \mathbf{0}$, pentru care $J(\mathbf{0}) = \ln 2 \approx 0.6931$, cu rata de învățare $\alpha = 0.001$. Toate predicțiile inițiale sunt $\hat{y}^{(i)} = 0.5$, deci, conform (3.18), componentele gradientului sunt:

$$g_0 = \frac{1}{8}\sum_{i=1}^{8}\left(0.5 - y^{(i)}\right) = 0.5 - \frac{4}{8} = 0$$

$$g_1 = \frac{1}{8}\sum_{i=1}^{8}\left(0.5 - y^{(i)}\right) s^{(i)} = 0.5 \cdot \frac{412}{8} - \frac{241}{8} = 25.75 - 30.125 = -4.375$$

unde 412 este suma tuturor suprafețelor, iar 241 suma suprafețelor apartamentelor de clasă pozitivă. Componenta g_0 este nulă deoarece exact jumătate dintre apartamente sunt de clasă pozitivă. După primul pas, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha\,\mathbf{g} = (0, 0.004375)^t$, iar J scade la aproximativ 0.6808. Continuând iterațiile, algoritmul converge către

$$\boldsymbol{\theta}^* \approx (-9.2253, 0.1792)^t, \qquad J(\boldsymbol{\theta}^*) \approx 0.4127$$

Valoarea minimă a lui J rămâne departe de 0, deoarece datele nu sunt liniar separabile: niciun model nu poate atribui probabilități apropiate de 1 tuturor apartamentelor de clasă pozitivă și, în același timp, probabilități apropiate de 0 tuturor celor de clasă negativă.

Convergența este însă lentă: pentru ca J să ajungă la cel mult 1% peste $J(\boldsymbol{\theta}^*)$ sunt necesare 376550 de iterații. Ca și la regresia liniară (secțiunea 2.3), cauza este diferența dintre ordinele de mărime ale trăsăturilor: $x_0 = 1$, iar $x_1 = s$ ia valori între 30 și 75. Dacă suprafața este standardizată ca în capitolul 2, cu media 51.5 și abaterea standard 13.23 calculate pe setul de antrenare, se poate folosi rata de învățare $\alpha = 1$, iar algoritmul pornit din $\boldsymbol{\theta} = \mathbf{0}$ atinge același nivel al lui J după doar 23 de iterații.

3.2.8 Etapa de predicție

După învățare, modelul probabilist dat de (3.3) este mai departe folosit pentru a face clasificare astfel: dacă pentru un vector de intrare $\mathbf{x}$ avem

$$P(y = 1|\mathbf{x}; \boldsymbol{\theta}) \geq P(y = 0|\mathbf{x}; \boldsymbol{\theta}) \tag{3.29}$$

atunci se decide că obiectul descris de vectorul $\mathbf{x}$ este din clasa 1, pozitivă; altfel este din clasa 0 – negativă.

Având în vedere că $P(y = 0|\mathbf{x}; \boldsymbol{\theta}) = 1 - P(y = 1|\mathbf{x}; \boldsymbol{\theta})$, decizia bazată pe inecuația (3.29) se reformulează echivalent ca: vom clasifica obiectul descris

de vectorul $\mathbf{x}$ ca fiind de clasă pozitivă dacă $P(y = 1|\mathbf{x}; \boldsymbol{\theta}) \geq 0.5$ și negativă altfel.

Unii autori consideră că dacă avem o intrare $\mathbf{x}$ pentru care $P(y = 1|\mathbf{x}; \boldsymbol{\theta}) = P(y = 0|\mathbf{x}; \boldsymbol{\theta}) = 0.5$, atunci modelul nu ar trebui să facă o clasificare a intrării: este tot atât de probabil să fie de clasă pozitivă pe cât e de probabil să fie de clasă negativă. În inecuația (3.29) am ales – în mod mai degrabă arbitrar – ca situația cu egalitate să fie tratată ca un caz pozitiv.

Se poate folosi și alt prag de discriminare pentru clase decât 0.5, de exemplu: obiectul descris de $\mathbf{x}$ este de clasă pozitivă dacă $P(y = 1|\mathbf{x}; \boldsymbol{\theta}) \geq 0.8$ și de clasă negativă altfel. O astfel de setare de prag se efectuează pentru date de antrenare în care clasele pozitive și cele negative sunt puternic debalansate (mult mai multe exemple de tip pozitiv decât negativ, sau invers), sau pentru cazul în care penalizarea care se plătește pentru o clasificare eronată de tip pozitiv (un fals pozitiv) este foarte mare față de penalizarea pentru clasificarea eronată de tip negativ (fals negativ) – sau invers. Pentru ultimul caz, un exemplu este: e mai grav dacă un e-mail legitim este clasificat ca fiind de tip spam și scos din inbox, față de cazul în care un e-mail spam este clasificat eronat ca fiind legitim: vom impune ca $P(\textit{spam}|\textit{e-mail})$ să fie comparată cu un prag t mare (de exemplu 0.8, sau 0.99) pentru a decide că e vorba într-adevăr de spam.

3.2.9 Suprafața de decizie a regresiei logistice binare

În ciuda caracterului neliniar al funcției de regresie logistică (3.3), se arată ușor că suprafața care desparte regiunea $\mathbf{x}$ de clasă pozitivă și cea de clasă negativă este o varietate liniară[6], dacă pragul este 0.5.

Inegalitatea (3.29) se scrie echivalent:

$$\begin{aligned} P(y = 1|\mathbf{x}; \boldsymbol{\theta}) \geq P(y = 0|\mathbf{x}; \boldsymbol{\theta}) \quad &\iff \quad \tfrac{1}{1+\exp(-\boldsymbol{\theta}^t\mathbf{x})} \geq \tfrac{\exp(-\boldsymbol{\theta}^t\mathbf{x})}{1+\exp(-\boldsymbol{\theta}^t\mathbf{x})} \\ &\iff \quad 1 \geq \exp(-\boldsymbol{\theta}^t\mathbf{x}) \\ &\overset{\text{(logaritmând)}}{\iff} \quad 0 \geq -\boldsymbol{\theta}^t\mathbf{x} \\ &\iff \quad \boldsymbol{\theta}^t\mathbf{x} \geq 0 \end{aligned} \tag{3.30}$$

Am obținut deci că, dacă $\mathbf{x}$ are proprietatea că $\boldsymbol{\theta}^t\mathbf{x} \geq 0$, atunci $\mathbf{x}$ este clasificat ca fiind de clasă 1, altfel este de clasă 0. Separarea dintre cele două clase se face de către varietatea liniară $\boldsymbol{\theta}^t\mathbf{x} = 0$: dacă $\mathbf{x}$ e în partea pozitivă a varietății liniare $\boldsymbol{\theta}^t\mathbf{x} = 0$ sau chiar pe această varietate, atunci e de clasă 1, altfel e de clasă 0.

Mai sus s-a făcut comparația între probabilitățile de clasă pozitivă și negativă, ceea ce am arătat că e echivalent cu a compara probabilitatea de

[6]Prin abuz de limbaj se folosește și denumirea „hiperplan”; în timp ce un hiperplan obligatoriu trebuie să treacă prin origine, varietatea liniară este o formă liniară fără această constrângere.

clasă pozitivă cu pragul 0.5. Invităm cititorul să verifice că pentru un prag diferit de 0.5 suprafața de separare este tot varietate liniară.

Dacă se permite ca în componența vectorului $\mathbf{x}$ să intre și forme pătratice, cubice etc. ale trăsăturilor originare, atunci suprafața de decizie poate fi mai complicată. De exemplu, să plecăm de la vectorul (x_1, x_2) și să considerăm extinderea lui cu trăsături polinomiale, obținând $\mathbf{x} = (x_0 = 1, x_1, x_2, x_1^2, x_2^2, x_1x_2)^t$; rezultă că $\boldsymbol{\theta} \in \mathbb{R}^6$; avem $\boldsymbol{\theta}^t\mathbf{x} = \theta_0 + \theta_1x_1 + \theta_2x_2 + \theta_3x_1^2 + \theta_4x_2^2 + \theta_5x_1x_2$. Pentru valorile[7] $\theta_0 = -4, \theta_1 = \theta_2 = \theta_5 = 0, \theta_3 = \theta_4 = 1$ se obține ecuația suprafeței de decizie $x_1^2 + x_2^2 = 4$, reprezentând un cerc (figura 3.5); în funcție de poziția față de cerc (înăuntrul sau în afara lui), obiectul de coordonate $(x_1, x_2)^t$ este estimat ca fiind de clasă negativă, respectiv pozitivă. Am arătat deci că suprafața de separare poate fi neliniară, dacă se introduc trăsături suplimentare.

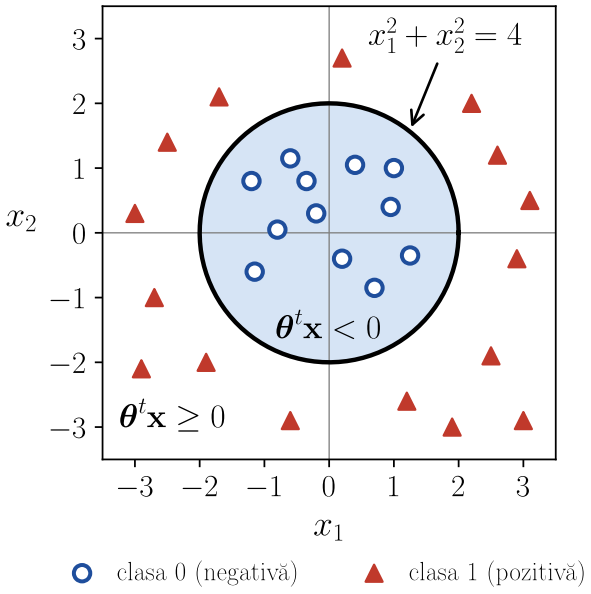


Figura 3.5: Suprafața de decizie circulară $x_1^2 + x_2^2 = 4$, obținută cu trăsături polinomiale, pentru $\theta_0 = -4$, $\theta_3 = \theta_4 = 1$ și ceilalți coeficienți nuli. Punctele din interiorul cercului sunt clasificate ca negative, cele din exterior ca pozitive.

Aceleași trăsături polinomiale permit și granițe eliptice. Pentru valorile $\theta_0 = -9$, $\theta_1 = \theta_2 = \theta_5 = 0$, $\theta_3 = 1$, $\theta_4 = 9$ se obține ecuația suprafeței de decizie $x_1^2 + 9x_2^2 = 9$, adică $\frac{x_1^2}{3^2} + \frac{x_2^2}{1^2} = 1$, o elipsă cu semiaxele 3 și 1 (figura 3.6). Diferența față de cerc este că ponderile lui x_1^2 și x_2^2 nu mai sunt egale: cu cât ponderea unei trăsături e mai mare, cu atât granița se strânge

[7] În mod normal, valorile lui $\boldsymbol{\theta}$ se determină prin procesul de antrenare.

mai mult pe axa respectivă. Dacă $\theta_5 \neq 0$, termenul mixt x_1x_2 rotește elipsa față de axele de coordonate. Modelul rămâne liniar în $\boldsymbol{\theta}$ și în trăsăturile extinse $\mathbf{x}$; neliniaritatea apare doar în raport cu trăsăturile originare (x_1, x_2).

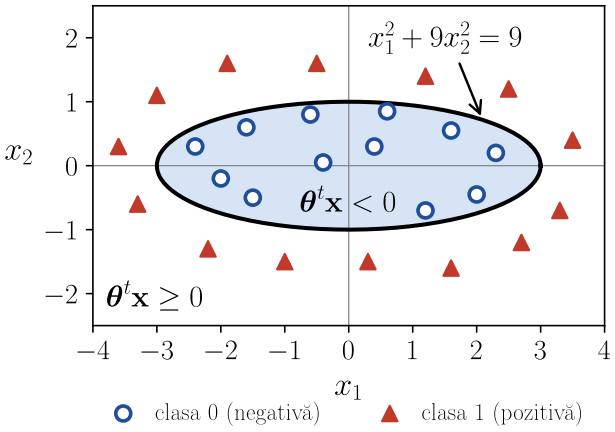


Figura 3.6: Suprafața de decizie eliptică $x_1^2 + 9x_2^2 = 9$, obținută cu trăsături polinomiale, pentru $\theta_0 = -9$, $\theta_3 = 1$, $\theta_4 = 9$ și ceilalți coeficienți nuli. Punctele din interiorul elipsei sunt clasificate ca negative, cele din exterior ca pozitive.

3.2.10 Exemplu numeric: predicția și pragul de decizie

Pentru modelul $\boldsymbol{\theta}^* \approx (-9.2253, 0.1792)^t$ obținut în secțiunea 3.2.7, apartamentul de 80 m^2 din figura 2.2 are

$$P(y = 1|\mathbf{x}; \boldsymbol{\theta}^*) = \sigma(-9.2253 + 0.1792 \cdot 80) \approx \sigma(5.11) \approx 0.994$$

deci este clasificat ca fiind de clasă pozitivă: modelul estimează că prețul lui depășește 80000 €. Pentru o suprafață de 56 m^2 probabilitatea estimată este 0.691, deci clasa pozitivă, iar pentru 50 m^2 este 0.433, deci clasa negativă – deși apartamentul de 50 m^2 din setul de antrenare este de clasă pozitivă. Pe setul de antrenare, modelul greșește exact la cele două apartamente adăugate în tabelul 3.1: cel de 50 m^2 este clasificat ca negativ, iar cel de 58 m^2, cu probabilitatea estimată 0.762, ca pozitiv. Datele nefiind liniar separabile, niciun prag de suprafață nu poate clasifica toate cele opt apartamente corect.

Pentru o singură trăsătură, $\boldsymbol{\theta}^t\mathbf{x} = \theta_0 + \theta_1 s$, iar varietatea liniară $\theta_0^* + \theta_1^* s = 0$ care separă clasele (secțiunea 3.2.9) se reduce la un punct: $s = -\theta_0^*/\theta_1^* \approx 51.5$ m^2. Cu pragul 0.5, un apartament este deci clasificat ca fiind de clasă pozitivă dacă $s \geq 51.5$ m^2.

Pentru un prag oarecare t, nu neapărat 0.5, condiția $P(y = 1|\mathbf{x}; \boldsymbol{\theta}^*) \geq t$

este echivalentă cu $\theta_0^* + \theta_1^* s \geq \text{logit}(t)$, adică

$$s \geq \frac{\text{logit}(t) - \theta_0^*}{\theta_1^*}$$

Pentru $t = 0.5$ avem $\text{logit}(0.5) = 0$ și regăsim granița de mai sus. Pentru pragul $t = 0.8$ avem $\text{logit}(0.8) = \ln 4 \approx 1.386$, iar granița se mută la $s \approx 59.2$ m^2, tot un punct (figura 3.7); de exemplu, apartamentul de 56 m^2, cu probabilitatea estimată 0.691, devine de clasă negativă. Pe setul de antrenare, cu pragul 0.5 apar un fals pozitiv[8] (58 m^2) și un fals negativ (50 m^2); cu pragul 0.8 falsul pozitiv dispare, dar apare încă un fals negativ: apartamentul de 54 m^2, cu probabilitatea estimată 0.610.

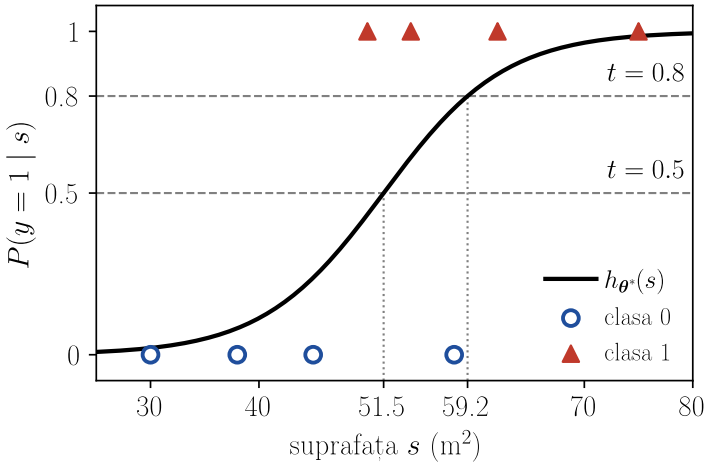


Figura 3.7: Apartamentele din tabelul 3.1, probabilitatea estimată $h_{\boldsymbol{\theta}^*}(s)$ și granițele dintre clase pentru pragurile $t = 0.5$ ($s \approx 51.5$ m^2) și $t = 0.8$ ($s \approx 59.2$ m^2).

3.3 Regresia logistică multinomială

3.3.1 Setul de antrenare

Setul de antrenare e similar cu cel de la regresia logistică binară, cu modificări pentru valorile lui $y^{(i)}$, care acum pot fi din mulțimea $\{1, \ldots, K\}$. Formal, setul de antrenare se scrie ca:

$$\mathscr{S} = \left\{ \left(\mathbf{x}^{(i)}, y^{(i)} \right) \mid \mathbf{x}^{(i)} \in \mathbb{R}^{n+1}, y^{(i)} \in \{1, \ldots, K\}, 1 \leq i \leq m \right\} \tag{3.31}$$

[8]Fals pozitiv: modelul clasifică un exemplu ca fiind de clasă pozitivă, dar în realitateeste de clasă negativă. Fals negativ: modelul clasifică un exemplu ca fiind de clasă negativă, dar exemplul este de clasă pozitivă.

unde vectorul de intrare $\mathbf{x}^{(i)}$ este vector de trăsături; ca și la regresia logistică binară, pe prima componentă avem valoarea fixă $x_0^{(i)} = 1$; $y^{(i)}$ este indicele clasei exemplului i, număratoarea claselor începând acum de la 1.

3.3.2 Funcția softmax

În cazul regresiei logistice cu două clase s-a folosit sigmoida logistică, producându-se valori de ieșire ce pot fi folosite ca probabilități condiționate. Pentru mai mult de două clase, estimarea de probabilitate condiționată se face cu funcția softmax, care transformă un vector oarecare de K numere într-un vector de K valori din intervalul $(0, 1)$ și care însumate dau 1 – adică un vector stohastic.

Pornind de la vectorul $\mathbf{z} = (z_1, \ldots, z_K)^t \in \mathbb{R}^K$, el e transformat de funcția *softmax* astfel:

$$\text{softmax}(\mathbf{z}) = (\text{softmax}(\mathbf{z}; 1), \ldots, \text{softmax}(\mathbf{z}; K))^t \tag{3.32}$$

unde $\text{softmax}(\mathbf{z}; k)$ este:

$$\text{softmax}(\mathbf{z}; k) = \frac{\exp(z_k)}{\sum_{l=1}^{K} \exp(z_l)} \tag{3.33}$$

pentru $1 \leq k \leq K$. Se verifică ușor că $\text{softmax}(\mathbf{z}; k) \in (0, 1)$ pentru orice k, $1 \leq k \leq K$ și $\sum_{k=1}^{K} \text{softmax}(\mathbf{z}; k) = 1$, adică vectorul $\text{softmax}(\mathbf{z})$ este stohastic. Vom folosi funcția softmax pentru producerea de estimări de probabilități condiționate de intrarea curentă $\mathbf{x}$.

Vom reîntâlni funcția softmax în capitolele de rețele neurale artificiale, unde se folosește pentru a produce probabilități de clasă pentru mai mult de două clase.

3.3.3 Exemplu numeric: funcția softmax

Pentru $K = 3$ și $\mathbf{z} = (2, 1, -1)^t$, valorile funcției exponențiale sunt $e^2 \approx 7.389$, $e^1 \approx 2.718$ și $e^{-1} \approx 0.368$, cu suma 10.475, deci

$$\text{softmax}(\mathbf{z}) \approx \left(\frac{7.389}{10.475}, \frac{2.718}{10.475}, \frac{0.368}{10.475}\right)^t \approx (0.705, 0.259, 0.035)^t$$

Componentele sunt în intervalul $(0, 1)$, au suma 1 și păstrează ordinea valorilor din $\mathbf{z}$: cea mai mare componentă a lui $\mathbf{z}$ primește cea mai mare probabilitate.

Remarcăm că softmax acceptă valori oricât de mari sau de mici, inclusiv negative: componenta $z_3 = -1 < 0$ primește probabilitatea 0.035, mică, dar strict pozitivă. Motivul este că funcția exponențială transformă orice

număr real într-unul strict pozitiv, $e^{z_k} > 0$, iar împărțirea la suma lor aduce rezultatele în $(0, 1)$. Aceasta este o capacitate importantă, deoarece valorile $\boldsymbol{\theta}_k^t \mathbf{x}$ produse de model pot avea orice semn. O normalizare directă, prin împărțirea la suma componentelor, nu ar funcționa: cum $\sum_k z_k = 2$, s-ar obține $(1, 0.5, -0.5)^t$, cu o „probabilitate" negativă; în plus, pentru un vector cu suma 0 împărțirea nici nu ar fi posibilă.

3.3.4 Reprezentarea modelului

Pentru cazul în care se cere discriminarea pentru K clase, $K > 2$, regresia logistică se extinde astfel încât să construiască K funcții (modele) simultan, pentru vectorul de intrare $\mathbf{x} = (x_0 = 1, x_1, \ldots, x_n)^t$:

$$h_{\boldsymbol{\Theta}}(\mathbf{x}) = \begin{pmatrix} P(y = 1|\mathbf{x}; \boldsymbol{\Theta}) \\ P(y = 2|\mathbf{x}; \boldsymbol{\Theta}) \\ \vdots \\ P(y = K|\mathbf{x}; \boldsymbol{\Theta}) \end{pmatrix} = \begin{pmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_K \end{pmatrix} = \hat{\mathbf{y}} \tag{3.34}$$

Facem o comparație între valoarea de ieșire a acestui model multinomial și ieșirea produsă de regresia logistică binară: pentru două clase e suficientă construirea unei singure funcții, sigmoida logistică, producând o singură valoare. Pentru $K > 2$ clase ar fi suficientă construirea a $K - 1$ funcții, dar pentru comoditate se preferă construirea a K funcții, oricare dintre ele fiind acceptată ca redundantă: suma celor K funcții fiind 1, oricare dintre ele poate fi determinată ca unu minus suma celorlalte $K - 1$. Discutăm acest aspect în secțiunile 3.5.1 și 3.5.3.

Ponderile instruibile care definesc modelul de clasificare se grupează într-o matrice $\boldsymbol{\Theta} \in \mathbb{R}^{(n+1)\times K}$, construită ca:

$$\boldsymbol{\Theta} = \begin{pmatrix} | & | & | & | \\ \boldsymbol{\theta}_1 & \boldsymbol{\theta}_2 & \cdots & \boldsymbol{\theta}_K \\ | & | & | & | \end{pmatrix} \tag{3.35}$$

unde vectorul coloană $\boldsymbol{\theta}_k \in \mathbb{R}^{n+1}$ conține ponderile (parametrii instruibili) pentru clasa k, $1 \leq k \leq K$.

Modelul care conține toate cele K modele, câte unul pentru fiecare clasă, are forma:

$$h_{\mathbf{\Theta}}(\mathbf{x}) = \begin{pmatrix} P(y=1|\mathbf{x};\mathbf{\Theta}) \\ P(y=2|\mathbf{x};\mathbf{\Theta}) \\ \vdots \\ P(y=K|\mathbf{x};\mathbf{\Theta}) \end{pmatrix} = \text{softmax} \begin{pmatrix} \boldsymbol{\theta}_1^t \mathbf{x} \\ \boldsymbol{\theta}_2^t \mathbf{x} \\ \vdots \\ \boldsymbol{\theta}_K^t \mathbf{x} \end{pmatrix} = \tag{3.36}$$

$$= \text{softmax}(\mathbf{\Theta}^t \mathbf{x}) = \begin{pmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_K \end{pmatrix} = \hat{\mathbf{y}} \tag{3.37}$$

pentru un vector de intrare $\mathbf{x} \in \mathbb{R}^{n+1}$. Valorile $\boldsymbol{\theta}_k^t \mathbf{x}$ se mai numesc și logiți (engl: logits).

Probabilitatea ca un vector $\mathbf{x}$ să fie de clasă k $(1 \leq k \leq K)$ este:

$$P(y=k|\mathbf{x};\mathbf{\Theta}) = \frac{\exp\left(\boldsymbol{\theta}_k^t \mathbf{x}\right)}{\sum_{l=1}^{K} \exp\left(\boldsymbol{\theta}_l^t \mathbf{x}\right)} = \hat{y}_k \in (0,1) \tag{3.38}$$

Predicția de probabilități condiționate – utilizarea modelului – este descrisă în detaliu în secțiunea 3.3.9.

Antrenarea vizează determinarea acelei matrice $\mathbf{\Theta}$ pentru care modelul învață să recunoască cât mai bine datele din setul de antrenare (eventual adăugându-se și un termen de regularizare, a se vedea capitolul 4). Mai clar, dacă un obiect $\mathbf{x}$ din setul de antrenare $\mathscr{S}$ este de clasă k, atunci vrem ca valoarea $P(y=k|\mathbf{x};\mathbf{\Theta})$ să fie cât mai aproape de 1.

3.3.5 Funcția de cost

Ca și în cazul funcției de cost (de eroare) pentru regresia logistică, se va cuantifica în ce măsură diferă clasa reală a unei intrări de predicția dată de modelul $h_{\mathbf{\Theta}}$. Introducem funcția indicator $I(\cdot)$ care pentru o valoare logică returnează 1 dacă argumentul este adevărat și 0 altfel:

$$I(\textit{valoare_logică}) = \begin{cases} 1 & \text{dacă } \textit{valoare_logică} = \textit{adevărat} \\ 0 & \text{dacă } \textit{valoare_logică} = \textit{fals} \end{cases} \tag{3.39}$$

Funcția de eroare pentru regresia logistică multinomială se definește ca:

$$J(\mathbf{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K}\left[I\left(y^{(i)}=k\right)\cdot\ln\hat{y}_k^{(i)}\right] \tag{3.40}$$

$$= -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K}\left[I\left(y^{(i)}=k\right)\cdot\ln P\left(y^{(i)}=k|\mathbf{x}^{(i)};\mathbf{\Theta}\right)\right] \tag{3.41}$$

$$= -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K}\left[I\left(y^{(i)}=k\right)\cdot\ln\frac{\exp\left(\boldsymbol{\theta}_k^t\mathbf{x}^{(i)}\right)}{\sum\limits_{l=1}^{K}\exp\left(\boldsymbol{\theta}_l^t\mathbf{x}^{(i)}\right)}\right] \tag{3.42}$$

și se numește entropie încrucișată (engl: cross-entropy).

Pentru reprezentarea clasei curente $y^{(i)}$ se folosește frecvent așa-numita codificare one-hot (unul din K), astfel: dacă $y^{(i)}=k$, atunci codificarea sa $\mathbf{y}_{\text{ohe}}^{(i)}$ este un vector coloană cu K valori, având pe poziția[9] k valoarea 1 și 0 în rest. Cu această codificare se rescrie funcția de eroare mai compact ca:

$$J(\mathbf{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\left(\mathbf{y}_{\text{ohe}}^{(i)}\right)^t\cdot\ln\left(\hat{\mathbf{y}}^{(i)}\right) \tag{3.43}$$

unde logaritmul se aplică pe fiecare componentă[10] a vectorului $\hat{\mathbf{y}}^{(i)}$.

În cazul în care se folosesc biblioteci de calcul vectorizat, se poate folosi produsul Hadamard în locul celui matricial: dacă $\mathbf{A}$ și $\mathbf{B}$ sunt două matrice de tip $m\times n$, atunci produsul Hadamard (sau punctual) al lor este $\mathbf{C}$ de același tip, $\mathbf{C}=\mathbf{A}\odot\mathbf{B}$, $c_{ij}=a_{ij}\cdot b_{ij}$, $1\le i\le m, 1\le j\le n$. Dacă considerăm matricea $\mathbf{Y}_{\text{ohe}}$ de dimensiune $K\times m$ având drept coloana i vectorul $\mathbf{y}_{\text{ohe}}^{(i)}$:

$$\mathbf{Y}_{\text{ohe}} = \begin{pmatrix} | & | & \cdots & | \\ \mathbf{y}_{\text{ohe}}^{(1)} & \mathbf{y}_{\text{ohe}}^{(2)} & \cdots & \mathbf{y}_{\text{ohe}}^{(m)} \\ | & | & \cdots & | \end{pmatrix} \tag{3.44}$$

și matricea $\hat{\mathbf{Y}}$ de aceeași dimensiune cu toate cele m coloane $\hat{\mathbf{y}}^{(i)}$:

$$\hat{\mathbf{Y}} = \begin{pmatrix} | & | & \cdots & | \\ \hat{\mathbf{y}}^{(1)} & \hat{\mathbf{y}}^{(2)} & \cdots & \hat{\mathbf{y}}^{(m)} \\ | & | & \cdots & | \end{pmatrix} \tag{3.45}$$

atunci putem calcula matricea $-\frac{1}{m}\cdot\mathbf{Y}_{\text{ohe}}\odot\ln\hat{\mathbf{Y}}$ pentru care se calculează apoi suma elementelor folosind funcții eficiente din bibliotecă.

[9] Indicele primei poziții din vectorul $\mathbf{y}_{\text{ohe}}^{(i)}$ este 1.

[10] Datorită valorilor apropiate de 0 pe care le pot lua componentele lui $\hat{\mathbf{y}}^{(i)}$, la implementare se impune o aducere a valorilor mici, apropiate de 0, la niște praguri convenabile (engl: clipping).

Alternativ, putem folosi faptul că suma tuturor elementelor unei matrice $\mathbf{S}$ de dimensiune $m \times n$ se poate obține cu produsul matricial:

$$\sum_{i=1}^{m}\sum_{j=1}^{n} S_{ij} = \mathbb{1}_m^t \cdot \mathbf{S} \cdot \mathbb{1}_n \tag{3.46}$$

unde $\mathbb{1}_p$ este vector coloană de p elemente egale cu 1. Funcția de eroare din ecuația (3.40) se rescrie ca:

$$J(\mathbf{\Theta}) = -\frac{1}{m}\mathbb{1}_K^t \cdot \left(\mathbf{Y}_{\text{ohe}} \odot \ln \hat{\mathbf{Y}}\right) \cdot \mathbb{1}_m \tag{3.47}$$

Secțiunea 3.5 conține alte clarificări legate de calculul funcției de eroare.

3.3.6 Gradientul funcției de cost

Determinarea lui $\mathbf{\Theta}^* = \arg\min_{\mathbf{\Theta}\in\mathbb{R}^{(n+1)\times K}} J(\mathbf{\Theta})$ se face prin metoda gradientului descendent. Formula gradientului este, pentru $1 \leq k \leq K$:

$$\nabla_{\boldsymbol{\theta}_k} J(\mathbf{\Theta}) = \frac{1}{m}\sum_{i=1}^{m}\left[\mathbf{x}^{(i)}\left(\hat{y}_k^{(i)} - I(y^{(i)}=k)\right)\right] = \tag{3.48}$$

$$= \frac{1}{m}\sum_{i=1}^{m}\left[\mathbf{x}^{(i)}\left(P(y^{(i)}=k|\mathbf{x}^{(i)};\mathbf{\Theta}) - I(y^{(i)}=k)\right)\right] \tag{3.49}$$

pe care o demonstrăm în cele ce urmează: având în vedere că gradientul e operator liniar, calculăm gradientul pentru un termen de cost corespunzător unei perechi de antrenare $\left(\mathbf{x}^{(i)}, y^{(i)}\right)$, $J_i(\mathbf{\Theta})$:

$$J_i(\mathbf{\Theta}) = -\sum_{l=1}^{K} I\left(y^{(i)}=l\right)\cdot \ln \frac{\exp\left(\boldsymbol{\theta}_l^t\mathbf{x}^{(i)}\right)}{\sum_{j=1}^{K}\exp\left(\boldsymbol{\theta}_j^t\mathbf{x}^{(i)}\right)} = \tag{3.50}$$

$$= -\sum_{l=1}^{K} I\left(y^{(i)}=l\right)\cdot \ln\left(\exp\left(\boldsymbol{\theta}_l^t\mathbf{x}^{(i)}\right)\right) + \tag{3.51}$$

$$+\sum_{l=1}^{K} I\left(y^{(i)}=l\right)\cdot \ln\left(\sum_{j=1}^{K}\exp\left(\boldsymbol{\theta}_j^t\mathbf{x}^{(i)}\right)\right) = \tag{3.52}$$

$$= -\sum_{l=1}^{K}\left[I\left(y^{(i)}=l\right)\cdot \boldsymbol{\theta}_l^t\mathbf{x}^{(i)}\right] + \tag{3.53}$$

$$+\sum_{l=1}^{K}\left[I\left(y^{(i)}=l\right)\cdot \ln\left(\sum_{j=1}^{K}\exp\left(\boldsymbol{\theta}_j^t\mathbf{x}^{(i)}\right)\right)\right] \tag{3.54}$$

În suma din linia (3.54), termenul cu logaritm nu depinde de indicele de însumare l, deci poate fi scos în fața sumei; mai departe, pentru suma rămasă, pentru exact un indice l avem $I\left(y^{(i)}=l\right)=1$ și în rest indicatorii sunt 0, deci suma e 1. $J_i(\boldsymbol{\Theta})$ devine:

$$J_i(\boldsymbol{\Theta}) = -\sum_{l=1}^{K}\left[I\left(y^{(i)}=l\right)\cdot\boldsymbol{\theta}_l^t\mathbf{x}^{(i)}\right]+\ln\left(\sum_{j=1}^{K}\exp\left(\boldsymbol{\theta}_j^t\mathbf{x}^{(i)}\right)\right) \tag{3.55}$$

Aplicând gradientul, obținem:

$$\begin{aligned}\nabla_{\boldsymbol{\theta}_k}J_i(\boldsymbol{\Theta}) &= -\nabla_{\boldsymbol{\theta}_k}\sum_{l=1}^{K}\left[I\left(y^{(i)}=l\right)\cdot\boldsymbol{\theta}_l^t\mathbf{x}^{(i)}\right]+\nabla_{\boldsymbol{\theta}_k}\ln\left(\sum_{j=1}^{K}\exp\left(\boldsymbol{\theta}_j^t\mathbf{x}^{(i)}\right)\right)\\ &= -I\left(y^{(i)}=k\right)\mathbf{x}^{(i)}+\frac{\nabla_{\boldsymbol{\theta}_k}\sum\limits_{j=1}^{K}\exp\left(\boldsymbol{\theta}_j^t\mathbf{x}^{(i)}\right)}{\sum\limits_{j=1}^{K}\exp\left(\boldsymbol{\theta}_j^t\mathbf{x}^{(i)}\right)} &(3.56)\\ &= -I\left(y^{(i)}=k\right)\mathbf{x}^{(i)}+\frac{\mathbf{x}^{(i)}\exp\left(\boldsymbol{\theta}_k^t\mathbf{x}^{(i)}\right)}{\sum\limits_{j=1}^{K}\exp\left(\boldsymbol{\theta}_j^t\mathbf{x}^{(i)}\right)} &(3.57)\\ &= -I\left(y^{(i)}=k\right)\mathbf{x}^{(i)}+\mathbf{x}^{(i)}P(y^{(i)}=k|\mathbf{x}^{(i)};\boldsymbol{\Theta}) &(3.58)\\ &= \mathbf{x}^{(i)}\left(P(y^{(i)}=k|\mathbf{x}^{(i)};\boldsymbol{\Theta})-I\left(y^{(i)}=k\right)\right) &(3.59)\end{aligned}$$

sau forma mai compactă

$$\nabla_{\boldsymbol{\theta}_k}J_i(\boldsymbol{\Theta}) = \mathbf{x}^{(i)}\left(\hat{y}_k^{(i)}-I\left(y^{(i)}=k\right)\right). \tag{3.60}$$

Calculând media celor m termeni $\nabla_{\boldsymbol{\theta}_k}J_i(\boldsymbol{\Theta})$ se obține formula gradientului total $\nabla_{\boldsymbol{\theta}_k}J(\boldsymbol{\Theta})$ din ecuația (3.49).

Factorul $\hat{y}_k^{(i)}-I\left(y^{(i)}=k\right)$ este derivata parțială a lui J_i în raport cu logitul $\boldsymbol{\theta}_k^t\mathbf{x}^{(i)}$; vectorul acestor derivate, pentru $k=1,\ldots,K$, este $\hat{\mathbf{y}}^{(i)}-\mathbf{y}_{\text{ohe}}^{(i)}$.

Calculul de gradienți pentru toată matricea $\boldsymbol{\Theta}$ este:

$$\nabla_{\boldsymbol{\Theta}}J(\boldsymbol{\Theta}) = \frac{1}{m}\mathbf{X}^t\cdot(\hat{\mathbf{Y}}^t-\mathbf{Y}_{\text{ohe}}^t) \tag{3.61}$$

unde $\hat{\mathbf{Y}}$ și $\mathbf{Y}_{\text{ohe}}$ sunt date la pagina 63.

Spre deosebire de regresia liniară, nu există o variantă algebrică (o formă analitică) de determinare a valorilor optime $\boldsymbol{\Theta}^*$.

3.3.7 Exemplu numeric: funcția de cost și gradientul pentru trei clase

Fie $K = 3$ și un exemplu $\mathbf{x}^{(i)}$ cu logiții $\boldsymbol{\Theta}^t \mathbf{x}^{(i)} = (2, 1, -1)^t$, ca în secțiunea 3.3.3, $\hat{\mathbf{y}}^{(i)} \approx (0.705, 0.259, 0.035)^t$. Dacă exemplul este de clasă $y^{(i)} = 1$, atunci $\mathbf{y}_{\text{ohe}}^{(i)} = (1, 0, 0)^t$, iar termenul corespunzător din suma (3.43) este $-\ln 0.705 \approx 0.349$: modelul acordă cea mai mare probabilitate clasei corecte, iar eroarea este mică. Dacă exemplul este de clasă $y^{(i)} = 3$, atunci $\mathbf{y}_{\text{ohe}}^{(i)} = (0, 0, 1)^t$, iar termenul este $-\ln 0.035 \approx 3.349$, de aproape zece ori mai mare: funcția de eroare penalizează puternic o predicție greșită făcută cu încredere. În ambele cazuri, la eroare contribuie doar probabilitatea clasei corecte; probabilitățile celorlalte clase o influențează doar indirect, prin numitorul funcției softmax.

Pentru $y^{(i)} = 1$, vectorul derivatelor parțiale ale lui J_i în raport cu logiții (secțiunea 3.3.6) este $\hat{\mathbf{y}}^{(i)} - \mathbf{y}_{\text{ohe}}^{(i)} \approx (-0.295, 0.259, 0.035)^t$. La un pas al metodei gradientului descendent, $\boldsymbol{\theta}_1$ se modifică astfel încât logitul clasei corecte să crească, iar $\boldsymbol{\theta}_2$ și $\boldsymbol{\theta}_3$ astfel încât logiții celorlalte clase să scadă, cu atât mai mult cu cât probabilitatea estimată pentru clasa respectivă este mai mare.

3.3.8 Algoritmul de antrenare pentru regresia logistică multinomială

Modificarea vectorului $\boldsymbol{\theta}_k$ este:

$$\boldsymbol{\theta}_k \leftarrow \boldsymbol{\theta}_k - \alpha \cdot \nabla_{\boldsymbol{\theta}_k} J(\boldsymbol{\Theta}) \tag{3.62}$$

$$= \boldsymbol{\theta}_k - \frac{\alpha}{m} \sum_{i=1}^{m} \left[\mathbf{x}^{(i)} \left(\hat{y}_k^{(i)} - I(y^{(i)} = k) \right) \right] \tag{3.63}$$

simultan pentru toți $1 \leq k \leq K$. Modificarea se face iterativ, până când norma gradientului devine mică, sau matricea $\boldsymbol{\Theta}$ se stabilizează[11], funcția de eroare evoluează prea puțin[12], sau se atinge un număr maxim de iterații.

Metoda gradientului descendent pentru regresia logistică multinomială este dată în algoritmul 3.2.

[11] Respectiv: norma Frobenius a diferenței dintre două versiuni succesive ale matricei $\boldsymbol{\Theta}$ devine mai mică decât un prag ε mic și pozitiv, prestabilit.

[12] Adică $|J(\boldsymbol{\Theta}_t) - J(\boldsymbol{\Theta}_{t-1})| < \varepsilon$, pentru un ε dat, pozitiv și mic.

Algoritmul 3.2 Metoda gradientului descendent pentru regresia logistică multinomială

Intrare: setul de antrenare $\mathscr{S}$, cu $y^{(i)} \in \{1, \ldots, K\}$, rata de învățare $\alpha > 0$, pragul $\varepsilon > 0$, numărul maxim de iterații T
Ieșire: matricea de parametri $\boldsymbol{\Theta} = (\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_K)$

1: inițializează $\boldsymbol{\Theta}$ cu valori aleatoare mici în jurul lui 0 sau cu 0; $t \leftarrow 0$
2: **repetă**
3: $\quad J \leftarrow J(\boldsymbol{\Theta})$ ▷ valoarea funcției de cost, pentru monitorizare
4: $\quad \mathbf{g}_k \leftarrow \nabla_{\boldsymbol{\theta}_k} J(\boldsymbol{\Theta})$, pentru $k = 1, \ldots, K$ ▷ gradientul (3.49)
5: $\quad \boldsymbol{\theta}_k \leftarrow \boldsymbol{\theta}_k - \alpha\, \mathbf{g}_k$, pentru $k = 1, \ldots, K$ ▷ modificarea ponderilor
6: $\quad t \leftarrow t + 1$
7: **până la** îndeplinirea condiției $\sqrt{\sum_{k=1}^{K} \|\mathbf{g}_k\|^2} < \varepsilon$ sau $t = T$
8: **returnează** $\boldsymbol{\Theta}$

Matricial, cu matricele $\hat{\mathbf{Y}}$ și $\mathbf{Y}_{\text{ohe}}$ de la pagina 63, metoda se rescrie ca în algoritmul 3.3.

Algoritmul 3.3 Metoda gradientului descendent pentru regresia logistică multinomială, în formă vectorizată

Intrare: matricea de design $\mathbf{X} \in \mathbb{R}^{m \times (n+1)}$, cu prima coloană plină cu 1, matricea $\mathbf{Y}_{\text{ohe}} \in \mathbb{R}^{K \times m}$ a etichetelor codificate one-hot, rata de învățare $\alpha > 0$, pragul $\varepsilon > 0$, numărul maxim de iterații T
Ieșire: matricea de parametri $\boldsymbol{\Theta} \in \mathbb{R}^{(n+1) \times K}$

1: inițializează $\boldsymbol{\Theta}$ cu valori aleatoare mici în jurul lui 0 sau cu 0; $t \leftarrow 0$
2: **repetă**
3: $\quad \hat{\mathbf{Y}} \leftarrow \text{softmax}(\boldsymbol{\Theta}^t \mathbf{X}^t)$ ▷ softmax pe fiecare coloană
4: $\quad J \leftarrow J(\boldsymbol{\Theta})$ ▷ valoarea funcției de cost, pentru monitorizare
5: $\quad \mathbf{G} \leftarrow \frac{1}{m} \mathbf{X}^t (\hat{\mathbf{Y}}^t - \mathbf{Y}_{\text{ohe}}^t)$ ▷ gradientul $\nabla_{\boldsymbol{\Theta}} J(\boldsymbol{\Theta})$
6: $\quad \boldsymbol{\Theta} \leftarrow \boldsymbol{\Theta} - \alpha\, \mathbf{G}$ ▷ modificarea ponderilor
7: $\quad t \leftarrow t + 1$
8: **până la** îndeplinirea condiției $\|\mathbf{G}\|_F < \varepsilon$ sau $t = T$
9: **returnează** $\boldsymbol{\Theta}$

3.3.9 Predicția pentru regresia logistică multinomială

După antrenare, modelul de regresie logistică multinomială poate fi folosit pentru a estima probabilitatea condiționată a unui vector de intrare $\mathbf{x} \in \mathbb{R}^{n+1}$, iar în final se dă clasa de apartenență. Avem:

$$P(y = k | \mathbf{x}; \boldsymbol{\Theta}) = \frac{\exp\left(\boldsymbol{\theta}_k^t \mathbf{x}\right)}{\sum_{l=1}^{K} \exp\left(\boldsymbol{\theta}_l^t \mathbf{x}\right)} = \hat{y}_k \in (0, 1)$$

pentru toți k între 1 și K și se alege acel indice de clasă $k^{(\max)}$ care realizează probabilitatea maximă:

$$k^{(\max)} = \arg \max_{1 \leq k \leq K} P(y = k|\mathbf{x}; \boldsymbol{\Theta}) = \arg \max_{1 \leq k \leq K} \hat{y}_k \tag{3.64}$$

Predicția făcută de model este că $\mathbf{x}$ aparține clasei $k^{(\max)}$. O decizie similară era folosită și pentru regresia logistică binară.

3.4 Interpretarea coeficienților

La regresia liniară, coeficienții modelului au o interpretare directă: în exemplul cu apartamente din capitolul 2, coeficientul suprafeței este prețul pe metru pătrat. Și coeficienții regresiei logistice pot fi interpretați, dar nu în termeni de probabilitate, ci în termeni de șanse.

3.4.1 Șanse și logaritmul șanselor

Pentru un eveniment de probabilitate $p \in (0, 1)$, *șansele* (engl: odds) sunt raportul $\frac{p}{1-p}$ dintre probabilitatea ca evenimentul să se producă și probabilitatea ca el să nu se producă. De exemplu, $p = 0.8$ corespunde unor șanse de $0.8/0.2 = 4$ (patru la unu), iar $p = 0.5$ unor șanse egale cu 1. Logaritmul șanselor este chiar funcția logit din (3.5), inversa sigmoidei logistice. Aplicând-o ambilor membri ai ecuației (3.3), obținem:

$$\ln \frac{P(y = 1|\mathbf{x}; \boldsymbol{\theta})}{1 - P(y = 1|\mathbf{x}; \boldsymbol{\theta})} = \text{logit}\left(h_{\boldsymbol{\theta}}(\mathbf{x})\right) = \boldsymbol{\theta}^t \mathbf{x} = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n \tag{3.65}$$

Așadar, regresia logistică este un model liniar pentru logaritmul șanselor clasei pozitive. Suprafața de decizie $\boldsymbol{\theta}^t \mathbf{x} = 0$ din (3.30) este exact mulțimea punctelor în care logaritmul șanselor este 0, adică cele două clase au probabilități egale.

3.4.2 Interpretarea fiecărui coeficient

Să presupunem că trăsătura x_j, $1 \leq j \leq n$, crește cu o unitate, iar celelalte trăsături rămân neschimbate. Din (3.65), logaritmul șanselor crește cu θ_j, deci șansele se înmulțesc cu e^{θ_j}:

$$\frac{\text{șansele pentru } (x_j + 1)}{\text{șansele pentru } x_j} = \frac{e^{\boldsymbol{\theta}^t \mathbf{x} + \theta_j}}{e^{\boldsymbol{\theta}^t \mathbf{x}}} = e^{\theta_j} \tag{3.66}$$

Valoarea e^{θ_j} se numește *raportul șanselor* (engl: odds ratio) asociat trăsăturii x_j. Rezultă că:

- semnul lui θ_j dă sensul influenței: pentru $\theta_j > 0$, creșterea lui x_j mărește probabilitatea clasei pozitive, pentru $\theta_j < 0$ o micșorează, iar pentru $\theta_j = 0$ trăsătura x_j nu influențează modelul;

- efectul asupra șanselor este multiplicativ și constant: nu depinde nici de valoarea lui x_j, nici de valorile celorlalte trăsături;
- efectul asupra probabilității nu este constant. Folosind (3.7),

$$\frac{\partial h_{\boldsymbol{\theta}}(\mathbf{x})}{\partial x_j} = \theta_j \, h_{\boldsymbol{\theta}}(\mathbf{x}) \, (1 - h_{\boldsymbol{\theta}}(\mathbf{x})) \, ,$$

  care este maxim în valoare absolută, egal cu $|\theta_j|/4$, pe suprafața de decizie, unde $h_{\boldsymbol{\theta}}(\mathbf{x}) = 0.5$, și aproape zero acolo unde modelul este foarte sigur, adică $h_{\boldsymbol{\theta}}(\mathbf{x})$ este aproape de 0 sau de 1. Altfel spus, efectul asupra probabilității este mai mare acolo unde modelul este nesigur și mai mic acolo unde modelul este sigur;
- termenul liber θ_0 este logaritmul șanselor clasei pozitive atunci când $x_1 = \cdots = x_n = 0$; ca și la regresia liniară, are o interpretare concretă doar dacă valoarea 0 a trăsăturilor are sens.

3.4.3 Exemplu numeric: interpretarea modelului pentru apartamente

Considerăm modelul $\boldsymbol{\theta}^*$ instruit în secțiunea 3.2.7 pe apartamentele din tabelul 3.1, care estimează probabilitatea ca un apartament să coste peste 80000 €, având ca singură trăsătură suprafața s, măsurată în m^2, cu coeficienții $\theta_0^* \approx -9.2253$ și $\theta_1^* \approx 0.1792$. Atunci:

- $e^{0.1792} \approx 1.196$: fiecare metru pătrat în plus înmulțește șansele ca apartamentul să fie scump cu aproximativ 1.2, adică le crește cu aproximativ 20%; 10 m^2 în plus le înmulțesc cu $e^{1.792} \approx 6$;
- suprafața de decizie se află acolo unde logaritmul șanselor este 0, adică la $s = -\theta_0^*/\theta_1^* \approx 51.5$ m^2;
- după cum se vede în tabelul 3.2, creșterea suprafeței de la 50 la 60 m^2 ridică probabilitatea de la 0.43 la 0.82, iar creșterea de la 70 la 80 m^2 doar de la 0.965 la 0.994, deși în ambele cazuri șansele se înmulțesc cu același factor, aproximativ 6.

3.4.4 Precauții la interpretare

- *Unitățile de măsură.* Valoarea lui θ_j depinde de unitatea în care se măsoară x_j: pentru suprafața exprimată în dm^2 în loc de m^2, coeficientul ar fi de 100 de ori mai mic. Coeficienții a două trăsături pot fi comparați ca mărime doar dacă trăsăturile sunt standardizate (capitolul 2); în acest caz, e^{θ_j} este factorul cu care se înmulțesc șansele la o creștere cu o abatere standard a trăsăturii x_j.

s (m^2)	$\boldsymbol{\theta}^t\mathbf{x}$	$h_{\boldsymbol{\theta}}(\mathbf{x})$	șansele
50	-0.27	0.433	0.77
60	1.52	0.821	4.59
70	3.32	0.965	27.5
80	5.11	0.994	165.1

Tabelul 3.2: Logaritmul șanselor, probabilitatea estimată și șansele pentru modelul cu $\theta_0^* \approx -9.2253$ și $\theta_1^* \approx 0.1792$. La fiecare 10 m^2, logaritmul șanselor crește cu $10\,\theta_1^* \approx 1.79$, iar șansele se înmulțesc cu $e^{1.792} \approx 6$.

- *Trăsături corelate.* Interpretarea presupune că celelalte trăsături rămân neschimbate. Când trăsăturile sunt puternic corelate – de exemplu suprafața și numărul de camere – o astfel de modificare izolată este nerealistă, iar coeficienții pot avea valori instabile, chiar cu semne neașteptate.

- *Date liniar separabile.* Dacă clasele din setul de antrenare pot fi separate perfect de o varietate liniară, coeficienții cresc nelimitat în valoare absolută pe parcursul instruirii, iar interpretarea lor nu mai are sens; regularizarea, prezentată în capitolul 4, menține coeficienții finiți.

- *Asociere, nu cauzalitate.* Un coeficient arată cum se modifică probabilitatea estimată de model atunci când trăsătura se modifică; el nu arată că trăsătura este cauza apartenenței la o clasă.

3.4.5 Cazul multinomial

Pentru regresia logistică multinomială, din (3.38) rezultă că logaritmul raportului dintre probabilitățile a două clase k și l este o funcție liniară de intrare, deoarece numitorii se simplifică:

$$\ln \frac{P(y=k|\mathbf{x};\boldsymbol{\Theta})}{P(y=l|\mathbf{x};\boldsymbol{\Theta})} = (\boldsymbol{\theta}_k - \boldsymbol{\theta}_l)^t\,\mathbf{x} \tag{3.67}$$

Prin urmare, au interpretare diferențele dintre vectorii de ponderi, nu vectorii $\boldsymbol{\theta}_k$ luați separat: după cum se arată în secțiunea 3.5.1, scăzând același vector din toți vectorii $\boldsymbol{\theta}_k$ se obține același model. De regulă se alege o clasă de referință, de exemplu clasa K, și se interpretează diferențele $\boldsymbol{\theta}_k - \boldsymbol{\theta}_K$. Notând cu θ_{jk} componenta j a vectorului $\boldsymbol{\theta}_k$, adică elementul de pe linia j și coloana k a matricei $\boldsymbol{\Theta}$ (liniile fiind numerotate de la 0, ca și componentele vectorilor $\boldsymbol{\theta}_k$), valoarea $e^{\theta_{jk}-\theta_{jK}}$ este factorul cu care se înmulțește raportul $P(y=k|\mathbf{x};\boldsymbol{\Theta})/P(y=K|\mathbf{x};\boldsymbol{\Theta})$ atunci când x_j crește cu o unitate. Pentru $K=2$ se regăsește cazul binar, cu $\boldsymbol{\theta} = \boldsymbol{\theta}_1 - \boldsymbol{\theta}_2$ (secțiunea 3.5.3).

3.5 Precizări

Elementele din această secțiune aduc precizări asupra modelelor de regresie logistică, precum și discuții legate de eficientizarea calculelor.

3.5.1 Redundanța parametrilor pentru regresia logistică multinomială

Pentru regresia logistică cu două clase a fost suficient un singur vector de parametri $\boldsymbol{\theta}$. Pentru modelul de regresie logistică multinomială cu K clase se folosesc K vectori $\boldsymbol{\theta}$ – o discrepanță vizibilă, ne-am fi așteptat să fie suficienți $K-1$ vectori. Discuția care urmează arată că, într-adevăr, regresia logistică multinomială are redundanță de parametri.

Pentru o dată de intrare $\mathbf{x}$, modelul estimează probabilitatea condiționată ca:

$$P(y=k|\mathbf{x};\boldsymbol{\Theta}) = \frac{\exp(\boldsymbol{\theta}_k^t\mathbf{x})}{\sum_{j=1}^{K}\exp\left(\boldsymbol{\theta}_j^t\mathbf{x}\right)} \tag{3.68}$$

Dacă scădem din fiecare vector $\boldsymbol{\theta}_j$ un același vector $\boldsymbol{\psi}$, obținem:

$$\frac{\exp\left((\boldsymbol{\theta}_k-\boldsymbol{\psi})^t\mathbf{x}\right)}{\sum_{j=1}^{K}\exp\left((\boldsymbol{\theta}_j-\boldsymbol{\psi})^t\mathbf{x}\right)} = \frac{\exp(-\boldsymbol{\psi}^t\mathbf{x})\exp\left(\boldsymbol{\theta}_k^t\mathbf{x}\right)}{\exp(-\boldsymbol{\psi}^t\mathbf{x})\sum_{j=1}^{K}\exp\left(\boldsymbol{\theta}_j^t\mathbf{x}\right)} = \tag{3.69}$$

$$= \frac{\exp(\boldsymbol{\theta}_k^t\mathbf{x})}{\sum_{j=1}^{K}\exp\left(\boldsymbol{\theta}_j^t\mathbf{x}\right)} = P(y=k|\mathbf{x};\boldsymbol{\Theta}) \tag{3.70}$$

deci predicțiile vor fi aceleași ca la setul de parametri originar; mai mult, dacă avem vectorii $\boldsymbol{\theta}_1,\ldots,\boldsymbol{\theta}_K$ care minimizează funcția de cost J, atunci aceeași valoare minimă poate fi atinsă cu $\boldsymbol{\theta}_1-\boldsymbol{\psi},\ldots,\boldsymbol{\theta}_K-\boldsymbol{\psi}$ pentru orice vector $\boldsymbol{\psi}$. Dacă pe post de $\boldsymbol{\psi}$ se ia oricare dintre vectorii $\boldsymbol{\theta}_k$, de exemplu $\boldsymbol{\theta}_K$, atunci vectorul de parametri devine neredundant, $\boldsymbol{\theta}_1-\boldsymbol{\theta}_K, \boldsymbol{\theta}_2-\boldsymbol{\theta}_K,\ldots,\mathbf{0}$ cu același comportament ca mai înainte. Sunt deci suficienți $K-1$ vectori de ponderi. Acceptăm însă redundanța, pentru simplitatea formulelor rezultate. Redundanța este demonstrată și mai jos pentru cazul particular $K=2$, când se regăsește regresia logistică binară.

O observație interesantă e: chiar și în cazul cu redundanță funcția de eroare J rămâne funcție convexă, cu o infinitate de puncte de minim în care J are aceeași valoare, deci situația din figura 3.3 nu apare.

3.5.2 Exemplu numeric: redundanța parametrilor

Scăzând vectorul $\boldsymbol{\psi}$ din toți vectorii $\boldsymbol{\theta}_k$, din fiecare logit se scade aceeași valoare $\boldsymbol{\psi}^t\mathbf{x}$. Pentru logiții $\mathbf{z}=(2,1,-1)^t$ din secțiunea 3.3.3 și $\boldsymbol{\psi}^t\mathbf{x}=-10$

se obțin logiții $(12, 11, 9)^t$, iar

$$\text{softmax}\left((12, 11, 9)^t\right) \approx (0.705, 0.259, 0.035)^t = \text{softmax}(\mathbf{z})$$

adică probabilitățile estimate nu se schimbă, conform (3.70).

3.5.3 Relația dintre cele două tipuri de regresii logistice

Pentru regresia logistică multinomială, e de așteptat să regăsim regresia logistică binară pentru cazul $K = 2$. Într-adevăr, ecuația (3.37) devine:

$$h_{\boldsymbol{\Theta}}(\mathbf{x}) = \frac{1}{\exp(\boldsymbol{\theta}_1^t\mathbf{x}) + \exp(\boldsymbol{\theta}_2^t\mathbf{x})} \begin{pmatrix} \exp(\boldsymbol{\theta}_1^t\mathbf{x}) \\ \exp(\boldsymbol{\theta}_2^t\mathbf{x}) \end{pmatrix} \tag{3.71}$$

$$= \frac{1}{\exp(\boldsymbol{\theta}_1^t\mathbf{x}) + \exp(\boldsymbol{\theta}_2^t\mathbf{x})} \frac{\exp(-\boldsymbol{\theta}_1^t\mathbf{x})}{\exp(-\boldsymbol{\theta}_1^t\mathbf{x})} \begin{pmatrix} \exp(\boldsymbol{\theta}_1^t\mathbf{x}) \\ \exp(\boldsymbol{\theta}_2^t\mathbf{x}) \end{pmatrix} \tag{3.72}$$

$$= \frac{1}{\exp\left((\boldsymbol{\theta}_1 - \boldsymbol{\theta}_1)^t\mathbf{x}\right) + \exp\left((\boldsymbol{\theta}_2 - \boldsymbol{\theta}_1)^t\mathbf{x}\right)} \begin{pmatrix} \exp((\boldsymbol{\theta}_1 - \boldsymbol{\theta}_1)^t\mathbf{x}) \\ \exp((\boldsymbol{\theta}_2 - \boldsymbol{\theta}_1)^t\mathbf{x}) \end{pmatrix} \tag{3.73}$$

Notând $\boldsymbol{\theta} = \boldsymbol{\theta}_1 - \boldsymbol{\theta}_2$, avem:

$$h_{\boldsymbol{\Theta}}(\mathbf{x}) = \frac{1}{1 + \exp\left(-\boldsymbol{\theta}^t\mathbf{x}\right)} \begin{pmatrix} 1 \\ \exp\left(-\boldsymbol{\theta}^t\mathbf{x}\right) \end{pmatrix} \tag{3.74}$$

$$= \begin{pmatrix} \frac{1}{1+\exp(-\boldsymbol{\theta}^t\mathbf{x})} \\ 1 - \frac{1}{1+\exp(-\boldsymbol{\theta}^t\mathbf{x})} \end{pmatrix} \tag{3.75}$$

$$= \begin{pmatrix} P(y = 1|\mathbf{x}; \boldsymbol{\theta}) \\ 1 - P(y = 1|\mathbf{x}; \boldsymbol{\theta}) \end{pmatrix} \tag{3.76}$$

Tot pentru $K = 2$, se verifică ușor că funcția de eroare (3.42) devine cea din (3.15).

3.5.4 Calculul numeric al funcției softmax

La implementarea directă a funcției softmax se poate întâmpla ca valoarea funcției exponențiale să depășească posibilitățile de reprezentare numerică, pentru valori z_l pozitive mari. Un truc de calcul frecvent aplicat în practică este următorul: în loc de a se calcula funcția softmax precum în ecuația (3.33), se calculează prima dată valoarea maximă din $(z_1, \ldots, z_K)^t$, fie ea M, apoi se calculează softmax pentru vectorul de valori $\mathbf{z}' = (z_1 - M, \ldots, z_K - M)^t$. Avem că:

$$\text{softmax}(\mathbf{z}'; k) = \frac{\exp(z_k - M)}{\sum_{i=1}^{K} \exp(z_i - M)} = \frac{\exp(z_k)/\exp(M)}{\sum_{i=1}^{K} [\exp(z_i)/\exp(M)]} =$$

$$= \frac{\exp(z_k)}{\sum_{i=1}^{K} \exp(z_i)} = \text{softmax}(\mathbf{z}; k) \tag{3.77}$$

Rezultatul e același ca înainte de scăderea lui M, dar calculul se face cu exponenți mai mici, dacă $M > 0$ (echivalent cu $z_l - M < z_l$). Se evită astfel depășirea de reprezentare (engl: overflow) dacă vectorul $\mathbf{z}$ are componente pozitive foarte mari.

3.5.5 Exemplu numeric: depășirea de reprezentare la calculul funcției softmax

Pentru $\mathbf{z} = (1000, 1001, 1002)^t$, valoarea $\exp(1000)$ depășește cel mai mare număr reprezentabil în virgulă mobilă cu dublă precizie, aproximativ $1.8 \cdot 10^{308}$. Calculul direct dă ∞ pentru toate exponențialele, iar raportul ∞/∞ dă valoarea nedefinită NaN (engl: not a number). Cu $M = 1002$ obținem $\mathbf{z}' = (-2, -1, 0)^t$, $(e^{-2}, e^{-1}, e^0)^t \approx (0.1353, 0.3679, 1)^t$, cu suma 1.5032, deci $\text{softmax}(\mathbf{z}) = \text{softmax}(\mathbf{z}') \approx (0.090, 0.245, 0.665)^t$.

3.5.6 Trucul „log sum exp”

Funcția de eroare pentru regresia logistică multinomială are forma:

$$J(\mathbf{\Theta}) = -\frac{1}{m}\left[\sum_{i=1}^{m}\sum_{k=1}^{K} I\left(y^{(i)} = k\right) \cdot \ln \frac{\exp\left(\boldsymbol{\theta}_k^t \mathbf{x}^{(i)}\right)}{\sum_{j=1}^{K} \exp\left(\boldsymbol{\theta}_j^t \mathbf{x}^{(i)}\right)}\right] \tag{3.78}$$

Termenul cu logaritm permite simplificări de calcule:

$$\ln \frac{\exp\left(\boldsymbol{\theta}_k^t \mathbf{x}^{(i)}\right)}{\sum_{j=1}^{K} \exp\left(\boldsymbol{\theta}_j^t \mathbf{x}^{(i)}\right)} = \ln\left(\exp\left(\boldsymbol{\theta}_k^t \mathbf{x}^{(i)}\right)\right) - \ln\left(\sum_{j=1}^{K} \exp\left(\boldsymbol{\theta}_j^t \mathbf{x}^{(i)}\right)\right) \tag{3.79}$$

$$= \boldsymbol{\theta}_k^t \mathbf{x}^{(i)} - \ln\left(\sum_{j=1}^{K} \exp\left(\boldsymbol{\theta}_j^t \mathbf{x}^{(i)}\right)\right) \tag{3.80}$$

Pentru al doilea termen din (3.80) se folosește un truc asemănător celui din secțiunea 3.5.4, pentru a evita calculul funcției exponențiale cu exponent mare. Trucul „log sum exp” se bazează pe identitatea:

$$\ln\left(\sum_{j=1}^{K} \exp(z_j)\right) = M + \ln\left(\sum_{j=1}^{K} \exp(z_j - M)\right) \tag{3.81}$$

pentru un M oarecare, demonstrată în cele ce urmează:

$$\ln\left(\sum_{j=1}^{K}\exp(z_j)\right) = \ln\left(\sum_{j=1}^{K}\left(\exp(z_j)\exp(M)\exp(-M)\right)\right) = \tag{3.82}$$

$$= \ln\left(\exp(M)\cdot\sum_{j=1}^{K}\left(\exp(z_j)\exp(-M)\right)\right) = \tag{3.83}$$

$$= \ln\exp(M) + \ln\left(\sum_{j=1}^{K}\exp(z_j - M)\right) = \tag{3.84}$$

$$= M + \ln\left(\sum_{j=1}^{K}\exp(z_j - M)\right) \tag{3.85}$$

Pentru a reduce din magnitudinea exponenților considerăm, în mod convenabil, $M = \max_j z_j$ și reducem riscul de depășire de reprezentare numerică (engl: overflow).

Trucurile din secțiunile 3.5.4 și 3.5.6 explică și de ce în unele implementări de biblioteci pentru machine learning, pentru funcția de eroare se cer valorile vectorului $\mathbf{z}$ (logiții) și nu valorile calculate de softmax.

3.5.7 Exemplu numeric: trucul „log sum exp”

Pentru $\mathbf{z} = (1000, 1001, 1002)^t$ din secțiunea 3.5.5, calculul direct al lui $\ln\left(\sum_j \exp(z_j)\right)$ dă ∞, pe când cu $M = 1002$ obținem

$$\ln\left(\sum_{j=1}^{3}\exp(z_j)\right) = 1002 + \ln\left(e^{-2} + e^{-1} + 1\right) \approx 1002.4076$$

Conform (3.80), logaritmii probabilităților estimate sunt $z_k - 1002.4076$, adică aproximativ $(-2.4076, -1.4076, -0.4076)^t$; ei se obțin fără a calcula vreo exponențială mare și fără a logaritma valori apropiate de 0.

Capitolul 4

Regularizarea și selecția modelelor

4.1 Underfitting și overfitting pentru regresia liniară

Pentru problema estimării prețului unui apartament, să presupunem că există cinci perechi de valori în setul de antrenare, o pereche fiind constituită din variabila predictivă suprafață – notată mai jos cu x – și variabila de ieșire preț; punctele sunt reprezentate în figura 4.1. Să considerăm trei modele de predicție:

1. polinom de gradul întâi, unde prețul estimat este de forma:

$$\text{preț} = \theta_0 + \theta_1 x \tag{4.1}$$

2. polinom de gradul al doilea, unde modelul are forma:

$$\text{preț} = \theta_0 + \theta_1 x + \theta_2 x^2 \tag{4.2}$$

3. polinom de gradul al patrulea, de forma:

$$\text{preț} = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4 \tag{4.3}$$

unde x este suprafața; pe baza trăsăturii x am introdus trăsăturile noi x^2, x^3, x^4. Primul model a fost discutat în capitolul 2, iar celelalte două sunt polinoame de grad mai mare. Modelele rămân liniare, deoarece parametrii θ_0, θ_1 etc. sunt la puterea 1. Se pune problema determinării coeficienților $\theta_0, \theta_1, \ldots$. Modelul este determinat prin regresie liniară, pe baza datelor cu mai multe trăsături (multivariate).

Graficele celor trei forme polinomiale sunt date în figura 4.1.

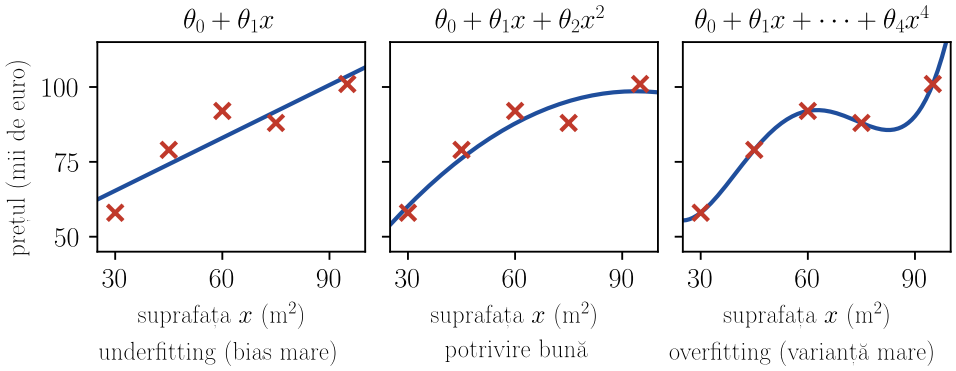


Figura 4.1: Trei polinoame pentru aproximarea prețului pornind de la suprafață, notată x; adaptare după [7].

Intuitiv, polinomul de gradul întâi nu aproximează bine evoluția prețului în funcție de suprafață. Spunem că modelul dat de primul polinom suferă de „underfitting”[1], fiind prea simplu pentru problema în cauză. Se mai spune despre un asemenea model că are bias mare (engl: high bias), deoarece face o presupunere nepotrivită pentru problema tratată.

Pentru polinomul de gradul al patrulea, dacă nu se găsesc două prețuri pe aceeași verticală (adică nu avem două apartamente cu suprafețe egale vândute cu prețuri diferite), se pot determina coeficienții $\theta_0, \ldots, \theta_4$ astfel încât curba să treacă prin toate cele cinci puncte (interpolare polinomială, vezi de exemplu polinoamele Lagrange–Hermite). Remarcăm însă forma nemonotonă, cu variații mari ale predicției. În acest caz modelul suferă de supraspecializare (engl: overfitting), fiind construit prea fidel pe datele din setul de antrenare. Deși polinomul de gradul al patrulea reproduce exact prețurile din setul de antrenare, în rest predicțiile lui nu sunt credibile: remarcăm scăderea prețului între al treilea și al patrulea punct din grafic. În plus, dacă aceste date conțin zgomot, atunci modelul va reproduce și zgomotul din setul de date. Se mai spune că modelul are variantă (variabilitate) mare (engl: high variance), din cauza faptului că e prea complex pentru problema tratată. Complexitatea crescută e dată de numărul mare de coeficienți θ, corespunzători gradului mare al polinomului de aproximare.

Pentru problema dată, polinomul de gradul al doilea prezintă cea mai credibilă formă, chiar dacă nu trece prin cele cinci puncte din setul de antrenare. Este un compromis între capacitatea de a reproduce setul de antrenare și capacitatea de generalizare, aceasta din urmă fiind abilitatea de a prezice valori de ieșire pentru cazuri care nu sunt din setul de date de antrenare.

Pe scurt, un model care suferă de „underfitting” este incapabil atât să

[1]Ipoteza simplificatoare – aici, că dependența este liniară – face ca modelul să nu producă estimări bune.

reprezinte setul de antrenare, cât și să facă estimări pentru alte valori. Un model care suferă de „overfitting” poate învăța foarte bine datele din setul de antrenare, dar nu reușește să facă estimări prea bune în afara lui; în ambele cazuri spunem că modelele nu generalizează bine. Pentru modele de predicție, în final, ne interesează cât de bine pot fi estimate situațiile din afara setului de antrenare.

Exemplificarea s-a făcut plecând de la o variabilă predictivă x reprezentând suprafața, din care am obținut alte trăsături (atribute): x^2, x^3, x^4. Mai general, putem să presupunem că avem trăsături de intrare definite în domeniul problemei; în cazul nostru, pot fi distanța dintre apartament și mijloacele de transport, școli sau magazine, gradul de poluare a zonei etc. Trebuie însă să fim capabili să detectăm cazurile de „overfitting” și „underfitting” și să le tratăm; modul în care se face acest lucru, pe baza unui set de validare, este prezentat în secțiunile 4.5.5 și 4.5.6.

O modalitate de evitare a „overfitting-ului” este reducerea numărului de trăsături: pentru problema noastră se evită folosirea variabilelor x^3, x^4, adică o alegere mai judicioasă a modelului de predicție. O altă opțiune, des folosită și tratată în acest capitol, este regularizarea: păstrăm variabilele predictive (inclusiv x^3, x^4), dar impunem constrângeri asupra parametrilor θ_j. Aceste remedii, împreună cu altele, sunt discutate în secțiunea 4.5.6.

4.2 Regularizarea L_2 pentru modele de regresie

Să considerăm că predicția este dată de funcția polinomială

$$h_{\boldsymbol{\theta}}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4 \tag{4.4}$$

unde x este suprafața apartamentului. Vrem ca în funcția de eroare să includem o constrângere asupra coeficienților θ_3, θ_4; ei se înmulțesc cu cantitățile x^3, respectiv x^4, care determină o variație rapidă a funcției h; altfel zis, o modificare mică a cantității x duce la modificări majore ale lui $h_{\boldsymbol{\theta}}(x)$. Constrângerea pe care o impunem este deci ca valorile absolute ale lui θ_3 și θ_4 să fie cât mai apropiate de zero.

Pentru aceasta, vom include în expresia funcției de eroare $J(\cdot)$ și pătratele lui θ_3 și θ_4:

$$J(\boldsymbol{\theta}) = \left\{ \frac{1}{2m} \sum_{i=1}^{m} \left(h_{\boldsymbol{\theta}} \left(x^{(i)} \right) - y^{(i)} \right)^2 \right\} + 100 \cdot \theta_3^2 + 100 \cdot \theta_4^2 \tag{4.5}$$

Minimizarea lui J din ecuația (4.5) va urmări atât micșorarea diferenței dintre valorile estimate de model și cele reale (eroarea empirică), cât și reducerea valorilor absolute ale lui θ_3, θ_4. Exemplul de mai sus este gândit pentru a aduce funcția polinomială de gradul al patrulea la una mai apropiată de gradul al doilea, despre care am văzut că generalizează mai bine la problema considerată.

În general, nu știm care dintre coeficienții puterilor lui x ar trebui să aibă valori absolute mici. Ne dăm seama că nu e nevoie ca parametrul θ_0 să fie constrâns (nu se înmulțește cu nicio variabilă predictivă; dacă avem „overfitting", nu e din cauza lui); vom impune deci constrângeri doar asupra lui $\theta_1, \theta_2, \ldots$ – să aibă valori absolute mici; echivalent, pătratele lor să fie mici, deci suma pătratelor lor să nu fie prea mare. Scopul final este de a evita „overfitting-ul". Principiul se aplică și dacă se pleacă de la variabile de intrare oarecare, nu neapărat suprafață, suprafață^2, ... Ca atare, ecuația (4.5) se rescrie mai general ca:

$$J(\boldsymbol{\theta}) = \underbrace{\frac{1}{2m}\left[\sum_{i=1}^{m}\left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}\right)^2\right]}_{\text{eroare empirică}} + \underbrace{\frac{\lambda}{2}\sum_{j=1}^{n}\theta_j^2}_{\text{termen de regularizare}} \tag{4.6}$$

unde $\lambda > 0$ e coeficientul termenului de regularizare. Cu cât λ e mai mare, cu atât constrângerea impusă coeficienților $\theta_1, \ldots, \theta_n$ e mai pronunțată. La limită, dacă $\lambda \to \infty$, atunci se ajunge la $\theta_1 = \cdots = \theta_n = 0$, deci $h_{\boldsymbol{\theta}}(\mathbf{x}) = \theta_0$, ceea ce înseamnă aproape sigur „underfitting": modelul returnează mereu θ_0, indiferent de intrare, iar dreapta de regresie este orizontală. Se arată ușor că pentru $\lambda = \infty$ valoarea lui θ_0 este egală cu media valorilor $y^{(i)}$.

În formă matricială, funcția de eroare incluzând termenul de regularizare se scrie ca:

$$J(\boldsymbol{\theta}) = \frac{1}{2m}\left(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\right)^t\left(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\right) + \frac{\lambda}{2}\left\|\boldsymbol{\theta}[1:]\right\|_2^2 \tag{4.7}$$

unde $\|\mathbf{v}\|_2$ este norma L_2 (euclidiană) a vectorului $\mathbf{v}$:

$$\|\mathbf{v}\|_2 = \sqrt{\sum_i v_i^2} \tag{4.8}$$

În ecuația (4.7), $\boldsymbol{\theta}[1:]$ este vectorul $\boldsymbol{\theta}$ din care se omite prima componentă, θ_0. Folosirea normei euclidiene dă denumirea regularizării: regularizare L_2. Regresia liniară cu regularizare L_2 se numește și ridge regression [8].

Metoda gradientului descendent pentru funcția de eroare (4.6) este dată în algoritmul 4.1.

Algoritmul 4.1 Metoda gradientului descendent pentru regresia liniară cu regularizare L_2

Intrare: setul de antrenare $\mathscr{S}$, rata de învățare $\alpha > 0$, coeficientul de regularizare $\lambda > 0$, pragul $\varepsilon > 0$, numărul maxim de iterații T

Ieșire: vectorul de parametri $\boldsymbol{\theta} = (\theta_0, \theta_1, \ldots, \theta_n)^t$

1: inițializează $\boldsymbol{\theta}$ cu valori aleatoare mici în jurul lui 0 sau cu vectorul nul; $t \leftarrow 0$
2: **repetă**
3: $\quad J \leftarrow J(\boldsymbol{\theta})$ ▷ valoarea funcției de cost, pentru monitorizare
4: $\quad g_0 \leftarrow \frac{1}{m} \sum_{i=1}^{m} \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}\right) x_0^{(i)}$ ▷ θ_0 nu se regularizează
5: $\quad g_j \leftarrow \frac{1}{m} \sum_{i=1}^{m} \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}\right) x_j^{(i)} + \lambda\, \theta_j$, pentru $j = 1, \ldots, n$ (4.9)
6: $\quad \theta_j \leftarrow \theta_j - \alpha\, g_j$, pentru $j = 0, 1, \ldots, n$ ▷ modificarea ponderilor
7: $\quad t \leftarrow t + 1$
8: **până la** îndeplinirea condiției $\|\mathbf{g}\| < \varepsilon$ sau $t = T$, unde $\mathbf{g} = (g_0, \ldots, g_n)^t$
9: **returnează** $\boldsymbol{\theta}$

Atribuirile din linia 6 se efectuează simultan, pentru toți indicii j. În formă vectorială scriem:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \left[\frac{1}{m} \mathbf{X}^t (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) + \lambda (0, \theta_1, \ldots, \theta_n)^t\right] \tag{4.10}$$

unde vectorul coloană $(0, \theta_1, \ldots, \theta_n)^t$ se formează din vectorul de ponderi $\boldsymbol{\theta}$ punând pe prima poziție valoarea 0.

Înlocuind g_j din (4.9) în linia 6 a algoritmului 4.1, modificarea lui θ_j, $j \geq 1$, se rescrie:

$$\theta_j \leftarrow \theta_j (1 - \alpha\lambda) - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}\right) \cdot x_j^{(i)}\right] \tag{4.11}$$

Observăm mai sus o dublă modificare a ponderilor θ_j: pe de o parte se scade gradientul erorii empirice înmulțit cu rata de învățare α (gradient descendent), pe de altă parte se scalează ponderea θ_j cu factorul $1 - \alpha\lambda$, de regulă pozitiv și subunitar; al doilea efect se numește decădere a ponderilor (engl: weight decay). Pentru metoda gradientului descendent, în forma dată mai sus, fără inerție (engl: momentum), regularizarea L_2 este deci echivalentă cu weight decay. Pentru alte metode de optimizare afirmația nu mai este valabilă [8].

Pentru metoda ecuațiilor normale se poate arăta că regularizarea produce

următoarea valoare pentru $\boldsymbol{\theta}$:

$$\boldsymbol{\theta}^* = \left(\mathbf{X}^t\mathbf{X} + m\lambda \cdot \underbrace{\begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{pmatrix}}_{(n+1)\times(n+1)} \right)^{-1} \mathbf{X}^t\mathbf{y} \tag{4.12}$$

unde matricea care se înmulțește cu $m\lambda$ se obține din matricea unitate de ordinul $n+1$, înlocuind cu 0 primul element de pe diagonala principală – din același motiv ca până acum: termenul liber θ_0 nu se regularizează. Se poate arăta că pentru $\lambda > 0$ matricea care se inversează în (4.12) este nesingulară, indiferent de valorile din $\mathbf{X}$.

4.3 Regularizarea modelelor de regresie logistică

4.3.1 Regularizarea L_2 pentru regresia logistică binară

Pentru regresia logistică binară, formula modelului este:

$$P(y = 1|\mathbf{x}; \boldsymbol{\theta}) = \frac{1}{1 + \exp(-\boldsymbol{\theta}^t\mathbf{x})} \tag{4.13}$$

cu vectorul de ponderi $\boldsymbol{\theta} = (\theta_0, \theta_1, \ldots, \theta_n)^t$.

Dacă valorile parametrilor[2] $\theta_1, \ldots, \theta_n$ sunt lăsate neconstrânse, atunci valorile lor absolute pot crește și influența negativ performanța de generalizare a modelului: pentru variații mici ale datelor de intrare vom avea variații mari ale ieșirii modelului; mai mult, dacă luăm în considerare trăsături de intrare de forma $x_i \cdot x_j$, $x_i \cdot x_j \cdot x_k$ etc., modelul poate ajunge să aproximeze foarte bine perechile din setul de antrenare, dar cu o performanță slabă pe datele din setul de testare (secțiunea 4.5.4)[3]. Ca atare, se preferă impunerea unor constrângeri asupra parametrilor $\theta_1, \ldots, \theta_n$ astfel încât aceștia să fie cât mai mici în valoare absolută.

Pentru regularizare se modifică forma funcției de eroare astfel:

$$J(\boldsymbol{\theta}) = \underbrace{-\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)} \cdot \ln \hat{y}^{(i)} + (1 - y^{(i)}) \cdot \ln(1 - \hat{y}^{(i)})\right]}_{\text{eroare empirică}} + \underbrace{\frac{\lambda}{2}\sum_{j=1}^{n}\theta_j^2}_{\text{termen de regularizare}} \tag{4.14}$$

[2] Ca la regresia liniară, θ_0 nu se regularizează.

[3] Altfel zis: creștem numărul de trăsături din setul de antrenare, dar numărul de exemple din acest set rămâne același: m.

Matricial se scrie astfel:

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\left(\mathbf{y}^t \cdot \ln \hat{\mathbf{y}} + (\mathbb{1}_m - \mathbf{y})^t \cdot \ln(\mathbb{1}_m - \hat{\mathbf{y}})\right) + \frac{\lambda}{2}\|\boldsymbol{\theta}[1:]\|_2^2 \tag{4.15}$$

unde $\boldsymbol{\theta}[1:]$ este vectorul format din toate componentele lui $\boldsymbol{\theta}$ mai puțin prima, iar $\|\mathbf{v}\|_2$ este norma L_2 (sau euclidiană) a vectorului $\mathbf{v}$ din (4.8).

Modificarea adusă algoritmului de antrenare este simplă: mai trebuie inclusă și derivata parțială a termenului $\frac{\lambda}{2}\theta_j^2$ în raport cu θ_j, adică $\lambda\theta_j$. Metoda gradientului descendent este dată în algoritmul 4.2.

Algoritmul 4.2 Metoda gradientului descendent pentru regresia logistică binară cu regularizare L_2

Intrare: setul de antrenare $\mathscr{S}$, cu $y^{(i)} \in \{0, 1\}$, rata de învățare $\alpha > 0$, coeficientul de regularizare $\lambda > 0$, pragul $\varepsilon > 0$, numărul maxim de iterații T

Ieșire: vectorul de parametri $\boldsymbol{\theta} = (\theta_0, \theta_1, \ldots, \theta_n)^t$

1: inițializează $\boldsymbol{\theta}$ cu valori aleatoare mici în jurul lui 0 sau cu vectorul nul; $t \leftarrow 0$
2: **repetă**
3: $\quad J \leftarrow J(\boldsymbol{\theta})$ ▷ valoarea funcției de cost, pentru monitorizare
4: $\quad g_0 \leftarrow \frac{1}{m}\sum_{i=1}^{m}\left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}\right)x_0^{(i)}$ ▷ θ_0 nu se regularizează
5: $\quad g_j \leftarrow \frac{1}{m}\sum_{i=1}^{m}\left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}\right)x_j^{(i)} + \lambda\,\theta_j$, pentru $j = 1, \ldots, n$ (4.16)
6: $\quad \theta_j \leftarrow \theta_j - \alpha\, g_j$, pentru $j = 0, 1, \ldots, n$ ▷ modificarea ponderilor
7: $\quad t \leftarrow t + 1$
8: **până la** îndeplinirea condiției $\|\mathbf{g}\| < \varepsilon$ sau $t = T$, unde $\mathbf{g} = (g_0, \ldots, g_n)^t$
9: **returnează** $\boldsymbol{\theta}$

Algoritmul are aceeași formă ca algoritmul 4.1 pentru regresia liniară; diferă doar $h_{\boldsymbol{\theta}}$, care aici e sigmoida logistică din (3.3).

În formă matricială, modificarea ponderilor este:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha\left[\frac{1}{m}\mathbf{X}^t(\hat{\mathbf{y}} - \mathbf{y}) + \lambda(0, \theta_1, \ldots, \theta_n)^t\right] \tag{4.17}$$

Este valabilă și aici echivalența dintre regularizarea L_2 și weight decay, pentru metoda gradientului descendent.

4.3.2 Exemplu numeric: date liniar separabile

Folosim primele șase apartamente din tabelul 3.1: cele de 54, 62 și 75 m^2 sunt de clasă pozitivă, cele de 30, 38 și 45 m^2 de clasă negativă. Așa cum am observat în secțiunea 3.2.2, aceste date sunt liniar separabile: orice prag de suprafață dintre 45 și 54 m^2 desparte perfect cele două clase. Arătăm că în acest caz funcția de eroare (3.14) nu are punct de minim și urmărim ce se

întâmplă cu ponderile și cu probabilitățile estimate, cu și fără regularizare L_2.

Considerăm pragul de 49.5 m^2, aflat la jumătatea distanței dintre 45 și 54 m^2, și vectorii de parametri $\boldsymbol{\theta}_c = c \cdot (-49.5, 1)^t$, cu $c > 0$, pentru care $h_{\boldsymbol{\theta}_c}(s) = \sigma\left(c\,(s - 49.5)\right)$. Pentru orice $c > 0$, modelul clasifică corect toate cele șase apartamente. Deoarece $-\ln \sigma(z) = \ln\left(1 + e^{-z}\right)$ și $-\ln\left(1 - \sigma(z)\right) = \ln\left(1 + e^{z}\right)$, termenul apartamentului i din (3.14) are, pentru ambele clase, forma

$$\ln\left(1 + e^{-c\,|s^{(i)} - 49.5|}\right)$$

care scade când c crește și tinde la 0 pentru $c \to \infty$. Tabelul 4.1 dă valoarea lui J și cea mai mică probabilitate atribuită clasei corecte pentru câteva valori ale lui c.

c	$J(\boldsymbol{\theta}_c)$	Probabilitatea minimă a clasei corecte
0.5	$3.4 \cdot 10^{-2}$	0.9047
1	$3.7 \cdot 10^{-3}$	0.9890
2	$4.1 \cdot 10^{-5}$	0.99988
4	$5.1 \cdot 10^{-9}$	$1 - 1.5 \cdot 10^{-8}$

Tabelul 4.1: Funcția de eroare pentru $\boldsymbol{\theta}_c = c \cdot (-49.5, 1)^t$, pe primele șase apartamente din tabelul 3.1.

Valoarea lui J poate fi deci făcută oricât de mică, dar nu ajunge la 0 pentru niciun $\boldsymbol{\theta}$ finit: J are infimumul 0, dar nu are punct de minim. Metoda gradientului descendent poate micșora J doar mărind în continuare ponderile.

Antrenăm modelul cu algoritmul 4.2, mai întâi pentru $\lambda = 0$, caz în care el coincide cu algoritmul 3.1. Ca în secțiunea 3.2.7, suprafața este standardizată, $z = (s - 50.67)/15.01$, cu media și abaterea standard calculate pe cele șase apartamente, iar θ_0, θ_1 sunt coeficienții modelului în variabila z. Pornim de la $\boldsymbol{\theta} = \mathbf{0}$, cu rata de învățare $\alpha = 1$. Tabelul 4.2 dă parametrii, eroarea și probabilitățile estimate pentru apartamentele cele mai apropiate de frontieră, de 45 și 54 m^2, după t iterații. Ultima coloană este pragul de decizie s^*, suprafața pentru care $\hat{y} = 0.5$, adică $s^* = 50.67 - 15.01 \cdot \theta_0/\theta_1$.

Din tabel și din figura 4.2 se văd patru lucruri.

- Ponderea θ_1 crește la fiecare înmulțire cu 10 a numărului de iterații. Creșterea este lentă, logaritmică în t, dar nelimitată; la fel crește și θ_0.
- Eroarea scade la fiecare înmulțire cu 10 a lui t și tinde la 0 fără să-l atingă.
- Probabilitățile se apropie de 0 și 1 chiar și pentru apartamentele de 45 și 54 m^2, aflate la numai 4.5 m^2 de prag. După 10^6 iterații, valorile $\hat{y}$ se apropie de extremele de probabilitate 0 și 1; modelul este „sigur” pe predicții pe care cele șase exemple nu le pot susține.

t	θ_0	θ_1	$J(\boldsymbol{\theta})$	$\hat{y}(45)$	$\hat{y}(54)$	s^*
10	0.0416	2.0594	0.2186	0.3239	0.6222	50.36
10^2	0.3668	5.4181	$6.5 \cdot 10^{-2}$	0.1572	0.8278	49.65
10^3	0.8664	11.4755	$1.1 \cdot 10^{-2}$	0.0303	0.9682	49.53
10^4	1.4533	18.9944	$1.1 \cdot 10^{-3}$	0.0033	0.9966	49.52
10^5	2.0506	26.6756	$1.1 \cdot 10^{-4}$	$3.3 \cdot 10^{-4}$	0.99966	49.51
10^6	2.6477	34.3571	$1.1 \cdot 10^{-5}$	$3.3 \cdot 10^{-5}$	0.999966	49.51

Tabelul 4.2: Metoda gradientului descendent fără regularizare ($\lambda = 0$, $\alpha = 1$), pe primele șase apartamente din tabelul 3.1, cu suprafața standardizată; $\hat{y}(45)$ și $\hat{y}(54)$ sunt probabilitățile estimate pentru apartamentele de 45 și 54 m^2, iar s^* este pragul de decizie, în m^2.

- Pragul de decizie s^* tinde la 49.5 m^2, mijlocul intervalului dintre 45 și 54 m^2. Direcția vectorului $\boldsymbol{\theta}$ se stabilizează, deși norma lui crește nelimitat. Rezultatul este general: pe date liniar separabile, metoda gradientului descendent converge în direcție către separatorul de margine maximă[4].

Consecința practică este că modelul obținut depinde de momentul în care oprim algoritmul, nu de date. Criteriul $\|\mathbf{g}\| < \varepsilon$ din algoritmul 4.2 ajunge să fie îndeplinit doar pentru că gradientul scade odată cu J, iar valorile coeficienților, fixate practic de ε sau de T, nu mai pot fi interpretate ca în secțiunea 3.4.

Repetăm antrenarea cu regularizare L_2, pentru $\lambda = 0.01$ și $\lambda = 0.1$. Acum funcția de eroare are un punct de minim: termenul $\frac{\lambda}{2}\theta_1^2$ crește nelimitat odată cu $|\theta_1|$, astfel încât mărirea ponderii, care aduce un câștig tot mai mic la eroarea empirică, ajunge să fie penalizată mai mult decât câștigă. Algoritmul converge: parametrii nu se mai modifică la a patra zecimală după 365 de iterații pentru $\lambda = 0.01$ și după 48 de iterații pentru $\lambda = 0.1$. Rezultatele sunt în tabelul 4.3.

Ponderile rămân finite, iar probabilitățile pentru apartamentele de lângă frontieră rămân moderate, cu atât mai apropiate de 0.5 cu cât λ este mai mare. Prețul plătit este o eroare empirică mai mare: 0.1091 pentru $\lambda = 0.01$ și 0.2881 pentru $\lambda = 0.1$, față de valori oricât de apropiate de 0 fără regularizare. Valoarea potrivită a lui λ se alege, de regulă, după eroarea pe setul de validare (secțiunea 4.5.3), nu pe setul de antrenare, unde eroarea empirică este cea mai mică pentru $\lambda = 0$.

[4] D. Soudry, E. Hoffer, M. S. Nacson, S. Gunasekar și N. Srebro, „The implicit bias of gradient descent on separable data", *Journal of Machine Learning Research*, vol. 19, nr. 70, pp. 1–57, 2018.

λ	θ_0^*	θ_1^*	Eroarea empirică	$J(\boldsymbol{\theta}^*)$	$\hat{y}(45)$	$\hat{y}(54)$	s^*
0.01	0.2608	3.8135	0.1091	0.1819	0.2352	0.7517	49.64
0.1	0.0464	1.4904	0.2881	0.3991	0.3737	0.5933	50.20

Tabelul 4.3: Parametrii obținuți prin metoda gradientului descendent cu regularizare L_2 ($\alpha = 1$) pe aceleași date; eroarea empirică este primul termen din J, $\hat{y}(45)$ și $\hat{y}(54)$ sunt probabilitățile estimate pentru apartamentele de 45 și 54 m^2, iar s^* este pragul de decizie, în m^2.

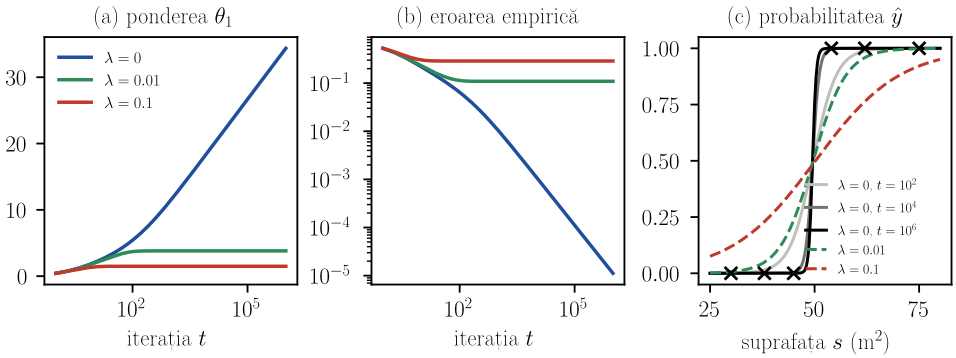


Figura 4.2: Regresia logistică pe primele șase apartamente din tabelul 3.1, date liniar separabile: (a) ponderea θ_1 și (b) eroarea empirică în funcție de numărul de iterații; (c) probabilitatea estimată în funcție de suprafață, fără regularizare după 10^2, 10^4 și 10^6 iterații și, punctat, cu regularizare L_2, după convergență.

4.3.3 Regularizarea L_2 pentru regresia logistică multinomială

Ca în cazul binar, se penalizează ponderile θ_{jk} $(1 \leq j \leq n, 1 \leq k \leq K)$, dar nu și termenii liberi θ_{0k} $(1 \leq k \leq K)$. Funcția de eroare devine:

$$J(\boldsymbol{\Theta}) = \underbrace{-\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K}\left[I\left(y^{(i)}=k\right)\cdot \ln \hat{y}_k^{(i)}\right]}_{\text{eroare empirică}} + \underbrace{\frac{\lambda}{2}\sum_{j=1}^{n}\sum_{k=1}^{K}\theta_{jk}^2}_{\text{termen de regularizare}} = \tag{4.18}$$

$$= -\frac{1}{m}\mathbb{1}_K^t \cdot \left(\mathbf{Y}_{\text{ohe}} \odot \ln \hat{\mathbf{Y}}\right)\cdot \mathbb{1}_m + \frac{\lambda}{2}\left\|\boldsymbol{\Theta}[1{:},:]\right\|_F^2$$

unde $\|\mathbf{A}\|_F$ e norma Frobenius[5] a matricei $\mathbf{A}_{p\times q}$:

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^{p}\sum_{j=1}^{q}|a_{ij}|^2} \tag{4.19}$$

iar $\boldsymbol{\Theta}[1{:},:]$ este matricea $\boldsymbol{\Theta}$ fără prima linie.

Ca mai sus, coeficientul λ este un hiperparametru care arată cât de mult contează regularizarea în cadrul funcției de eroare; pentru $\lambda = 0$ nu avem regularizare; dacă λ se alege foarte mare, atunci termenul de entropie încrucișată este neglijat în favoarea micșorării valorilor coeficienților θ_{jk}, ceea ce înseamnă că nepotrivirile dintre clasele estimate și cele reale contează cu atât mai puțin cu cât λ e mai mare. Evident, trebuie realizat un echilibru între aceste două extreme.

Gradientul funcției de eroare în raport cu $\boldsymbol{\theta}_k$, folosit în metoda gradientului descendent, este:

$$\nabla_{\boldsymbol{\theta}_k} J(\boldsymbol{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\left[\mathbf{x}^{(i)}\left(I(y^{(i)}=k) - \hat{y}_k^{(i)}\right)\right] + \lambda(0,\theta_{1k},\ldots,\theta_{nk})^t \tag{4.20}$$

unde $(0,\theta_{1k},\ldots,\theta_{nk})^t$ este vectorul coloană care are pe prima poziție scalarul 0 (termenii liberi nu se regularizează) și pe următoarele n poziții valorile $\theta_{1k},\ldots,\theta_{nk}$.

Metoda gradientului descendent, cu gradientul (4.20) calculat pentru fiecare coloană $\boldsymbol{\theta}_k$ a matricei $\boldsymbol{\Theta}$, este dată în algoritmul 4.3.

[5] În (4.19) avem pătratul unui modul, ceea ce pentru numere reale este redundant. Deoarece norma Frobenius este definită și pentru matrice cu numere complexe, păstrăm varianta cu modul.

Algoritmul 4.3 Metoda gradientului descendent pentru regresia logistică multinomială cu regularizare L_2

Intrare: setul de antrenare $\mathscr{S}$, cu $y^{(i)} \in \{1, \ldots, K\}$, rata de învățare $\alpha > 0$, coeficientul de regularizare $\lambda > 0$, pragul $\varepsilon > 0$, numărul maxim de iterații T

Ieșire: matricea de parametri $\boldsymbol{\Theta} = (\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_K)$

1: inițializează $\boldsymbol{\Theta}$ cu valori aleatoare mici în jurul lui 0 sau cu 0; $t \leftarrow 0$
2: **repetă**
3: $\quad J \leftarrow J(\boldsymbol{\Theta})$ ▷ valoarea funcției de cost, pentru monitorizare
4: $\quad \hat{\mathbf{y}}^{(i)} \leftarrow \text{softmax}(\boldsymbol{\Theta}^t \mathbf{x}^{(i)})$, pentru $i = 1, \ldots, m$
5: $\quad \mathbf{g}_k \leftarrow -\frac{1}{m} \sum_{i=1}^{m} \mathbf{x}^{(i)} \left(I(y^{(i)} = k) - \hat{y}_k^{(i)} \right)$, pentru $k = 1, \ldots, K$
6: $\quad \mathbf{g}_k \leftarrow \mathbf{g}_k + \lambda \, (0, \theta_{1k}, \ldots, \theta_{nk})^t$, pentru $k = 1, \ldots, K$
7: $\quad \boldsymbol{\theta}_k \leftarrow \boldsymbol{\theta}_k - \alpha \, \mathbf{g}_k$, pentru $k = 1, \ldots, K$ ▷ modificarea ponderilor
8: $\quad t \leftarrow t + 1$
9: **până la** îndeplinirea condiției $\sqrt{\sum_{k=1}^{K} \|\mathbf{g}_k\|^2} < \varepsilon$ sau $t = T$
10: **returnează** $\boldsymbol{\Theta}$

Linia 4 calculează probabilitățile estimate pentru toate exemplele. Liniile 5 și 6 calculează gradientul (4.20): prima, gradientul erorii empirice, ca în algoritmul 3.2; a doua adaugă termenul de regularizare, în care termenul liber θ_{0k} nu apare. Toți gradienții $\mathbf{g}_k$ sunt calculați înainte de modificarea ponderilor din linia 7, deci vectorii $\boldsymbol{\theta}_k$ se modifică simultan. Pentru $\lambda = 0$ se obține algoritmul 3.2.

Matricial, gradientul în raport cu întreaga matrice $\boldsymbol{\Theta}$ este:

$$\nabla_{\boldsymbol{\Theta}} J(\boldsymbol{\Theta}) = -\frac{1}{m} \mathbf{X}^t (\mathbf{Y}_{\text{ohe}}^t - \hat{\mathbf{Y}}^t) + \lambda \boldsymbol{\Theta}_0 \tag{4.21}$$

unde $\boldsymbol{\Theta}_0$ este matricea $\boldsymbol{\Theta}$ cu prima linie înlocuită cu zerouri.

Valoarea hiperparametrului λ se alege după eroarea pe setul de validare, ca în secțiunea 4.5.3. Valorile candidate se pot obține prin căutare pe o grilă de valori, căutare aleatoare, optimizare bayesiană, algoritmi genetici sau alte euristici de căutare.

4.4 Regularizarea cu coeficient variabil

În unele lucrări, coeficientul termenului de regularizare din oricare dintre ecuațiile (4.6), (4.14) și (4.18) se scrie ținând cont de numărul de exemple din setul de antrenare, m. De exemplu, funcția de eroare (4.6) se poate scrie ca:

$$J(\boldsymbol{\theta}) = \underbrace{\frac{1}{2m} \left[\sum_{i=1}^{m} \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2 \right]}_{\text{eroare empirică}} + \underbrace{\frac{\lambda}{2m} \sum_{j=1}^{n} \theta_j^2}_{\text{termen de regularizare}} \tag{4.22}$$

și similar pentru (4.14) și (4.18): coeficientul $\frac{\lambda}{2}$ devine $\frac{\lambda}{2m}$. Ca efect imediat, importanța relativă a termenului de regularizare scade pe măsură ce crește setul de antrenare. Motivația este că utilizarea mai multor date în antrenare are ea însăși un efect de regularizare a modelului (secțiunea 4.5.6), deci putem reduce penalizarea adăugată de termenul de regularizare, micșorând proporțional coeficientul ei. Corespunzător, coeficientul de decădere a ponderilor se modifică și el; de exemplu, (4.11) devine:

$$\theta_j \leftarrow \theta_j \left(1 - \frac{\alpha\lambda}{m}\right) - \alpha \left[\frac{1}{m}\sum_{i=1}^{m}\left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}\right) \cdot x_j^{(i)}\right] \tag{4.23}$$

iar prin metoda ecuațiilor normale se ajunge la:

$$\boldsymbol{\theta}^* = \left(\mathbf{X}^t\mathbf{X} + \lambda \underbrace{\begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{pmatrix}}_{(n+1)\times(n+1)}\right)^{-1} \mathbf{X}^t\mathbf{y} \tag{4.24}$$

4.5 Seturi de antrenare, validare și testare

În secțiunea 4.1 am analizat cele trei modele din figura 4.1 după cât de credibile sunt graficele lor. Pentru modele cu multe trăsături de intrare graficul nu mai poate fi desenat, deci avem nevoie de un criteriu numeric. În această secțiune arătăm cum se estimează capacitatea de generalizare a unui model, cum se aleg hiperparametrii, de exemplu λ, și cum se depistează și se corectează underfitting-ul și overfitting-ul.

4.5.1 Motivație

Eroarea empirică pe setul de antrenare nu măsoară capacitatea de generalizare. Polinomul de gradul al patrulea din figura 4.1 trece prin toate cele cinci puncte, deci are eroarea empirică 0, cea mai mică dintre cele trei modele, deși predicțiile lui sunt cele mai puțin credibile. La fel, în exemplul din secțiunea 4.3.2, eroarea empirică este cea mai mică pentru $\lambda = 0$, adică exact pentru modelul cu probabilitățile „prea sigure”. Dacă am alege modelul după eroarea pe setul de antrenare, am alege mereu modelul cel mai complex și cel mai puțin regularizat.

Ne interesează eroarea modelului pe date noi, care provin din aceeași distribuție ca datele de antrenare, dar nu au fost folosite la antrenare; o numim eroare de generalizare. Ea nu poate fi calculată exact, deoarece ar

necesita toate datele posibile, dar poate fi estimată pe exemple etichetate care au fost ținute deoparte la antrenare.

4.5.2 Împărțirea datelor

Datele etichetate disponibile se împart în trei seturi disjuncte:

- setul de antrenare $\mathscr{S}$ (engl: training set), folosit pentru determinarea parametrilor $\boldsymbol{\theta}$ cu algoritmii de învățare din capitolele anterioare;
- setul de validare $\mathscr{V}$ (engl: validation set), folosit pentru alegerea hiperparametrilor și pentru compararea modelelor, secțiunea 4.5.3;
- setul de testare $\mathscr{T}$ (engl: test set), folosit o singură dată, la final, pentru estimarea erorii de generalizare a modelului ales, secțiunea 4.5.4.

O împărțire uzuală este 60% din date pentru antrenare, 20% pentru validare și 20% pentru testare; de exemplu, din 1000 de apartamente, 600 sunt folosite la antrenare, 200 la validare și 200 la testare. Se folosesc și alte proporții, de exemplu 70%, 15% și 15%. Pentru seturi de date foarte mari, de ordinul milioanelor de exemple, câteva zeci de mii de exemple sunt suficiente pentru validare și testare, deci proporția acestor două seturi poate coborî la 1% fiecare.

Înainte de împărțire, exemplele se amestecă aleator. Datele pot fi ordonate, de exemplu după suprafață sau după data vânzării; fără amestecare, setul de testare ar conține doar apartamentele cele mai mari sau cele vândute ultimele, iar cele trei seturi nu ar mai proveni din aceeași distribuție. Excepție fac datele ordonate în timp pentru care se cere predicția valorilor viitoare: acolo setul de testare conține cele mai recente exemple.

Pentru probleme de clasificare se folosește de regulă împărțirea stratificată (engl: stratified split): proporția fiecărei clase este aceeași în cele trei seturi ca în datele inițiale. Dacă, de exemplu, doar 5% dintre exemple sunt de clasă pozitivă, o împărțire pur aleatoare a unui set mic de date poate lăsa setul de validare sau pe cel de testare cu foarte puține exemple pozitive, sau chiar fără niciunul, iar eroarea calculată pe el nu mai este reprezentativă. Împărțirea stratificată se face separat pentru fiecare clasă: exemplele fiecărei clase se împart în proporțiile alese, apoi părțile corespunzătoare se reunesc.

Seturile de validare și de testare trebuie să joace rolul unor date noi. Prin urmare, nicio informație din ele nu trebuie să ajungă în modelul antrenat; încălcarea acestei reguli se numește scurgere de informație (engl: data leakage). Cel mai frecvent caz apare la preprocesare. Media și abaterea standard folosite la standardizarea unei trăsături x se calculează doar pe setul de antrenare:

$$z = \frac{x - \mu_{\mathscr{S}}}{\sigma_{\mathscr{S}}} \tag{4.25}$$

unde $\mu_{\mathscr{S}}$ și $\sigma_{\mathscr{S}}$ sunt media și abaterea standard ale trăsăturii x pe $\mathscr{S}$. Aceleași valori $\mu_{\mathscr{S}}$ și $\sigma_{\mathscr{S}}$ se aplică apoi exemplelor din $\mathscr{V}$ și din $\mathscr{T}$, precum și datelor noi din etapa de predicție. Dacă media și abaterea standard s-ar calcula pe toate datele, modelul ar „vedea" indirect setul de testare, iar eroarea estimată pe el ar fi optimistă. Regula se aplică oricărei operații care învață ceva din date: scalarea la intervalul $[0, 1]$, selecția trăsăturilor, completarea valorilor lipsă cu media etc. În exemplele numerice din secțiunile 3.2.7 și 4.3.2 am procedat deja astfel: media și abaterea standard au fost calculate pe setul de antrenare.

Când datele sunt puține, un set de validare de 20% conține puține exemple, iar eroarea calculată pe el variază mult de la o împărțire la alta. În acest caz se folosește validarea încrucișată cu k partiții (engl: k-fold cross-validation): datele de antrenare și de validare se împart în k părți egale, de regulă $k = 5$ sau $k = 10$; modelul se antrenează de k ori, de fiecare dată pe $k - 1$ părți, și se evaluează pe partea rămasă, iar eroarea de validare este media celor k erori obținute [8].

4.5.3 Rolul setului de validare

Hiperparametrii, introduși în capitolul 2, nu sunt determinați de algoritmul de antrenare, ci sunt fixați înaintea lui. Exemple sunt rata de învățare α – conform [8] e cel mai important hiperparametru, gradul polinomului din (4.1)–(4.3), coeficientul de regularizare λ, și numărul maxim de iterații T. Setul de validare se folosește pentru alegerea lor:

1. se alege o mulțime de valori candidate; pentru α, de regulă pe o scară logaritmică, de exemplu $10^{-4}, 10^{-3}, \ldots, 10^{-1}, 1$; se fac alegeri similare pentru ceilalți hiperparametri;
2. pentru fiecare valoare candidată, modelul se antrenează pe $\mathscr{S}$;
3. pentru fiecare model obținut se calculează eroarea pe setul de validare, J_{val};
4. se alege valoarea pentru care J_{val} este minimă.

Pentru regresia liniară, eroarea pe setul de validare este:

$$J_{\text{val}}(\boldsymbol{\theta}) = \frac{1}{2|\mathscr{V}|} \sum_{(\mathbf{x},y)\in\mathscr{V}} \left(h_{\boldsymbol{\theta}}(\mathbf{x}) - y\right)^2 \tag{4.26}$$

unde $|\mathscr{V}|$ este numărul de exemple din setul de validare. Pentru regresia logistică se folosește, analog, entropia încrucișată (3.14) calculată pe $\mathscr{V}$, sau proporția de exemple clasificate greșit. Eroarea de antrenare J_{antr} și eroarea de testare J_{test} se definesc la fel, pe $\mathscr{S}$, respectiv pe $\mathscr{T}$. Toate trei conțin doar eroarea empirică, fără termenul de regularizare: acesta este un instrument

de antrenare, nu o măsură a calității predicțiilor; dacă ar fi inclus, valori diferite ale lui λ ar fi comparate pe funcții diferite.

După alegerea hiperparametrilor, modelul se poate antrena din nou pe reuniunea $\mathscr{S} \cup \mathscr{V}$, cu valorile alese, pentru a folosi toate datele disponibile.

Aceeași procedură se folosește pentru alegerea între modele diferite, de exemplu între cele trei polinoame din secțiunea 4.1. Numărul de iterații poate fi ales tot pe baza setului de validare: se urmărește J_{val} în timpul antrenării și se păstrează parametrii pentru care aceasta este minimă; procedeul se numește oprire timpurie (engl: early stopping) și este prezentat în capitolul 6, figura 6.15.

4.5.4 Rolul setului de testare

După ce hiperparametrii au fost aleși, eroarea modelului final pe setul de testare, J_{test}, este estimarea erorii lui de generalizare. De ce nu folosim în acest scop eroarea J_{val}, calculată deja? Hiperparametrii au fost aleși tocmai pentru a minimiza J_{val}: dintre mai multe modele candidate l-am păstrat pe cel cu eroarea de validare cea mai mică, inclusiv datorită unor particularități întâmplătoare ale exemplelor din $\mathscr{V}$. Valoarea minimă a lui J_{val} este deci o estimare optimistă a erorii de generalizare, cu atât mai optimistă cu cât am încercat mai multe valori candidate. Este același fenomen ca la setul de antrenare, unde parametrii $\boldsymbol{\theta}$ sunt aleși pentru a minimiza J_{antr}, dar la o scară mai mică: hiperparametrii sunt puțini, deci se potrivesc pe setul de validare mai puțin decât se potrivesc parametrii pe setul de antrenare.

Setul de testare dă o estimare nedeplasată numai dacă nu a influențat nicio decizie. De aceea el se folosește o singură dată, la final, după ce modelul și hiperparametrii au fost fixați. Dacă, văzând J_{test}, ne întoarcem și modificăm modelul, setul de testare devine de fapt un al doilea set de validare, iar eroarea calculată pe el nu mai este o estimare nedeplasată.

4.5.5 Depistarea underfitting-ului și a overfitting-ului

Underfitting-ul și overfitting-ul se depistează comparând eroarea pe setul de antrenare, J_{antr}, cu eroarea pe setul de validare, J_{val}:

- underfitting (bias mare): J_{antr} este mare, iar J_{val} este apropiată de J_{antr}; modelul nu reușește să învețe nici măcar datele pe care a fost antrenat;
- overfitting (varianță mare): J_{antr} este mică, dar J_{val} este mult mai mare decât J_{antr}; modelul reproduce bine setul de antrenare, dar nu generalizează;
- model potrivit: ambele erori sunt mici și apropiate între ele.

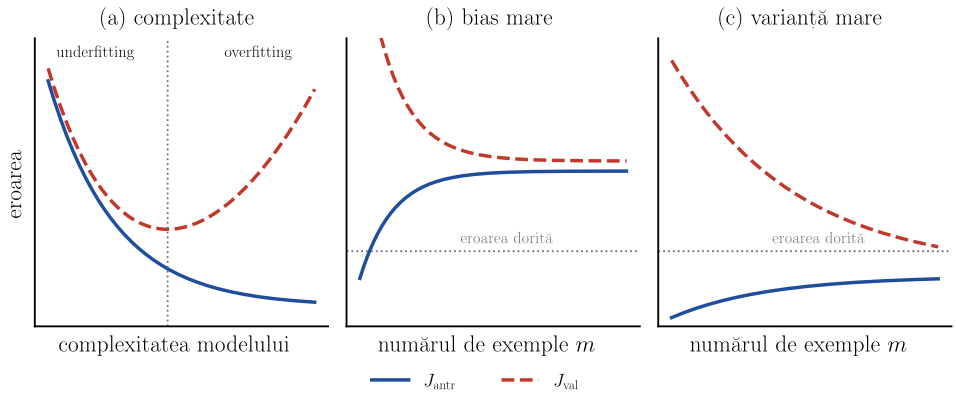


Figura 4.3: Curbe schematice ale erorii pe setul de antrenare (J_{antr}, linie continuă) și pe setul de validare (J_{val}, linie întreruptă): (a) în funcție de complexitatea modelului, cu minimul erorii de validare marcat prin linia punctată verticală; (b) curbe de învățare pentru un model cu bias mare (underfitting); (c) curbe de învățare pentru un model cu varianță mare (overfitting). Linia punctată orizontală din (b) și (c) este eroarea dorită.

„Mare" și „mic" se apreciază în raport cu eroarea considerată acceptabilă pentru problema dată, numită mai jos eroarea dorită: de exemplu, la estimarea prețului unui apartament, o abatere tipică de câteva mii de euro, sau eroarea pe care o face un expert uman.

Figura 4.3(a) arată evoluția celor două erori când crește complexitatea modelului, de exemplu gradul polinomului din secțiunea 4.1. Eroarea de antrenare scade monoton: un model mai complex poate reproduce mai bine datele de antrenare. Eroarea de validare scade la început, apoi crește din nou, deci are formă de U. În stânga minimului ei, ambele erori sunt mari și apropiate (underfitting); în dreapta lui, decalajul dintre ele crește (overfitting). Complexitatea potrivită corespunde minimului erorii de validare. Pentru coeficientul de regularizare λ imaginea este oglindită: un λ mare constrânge puternic parametrii și dă un model simplu, cu underfitting, iar un λ mic lasă loc overfitting-ului.

Curbele de învățare (engl: learning curves) arată cum se schimbă cele două erori când crește numărul de exemple de antrenare [7]. Modelul se antrenează pe primele m exemple din $\mathscr{S}$, pentru valori crescătoare ale lui m, și se reprezintă grafic J_{antr}, calculată pe cele m exemple folosite, și J_{val}, calculată pe tot setul de validare. Pentru m mic, modelul poate reproduce aproape exact puținele exemple de antrenare, deci J_{antr} este mică, dar generalizează slab, deci J_{val} este mare. Pe măsură ce m crește, exemplele de antrenare devin mai greu de reprodus toate, iar J_{antr} crește; în același timp modelul generalizează mai bine, iar J_{val} scade.

- Pentru un model cu bias mare, figura 4.3(b), cele două curbe ajung

repede la un platou comun, aflat mult deasupra erorii dorite. Exemplele suplimentare nu mai schimbă nimic: un model prea simplu rămâne prea simplu oricâte date ar primi. Dreapta din figura 4.1 ar rămâne o aproximare slabă a prețului oricâte apartamente am adăuga.

- Pentru un model cu variață mare, figura 4.3(c), între cele două curbe rămâne un decalaj mare, iar J_{val} continuă să scadă când m crește. Eroarea J_{antr} crește, deoarece este o medie calculată chiar pe cele m exemple folosite la antrenare: un model flexibil reproduce exact puține exemple, dar nu și multe exemple afectate de zgomot. De exemplu, polinomul de gradul al patrulea, cu cinci parametri, trece prin oricare cinci puncte cu abscise distincte, deci are $J_{\text{antr}} = 0$ pentru $m \leq 5$; pentru $m \geq 6$ el nu mai poate trece, în general, prin toate punctele, iar $J_{\text{antr}} > 0$. Creșterea este însă lentă, iar J_{antr} rămâne sub eroarea dorită: modelul are bias mic, deci poate reprezenta tendința generală a datelor, iar eroarea rămasă provine în principal din zgomot. Exemplele suplimentare ajută: cu cât sunt mai multe, cu atât e mai greu ca modelul să le reproducă exact pe toate, deci este forțat să surprindă tendința generală a datelor. Pe măsură ce m crește, decalajul se micșorează, iar cele două curbe tind spre aceeași valoare, cea mai mică eroare pe care o poate atinge acest tip de model.

4.5.6 Corectarea underfitting-ului și a overfitting-ului

Diagnosticul din secțiunea 4.5.5 arată în ce direcție trebuie modificat modelul. Remediile pentru cele două situații sunt, în mare, opuse.

Pentru underfitting (bias mare):

- se crește complexitatea modelului: se adaugă trăsături noi, fie derivate din cele existente (x^2, x^3, produse $x_i \cdot x_j$), fie din domeniul problemei, ca în exemplele din secțiunea 4.1 (distanța față de mijloacele de transport, școli sau magazine); pentru rețelele neurale din capitolul 6, se mărește numărul de neuroni sau de straturi;
- se micșorează coeficientul de regularizare λ;
- se verifică dacă algoritmul de antrenare a convers: o eroare de antrenare mare poate proveni și dintr-un număr de iterații T prea mic sau dintr-o rată de învățare α nepotrivită, nu din forma modelului.

Mai multe date de antrenare nu corectează underfitting-ul, după cum se vede în figura 4.3(b).

Pentru overfitting (variață mare):

- se folosesc mai multe date de antrenare, figura 4.3(c); este de regulă remediul cel mai eficient, dar și cel mai costisitor, deoarece datele

etichetate trebuie colectate; faptul că mai multe date au ele însele un efect de regularizare stă la baza regularizării cu coeficient variabil din secțiunea 4.4;

- se aplică regularizarea din secțiunea 4.2 sau, dacă ea este deja folosită, se mărește coeficientul λ;
- se reduce complexitatea modelului: mai puține trăsături, de exemplu renunțarea la x^3 și x^4 ca în secțiunea 4.1, sau un polinom de grad mai mic;
- se oprește antrenarea mai devreme, pe baza erorii de validare (oprirea timpurie, capitolul 6).

După fiecare modificare, modelul se evaluează din nou pe setul de validare, nu pe cel de testare (secțiunea 4.5.4).

Capitolul 5

Perceptronul liniar

5.1 Încadrare și exemplu

Un perceptron liniar este cel mai simplu tip de neuron, capabil să învețe să separe două clase de puncte care sunt liniar separabile. Învățarea este supervizată.

Pentru exemplificare, să presupunem că avem următoarele șase puncte din planul cartezian: $(-4, 2)$, $(-4, 6)$, $(-2, 4)$, $(3, -2)$, $(5, -2)$, $(4, 4)$. Primele trei puncte sunt de clasă negativă (*i.e.* -1) și restul de clasă pozitivă (*i.e.* $+1$). Reprezentarea lor este dată în figura 5.1. Observăm că, pentru exemplul considerat, putem găsi cel puțin o dreaptă care are de o parte doar punctele de clasă pozitivă, iar de cealaltă parte doar punctele de clasă negativă; de exemplu, dreapta d_1 de ecuație $4x - 3y + 12 = 0$. Spunem că cele două clase – pozitivă și negativă – sunt liniar separabile. Pot exista mai multe drepte care le separă: de exemplu, avem dreapta d_2 de ecuație $3x + y - 6 = 0$ – de asemenea reprezentată în aceeași figură. Se poate arăta că, dacă există o dreaptă de separare, atunci există o infinitate de drepte care separă aceleași mulțimi de puncte.

Din geometria analitică, pentru un plan (spațiu bidimensional), știm că o dreaptă de ecuație $ax + by + c = 0$ îl împarte în două semiplane. Pentru toate punctele de coordonate (x_p, y_p) aflate de o anumită parte a dreptei, expresia $ax_p + by_p + c$ este mereu strict mai mare decât zero; de cealaltă parte a dreptei, punctele produc doar valori strict negative; punctele aflate pe dreaptă, satisfăcând deci ecuația ei, vor produce valoarea 0. Pentru punctele figurate cu „+” în figura 5.2, toate valorile sunt strict pozitive, iar cele figurate cu „−” produc valori strict negative. Cele două semiplane se numesc semiplan pozitiv, respectiv semiplan negativ.

Am ales puncte din plan pentru a le putea reprezenta grafic. Considerente similare sunt valabile în spații euclidiene multidimensionale.

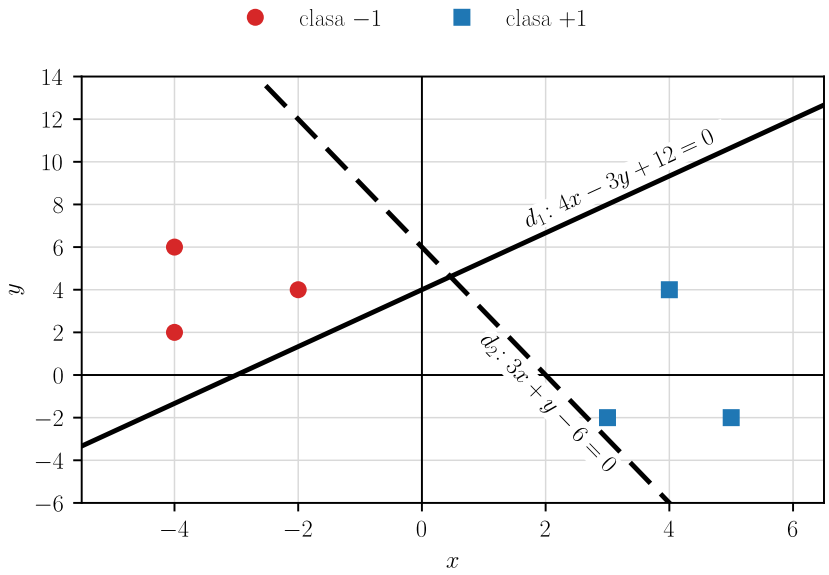


Figura 5.1: Pentru două mulțimi de puncte liniar separabile pot exista mai multe suprafețe de separare. În cazul dat, ambele drepte d_1 și d_2 separă punctele de clasă pozitivă de cele de clasă negativă.

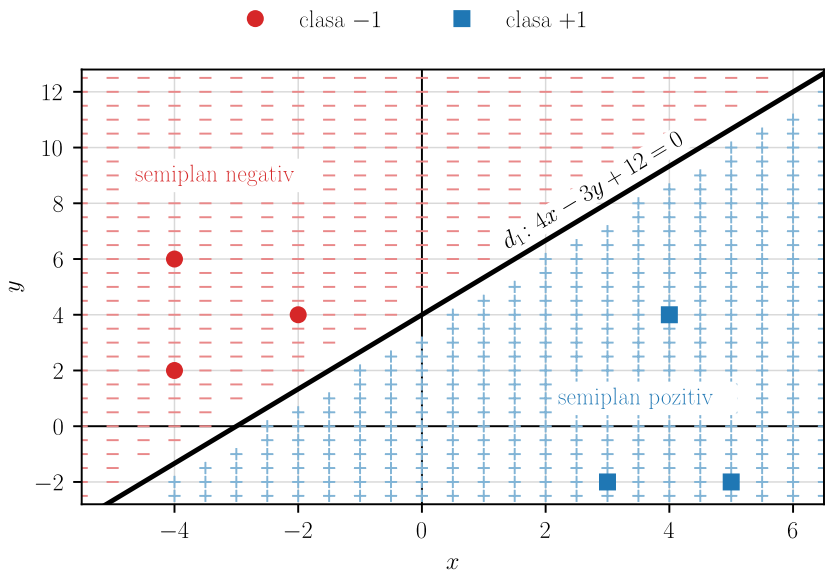


Figura 5.2: Orice dreaptă împarte un plan în două semiplane. Pentru o dreaptă de ecuație $d(x, y) = 0$ și pentru un semiplan fixat, orice punct de coordonate (x_p, y_p) din acel semiplan produce același semn pentru $d(x_p, y_p)$. Semnul rezultat dă natura semiplanului: pozitiv sau negativ.

5.2 Definiții și notații

În acest capitol, parametrii modelului se notează cu $\mathbf{w}$ și b. După extinderea vectorilor din relația (5.6), vectorul $\mathbf{w}$ are rolul vectorului de parametri $\boldsymbol{\theta}$ din capitolele 2–4, iar termenul liber $w_0 = b$ pe al lui θ_0. Etichetele claselor sunt -1 și $+1$, nu 0 și 1 ca la regresia logistică; această alegere simplifică regula de modificare a ponderilor (5.18).

Definiția 4 (Clase liniar separabile)**.** *Mulțimile $\mathscr{C}_1$ și $\mathscr{C}_2$ din spațiul $\mathbb{R}^n$ se numesc liniar separabile dacă există o funcție $y : \mathbb{R}^n \to \mathbb{R}$ de forma:*

$$y(\mathbf{x}) = \sum_{i=1}^{n} w_i \cdot x_i + b \tag{5.1}$$

astfel încât:

$$\begin{cases} y(\mathbf{x}) > 0 \text{ pentru } \mathbf{x} \in \mathscr{C}_1 \\ y(\mathbf{x}) < 0 \text{ pentru } \mathbf{x} \in \mathscr{C}_2 \end{cases} \tag{5.2}$$

În condițiile de mai sus, numim clasa $\mathscr{C}_1$ clasă pozitivă, iar $\mathscr{C}_2$ clasă negativă.

Plecând de la un set de antrenare format din mulțimile $\mathscr{C}_1$, $\mathscr{C}_2$ despre care se știe că sunt liniar separabile – dar pentru care funcția $y(\cdot)$ nu se cunoaște – perceptronul liniar determină ponderile (coeficienții) $b, w_1, \ldots, w_n$ pentru care condițiile din (5.2) sunt îndeplinite. Determinarea ponderilor se face prin învățare supervizată.

Ecuația $y(\mathbf{x}) = 0$, precum și mulțimile $\mathscr{C}_1$, $\mathscr{C}_2$ pe care y le separă, au interpretări geometrice simple. Punctele $\mathbf{x} \in \mathbb{R}^n$ pentru care $y(\mathbf{x}) = 0$ formează o varietate liniară, notată cu S, de dimensiune $n - 1$. Dacă $b = 0$, atunci $y(\mathbf{x}) = 0$ este un hiperplan (subspațiu vectorial de dimensiune $n - 1$). Prin abuz de limbaj, în literatură se folosește tot denumirea de hiperplan pentru suprafața $y(\mathbf{x}) = 0$, chiar și pentru $b \neq 0$.

Suprafața liniară $y(\mathbf{x}) = 0$ împarte spațiul $\mathbb{R}^n$ în două submulțimi (semispații); pentru $n = 2$, în două semiplane. Oricum am alege un punct $\mathbf{x}$ dintr-o submulțime (semispațiu) oarecare, semnul lui $y(\mathbf{x})$ este mereu același, iar la trecerea în cealaltă submulțime semnul se schimbă. Vorbim astfel de semispațiu pozitiv și semispațiu negativ.

Dacă $\mathbf{x}_1$ și $\mathbf{x}_2$ sunt două puncte de pe varietatea liniară S dată de $y(\mathbf{x}) = 0$, atunci:

$$y(\mathbf{x}_1) = 0 \Leftrightarrow \mathbf{w}^t \cdot \mathbf{x}_1 + b = 0 \tag{5.3}$$

$$y(\mathbf{x}_2) = 0 \Leftrightarrow \mathbf{w}^t \cdot \mathbf{x}_2 + b = 0 \tag{5.4}$$

Scăzând (5.4) din (5.3), obținem $\mathbf{w}^t \cdot (\mathbf{x}_1 - \mathbf{x}_2) = 0$, sau, echivalent, vectorul $\mathbf{w}$ este perpendicular pe varietatea liniară $y(\mathbf{x}) = 0$, a se vedea figura 5.3.

Distanța dintre un punct de coordonate $\mathbf{x}$ și suprafața S este:

$$d(\mathbf{x}, S) = \frac{|y(\mathbf{x})|}{\|\mathbf{w}\|} \tag{5.5}$$

unde $\mathbf{w} = (w_1, \ldots, w_n)^t$, iar $\|\mathbf{w}\|_2 = \sqrt{w_1^2 + \cdots + w_n^2}$ este norma euclidiană[1] a vectorului $\mathbf{w}$.

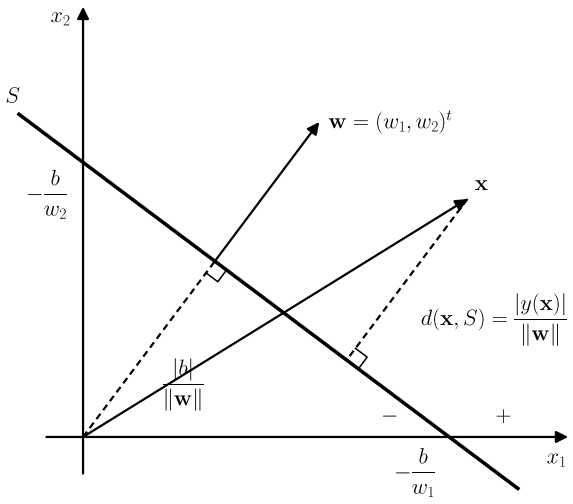


Figura 5.3: Suprafața de decizie S, de ecuație $y(\mathbf{x}) = \mathbf{w}^t \cdot \mathbf{x} + b = 0$, pentru $n = 2$. Vectorul de ponderi $\mathbf{w}$ este perpendicular pe S și arată spre semispațiul pozitiv $(+)$. Segmentele punctate sunt distanța de la punctul $\mathbf{x}$ la S și distanța de la origine la S.

Notăm termenul liber b din (5.1) cu w_0 și adăugăm vectorului $\mathbf{x}$ componenta $x_0 = 1$. Notând tot cu $\mathbf{w}$ și $\mathbf{x}$ vectorii extinși, $\mathbf{w} = (w_0, w_1, \ldots, w_n)^t$ și $\mathbf{x} = (x_0, x_1, \ldots, x_n)^t$, funcția y din (5.1) se rescrie:

$$y(\mathbf{x}) = \mathbf{w}^t \cdot \mathbf{x} \tag{5.6}$$

5.3 Modelul perceptronului

Setul de antrenare este:

$$\mathscr{S} = \left\{ \left(\mathbf{x}^{(i)}, y^{(i)} \right) \mid \mathbf{x}^{(i)} \in \mathbb{R}^{n+1}, y^{(i)} \in \{-1, 1\}, 1 \leq i \leq m \right\} \tag{5.7}$$

unde $y^{(i)}$ este eticheta vectorului de intrare $\mathbf{x}^{(i)}$; ca și în capitolele precedente, considerăm $\mathbf{x}_0^{(i)} = 1$. Partiționăm vectorii de intrare $\mathbf{x}^{(i)}$ în submulțimile:

- $\mathscr{C}_1$, clasa pozitivă: vectorii $\mathbf{x}^{(i)}$ pentru care $y^{(i)} = +1$;

[1] În acest capitol folosim norma euclidiană. În loc de $\|\mathbf{v}\|_2$ vom scrie mai simplu $\|\mathbf{v}\|$.

- $\mathscr{C}_2$, clasa negativă: vectorii $\mathbf{x}^{(i)}$ pentru care $y^{(i)} = -1$.

Presupunem că mulțimile $\mathscr{C}_1$ și $\mathscr{C}_2$ sunt liniar separabile; în acest caz, perceptronul poate învăța o suprafață de separare.

Graful de calcul pentru un perceptron este dat în figura 5.4.

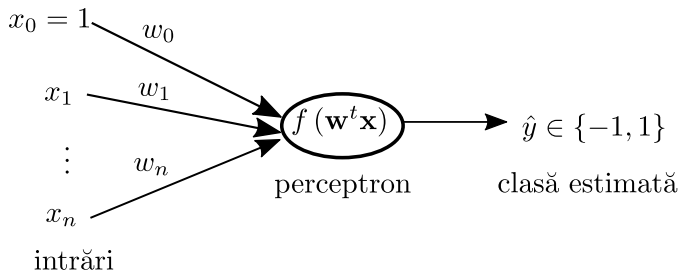


Figura 5.4: Graful de calcul al unui perceptron liniar.

Valoarea de stare a perceptronului este $z = \mathbf{w}^t \cdot \mathbf{x} = \sum_{i=0}^{n} w_i \cdot x_i$. Valoarea de ieșire (numită și activare) a perceptronului se calculează cu funcția de activare „treaptă”:

$$f(z) = \begin{cases} +1, & \text{dacă } z > 0 \\ -1, & \text{dacă } z < 0 \\ \text{nedefinit}, & \text{dacă } z = 0 \end{cases} \tag{5.8}$$

Valoarea produsă pentru ponderile $\mathbf{w}$ și intrarea $\mathbf{x}$ este:

$$\hat{y} = f(\mathbf{w}^t \cdot \mathbf{x}) \tag{5.9}$$

La predicția făcută cu ponderile $\mathbf{w}$ pentru un vector $\mathbf{x}$:

- dacă $f(\mathbf{w}^t \cdot \mathbf{x}) = 1$, adică $\mathbf{w}^t \cdot \mathbf{x} > 0$, spunem că perceptronul indică $\mathbf{x}$ ca fiind de clasă pozitivă;
- dacă $f(\mathbf{w}^t \cdot \mathbf{x}) = -1$, adică $\mathbf{w}^t \cdot \mathbf{x} < 0$, spunem că perceptronul indică $\mathbf{x}$ ca fiind de clasă negativă.

Valoarea funcției f este deliberat nedefinită în 0: dacă $\mathbf{w}^t \cdot \mathbf{x} = 0$, atunci punctul $\mathbf{x}$ se află pe suprafața de separare și despre un astfel de punct nu se poate spune că este de clasă pozitivă sau negativă.

5.4 Algoritmul de instruire a perceptronului

Plecând de la setul de antrenare $\mathscr{S}$ din (5.7), algoritmul caută ponderile $\mathbf{w} = (w_0, w_1, \ldots, w_n)^t$ astfel încât:

$$\begin{cases} \text{pentru } \mathbf{x}^{(i)} \in \mathscr{C}_1 \text{ să avem } \mathbf{w}^t \cdot \mathbf{x}^{(i)} > 0 \\ \text{pentru } \mathbf{x}^{(i)} \in \mathscr{C}_2 \text{ să avem } \mathbf{w}^t \cdot \mathbf{x}^{(i)} < 0 \end{cases} \tag{5.10}$$

adică o suprafață liniară care separă clasele $\mathscr{C}_1$ și $\mathscr{C}_2$.

Vom demonstra, în secțiunea 5.7, că, dacă mulțimile $\mathscr{C}_1$ și $\mathscr{C}_2$ sunt liniar separabile, atunci algoritmul de instruire determină o funcție de separare liniară.

Instruirea pornește de la valori aleatoare ale ponderilor $w_0, w_1, \ldots, w_n$; ele pot fi chiar toate 0. Notăm vectorul inițial de ponderi cu $\mathbf{w}(1)$. Printr-un proces iterativ vom obține vectorii de ponderi $\mathbf{w}(2), \ldots, \mathbf{w}(k), \ldots$. Vectorii de ponderi se vor stabiliza la un moment dat: $\mathbf{w}(l) = \mathbf{w}(l+1) = \ldots$ dacă avem două clase liniar separabile.

Învățarea (adaptarea) ponderilor $\mathbf{w}$ se face prin adăugarea succesivă a unor vectori de diferențe:

$$\mathbf{w}(1) \xrightarrow{+\Delta\mathbf{w}(1)} \mathbf{w}(2) \xrightarrow{+\Delta\mathbf{w}(2)} \mathbf{w}(3) \ldots \mathbf{w}(k) \xrightarrow{+\Delta\mathbf{w}(k)} \mathbf{w}(k+1) \ldots \tag{5.11}$$

Pentru vectorul curent de ponderi $\mathbf{w}(k)$, intrarea curentă $\mathbf{x}^{(i)}$ și eticheta asociată $y^{(i)}$, estimarea produsă de perceptron este $\hat{y}^{(i)} = f\left(\mathbf{w}(k)^t \mathbf{x}^{(i)}\right)$. Dacă pentru un vector de ponderi $\mathbf{w}(k)$ avem $\hat{y}^{(i)} = y^{(i)}$ pentru toate exemplele din setul de antrenare, atunci perceptronul recunoaște corect clasele datelor din $\mathscr{S}$. Altfel, vectorul $\mathbf{w}(k)$ trebuie modificat, ca mai jos.

Dacă pentru o pereche $\left(\mathbf{x}^{(i)}, y^{(i)}\right) \in \mathscr{S}$ avem că $\hat{y}^{(i)} = y^{(i)}$, atunci pentru acest caz vectorul de ponderi $\mathbf{w}(k)$ nu trebuie modificat: perceptronul clasifică corect intrarea $\mathbf{x}^{(i)}$. Dacă $\hat{y}^{(i)} \neq y^{(i)}$, atunci avem unul dintre următoarele două cazuri:

1. $\mathbf{x}^{(i)}$ este de clasă pozitivă, dar este clasificat momentan ca negativ sau ca nedefinit: $y^{(i)} = +1$, dar $\hat{y}^{(i)} = -1$ sau nedefinit;

2. $\mathbf{x}^{(i)}$ este de clasă negativă, dar este clasificat momentan ca pozitiv sau nedefinit: $y^{(i)} = -1$, dar $\hat{y}^{(i)} = +1$ sau nedefinit.

Pentru cazul 1 avem că $\mathbf{w}(k)^t \cdot \mathbf{x}^{(i)} \leq 0$, dar ar trebui ca produsul să fie strict pozitiv. Trebuie deci modificat vectorul $\mathbf{w}(k)$ astfel încât, pentru noul vector $\mathbf{w}(k+1)$, valoarea produsului scalar cu $\mathbf{x}^{(i)}$ să crească: $\mathbf{w}(k+1)^t \cdot \mathbf{x}^{(i)} > \mathbf{w}(k)^t \cdot \mathbf{x}^{(i)}$, în speranța că aceasta va duce la $\mathbf{w}(k+1)^t \cdot \mathbf{x}^{(i)} > 0$ și deci la $\hat{y}^{(i)} = f(\mathbf{w}(k+1)^t \cdot \mathbf{x}^{(i)}) = +1 = y^{(i)}$. Modificarea propusă $\Delta\mathbf{w}(k)$ este:

$$\Delta\mathbf{w}(k) = \alpha \cdot \mathbf{x}^{(i)} \tag{5.12}$$

unde α este un coeficient strict pozitiv. Noul vector de ponderi va fi:

$$\mathbf{w}(k+1) \leftarrow \mathbf{w}(k) + \Delta\mathbf{w}(k) = \mathbf{w}(k) + \alpha \cdot \mathbf{x}^{(i)} \tag{5.13}$$

Verificăm că vectorul $\mathbf{w}(k+1)$ duce la creșterea valorii produsului scalar dintre ponderi și vectorul de intrare.

Avem:

$$\mathbf{w}(k+1)^t \cdot \mathbf{x}^{(i)} = \left(\mathbf{w}(k) + \alpha \cdot \mathbf{x}^{(i)}\right)^t \cdot \mathbf{x}^{(i)} = \mathbf{w}(k)^t \cdot \mathbf{x}^{(i)} + \alpha \cdot \left(\mathbf{x}^{(i)}\right)^t \cdot \mathbf{x}^{(i)} =$$
$$= \mathbf{w}(k)^t \cdot \mathbf{x}^{(i)} + \alpha \left\|\mathbf{x}^{(i)}\right\|^2 \geq \mathbf{w}(k)^t \cdot \mathbf{x}^{(i)} + \alpha \quad (5.14)$$

deoarece $\left\|\mathbf{x}^{(i)}\right\| \geq 1$ (componenta $x_0^{(i)} = 1$). Prin urmare, obținem creșterea produsului scalar dintre ponderi și vectorul de intrare, așa cum ne-am propus.

Pentru cazul 2, facem modificarea ponderilor astfel:

$$\Delta\mathbf{w}(k) = -\alpha \cdot \mathbf{x}^{(i)} \quad (5.15)$$

deci

$$\mathbf{w}(k+1) \leftarrow \mathbf{w}(k) + \Delta\mathbf{w}(k) = \mathbf{w}(k) - \alpha \cdot \mathbf{x}^{(i)} \quad (5.16)$$

cu aceeași condiție pentru α. Se verifică, la fel ca în (5.14), că $\mathbf{w}(k+1)^t \cdot \mathbf{x}^{(i)} < \mathbf{w}(k)^t \cdot \mathbf{x}^{(i)}$.

Sintetizând:

$$\Delta\mathbf{w}(k) = \begin{cases} \mathbf{0}_{n+1}, & \text{dacă } y^{(i)} = \hat{y}^{(i)} \\ \alpha \cdot \mathbf{x}^{(i)}, & \text{dacă } y^{(i)} = +1 \text{ și } \hat{y}^{(i)} \neq +1 \\ -\alpha \cdot \mathbf{x}^{(i)}, & \text{dacă } y^{(i)} = -1 \text{ și } \hat{y}^{(i)} \neq -1 \end{cases} \quad (5.17)$$

$$= \begin{cases} \mathbf{0}_{n+1}, & \text{dacă } y^{(i)} = \hat{y}^{(i)} \\ \alpha y^{(i)} \mathbf{x}^{(i)}, & \text{dacă } y^{(i)} \neq \hat{y}^{(i)} \end{cases} \quad (5.18)$$

Este posibil ca o singură modificare a ponderilor să nu fie suficientă pentru ca întregul set $\mathscr{S}$ să fie clasificat corect. Dacă în decursul unei epoci de învățare – o epocă este o parcurgere completă a setului $\mathscr{S}$ – apare cel puțin o modificare, se va efectua o nouă epocă. Procesul se încheie când ponderile se stabilizează, adică întregul set este clasificat corect. Teorema 1 ne asigură că instruirea se termină după un număr finit de modificări de ponderi.

Pașii învățării sunt dați în algoritmul 5.1.

Algoritmul 5.1 Algoritmul de instruire a perceptronului

Intrare: setul de antrenare $\mathscr{S}$, cu clasele $\mathscr{C}_1$ și $\mathscr{C}_2$ liniar separabile, rata de învățare $\alpha > 0$

Ieșire: vectorul de ponderi $\mathbf{w}(k) = (w_0, w_1, \ldots, w_n)^t$

1: inițializează $\mathbf{w}(1)$ cu componente aleatoare sau cu vectorul nul; $k \leftarrow 1$
2: **repetă**
3: $corectClasificat \leftarrow$ adevărat
4: **pentru** $i = 1, \ldots, m$ **execută**
5: $\hat{y}^{(i)} \leftarrow f\left(\mathbf{w}(k)^t \cdot \mathbf{x}^{(i)}\right)$
6: **dacă** $\hat{y}^{(i)} \neq y^{(i)}$ **atunci**
7: $corectClasificat \leftarrow$ fals
8: $\mathbf{w}(k+1) \leftarrow \mathbf{w}(k) + \alpha y^{(i)} \mathbf{x}^{(i)}$ ▷ modificarea ponderilor
9: $k \leftarrow k + 1$
10: **sfârșit dacă**
11: **sfârșit pentru**
12: **până când** $corectClasificat =$ adevărat
13: **returnează** $\mathbf{w}(k)$

La linia 6 din algoritmul 5.1, pentru cazul $\mathbf{w}(k)^t \cdot \mathbf{x}^{(i)} = 0$, se consideră că nu avem egalitate între clasa prezisă și cea reală, deoarece funcția f returnează rezultat nedefinit – a se vedea ecuația (5.8) de la pagina 98.

Vectorul $\mathbf{w}(k)$ returnat este vectorul final de ponderi pentru perceptron, iar k este numărul de vectori de ponderi considerați.

Deși modifică ponderile imediat după fiecare clasificare greșită, algoritmul nu este incremental în sensul dat în secțiunea 8.1, având de regulă nevoie de mai multe epoci de învățare.

După ce algoritmul de învățare se oprește, clasificarea unui vector ca fiind de clasă pozitivă sau negativă se face ca în secțiunea 5.3.

5.5 Exemplu numeric

Considerăm setul de antrenare:

$$\begin{aligned}\mathscr{S} = \{&((-4,2),-1),((-4,6),-1),((-2,4),-1),\\ &((3,-2),1),((5,-2),1),((4,4),1)\}\end{aligned} \tag{5.19}$$

unde un element de forma $((x,y),c)$ se interpretează astfel: punctul de coordonate (x,y) este de clasă c. Pornim de la dreapta inițială d de ecuație $-2x-3y+7=0$, deci $\mathbf{w} = (7,-2,-3)^t$. Punctele și dreapta sunt reprezentate în figura 5.5. Observăm că dreapta d nu separă clasele. În algoritmul de instruire considerăm $\alpha = 1$.

Punctele sunt parcurse pe rând, în ordinea din $\mathscr{S}$. Pentru punctul $(-4,2)$, de clasă -1, perceptronul produce $\hat{y} = 1$. Se face modificarea

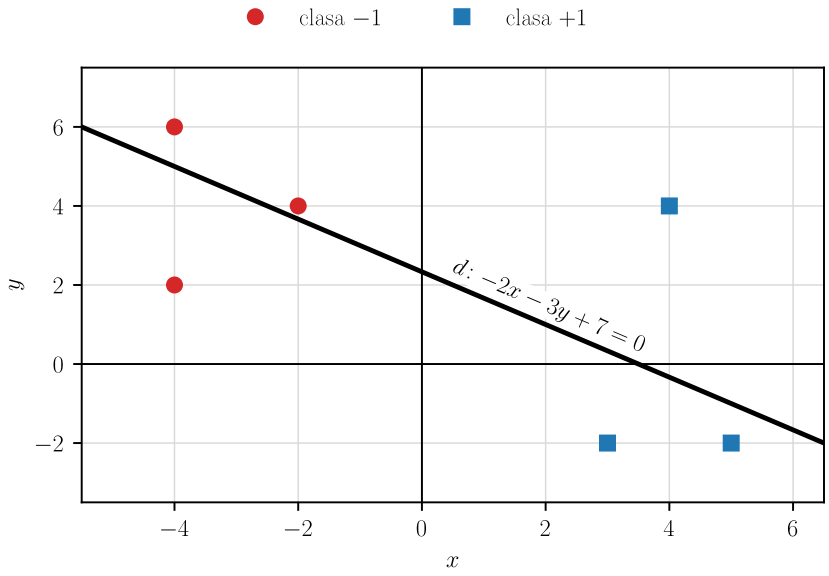


Figura 5.5: Se pornește de la dreapta d de ecuație $-2x - 3y + 7 = 0$. Ea nu clasifică corect datele de intrare.

dictată de algoritm și vectorul de ponderi $\mathbf{w}$ devine $\mathbf{w} = (6, 2, -5)^t$; dreapta corespunzătoare este reprezentată în figura 5.6. Observăm că nici această dreaptă nu separă corect clasele, deci este de așteptat să se modifice vectorul de ponderi.

Punctele de coordonate $(-4, 6)$, $(-2, 4)$, $(3, -2)$, $(5, -2)$ sunt corect clasificate de dreapta curentă. Punctul de coordonate $(4, 4)$ este incorect clasificat ca fiind negativ; se modifică ponderile și se obține $\mathbf{w} = (7, 6, -1)^t$, iar dreapta devine $d\colon 6x - y + 7 = 0$, reprezentată în figura 5.7. Algoritmul mai parcurge o dată setul de date de antrenare: deoarece nu mai sunt necesare modificări ale ponderilor $\mathbf{w}$, algoritmul se oprește la finalul celei de-a doua epoci.

5.6 Modificarea ponderilor prin metoda gradientului descendent

Algoritmii de instruire pentru regresia liniară și cea logistică au folosit gradienții unor funcții de eroare. Putem da și pentru perceptron o regulă de instruire cu gradienți. Notăm cu $\mathscr{M}(\mathbf{w})$ mulțimea indicilor exemplelor din $\mathscr{S}$ clasificate greșit de perceptronul cu ponderile $\mathbf{w}$, adică a celor pentru care $y^{(i)}\mathbf{w}^t \cdot \mathbf{x}^{(i)} \leq 0$; ca la linia 6 din algoritmul 5.1, punctele aflate pe suprafața de separare sunt considerate greșit clasificate. Funcția de eroare pentru tot setul de antrenare, numită *criteriul perceptronului* (a se vedea [3], secț. 1.6),

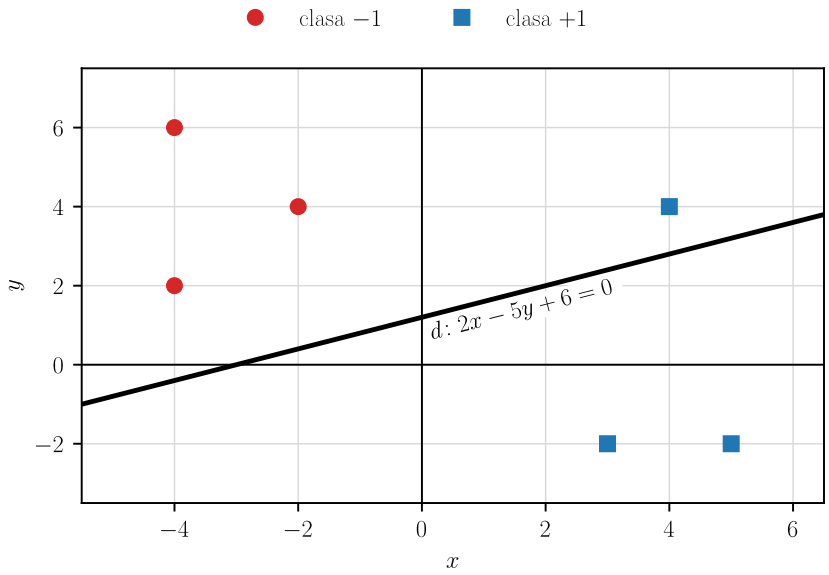


Figura 5.6: La prima eroare de clasificare, dreapta se schimbă în d: $2x - 5y + 6 = 0$. Doar un punct de clasă pozitivă este clasificat incorect.

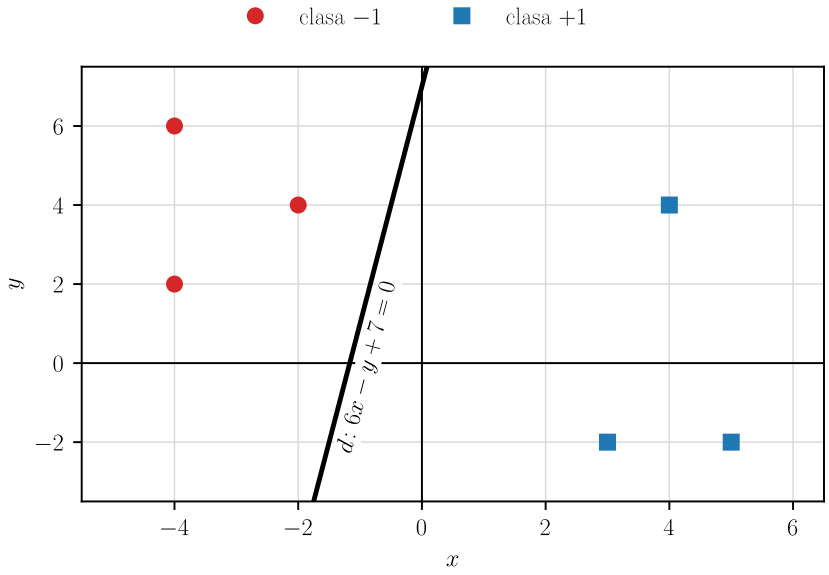


Figura 5.7: Algoritmul de instruire modifică dreapta în d: $6x - y + 7 = 0$. Toate punctele sunt corect clasificate.

este:

$$J(\mathbf{w}) = -\sum_{i \in \mathscr{M}(\mathbf{w})} y^{(i)} \mathbf{w}^t \cdot \mathbf{x}^{(i)} \tag{5.20}$$

Observăm că:

- fiecare termen al sumei este nenegativ, deoarece $y^{(i)} \mathbf{w}^t \cdot \mathbf{x}^{(i)} \leq 0$ pentru $i \in \mathscr{M}(\mathbf{w})$, deci $J(\mathbf{w}) \geq 0$;
- atunci când perceptronul clasifică corect tot setul de antrenare, mulțimea $\mathscr{M}(\mathbf{w})$ este vidă și $J(\mathbf{w}) = 0$;
- J se poate scrie ca sumă peste toate exemplele:

$$J(\mathbf{w}) = \sum_{i=1}^{m} J_i(\mathbf{w}), \quad \text{unde } J_i(\mathbf{w}) = \max\left(0, -y^{(i)} \mathbf{w}^t \cdot \mathbf{x}^{(i)}\right) \tag{5.21}$$

  de unde se vede că J este continuă și liniară pe porțiuni; funcția J_i nu este derivabilă în punctele $\mathbf{w}$ pentru care $\mathbf{w}^t \cdot \mathbf{x}^{(i)} = 0$;
- Din definiția (5.20) rezultă că $J(\mathbf{0}_{n+1}) = 0$, deși vectorul nul nu clasifică corect niciun exemplu; mai general, $J(\mathbf{w}) = 0$ ori de câte ori niciun exemplu nu se află strict de partea greșită a suprafeței de decizie. Prin urmare, $J(\mathbf{w}) = 0$ nu garantează clasificarea corectă: algoritmul 5.1 se oprește doar când $\mathscr{M}(\mathbf{w})$ este vidă. Pornind de la $\mathbf{w}(1) = \mathbf{0}_{n+1}$, algoritmul părăsește acest minim necorespunzător tocmai pentru că tratează cazul $\mathbf{w}^t \cdot \mathbf{x}^{(i)} = 0$ ca eroare.

Calculăm gradientul lui J_i în raport cu $\mathbf{w}$. Dacă exemplul i este clasificat corect, adică $y^{(i)} \mathbf{w}^t \cdot \mathbf{x}^{(i)} > 0$, atunci J_i este nulă într-o vecinătate a lui $\mathbf{w}$ și gradientul ei este $\mathbf{0}_{n+1}$. Pentru $i \in \mathscr{M}(\mathbf{w})$ avem $J_i(\mathbf{w}) = -y^{(i)} \mathbf{w}^t \cdot \mathbf{x}^{(i)}$ și:

$$\nabla_{\mathbf{w}} \left[-y^{(i)} \mathbf{w}^t \cdot \mathbf{x}^{(i)}\right] = -y^{(i)} \nabla_{\mathbf{w}} \left[\mathbf{w}^t \cdot \mathbf{x}^{(i)}\right] = -y^{(i)} \mathbf{x}^{(i)} \tag{5.22}$$

pentru ultimul pas, a se vedea [9] secț. 2.4.1, ec. (69); s-a mai folosit această proprietate și în capitolul de regresie liniară, ecuația (2.32) de la pagina 35. În punctele cu $\mathbf{w}^t \cdot \mathbf{x}^{(i)} = 0$, unde J_i nu este derivabilă, se folosește aceeași valoare, $-y^{(i)} \mathbf{x}^{(i)}$.

Aplicând metoda gradientului descendent pentru vectorul curent $\mathbf{x}^{(i)}$, modificăm[2] ponderile cu $-\alpha \cdot \nabla_{\mathbf{w}} J_i(\mathbf{w})$: pentru un exemplu corect clasificat nu se face nicio modificare, iar pentru unul greșit clasificat modificarea este $-\alpha \cdot \left(-y^{(i)} \mathbf{x}^{(i)}\right)$. Regăsim astfel termenul de modificare a ponderilor din algoritmul 5.1.

Avem o deosebire între algoritmul perceptronului și cei de la regresia liniară sau logistică: la aceștia din urmă funcția de eroare este calculată pe

[2] Ca până acum, $\alpha > 0$ este rata de învățare.

tot setul de antrenare, pe când în algoritmul perceptronului se calculează eroarea și se aplică gradientul ei pentru fiecare caz de clasificare greșită, ca în varianta *stochastic gradient descent* din secțiunea 6.9.2. Modificarea algoritmului perceptronului pentru a folosi funcția (5.20) și gradienții pe toată funcția J este imediată. Am preferat algoritmul 5.1, deoarece este forma clasică.

5.7 Convergența perceptronului

Vom demonstra că algoritmul de instruire a perceptronului se termină într-un număr finit de pași, dacă setul de antrenare $\mathscr{S}$ este compus din două mulțimi $\mathscr{C}_1$, $\mathscr{C}_2$ liniar separabile. Demonstrația urmează, cu altă notație, pe cea din [3], sect. 1.3.

Din condiția de separabilitate liniară rezultă că există un vector $\mathbf{w}_* \in \mathbb{R}^{n+1}$ astfel încât:

$$\begin{cases} \text{pentru } \mathbf{x}^{(i)} \in \mathscr{C}_1 \text{ să avem } \mathbf{w}_*^t \cdot \mathbf{x}^{(i)} > 0 \\ \text{pentru } \mathbf{x}^{(i)} \in \mathscr{C}_2 \text{ să avem } \mathbf{w}_*^t \cdot \mathbf{x}^{(i)} < 0 \end{cases} \tag{5.23}$$

(a se revedea condițiile 5.10), sau, mai pe scurt,

$$y^{(i)} \cdot \mathbf{w}_*^t \cdot \mathbf{x}^{(i)} > 0 \text{ pentru orice } i, 1 \leq i \leq m \tag{5.24}$$

Putem presupune că vectorul $\mathbf{w}_*$ este de normă 1; în caz contrar, îl normăm[3], iar inegalitatea din (5.24) rămâne adevărată.

Întrucât numărul de elemente din setul de antrenare este finit, putem găsi $\gamma > 0$ astfel încât:

$$y^{(i)} \cdot \mathbf{w}_*^t \cdot \mathbf{x}^{(i)} \geq \gamma \text{ pentru orice } i, 1 \leq i \leq m \tag{5.25}$$

de exemplu:

$$\gamma = \min_{1 \leq i \leq m} \left\{ y^{(i)} \cdot \mathbf{w}_*^t \cdot \mathbf{x}^{(i)} \right\} > 0 \tag{5.26}$$

Deoarece vectorul $\mathbf{w}_*$ nu este cunoscut (știm însă că există, pentru că $\mathscr{C}_1$ și $\mathscr{C}_2$ sunt liniar separabile), cantitatea γ este și ea necunoscută. Considerarea ei este totuși utilă pentru demonstrarea convergenței algoritmului de instruire a perceptronului.

Tot pentru că mulțimea de antrenare este finită, există $R > 0$ astfel încât $\|\mathbf{x}^{(i)}\| \leq R$ pentru orice i: toți vectorii de intrare din setul de antrenare sunt cuprinși în interiorul unei hipersfere centrate în origine și de rază R. Putem, de exemplu, să luăm

$$R = \max_{1 \leq i \leq m} \left\{ \|\mathbf{x}^{(i)}\| \right\} \tag{5.27}$$

Pentru a simplifica demonstrația teoremei de mai jos, presupunem că:

[3] Un vector nenul împărțit la norma lui produce un vector de normă 1. Se observă că norma lui $\mathbf{w}_*$ nu poate fi 0: dacă ar fi, atunci tot vectorul ar fi 0, iar inegalitatea (5.24) nu ar mai putea fi îndeplinită pentru niciun i.

1. inițial vectorul $\mathbf{w}(1)$ are toate elementele 0;
2. $\alpha = 1$; când $\mathbf{w}(1) = \mathbf{0}_{n+1}$, orice $\alpha > 0$ duce la aceleași clasificări, deoarece toți vectorii $\mathbf{w}(k)$ se înmulțesc cu α.

Teorema 1 (de convergență a perceptronului, Novikoff[4]). *Algoritmul de instruire a perceptronului efectuează cel mult R^2/γ^2 modificări ale ponderilor, după care returnează o suprafață liniară de separare.*

Demonstrație. Pentru $k \geq 1$, fie un $\mathbf{x}^{(i)}$ clasificat greșit de perceptron, adică $f(\mathbf{w}(k)^t \mathbf{x}^{(i)}) \neq y^{(i)}$, sau, echivalent:

$$f(\mathbf{w}(k)^t \mathbf{x}^{(i)}) \neq y^{(i)} \Leftrightarrow y^{(i)} \cdot \mathbf{w}(k)^t \mathbf{x}^{(i)} \leq 0 \tag{5.28}$$

Pentru această situație de clasificare greșită a unui vector din setul de antrenare, algoritmul modifică $\mathbf{w}(k)$ conform ecuației (5.18).

Avem:

$$\begin{aligned}
\mathbf{w}(k+1)^t \cdot \mathbf{w}_* &= (\mathbf{w}(k) + y^{(i)}\mathbf{x}^{(i)})^t \cdot \mathbf{w}_* && (5.29)\\
&= \mathbf{w}(k)^t \cdot \mathbf{w}_* + \underbrace{y^{(i)}(\mathbf{x}^{(i)})^t \cdot \mathbf{w}_*}_{\geq \gamma \text{ cf. } (5.25)} && (5.30)\\
&\geq \mathbf{w}(k)^t \cdot \mathbf{w}_* + \gamma && (5.31)
\end{aligned}$$

Prin inducție matematică și ținând cont că $\mathbf{w}(1) = \mathbf{0}_{n+1}$, se arată că:

$$\mathbf{w}(k+1)^t \cdot \mathbf{w}_* \geq k\gamma \tag{5.32}$$

Folosind inegalitatea Cauchy–Schwarz ($|\mathbf{a}^t \cdot \mathbf{b}| \leq \|\mathbf{a}\| \cdot \|\mathbf{b}\|$) și faptul că $z \leq |z|$ pentru orice z real, obținem:

$$\mathbf{w}(k+1)^t \cdot \mathbf{w}_* \leq |\mathbf{w}(k+1)^t \cdot \mathbf{w}_*| \leq \|\mathbf{w}(k+1)\| \cdot \underbrace{\|\mathbf{w}_*\|}_{=1} = \|\mathbf{w}(k+1)\| \tag{5.33}$$

și din (5.32) și (5.33) avem:

$$\|\mathbf{w}(k+1)\| \geq k\gamma \tag{5.34}$$

Pe de altă parte, avem că:

$$\begin{aligned}
\|\mathbf{w}(k+1)\|^2 &= \|\mathbf{w}(k) + y^{(i)}\mathbf{x}^{(i)}\|^2 && (5.35)\\
&= (\mathbf{w}(k) + y^{(i)}\mathbf{x}^{(i)})^t \cdot (\mathbf{w}(k) + y^{(i)}\mathbf{x}^{(i)}) && (5.36)\\
&= \|\mathbf{w}(k)\|^2 + \|y^{(i)}\mathbf{x}^{(i)}\|^2 + 2\mathbf{w}(k)^t \cdot \mathbf{x}^{(i)} \cdot y^{(i)} && (5.37)\\
&= \|\mathbf{w}(k)\|^2 + \underbrace{|y^{(i)}|^2}_{=1} \cdot \underbrace{\|\mathbf{x}^{(i)}\|^2}_{\leq R^2} + \underbrace{2\mathbf{w}(k)^t \cdot \mathbf{x}^{(i)} \cdot y^{(i)}}_{\leq 0 \text{ cf. } (5.28)} && (5.38)\\
&\leq \|\mathbf{w}(k)\|^2 + R^2 && (5.39)
\end{aligned}$$

[4] Novikoff, A. B. (1962). On convergence proofs for perceptrons. În Proceedings of the Symposium on the Mathematical Theory of Automata, Vol. XII, paginile 615–622. Polytechnic Press, Institute of Polytechnic Institute of Brooklyn.

Ținând cont că $\mathbf{w}(1) = \mathbf{0}_{n+1}$, prin inducție matematică se arată că:

$$\|\mathbf{w}(k+1)\|^2 \leq kR^2 \tag{5.40}$$

Din inegalitățile (5.34) și (5.40) rezultă:

$$k^2\gamma^2 \leq \|\mathbf{w}(k+1)\|^2 \leq kR^2 \tag{5.41}$$

și deci $k \leq R^2/\gamma^2$. □

Am obținut că indicele k al modificărilor de ponderi nu poate fi oricât de mare, deci algoritmul se termină în timp finit. La oprire, algoritmul returnează o suprafață de separare liniară care clasifică corect cazurile din setul de antrenare.

Comentarii:

1. Deoarece valoarea lui γ nu este cunoscută, rezultatul de mai sus nu ne spune care este numărul maxim de pași necesari. Totuși, am demonstrat că actualizarea ponderilor nu continuă la nesfârșit. Din criteriul de terminare a algoritmului deducem că la final avem un perceptron care separă $\mathscr{C}_1$ de $\mathscr{C}_2$.

2. Dacă vectorii $\mathbf{x}^{(i)}$ sunt departe de origine sau au altă scală decât intrarea constantă $\mathbf{x}_0^{(i)} = 1$, atunci R crește și convergența se încetinește. Standardizarea datelor de intrare, așa cum s-a discutat în capitolul 2, este din perspectivă teoretică și practică foarte utilă.

5.8 Algoritmul lui Gallant

Algoritmul lui Gallant[5] – sau algoritmul „buzunarului” (engl: pocket algorithm) – tratează cazul în care setul de antrenare nu este liniar separabil. Ideea algoritmului este de a păstra vectorul de ponderi $\mathbf{w}$ care face cele mai puține erori de clasificare pe setul de antrenare. Notăm cu $c(\mathbf{w})$ numărul de exemple din $\mathscr{S}$ clasificate corect de vectorul de ponderi $\mathbf{w}$. Vectorul curent de ponderi se modifică la fel ca în algoritmul 5.1. La finalul unei epoci se calculează $c(\mathbf{w})$ pentru vectorul curent de ponderi. Dacă numărul de clasificări corecte este mai mare decât pentru vectorul menținut până acum într-un „buzunar” (inițial, vectorul de ponderi de la care se pornește și numărul de clasificări corecte pe care le produce), atunci se actualizează conținutul „buzunarului”: vectorul curent de ponderi și numărul lui de clasificări corecte. Procesul se repetă pentru un număr prestabilit de epoci, T. La final se returnează vectorul de ponderi din „buzunar”. Pașii sunt dați în algoritmul 5.2.

[5] S. I. Gallant, „Perceptron-based learning algorithms”, *IEEE Transactions on Neural Networks*, vol. 1, nr. 2, pp. 179–191, 1990.

Algoritmul 5.2 Algoritmul buzunarului, varianta cu clichet

Intrare: setul de antrenare $\mathscr{S}$, nu neapărat liniar separabil, rata de învățare $\alpha > 0$, numărul de epoci T
Ieșire: vectorul de ponderi $\mathbf{w}_P$ din „buzunar”

1: inițializează $\mathbf{w}$ cu componente aleatoare sau cu vectorul nul
2: $\mathbf{w}_P \leftarrow \mathbf{w}$; $c_P \leftarrow c(\mathbf{w})$ ▷ conținutul inițial al „buzunarului”
3: **pentru** $j = 1, \ldots, T$ **execută** ▷ epoca j
4: **pentru** $i = 1, \ldots, m$ **execută**
5: **dacă** $f\left(\mathbf{w}^t \cdot \mathbf{x}^{(i)}\right) \neq y^{(i)}$ **atunci**
6: $\mathbf{w} \leftarrow \mathbf{w} + \alpha y^{(i)} \mathbf{x}^{(i)}$ ▷ ca în algoritmul 5.1
7: **sfârșit dacă**
8: **sfârșit pentru**
9: **dacă** $c(\mathbf{w}) > c_P$ **atunci** ▷ clichetul: c_P nu scade niciodată
10: $\mathbf{w}_P \leftarrow \mathbf{w}$; $c_P \leftarrow c(\mathbf{w})$
11: **sfârșit dacă**
12: **sfârșit pentru**
13: **returnează** $\mathbf{w}_P$

Conținutul „buzunarului” se schimbă doar dacă numărul de clasificări corecte crește; de aceea, varianta se numește „cu clichet” (engl: ratchet). În varianta originală a algoritmului, exemplele sunt alese aleator, iar în „buzunar” se păstrează vectorul de ponderi cu cel mai lung șir de clasificări corecte consecutive. Gallant a propus și varianta cu clichet, în care un astfel de vector intră în „buzunar” doar dacă, în plus, clasifică corect mai multe exemple din setul de antrenare decât vectorul aflat deja acolo. Algoritmul 5.2 păstrează doar această ultimă condiție și o verifică la finalul fiecărei epoci.

5.9 Comentarii

Spre deosebire de regresia logistică, perceptronul liniar nu produce o valoare care să exprime în ce măsură modelul consideră că un vector de intrare aparține clasei pozitive, $P(\mathscr{C}_1 \mid \mathbf{x})$. În schimb, are o demonstrație de convergență, atunci când clasele sunt liniar separabile.

La momentul apariției, perceptronul liniar al lui Rosenblatt[6] a stârnit așteptări foarte mari. Cartea *Perceptrons*[7] a lui Minsky și Papert, din 1969, arată însă limitele perceptronului cu un singur strat adaptiv: acesta nu poate rezolva probleme neseparabile liniar, cum este problema XOR, iar pentru proprietăți ca paritatea sau conexitatea are nevoie de trăsături de intrare tot mai complexe pe măsură ce crește dimensiunea intrării. Se știa că o

[6] F. Rosenblatt, „The perceptron: a probabilistic model for information storage and organization in the brain”, *Psychological Review*, vol. 65, nr. 6, pp. 386–408, 1958.
[7] M. Minsky și S. Papert, *Perceptrons: an introduction to computational geometry*, MIT Press, 1969.

rețea cu mai multe straturi de neuroni cu prag poate calcula XOR, dar nu se cunoștea o metodă de instruire pentru straturile ascunse; autorii și-au exprimat părerea, numită de ei „judecată intuitivă", că extinderea la mai multe straturi ar fi sterilă. Cartea a contribuit la descurajarea cercetărilor în domeniul rețelelor neurale artificiale. Revenirea s-a produs în 1986, când Rumelhart, Hinton și Williams[8] au propus o procedură de învățare pentru rețele cu mai multe straturi de neuroni neliniari care permitea abordarea claselor neseparabile liniar[9].

Setul de antrenare pentru problema XOR este următorul:

$$\{((0,0)^t, 0), ((1,1)^t, 0), ((1,0)^t, 1), ((0,1)^t, 1)\}$$

unde fiecare dintre cele patru tupluri conține o pereche de valori de intrare din $\{0,1\}^2$ împreună cu eticheta de clasă asociată, 0 sau 1. Reprezentarea este dată în figura 5.8.

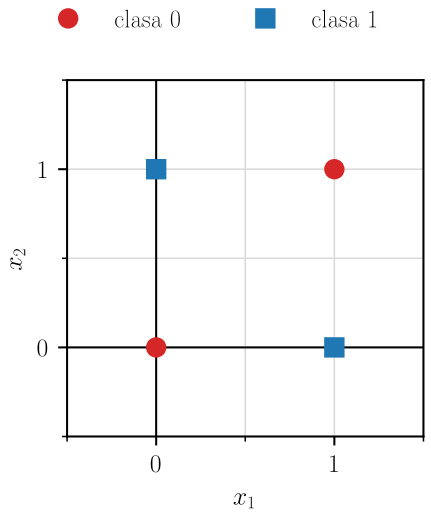


Figura 5.8: Problema XOR. Clasele sunt marcate cu forme diferite. Se poate demonstra că nu există o dreaptă în plan care să aibă de o parte a ei doar puncte de clasă 0 și de cealaltă parte doar puncte de clasă 1.

Demonstrăm algebric că nu există un vector de trei ponderi $(w_0, w_1, w_2)^t \in \mathbb{R}^3$ pentru care:

$$\text{sgn}(w_0 \cdot 1 + w_1 \cdot 0 + w_2 \cdot 1) = \text{sgn}(w_0 \cdot 1 + w_1 \cdot 1 + w_2 \cdot 0) = 1 \qquad (5.42)$$

[8] D. E. Rumelhart, G. E. Hinton și R. J. Williams, „Learning representations by back-propagating errors", *Nature*, vol. 323, nr. 6088, pp. 533–536, 1986.

[9] Procedura este algoritmul backpropagation, prezentat în secțiunea 6.9. Articolul lui Rumelhart, Hinton și Williams l-a popularizat, dar algoritmul backpropagation are antecedente: S. Linnainmaa a descris, în teza sa de master din 1970, modul invers al diferențierii automate, pe care se bazează algoritmul; P. J. Werbos a descris algoritmul în teza sa de doctorat, *Beyond regression: new tools for prediction and analysis in the behavioral sciences*, Universitatea Harvard, 1974.

iar

$$\text{sgn}(w_0 \cdot 1 + w_1 \cdot 0 + w_2 \cdot 0) = \text{sgn}(w_0 \cdot 1 + w_1 \cdot 1 + w_2 \cdot 1) = -1 \quad (5.43)$$

Incompatibilitatea sistemului de ecuații (5.42) și (5.43) se arată ușor: să presupunem că ar exista w_0, w_1, w_2 care să satisfacă (5.42) și (5.43), pe care le rescriem echivalent:

$$w_0 \cdot 1 + w_1 \cdot 0 + w_2 \cdot 1 > 0 \quad (5.44a)$$
$$w_0 \cdot 1 + w_1 \cdot 1 + w_2 \cdot 0 > 0 \quad (5.44b)$$
$$w_0 \cdot 1 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \quad (5.44c)$$
$$w_0 \cdot 1 + w_1 \cdot 1 + w_2 \cdot 1 < 0 \quad (5.44d)$$

Adunând primele două inecuații, apoi ultimele două, obținem:

$$2 \cdot w_0 + w_1 + w_2 > 0 \quad (5.45a)$$
$$2 \cdot w_0 + w_1 + w_2 < 0 \quad (5.45b)$$

ceea ce este o contradicție.

Pentru problema n-dimensională, clasa de ieșire pentru vectorul binar $\mathbf{x} \in \{0,1\}^n$ este 1 dacă numărul de componente 1 este impar, altfel 0. Și pentru cazul n-dimensional, cu $n \geq 2$, problema nu este rezolvabilă printr-un separator liniar. Să presupunem prin absurd că ar exista ponderile $w_0, w_1, \ldots, w_n$ ale unui astfel de separator. Ne restrângem la vectorii de intrare cu $x_3 = \cdots = x_n = 0$. Pentru aceștia, numărul de componente 1 este impar exact atunci când (x_1, x_2) este $(1,0)$ sau $(0,1)$, deci clasele sunt cele din problema XOR. În plus, $w_0 + w_1 x_1 + \cdots + w_n x_n = w_0 + w_1 x_1 + w_2 x_2$, deci ponderile $(w_0, w_1, w_2)^t$ ar separa clasele problemei XOR. Aceasta contrazice incompatibilitatea sistemului format din (5.42) și (5.43), demonstrată mai sus.

Capitolul 6

Perceptronii multistrat

Rețelele neurale multistrat — sau perceptronii multistrat, rețele dens conectate, eng: multilayer perceptrons (MLP), densely connected networks — sunt folosite pentru probleme de regresie, de clasificare și de estimare de probabilități condiționate. Instruirea este supervizată. Sunt cea mai populară variantă de rețele neurale artificiale și fac parte din familia rețelelor cu propagare înainte.

6.1 Motivație pentru rețele neurale multistrat

Conform celor din cursul precedent, un perceptron liniar este capabil să găsească un hiperplan de separare pentru două mulțimi, dacă – și numai dacă – ele sunt liniar separabile. Există însă exemple de clase de puncte care nu sunt liniar separabile — și deci nerezolvabile de către perceptronul liniar — dar pentru care separare se poate, totuși, face. În plus, dorim să rezolvăm și altfel de probleme decât de clasificare binară: regresie (estimare de valoare de ieșire de tip continuu), estimare de probabilitate condiționată, clasificarea oricâtor clase. Cursul de față descrie un model bazat pe straturi de neuroni cu funcții de activare neliniare, împreună cu algoritmul de învățare asociat, în care se pot rezolva toate aceste tipuri de probleme.

În capitolul anterior s–a dat un exemplu clasic de două mulțimi care nu sunt liniar separabile și deci, un perceptron liniar nu le poate discrimina. Un alt exemplu este dat în figura 6.1.

Intuim că un singur neuron e prea puțin pentru probleme complexe de separare. Totodată, concatenarea mai multor neuroni cu funcție de activare liniară este echivalentă cu produsul dintre vectorul de intrare si o matrice rezultată din înmulțirea unei succesiuni de matrice de ponderi – vectorii de valori de bias pot fi absorbiți în matrice. Datorită faptului că înmulțirea de matrice produce tot o matrice, operația este echivalentă cu înmulțirea unei matrice cu vectorul de intrare. Obținem de aici că succesiunea de neuroni cu funcție de activare liniară este echivalentă cu un singur strat de neuroni cu

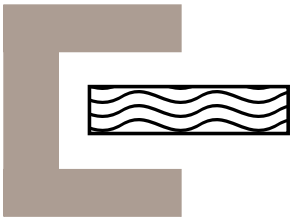


Figura 6.1: Două clase de puncte ce nu sunt liniar separabile

funcție de activare liniară.

Vom folosi deci mai mulți neuroni, iar funcțiile lor de activare vor fi neliniare.

6.2 Notații

Tabelul 6.1 conține notațiile care se folosesc în acest curs.

6.3 Setul de antrenare

Rețelele din acest capitol sunt pentru instruire de tip supervizat. Setul de antrenare este:

$$\mathscr{S} = \left\{ \left(\mathbf{x}^{(1)}, \mathbf{y}^{(1)} \right), \left(\mathbf{x}^{(2)}, \mathbf{y}^{(2)} \right), \ldots, \left(\mathbf{x}^{(p)}, \mathbf{y}^{(p)} \right) \right\} \tag{6.1}$$

unde $\mathbf{x}^{(i)} \in \mathbb{R}^n$ iar $\mathbf{y}^{(i)}$ este eticheta asociată intrării i (eng: ground truth), după caz:

- pentru o problemă de regresie: vector din $\mathbb{R}^m$;
- pentru o problemă de clasificare sau estimare de probabilități pentru m clase: vectori de forma $(1, 0, \ldots, 0)^t$, $(0, 1, 0, \ldots, 0)^t$, ..., $(0, 0, \ldots, 0, 1)^t$ cu m valori binare, din care cea de pe poziția aferentă clasei curente este unu iar restul sunt zero[1].

6.4 Rețeaua neurală MLP

6.4.1 Arhitectură

Există mai multe modalități de dispunere a neuronilor artificiali într–o rețea neurală; noi vom folosi o arhitectură de tip multistrat, feedforward, numită perceptron multistrat – chiar dacă neuronii folosiți nu sunt perceptroni, ci neuroni cu funcție de activare neliniară. O rețea multistrat se compune din minim trei straturi:

[1] Așa–numita codificare *one–hot* sau *1-din-m*.

Noțiune sau notație	Explicație
p	numărul de perechi din setul de antrenare
$\mathbf{x}^{(i)}$	vector de intrare din setul de antrenare, $\mathbf{x}^{(i)} = \left(x_1^{(i)}, \ldots, x_n^{(i)}\right)^t$, $1 \leq i \leq p$
$\mathbf{y}^{(i)}$	ieșirea asociată intrării $\mathbf{x}^{(i)}$, din setul de antrenare, $\mathbf{y}^{(i)} = \left(y_1^{(i)}, \ldots, y_m^{(i)}\right)^t$, $1 \leq i \leq p$
L	numărul de straturi din rețeaua neurală, inclusiv straturile de intrare și de ieșire
nod	neuron – dacă apare în stratul $1, 2, \ldots, L-1$ – sau nod de intrare – dacă apare în primul strat (de indice 0)
n_l	numărul de noduri din stratul l, $0 \leq l \leq L-1$; $n_0 = n$, $n_{L-1} = m$
$z_i^{[l]}$	starea neuronului i din stratul l, $1 \leq l \leq L-1$, $0 \leq i \leq n_l$
$\mathbf{z}^{[l]}$	vectorul conținând stările neuronilor din stratul l, $\mathbf{z}^{[l]} = \left(z_1^{[l]}, \ldots, z_{n_l}^{[l]}\right)^t$, $1 \leq l \leq L-1$
$a_i^{[l]}$	activarea, valoarea de ieșire a celui de al i–lea nod din stratul l, $0 \leq l \leq L-1$, $1 \leq i \leq n_l$
$\mathbf{a}^{[l]}$	vectorul cu activările nodurilor din stratul l, $0 \leq l \leq L-1$, $\mathbf{a}^{[l]} = \left(a_1^{[l]}, \ldots, a_{n_l}^{[l]}\right)^t$
$w_{ij}^{[l]}$	ponderea legăturii între neuronul i de pe stratul l și nodul j de pe stratul $l-1$, $1 \leq l \leq L-1$, $1 \leq i \leq n_l$, $1 \leq j \leq n_{l-1}$
$\mathbf{W}^{[l]}$	matricea de ponderi dintre straturile $l-1$ și l, $1 \leq l \leq L-1$, $\mathbf{W}_{ij}^{[l]} = w_{ij}^{[l]}$, $1 \leq i \leq n_l$, $1 \leq j \leq n_{l-1}$
$\mathbf{W}_i^{[l]}$	linia i a matricei $\mathbf{W}^{[l]}$, $1 \leq l \leq L-1$, $1 \leq i \leq n_l$
$b_i^{[l]}$	ponderea de bias pentru neuronul i din stratul l, $1 \leq l \leq L-1$, $1 \leq i \leq n_l$
$\mathbf{b}^{[l]}$	vectorul ponderilor de bias către stratul l, $\mathbf{b}^{[l]} = \left(b_1^{[l]}, \ldots, b_{n_l}^{[l]}\right)^t$, $1 \leq l \leq L-1$
$f^{[l]}$	funcție de activare a neuronilor din stratul l, $1 \leq l \leq L-1$
$\mathbf{W}$	secvența de matrice de ponderi $(\mathbf{W}^1, \mathbf{W}^2, \ldots, \mathbf{W}^{L-1})$
$\mathbf{b}$	secvența de vectori de ponderi de bias $(\mathbf{b}^1, \mathbf{b}^2, \ldots, \mathbf{b}^{L-1})$
$J\left(\mathbf{W}, \mathbf{b}; \mathbf{x}^{(i)}, \mathbf{y}^{(i)}\right)$	eroarea empirică pentru perechea de vectori $\left(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\right)$
$J\left(\mathbf{W}, \mathbf{b}\right)$	eroare empirică medie pentru set de vectori cu etichete cunoscute
$\hat{\mathbf{y}}^{(i)}$	vector coloană de ieșire corespunzător intrării $\mathbf{x}^{(i)}$, calculat de rețea
$\boldsymbol{\delta}^l$	vectorul cu semnalul de eroare pentru stratul l, $1 \leq l \leq L-1$
$\odot$	produs Hadamard

Tabelul 6.1: Notații folosite pentru MLP

- strat de intrare ce preia valorile de intrare; nu are rol computațional, nu este format din neuroni[2];
- măcar un strat ascuns, compus din neuroni;
- strat de ieșire, de asemenea compus din neuroni, produce valori estimate care sunt apoi comparate cu ieșirile dorite.

Începem numerotarea straturilor de la 0. Neuronii din straturile ascunse produc trăsături noi pe baza vectorilor de intrare și a funcțiilor de activare neliniare, trăsături care sunt mai apoi necesare rețelei neurale pentru producerea unei estimări. Este posibil ca o rețea să aibă mai mult de un neuron în stratul de ieșire, așa cum se vede în figura 6.3.

Instruirea e mai eficientă dacă pe lângă valorile de intrare și pe lângă valorile calculate de un strat de neuroni se mai furnizează o valoare constantă, de regulă +1, înmulțită cu o pondere de *bias*[3]. Ponderile dintre straturi precum și aceste ponderi de *bias* sunt instruibile, adică se vor modifica prin procesul de învățare[4].

O reprezentare de rețea neurală cu trei straturi și o ieșire este dată în figura 6.2; o rețea cu 4 straturi și două ieșiri este reprezentată în figura 6.3. Nu există o relație anume între numărul de straturi ascunse, numărul de neuroni de pe aceste straturi și numărul de noduri de intrare și ieșire.

Vom considera că avem $L \geq 3$ straturi și în fiecare strat l ($1 \leq l \leq L-1$) un număr de n_l noduri. Stratul de intrare are $n_0 = n$ noduri, numărul de neuroni din stratul de ieșire este $n_{L-1} = m$ dat de: numărul de clase pentru care se face recunoașterea (la problemă de clasificare sau estimare de probabilitate condiționată) respectiv numărul de ieșiri care se doresc a fi aproximate (la regresie).

Pentru orice rețea MLP:

- valorile x_1, x_2, x_3 sunt componentele vectorului de intrare, aici considerat ca vector coloană, $\mathbf{x} = (x_1, x_2, x_3)^t$; se mai consideră încă o intrare cu valoarea constantă +1, care se va înmulți cu ponderea de *bias*;
- valoarea $a_i^{[l]}$ este ieșirea (activare) nodului i din stratul l, $0 \leq l \leq L-1$, $1 \leq i \leq n_l$. Pentru stratul de intrare activările sunt chiar intrările, *i.e.* $a_i^{[0]} = x_i$;
- valoarea $h_{\mathbf{W},\mathbf{b}}(\mathbf{x})$ este ieșirea calculată de către rețea pentru vectorul curent de intrare $\mathbf{x}$; $h_{\mathbf{W},\mathbf{b}}(\mathbf{x}) \in \mathbb{R}$ pentru figura 6.2; $h_{\mathbf{W},\mathbf{b}}(\mathbf{x}) \in \mathbb{R}^2$ pentru

[2]Motiv pentru care unii autori nu îl consideră un strat propriu–zis; frecvent se folosește exprimarea că o rețea are "k" straturi ascunse, cele de intrare și ieșire existând oricum. În cele ce urmează considerăm intrarea ca formând un strat.

[3]Unii autori consideră valoarea constantă -1 în loc de +1; nu e esențial, deoarece ponderile sunt determinate prin instruire și pot fi pozitive sau negative.

[4]O discuție asupra necesității considerării bias–ului este la https://web.archive.org/web/20210207195343/ftp://ftp.sas.com/pub/neural/FAQ2.html

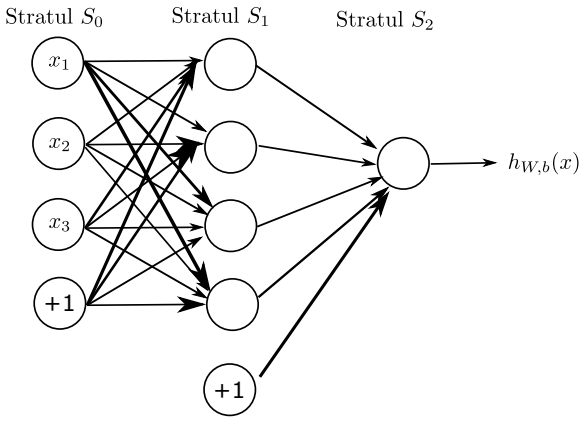


Figura 6.2: Rețea MLP cu 3 straturi. Poate fi folosită pentru estimarea unei valori continue (problemă de regresie) sau pentru discriminare/estimare de probabilitate condiționată pentru două clase.

figura 6.3. Rețeaua din figura 6.2 poate fi folosită pentru estimarea unei valori continue (problemă de regresie) sau pentru estimare de probabilitate condiționată pentru două clase. Rețeaua din figura 6.3 se poate folosi pentru estimarea a două valori de ieșire, numere reale – problemă de regresie.

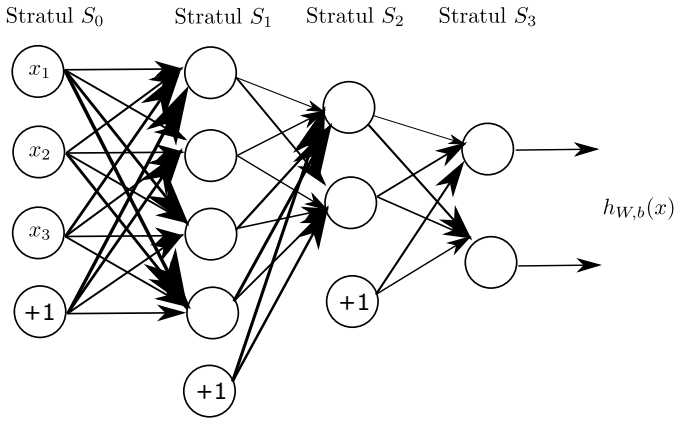
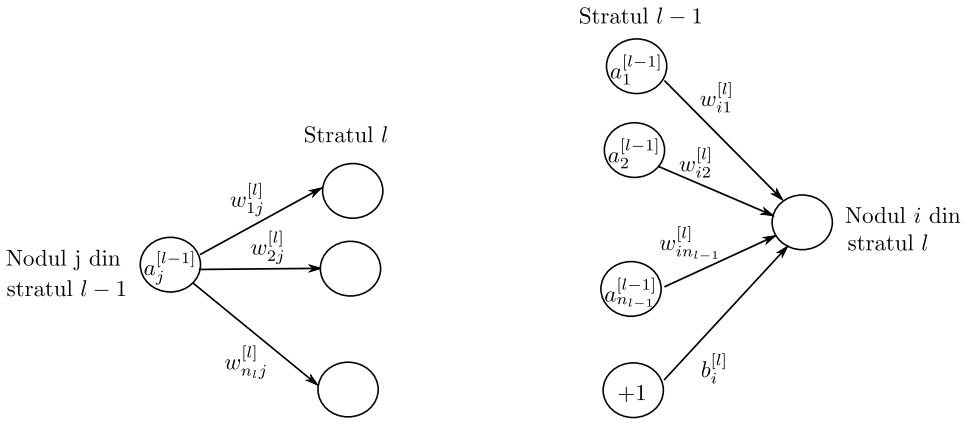


Figura 6.3: Rețea MLP cu 4 straturi ce se poate folosi pentru estimarea a două valori de ieșire.

Perechea $(\mathbf{W}, \mathbf{b})$ este formată din lista matricelor ponderilor și lista vectorilor coeficienților de bias din rețea. Folosim următoarele notații:

- ponderile dintre stratul de intrare și stratul ascuns sunt conținute în matricea $\mathbf{W}^{[1]}$: $w_{ij}^{[1]}$ este ponderea legăturii dintre neuronul i al stratului al doilea (de indice 1) și nodul j din stratul de intrare; se remarcă



(a) Ponderile dintre nodul j din stratul $l-1$ și nodurile din stratul l.

(b) Valoarea de bias și ponderile dintre stratul $l-1$ și nodul i din stratul l.

Figura 6.4: Legăturile între noduri din straturile $l-1$ și l.

ordinea indicilor inferiori, utilă mai departe pentru operațiile folosite;

- în general, notăm cu $w_{ij}^{[l]}$ ponderea legăturii dintre al i–lea neuron din stratul de indice l și al j–lea nod (neuron sau nod de intrare) din stratul $l-1$ ($1 \leq l \leq L-1$); a se vedea figurile 6.4a, 6.4b. Ele formează matricea de ponderi:

$$\mathbf{W}^{[l]} = \begin{pmatrix} w_{11}^{[l]} & \dots & w_{1n_{l-1}}^{[l]} \\ \vdots & \ddots & \vdots \\ w_{n_l 1}^{[l]} & \dots & w_{n_l n_{l-1}}^{[l]} \end{pmatrix} \tag{6.2}$$

  cu $n_l \times n_{l-1}$ ponderi;

- valoarea ponderii de bias dintre intrarea constantă $+1$ din stratul de intrare și neuronul i din primul strat ascuns este $b_i^{[1]}$, $1 \leq i \leq n_2$;

- în general, ponderea de bias provenind din stratul $l-1$ ($1 \leq l \leq L-1$) și pentru neuronul i din stratul l este notată cu $b_i^{[l]}$, $1 \leq i \leq n_l$, — figura 6.4b. Coeficienții $b_i^{[l]}$ formează vectorul coloană $\mathbf{b}^{[l]}$ cu n_l coeficienți.

Datorită faptului că avem legături între oricare doi neuroni aflati pe straturi succesive, rețelele MLP se mai numesc și *dens conectate* (eng: fully connected networks). Există alte tipuri de rețele (de exemplu cele de tip convoluțional, capitolul 9) care folosesc legături mai rare.

6.4.2 Funcții de activare

Fiecare neuron agregă valorile din nodurile din stratul anterior – incluzând și termenul constant +1 înmulțit cu coeficientul de bias. Neuronul de indice i din stratul $l \geq 1$ are starea calculată ca:

$$z_i^{[l]} = w_{i1}^{[l]} \cdot a_1^{[l-1]} + w_{i2}^{[l]} \cdot a_2^{[l-1]} + \cdots + w_{i,n_{l-1}}^{[l]} \cdot a_{n_{l-1}}^{[l-1]} + b_i^{[l]} \tag{6.3}$$

$$= \mathbf{W}_i^{[l]} \cdot \mathbf{a}^{[l-1]} + b_i^{[l]},\ 1 \leq i \leq n_l \tag{6.4}$$

unde: $\mathbf{W}_i^{[l]}$ este linia i a matricei $\mathbf{W}^{[l]}$, $a_i^{[l-1]}$ este, după caz: valoarea de ieșire a neuronului i din stratul $l-1$ (dacă $l > 1$) sau valoarea x_i din vectorul de intrare curent (dacă $l = 1$); evident, vectorul $\mathbf{a}^{[l]}$ este $(a_1^{[l]}, \ldots, a_{n_l}^{[l]})^t$. Notând cu $\mathbf{z}^{[l]}$ vectorul coloană $\left(z_1^{[l]}, z_2^{[l]}, \ldots, z_{n_l}^{[l]}\right)^t$, putem scrie matricial:

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]} \cdot \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]},\ 1 \leq l \leq L-1 \tag{6.5}$$

Pe baza stării $z_i^{[l]}$ a neuronului i din stratul l se calculează ieșirea (activarea) – sa folosind funcția de activare $f^{[l]}$:

$$a_i^{[l]} = f^{[l]}\left(z_i^{[l]}\right) \tag{6.6}$$

pentru $1 \leq l \leq L-1, 1 \leq i \leq n_l$. Dacă folosim notația $f^{[l]}\left((z_1, z_2, \ldots, z_k)^t\right) \stackrel{\text{def}}{=} \left(f^{[l]}(z_1), f^{[l]}(z_2), \ldots, f^{[l]}(z_k)\right)^t$ – adică se aplică funcția $f^{[l]}$ pe fiecare valoare din vectorul argument – atunci putem scrie mai compact ecuația (6.6) sub forma:

$$\mathbf{a}^{[l]} = f^{[l]}\left(\mathbf{z}^{[l]}\right) \tag{6.7}$$

Funcția de activare $f^{[l]}(\cdot)$ este necesară în pasul de propagare înainte; pentru pasul de propagare înapoi a erorii este folosită derivata ei. Alegeri populare pentru funcția de activare sunt:

1. **funcția logistică sigmoidă:**

$$f = \sigma : \mathbb{R} \to (0,1), f(z) = \sigma(z) = \frac{1}{1+\exp(-z)} \tag{6.8}$$

Derivata acestei funcții este:

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z)) \tag{6.9}$$

2. **funcția tangentă hiperbolică**:

$$f = \tanh : \mathbb{R} \to (-1,1),$$

$$f(z) = \tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)} \tag{6.10}$$

$$= \frac{2}{1+\exp(-2z)} - 1 \tag{6.11}$$

a cărei derivată este:

$$\tanh'(z) = 1 - \tanh^2(z) \tag{6.12}$$

Se arată ușor că între cele două funcții de activare tanh și σ există relația:

$$\tanh(z) = 2 \cdot \sigma(2z) - 1 \tag{6.13}$$

În practică funcția tanh dă rezultate mai bune decât sigmoida logistică. O explicație teoretică se găsește în [8]; rezultate empirice sunt în [10].

Atât sigmoida logistică, cât și tangenta hiperbolică au problema gradienților mici (eng. vanishing gradients): pentru acei z pentru care $\sigma(z) \approx 0$, $\sigma(z) \approx 1$, $\tanh(z) \approx -1$, $\tanh(z) \approx 1$ avem că derivatele lor sunt apropiate de zero. Întrucât derivatele sunt mai departe folosite prin înmulțire, modificările ponderilor vor fi nesemnificative și învățarea va stagna.

3. **funcția liniară:**

$$f(z) = a \cdot z + b \tag{6.14}$$

cu derivata $f'(z) = a$; frecvent se iau $a = 1$, $b = 0$, obținând funcție identitate, $f(z) = z$; funcțiile liniare se practică, re regulă, în ultimul strat. Este utilizată dacă se dorește ca la ieșire valorile să fie în afara intervalelor $(0, 1)$ și $(-1, 1)$, cum se întâmplă la funcțiile de activare de mai sus.

4. **funcția softmax:**

$$softmax(\mathbf{z}; i) = \frac{\exp(z_i)}{\sum_{j=1}^{m} \exp(z_j)} \tag{6.15}$$

unde i este indicele neuronului, iar m este numărul total de valori din vectorul $\mathbf{z}$. Funcția softmax a mai fost folosită la regresia logistică pentru cazul a mai mult de două clase; $\mathbf{z}$ este vector cu valori de stare a neuronilor, $\mathbf{z} = (z_1, \ldots, z_m)^t$.

Funcția softmax este utilă pentru a transforma un vector de valori oarecare în distribuție de probabilitate: se arată ușor că $softmax(\mathbf{z}; c) \in (0, 1)\ \forall c$ și $\sum_{c=1}^{m} softmax(\mathbf{z}; c) = 1$. De regulă, $softmax$ se folosește pentru stratul de ieșire și valorile furnizate se interpretează convenabil drept probabilitatea ca intrarea curentă să fie de clasă c, $1 \leq c \leq m$; clasificarea se face găsind acel indice $1 \leq c \leq m$ pentru care $softmax(\mathbf{z}; c)$ este maxim. Se utilizează în stratul de ieșire a rețelei neurale de clasificare sau estimare de probabilitate.

Derivatele parțiale ale funcției softmax sunt:

$$\frac{\partial softmax(\mathbf{z};i)}{\partial z_j} = \begin{cases} softmax(\mathbf{z};i) \cdot (1 - softmax(\mathbf{z};i)) & \text{dacă } i = j \\ -softmax(\mathbf{z};i) \cdot softmax(\mathbf{z};j) & \text{dacă } i \neq j \end{cases} \tag{6.16}$$

Putem folosi funcția delta al lui Kronecker: $\delta_{ij} = 1$ dacă $i = j$ și 0 altfel și rescriem (6.16) ca:

$$\frac{\partial softmax(\mathbf{z};i)}{\partial z_j} = softmax(\mathbf{z};i) \cdot (\delta_{ij} - softmax(\mathbf{z};j)) \tag{6.17}$$

5. **funcția Rectified Linear Unit (ReLU):**

$$ReLU(z) = \max(0, z) = \begin{cases} 0 & \text{dacă } z < 0 \\ z & \text{dacă } z \geq 0 \end{cases} \tag{6.18}$$

Derivatele pe subintervale sunt ușor de calculat și cu evaluare rapidă. În plus, spre deosebire de sigmoida logistică și de tangenta hiperbolică, ele nu saturează.

Reprezentarea grafică e dată în figura 6.5.

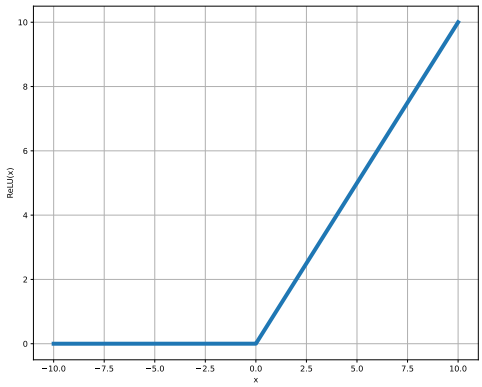


Figura 6.5: Graficul funcției de activare ReLU

Chiar dacă funcția este liniară pe porțiuni, ea este neliniară în ansamblu. Faptul că doar într–un punct nu e derivabilă nu deranjează în practică. Derivata funcției ReLU în origine se ia în mod convenabil[5]; se consideră

[5]Convine mai mult să considerăm derivata în 0 ca fiind 1: gradient egal cu 0 ar însemna că nu se face modificarea ponderilor. O discuție mai amplă și valori candidat pentru $f'(0)$ se găsește la https://sebastianraschka.com/faq/docs/relu-derivative.html

că în practică șansa de a fi nevoie să se calculeze derivata exact în origine este nesemnificativă. Avem deci:

$$ReLU'(z) = \begin{cases} 0 & \text{dacă } z < 0 \\ 1 & \text{dacă } z \geq 0 \end{cases} \tag{6.19}$$

Teoretic, există posibilitatea ca o parte din neuronii care folosesc ReLU să producă activare 0; asta se poate întâmpla, de exemplu, dacă valorile de bias sunt negative și foarte mici. E foarte probabil ca un astfel de neuron să devină insensibil la orice intrare și să producă mereu 0; spunem despre el că e un ReLU mort. Dacă toate activările sunt 0, rețeaua va produce o valoare constantă indiferent de intrare. Chiar și la învățare avem problemă, căci gradientul pentru un ReLU mort este 0, deci ponderile neuronului lui nu se modifică.

6. **funcția Parametric ReLU (PReLU)**, reprezentând o ușoară generalizare a funcției ReLU:

$$PReLU(z) = \begin{cases} \alpha z & \text{dacă } z < 0 \\ z & \text{dacă } z \geq 0 \end{cases} \tag{6.20}$$

cu $\alpha > 0$. Graficul funcției pentru $\alpha = 0.1$ este dat în figura 6.6. Derivatele pe subintervale sunt:

$$PReLU'(z) = \begin{cases} \alpha & \text{dacă } z < 0 \\ 1 & \text{dacă } z \geq 0 \end{cases} \tag{6.21}$$

Pentru $\alpha = 0.01$ se obține un caz particular vehiculat în literatură, Leaky ReLU.

PRelu rezolvă problema "dying ReLU", deoarece permite modificarea ponderilor, chiar dacă starea neuronului este negativă.

7. **funcția Exponential Linear Units (ELU)** este definită ca:

$$f(z) = \begin{cases} \alpha(e^z - 1) & \text{dacă } z < 0 \\ z & \text{dacă } z \geq 0 \end{cases} \tag{6.22}$$

cu $0 < \alpha \ll 1$. Graficul este dat în figura 6.7. Derivata lui ELU este:

$$f'(z) = \begin{cases} \alpha \cdot e^z & \text{dacă } z < 0 \\ 1 & \text{dacă } z \geq 0 \end{cases} \tag{6.23}$$

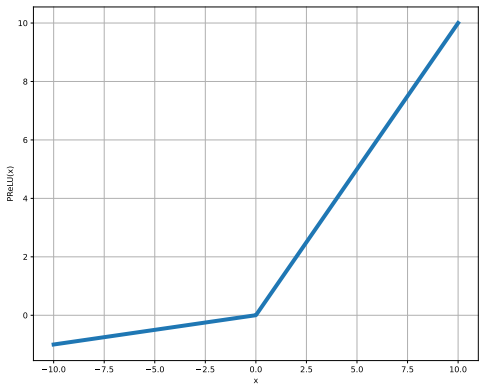


Figura 6.6: Graficul funcției de activare PReLU pentru $\alpha = 0.1$

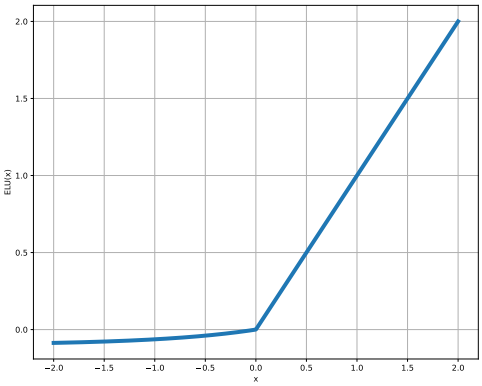


Figura 6.7: Graficul funcției de activare ELU pentru $\alpha = 0.1$

8. **funcția Swish**:

$$Swish(z) = z \cdot \sigma(\beta z) = \frac{z}{1 + \exp(-\beta z)} \tag{6.24}$$

unde σ este logistica sigmoidă, β este parametru constant sau antrenabil. Se arată[6] că pentru rețele cu multe straturi Swish funcționează mai bine decât ReLU. Reprezentarea ei este dată în figura 6.8. Derivata ei este:

$$Swish'(z) = \sigma(\beta z)(1 + \beta z - \beta z \sigma(\beta z))) \tag{6.25}$$

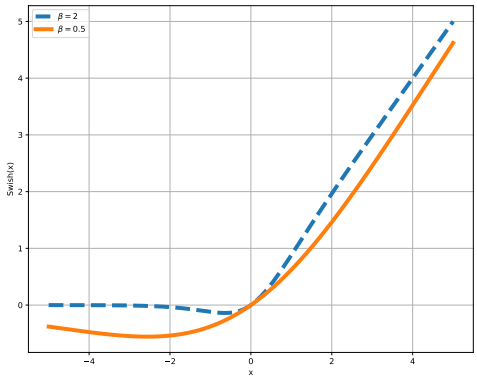


Figura 6.8: Graficul funcției de activare Swish pentru $\beta \in \{0.5, 2\}$

Lista funcțiilor de activare de mai sus e deschisă. Este admis ca funcția de activare să difere de la strat la strat sau de la neuron la neuron. În practică se preferă folosirea aceleiași funcții de activare în toată rețeaua, exceptând eventual ultimul strat.

6.5 Pasul de propagare înainte

Odată ce arhitectura rețelei e fixată – numărul de straturi ascunse și numărul de neuroni în fiecare strat, precum și funcțiile de activare – se poate trece la instruirea și apoi utilizarea ei. Pasul de propagare înainte preia un vector de intrare $\mathbf{x} = (x_1, \ldots, x_n)^t$ și produce modificări în starea neuronilor rețelei pornind de la intrare, acționând succesiv asupra straturilor $1, \ldots, L-1$. Aceasta dă și numele familiei din care face parte rețeaua: "cu propagare înainte" – eng. "feedforward". Ieșirile din ultimul strat sunt folosite pentru predicție – regresie, estimare de probabilitate condiționată sau clasificare.

[6] Prajit Ramachandran, Barret Zoph, Quoc V. Le, "Searching for Activation Functions", https://arxiv.org/abs/1710.05941

După cum s–a mai afirmat, stratul de intrare nu are rol computațional; valoare sa de ieșire este chiar vectorul de intrare – considerat ca vector coloană – $\mathbf{x}$ furnizat rețelei:

$$\mathbf{a}^{[0]} = \mathbf{x} \tag{6.26}$$

Dacă se cunosc valorile de ieșire (numite și activări) ale nodurilor din stratul $l-1$ se pot calcula stările neuronilor din stratul l și apoi activările lor, astfel:

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]} \cdot \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]} \tag{6.27}$$

$$\mathbf{a}^{[l]} = f^{[l]}(\mathbf{z}^{[l]}) \tag{6.28}$$

pentru $l = 1, \ldots L-1$, cu $f^{[l]}(\cdot)$ funcție de activare pentru stratul l. Vom nota cu $\hat{\mathbf{y}}$ vectorul de m valori de ieșire produs de către rețea:

$$\hat{\mathbf{y}} = \mathbf{a}^{[L-1]} \tag{6.29}$$

Pentru cazul în care se lucrează pe un set de date format din perechi vectori intrare–ieșire, vectorii coloană de date de intrare se pot concatena pe orizontală într–o matrice $\mathbf{X}$. De exemplu, pentru mulțimea $\left\{\left(\mathbf{x}^{(1)}, \mathbf{y}^{(1)}\right), \left(\mathbf{x}^{(2)}, \mathbf{y}^{(2)}\right), \ldots, \left(\mathbf{x}^{(k)}, \mathbf{y}^{(k)}\right)\right\}$, matricea $\mathbf{X}$ se formează ca:

$$\mathbf{X} = \begin{pmatrix} | & \cdots & | \\ \mathbf{x}^{(1)} & \cdots & \mathbf{x}^{(k)} \\ | & \cdots & | \end{pmatrix} \tag{6.30}$$

În cele ce urmează, se poate să avem $\mathbf{X}$ ca fiind matricea formată cu toate datele din setul de antrenare, sau doar cu un eșantion al acestora de k vectori (eng. mini-batch). În particular, acest minibatch poate fi format dintr-un singur vector de date, $\mathbf{x}^{(i)}$.

Propagarea înainte presupune că la intrare este furnizat un set de date $\mathbf{X}$ de k coloane. Intrarea dată este propagată înainte prin rețea și transformată succesiv:

$$\mathbf{Z}^{[1]} = \mathbf{W}^{[1]} \cdot \mathbf{X} + \mathbf{b}^{[1]} = \mathbf{W}^{[1]} \cdot \mathbf{A}^{[0]} + \mathbf{b}^{[1]} \tag{6.31}$$

$$\mathbf{A}^{[1]} = f^{[1]}\left(\mathbf{Z}^{[1]}\right) \tag{6.32}$$

$$\mathbf{Z}^{[2]} = \mathbf{W}^{[2]} \cdot \mathbf{A}^{[1]} + \mathbf{b}^{[2]} \tag{6.33}$$

$$\mathbf{A}^{[2]} = f^{[2]}\left(\mathbf{Z}^{[2]}\right) \tag{6.34}$$

$$\cdots$$

$$\mathbf{Z}^{[L-1]} = \mathbf{W}^{[L-1]} \cdot \mathbf{A}^{[L-2]} + \mathbf{b}^{[L-1]} \tag{6.35}$$

$$\mathbf{A}^{[L-1]} = f^{[L-1]}\left(\mathbf{Z}^{[L-1]}\right) \tag{6.36}$$

$\mathbf{Z}^{[l]}$ și $\mathbf{A}^{[l]}$ sunt matrice cu n_l linii și k coloane indentice. Pentru adunările din (6.31), (6.33), (6.35) se consideră că se aplică mecanismul de "broadcasting": vectorii coloană $\mathbf{b}^{[l]}$ produc prin copiere matrice cu k coloane.

Matricea $\mathbf{A}^{[L-1]}$ conține k coloane cu predicții, corespunzătoare celor k vectori de intrare din $\mathbf{X}$.

În practică, se recomandă ca operațiile date mai sus să fie implementate folosind biblioteci optimizate de algebră liniară, ce permit înmulțirea eficientă de matrice și calcul pe CPU sau GPU — Octave, Matlab, NumPy, PyTorch, Tensorflow, CuPy, JAX etc.

6.6 Funcții de cost

O pereche $(\mathbf{x} \in \mathbb{R}^n, \mathbf{y} \in \mathbb{R}^m)$ va produce o valoare de eroare astfel: se furnizează vectorul $\mathbf{x}$ ca intrare în rețea și se calculează un vector de ieșire $\hat{\mathbf{y}}$, reprezentând estimarea produsă de rețea pentru intrarea furnizată; se folosește o funcție de cost, sau de eroare, $J(\mathbf{W}, \mathbf{b}; \mathbf{x}, \mathbf{y})$ care se dorește a fi cu atât mai mică cu cât vectorul $\hat{\mathbf{y}}$ e mai apropiat de $\mathbf{y}$, și cu atât mai mare cu cât cei doi vectori sunt mai depărtați. În plus, se mai consideră un termen de regularizare care împiedică ponderile să devină prea mari în valoare absolută, caz asociat de regulă cu un comportament instabil al rețelei: variații mici ale intrării duc la salturi mari în straturile ascunse și la ieșire.

Să considerăm că avem un set de date pentru care dorim să calculăm funcția de eroare (de cost). Vectorii de intrare sunt organizați în batch ca în ecuația (6.30), unde pentru fiecare vector coloană $\mathbf{x}^{(i)}$ avem o etichetă $\mathbf{y}^{(i)}$ asociată.

Forma generală a funcției de eroare calculată pentru setul de date etichetate $\mathbf{X}$ cu k date etichetate este:

$$J(\mathbf{W}, \mathbf{b}) = \underbrace{\left[\frac{1}{k}\sum_{i=1}^{k} \overbrace{J\left(\mathbf{W}, \mathbf{b}; \mathbf{x}^{(i)}, \mathbf{y}^{(i)}\right)}^{\text{eroare empirică pentru}\left(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\right)}\right]}_{\text{Eroarea empirică medie pe setul } \mathbf{X}} + \underbrace{\frac{\lambda}{2}\sum_{l=1}^{L-1}\sum_{i=1}^{n_l}\sum_{j=1}^{n_{l-1}}\left(w_{ij}^{[l]}\right)^2}_{\text{Termen de regularizare } J_{reg}} \tag{6.37}$$

unde $\lambda > 0$ este coeficientul de regularizare. Termenul de regularizare este aici bazat pe norma L_2, o sumă de pătrate de norme Frobenius ale matricelor $\mathbf{W}^{[1]}, \ldots, \mathbf{W}^{[L-1]}$:

$$\left\|\mathbf{W}^{[l]}\right\|_F^2 \overset{\text{def}}{=} \sum_{i=1}^{n_l}\sum_{j=1}^{n_{l-1}}\left(w_{ij}^{[l]}\right)^2 \tag{6.38}$$

Regularizarea impune o presiune asupra ponderilor – ele sunt direcționate spre 0. O subliniere importantă este că ponderile de bias nu sunt regularizate;

de aceea vectorii $b^{[1]}, \ldots, b^{[L-1]}$ nu apar în termenul de regularizare; la fel s-a întâmplat și la modelele studiate anterior.

Cu norma Frobenius, termenul de regularizare J_{reg} din (6.37) se scrie succint:

$$J_{reg}(\mathbf{W}) = \frac{\lambda}{2} \sum_{l=1}^{L-1} \left\| \mathbf{W}^{[l]} \right\|_F^2 \tag{6.39}$$

6.6.1 Funcția de cost pentru problemă de regresie

În cazul unei probleme de regresie, cea mai utilizată funcție de eroare $J(\mathbf{W}, \mathbf{b}; \mathbf{x}, \mathbf{y})$ ce măsoară calitatea predicției $\hat{\mathbf{y}}$ produse pentru perechea $(\mathbf{x}, \mathbf{y})$ este jumătate din eroarea L_2 pătratică:

$$J(\mathbf{W}, \mathbf{b}; \mathbf{x}, \mathbf{y}) = \frac{1}{2} \cdot \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2 \tag{6.40}$$

unde $\hat{\mathbf{y}} \in \mathbb{R}^m$ este vectorul de ieșire din rețeaua neurală corespunzând intrării $\mathbf{x}$, iar $\|\mathbf{v}\|_2$ este norma L_2 (Euclidiană) a vectorului $\mathbf{v} = (v_1, \ldots, v_q)^t$:

$$\|\mathbf{v}\|_2 = \sqrt{\sum_{i=1}^{q} v_i^2}$$

În acest caz funcția de eroare pentru un set de date $\mathbf{X}$ cu k perechi $\left(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\right)$ și incluzând termen de regularizare devine:

$$J(\mathbf{W}, \mathbf{b}) = \underbrace{\left[\frac{1}{2k} \sum_{i=1}^{k} \left\| \mathbf{y}^{(i)} - \hat{\mathbf{y}}^{(i)} \right\|_2^2 \right]}_{\text{Eroarea empirică medie pe setul } \mathbf{X}} + \underbrace{\frac{\lambda}{2} \sum_{l=1}^{L-1} \left\| \mathbf{W}^{[l]} \right\|_F^2}_{\text{Termen de regularizare}} \tag{6.41}$$

Termenul $\frac{1}{k} \sum_{i=1}^{k} \left\| \mathbf{y}^{(i)} - \hat{\mathbf{y}}^{(i)} \right\|_2^2$ se numește eroare pătratică medie (mean squared error, MSE). Coeficientul $\frac{1}{2}$ din (6.40), respectiv din eroarea empirică medie (6.41) este folosit aici din motive estetice[7], și atunci eroarea empirică medie e chiar eroarea pătratică medie.

Deși larg folosită pentru probleme de regresie, la o analiză mai atentă se observă ca dă mai multă importanță erorilor mai mari. De exemplu, dă aceeași importanță unei norme de diferență $\|\mathbf{y} - \hat{\mathbf{y}}\|_2 = 10$ ca la 100 de cazuri pentru care $\|\mathbf{y} - \hat{\mathbf{y}}\|_2 = 1$. Se tinde deci a se preocupa de cazuri ce produc erori mari, dar puține, în detrimentul unor cazuri frecvente cu erori mai mici. Funcția de eroare este dominată de valorile de eroare extreme.

[7] La derivare, numitorul 2 se reduce cu coeficientul 2 obținut din derivarea formei pătratice. Coeficientul 1/2 este, totuși, opțional.

O altă funcție de eroare empirică frecvent folosită este eroarea absolută medie (mean absolute error, MAE; eroarea L_1), definită ca (excludem pentru moment termenul de regularizare):

$$J\left(\mathbf{W},\mathbf{b};\mathbf{x},\mathbf{y}\right)=\frac{1}{k}\|\mathbf{y}-\hat{\mathbf{y}}\|_1=\frac{1}{k}\sum_{i=1}^{k}|y_i-\hat{y}_i| \tag{6.42}$$

MAE are avantajul că nu e atât de afectată de erorile mari comparativ cu MSE; pentru același exemplu: o eroare de 10 este echivalentă cu doar 10 erori de 1. Dezavantajul ei este că nu e derivabilă peste tot.

O variantă care combină diferențiabilitatea lui MSE și senzitivitate redusă la erori extreme a lui MAE este funcția de eroare a lui Huber, exprimată în continuare pentru perechea $(\mathbf{y},\hat{\mathbf{y}})$:

$$HL(\mathbf{y},\hat{\mathbf{y}})=\begin{cases}\frac{1}{2}\left\|\mathbf{y}-\hat{\mathbf{y}}\right\|_2^2 & \text{dacă } \left\|\mathbf{y}-\hat{\mathbf{y}}\right\|_2\leq\delta\\ \delta\left(\left\|\mathbf{y}-\hat{\mathbf{y}}\right\|_1-\frac{\delta}{2}\right) & \text{dacă } \left\|\mathbf{y}-\hat{\mathbf{y}}\right\|_2>\delta\end{cases} \tag{6.43}$$

unde $\delta>0$ este un hiper–parametru care stabilește zona de trecere de la o funcție la alta; valoarea adecvată a lui δ poate fi determinată prin încercări repetate. Indiferent de valoarea lui δ, funcția de eroare a lui Huber este diferențiabilă.

Reprezentările MAE, MSE si Huber pentru o diferență $\left\|\mathbf{y}-\hat{\mathbf{y}}\right\|_2$ fixată sunt date în figura 6.9.

6.6.2 Funcția de cost pentru discriminarea a două clase

Dacă problema este de discriminare a două clase, atunci stratul de ieșire conține un singur neuron, cu funcția de activare sigmoidă logistică – idee preluată de la regresia logistică binară. Funcția de eroare pentru o singură pereche $(\mathbf{x},y)$ este[8]:

$$J(\mathbf{W},\mathbf{b};\mathbf{x},y)=-y\ln\hat{y}-(1-y)\ln\left(1-\hat{y}\right) \tag{6.44}$$

deci funcția de eroare medie pentru setul de date $\mathbf{X}$, incluzând termenul de regularizare va fi:

$$\begin{aligned}J(\mathbf{W},\mathbf{b}) &= \underbrace{-\frac{1}{k}\sum_{i=1}^{k}\left[y^{(i)}\ln\hat{y}^{(i)}+\left(1-y^{(i)}\right)\ln\left(1-\hat{y}^{(i)}\right)\right]}_{\text{Eroarea empirică medie pe setul }\mathbf{X}}+\\ &+ \underbrace{\frac{\lambda}{2}\sum_{l=1}^{L-1}\left\|\mathbf{W}^{[l]}\right\|_F^2}_{\text{Termen de regularizare}}\end{aligned} \tag{6.45}$$

[8]Reamintim că pentru probleme de clasificare binară eticheta de clasă este $y\in\{0,1\}$. Ieșirea $\hat{y}$ este un număr in intervalul $(0,1)$.

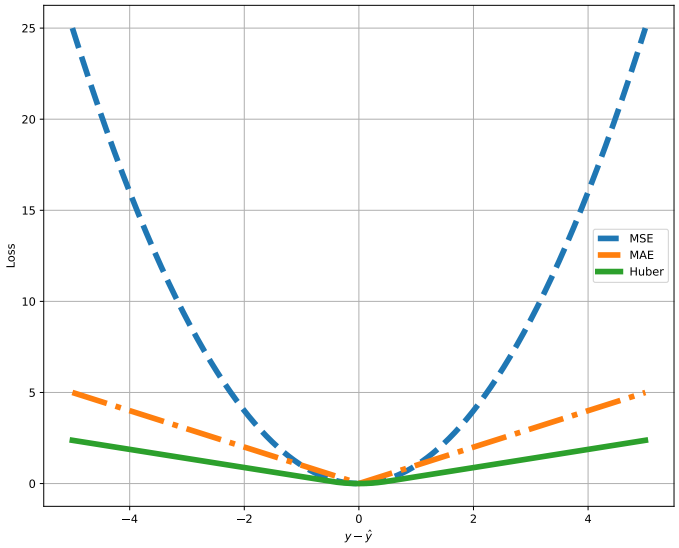


Figura 6.9: Functiile mean squared error, mean absolute error, Huber ($\delta = 0.5$) pentru cazul unidimensional

Pentru forma vectorizată a erorii empirice medii (prima sumă din ecuația de mai sus) a se vedea ecuația (4.15), pagina 81.

6.6.3 Funcția de cost pentru mai mult de două clase independente

Pentru anumite probleme se poate cere să se determine dacă intrarea curentă conține sau face parte din niște clase independente. De exemplu, pentru o imagine se cere să se determine dacă conține obiecte de clasă 1, clasă 2, ..., clasă m. Se va crea o rețea neurală cu m neuroni de ieșire, neuronul k dând estimarea ca intrarea să conțină element/să fie de clasă k.

Funcția de eroare pentru o singură pereche $(\mathbf{x}, \mathbf{y})$ este:

$$J(\mathbf{W}, \mathbf{b}; \mathbf{x}, \mathbf{y}) = -\sum_{j=1}^{m} [y_j \ln \hat{y}_j + (1 - y_j) \ln (1 - \hat{y}_j)] \tag{6.46}$$

unde vectorul $\mathbf{y} = (y_1, y_2, \ldots, y_m)^t$ conține valori 0 și 1. Funcția de eroare totală $J(\mathbf{W}, \mathbf{b})$ calculată pentru setul de date $\mathbf{X}$ și incluzând termenul de

regularizare devine:

$$J(\mathbf{W},\mathbf{b}) = \underbrace{-\frac{1}{k}\sum_{i=1}^{k}\sum_{j=1}^{m}\left[y_j^{(i)}\ln\hat{y}_j^{(i)}+\left(1-y_j^{(i)}\right)\ln\left(1-\hat{y}_j^{(i)}\right)\right]}_{\text{Eroarea empirică medie pe setul } \mathbf{X}} + \\ + \underbrace{\frac{\lambda}{2}\sum_{l=1}^{L-1}\left\|\mathbf{W}^{[l]}\right\|_F^2}_{\text{Termen de regularizare}} \tag{6.47}$$

6.6.4 Funcția de cost pentru clasificare cu mai mult de două clase

Pentru probleme de clasificare se preferă utilizarea funcției de eroare cross–entropy iar în stratul de ieșire funcția de activare să fie softmax. Funcția de eroare pentru o singură pereche $(\mathbf{x},\mathbf{y})$ este:

$$J(\mathbf{W},\mathbf{b};\mathbf{x},\mathbf{y}) = -\sum_{j=1}^{m} y_j \log\hat{y}_j \tag{6.48}$$

unde folosim, ca mai sus, codificarea one–hot. Eroarea pentru setul de date $\mathbf{X}$ și cu termen de regularizare este:

$$J(\mathbf{W},\mathbf{b}) = \underbrace{-\frac{1}{k}\sum_{i=1}^{k}\sum_{j=1}^{m} y_j^{(i)}\log\hat{y}_j^{(i)}}_{\text{Eroarea empirică medie pe setul } \mathbf{X}} + \underbrace{\frac{\lambda}{2}\sum_{l=1}^{L-1}\left\|\mathbf{W}^{[l]}\right\|_F^2}_{\text{Termen de regularizare}} \tag{6.49}$$

Pentru forma vectorizată a erorii empirice se poate vedea ecuația (3.47), pagina 64.

6.7 Inițializarea ponderilor rețelei

Valorile inițiale ale ponderilor $\mathbf{W}$ și $\mathbf{b}$ sunt setate aleator, în jurul lui zero. Este necesar ca valorile ponderilor să nu fie toate egale; dacă ar fi toate egale, fiecare neuron dintr-un strat ar avea exact aceeași stare ca și ceilalți din stratul lui, deoarece fiecare neuron e legat la aceleași intrări ca și ceilați din stratul lui; mai departe, ponderile cu care se înmulțesc intrările fiind egale, valoarea de activare a fiecărui neuron de pe acel strat e aceeași; argumentul e valabil începând cu primul strat ascuns. Am ajunge deci ca neuronii de pe același strat să calculeze exact aceleași valori, ceea ce e redundant și inutil, iar pentru stratul de ieșire s–ar prezice valori egale pe toți neuronii de ieșire. Efectul de simetrie obținut cu ponderi egale în $\mathbf{W}$ se elimină prin inițializare cu numere aleatoare. În ce privește ponderile de bias – din $\mathbf{b}$ – ele se pot

inițializa cu 0; inițializarea aleatoare a ponderilor $\mathbf{W}$ este suficientă pentru "spargerea simetriei".

Strategii rafinate de inițializare pentru ponderile sunt cele propuse de Xavier Glorot *et al.*[9] și He *et al.*[10].

Pentru arhitecturile de tip deep learning se preferă o preantrenare nesupervizată a ponderilor [8], [11] sau preluarea unor ponderi care au fost antrenate pentru un set de date similar cu cel curent – transfer de învățare (transfer learning).

6.8 Grafuri computaționale și calcul de derivate parțiale

6.8.1 Derivarea funcțiilor compuse

Pentru algoritmul backpropagation avem nevoie de calcul de derivate parțiale de funcții compuse.

Reamintim definiția derivatei unei funcții $f : \mathbb{R} \to \mathbb{R}$:

$$f'(x) = \lim_{\delta \to 0} \frac{f(x+\delta) - f(x)}{\delta}$$

ceea ce ne permite ca pentru δ valori mici (în jurul lui 0) să folosim aproximarea:

$$f(x+\delta) \approx f(x) + \delta f'(x)$$

Aproximarea permite sa ne dăm seama că o modificare cu δ a lui x face ca valoarea lui $f(x)$ să se modifice cu $\delta f'(x)$.

Reamintim formula de derivare a funcțiilor compuse: dacă avem $f : B \to C$, $g : A \to B$ și notăm $h(x) = f(g(x))$, $h : A \to C$, atunci derivata lui h în raport cu x se calculează ca[11]:

$$\frac{\partial h}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x} \tag{6.50}$$

Pentru funcții multivariate $f : \mathbb{R}^m \to \mathbb{R}$, $g_i, h : \mathbb{R}^n \to \mathbb{R}$ de forma[12]:

$$h(x_1, \ldots, x_n) = f\left(g_1(x_1, \ldots, x_n), \ldots, g_m(x_1, \ldots, x_n)\right) \tag{6.51}$$

[9] Glorot, Xavier and Yoshua Bengio. "Understanding the difficulty of training deep feedforward neural networks." International Conference on Artificial Intelligence and Statistics (2010).

[10] K. He, X. Zhang, S. Ren and J. Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," 2015 IEEE International Conference on Computer Vision (ICCV), 2015, pp. 1026-1034, doi: 10.1109/ICCV.2015.123.

[11] Chiar și pentru funcții de o singură variabilă vom folosi notația de derivare ∂, pentru omogenitatea notațiilor.

[12] Aici, m și n din definițiile f, g_i, h nu au legătură cu dimensiunile vectorilor din setul de antrenare.

derivatele parțiale se calculează astfel:

$$\frac{\partial h}{\partial x_i} = \frac{\partial f}{\partial g_1} \cdot \frac{\partial g_1}{\partial x_i} + \frac{\partial f}{\partial g_2} \cdot \frac{\partial g_2}{\partial x_i} + \cdots + \frac{\partial f}{\partial g_m} \cdot \frac{\partial g_m}{\partial x_i} = \sum_{j=1}^{m} \frac{\partial f}{\partial g_j} \cdot \frac{\partial g_j}{\partial x_i} \tag{6.52}$$

În mod clar, formula (6.52) generalizează (6.50).

Exemplificăm (6.51), (6.52): plecăm de la $g_1, g_2 : \mathbb{R} \to \mathbb{R}$, $f : \mathbb{R}^2 \to \mathbb{R}$ diferențiabile. Pentru simplitate, notăm cu h funcția definită prin $h(x) = f(g_1(x), g_2(x))$. Graful computațional este dat în figura 6.10.

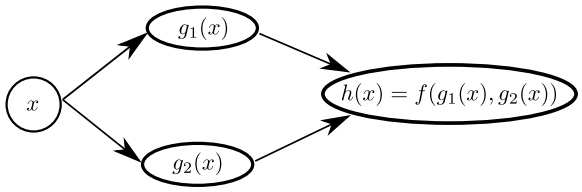


Figura 6.10: Graf computațional pentru funcție multivariată și compusă. Variabila x influențează funcția h prin două căi, exemplificând necesitatea folosirii formulei (6.52).

Vrem să calculăm $\partial h/\partial x$. Vom da o motivare intuitivă a egalității

$$\frac{\partial h}{\partial x} = \frac{\partial f}{\partial g_1} \cdot \frac{\partial g_1}{\partial x} + \frac{\partial f}{\partial g_2} \cdot \frac{\partial g_2}{\partial x} \tag{6.53}$$

care e un caz particular al (6.52).

Folosind definția derivatei, avem următoarele aproximări pentru δ apropiat de 0:

$$g_1(x + \delta) \approx g_1(x) + \delta \frac{\partial g_1}{\partial x}(x) \tag{6.54}$$

$$g_2(x + \delta) \approx g_2(x) + \delta \frac{\partial g_2}{\partial x}(x) \tag{6.55}$$

iar pentru δ_1, δ_2 apropiați de 0:

$$\begin{aligned} f(g_1(x) + \delta_1, g_2(x) + \delta_2) &\approx f(g_1(x), g_2(x)) + && (6.56)\\ &+ \delta_1 \frac{\partial f}{\partial g_1}(g_1(x), g_2(x)) + && (6.57)\\ &+ \delta_2 \frac{\partial f}{\partial g_2}(g_1(x), g_2(x)) && (6.58) \end{aligned}$$

Avem deci că

$$\begin{aligned} h(x + \delta) &= f(g_1(x + \delta), g_2(x + \delta)) && (6.59)\\ &\approx f\left(g_1(x) + \delta \frac{\partial g_1}{\partial x}(x), g_2(x) + \delta \frac{\partial g_2}{\partial x}(x)\right) && (6.60) \end{aligned}$$

și folosind (6.56)–(6.58) obținem:

$$h(x+\delta) \approx h(x) \quad + \quad \delta \frac{\partial g_1}{\partial x}(x) \frac{\partial f}{\partial g_1}(g_1(x), g_2(x)) \\ + \quad \delta \frac{\partial g_2}{\partial x}(x) \frac{\partial f}{\partial g_2}(g_1(x), g_2(x)) \tag{6.61}$$

ce se transcrie ca

$$\begin{aligned} \frac{\partial h}{\partial x}(x) &\approx \frac{h(x+\delta) - h(x)}{\delta} \\ &\approx \frac{\partial g_1}{\partial x}(x) \cdot \frac{\partial f}{\partial g_1}(g_1(x), g_2(x)) + \\ &+ \frac{\partial g_2}{\partial x}(x) \cdot \frac{\partial f}{\partial g_2}(g_1(x), g_2(x)) \end{aligned} \tag{6.62}$$

ceea ce e similar cu (6.53). Există, desigur, o demonstrație riguroasă pentru (6.52).

Formula (6.52) este utilă pentru cazul în care de la o variabilă – x în cazul nostru – la rezultat – $h(x)$ – există mai mult de o cale în graful computațional – via g_1 și g_2 în cazul dat. Pentru o rețea neurală cu măcar un strat ascuns, orice pondere $w_{ij}^{[k]}$ de dinainte de ultimul strat ascuns intervine în calculul ieșirii rețelei – și deci al funcției de eroare – prin intermediul tuturor căilor de la neuronul i din stratul k la stratul de ieșire. Pentru un exemplu, a se vedea secțiunea 6.8.3.

6.8.2 Exemplul 1: căi unice

Exemplificăm și figurăm în cele ce urmează calculul de derivate parțiale, așa cum o efecturază metoda backpropagation pentru funcția $f : \mathbb{R}^3 \to \mathbb{R}$, $f(x, y, z) = x \cdot (y + z)$. Notăm cu q funcția $q(y, z) = y + z$, deci $f(x, y, z) = x \cdot q(y, z)$. Graful computațional pentru calculul valorii lui f este în figura 6.11. Dorim să calculăm derivatele parțiale ale lui f în raport cu x, y, z.

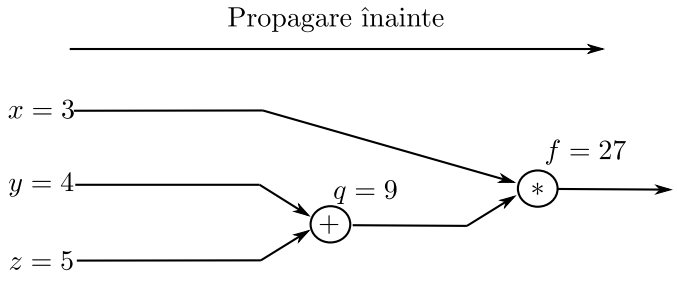


Figura 6.11: Graful computațional pentru calculul valorii funcției $f(x, y, z) = x \cdot (y + z) = x \cdot q(y, z)$ pentru $x = 3, y = 4, z = 5$. Valorile $q(y, z)$ și $f(x, y, z)$ se calculează de la stânga spre dreapta, prin propagare înainte.

Fixăm $x = 3, y = 4, z = 5$. Calculul valorilor q, f este dat în graful computațional din figura 6.11. Valorile lui q și f se calculează de la stânga spre dreapta, prin propagare înainte, în sensul dat de săgețile negre.

Derivatele parțiale se calculează de la dreapta spre stânga (propagare inversă, eng. backropagation) și este calculul este arătat în figura 6.12. .

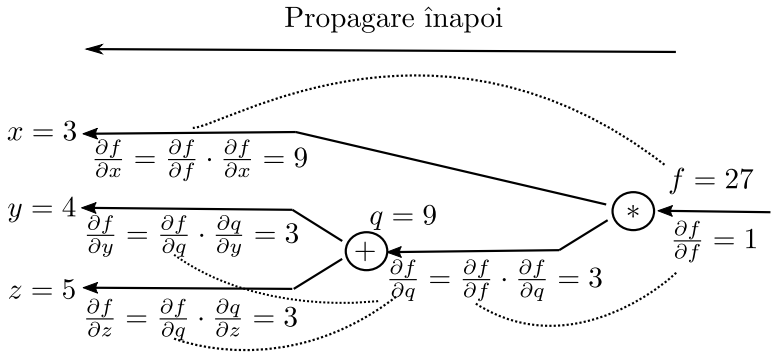


Figura 6.12: Graful computațional pentru calculul derivatelor parțiale $\partial f/\partial x$, $\partial f/\partial y$, $\partial f/\partial h$, pentru $x = 3, y = 4, z = 5$. Derivatele parțiale se calculează de la dreapta spre stânga. Utilizarea repetată a derivatelor parțiale "din amonte" este arătată prin linie punctată.

Astfel, calculăm începând de la ieșirea grafului computațional către variabilele de intrare, în ordine:

1. Un calcul imediat, dar util pentru a asigura omogenitatea implementării calculelor ce urmează:
$$\frac{\partial f}{\partial f} = 1 \tag{6.63}$$

2. Calculăm derivata parțială $\partial f/\partial q$, făcând înmulțind derivata "din amonte" $\partial f/\partial f$ cu derivata locală $\partial f/\partial q$. Derivatele locale sunt derivatele funcției reprezentate de nodul curent în raport cu fiecare din intrările sale. Una din derivatele locale este:
$$\frac{\partial f}{\partial q} = \frac{\partial (x \cdot q)}{\partial q} = x = 3 \tag{6.64}$$
Avem deci
$$\frac{\partial f}{\partial q} = \frac{\partial f}{\partial f} \cdot \frac{\partial f}{\partial q} = 1 \cdot 3 = 3 \tag{6.65}$$

3. Pentru calculul derivatei parțiale $\partial f/\partial y$, folosind regula derivării funcțiilor compuse, avem: $\partial f/\partial y = \partial f/\partial q \cdot \partial q/\partial y$, deci păstrăm principiul: se înmulțește derivata parțială din amonte – și deja calculată la pasul anterior, $\partial f/\partial q$ – cu derivata locală $\partial q/\partial y$:
$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial y} = 3 \cdot \frac{\partial (y + z)}{\partial y} = 3 \cdot 1 = 3 \tag{6.66}$$

4. Similar, pentru calculul derivatei parțiale $\partial f/\partial z$ avem:

$$\frac{\partial f}{\partial z} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial z} = 3 \cdot 1 = 3 \tag{6.67}$$

5. Pentru calculul lui $\partial f/\partial x$ avem:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial f} \cdot \frac{\partial f}{\partial x} = 1 \cdot \frac{\partial (x \cdot q)}{\partial x} = q = 9 \tag{6.68}$$

Sumarizând, calculăm derivatele parțiale de la ieșire către intrare și folosim derivatele calculate până la acel pas pentru a determina derivatele aflate din ce în ce mai aproape de intrarea în graful computațional. Principiul din figura 6.12 stă la baza unor algoritmi de calcul automat al gradienților – bibliotecile de tip *autograd*.

6.8.3 Exemplul 2: căi multiple

Considerăm funcția $f : \mathbb{R}^3 \to \mathbb{R}$, $f(x, y, z) = (x + y) \cdot (y + z)$. Notăm expresiile intermediare: $p(x, y) = x + y$, $q(y, z) = y + z$ și deci $f(x, y, z) = p(x, y) \cdot q(y, z)$. Fixăm $x = -1$, $y = 3$ și $z = 5$. Graful computațional e dat în figura 6.13.

Se observă că y contribuie la rezultatul final prin două căi - via p și q, deci vom folosi rezultatul din ecuația (6.52).

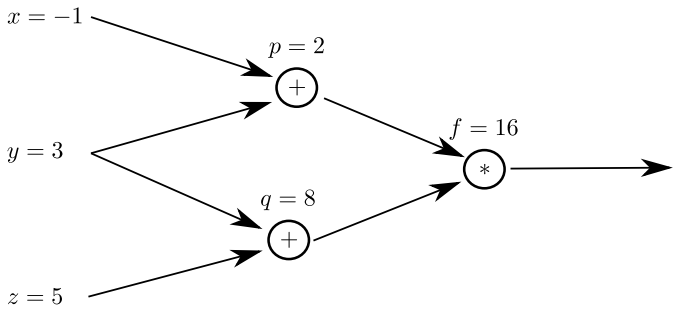


Figura 6.13: Graf computațional pentru $f(x, y, z) = (x + y) \cdot (y + z)$, $x = -1$, $y = 3$, $z = 5$. Se fac calcule intermediare dec către nodurile p și q.

Vrem să calculăm derivatele parțiale $\partial f/\partial x$, $\partial f/\partial y$, $\partial f/\partial z$. Calculăm aici derivatele locale, urmând a le folosi mai departe:

$$\frac{\partial f}{\partial p} = \frac{\partial (p \cdot q)}{\partial p} = q = 8 \qquad \frac{\partial f}{\partial q} = \frac{\partial (p \cdot q)}{\partial q} = p = 2 \qquad \frac{\partial p}{\partial x} = \frac{\partial (x + y)}{\partial x} = 1$$

$$\frac{\partial p}{\partial y} = \frac{\partial (x + y)}{\partial y} = 1 \qquad \frac{\partial q}{\partial y} = \frac{\partial (y + z)}{\partial y} = 1 \qquad \frac{\partial q}{\partial z} = \frac{\partial (y + z)}{\partial z} = 1$$

Gradienții lui f se calculează de la dreapta spre stânga. Se începe cu evidentul $\partial f/\partial f = 1$, iar restul de calcule e dat în figura 6.14. Derivata parțială $\partial f/\partial y$ este deosebită de celelalte: pentru că y dă valoarea ei atât lui p cât și lui q, ea primește "din amonte" gradienți, via nodurile p și q, pe care îi însumează. Însumarea e necesară conform formulei (6.52):

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial p} \cdot \frac{\partial p}{\partial y} + \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial y} = 8 + 2 = 10 \tag{6.69}$$

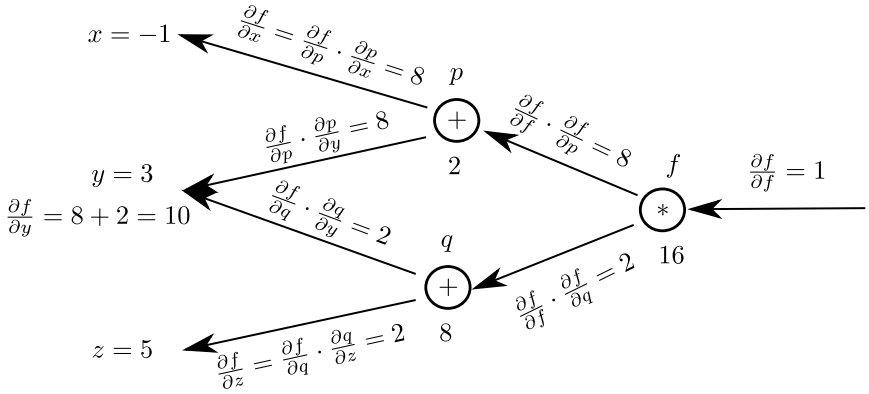


Figura 6.14: Calculul gradienților pentru $f(x, y, z) = (x + y) \cdot (y + z)$ folosind gradienții "din amonte" și gradienții locali. Variabila y influențează valoarea funcției f prin două căi.

6.9 Algoritmul backpropagation

6.9.1 Scopul algoritmului

Revenim la rețelele multistrat. Se dorește modificarea ponderilor din matricele $\mathbf{W}^{[l]}$ și a coeficienților de bias $\mathbf{b}^{[l]}$ astfel încât valoarea funcției de eroare $J(\mathbf{W}, \mathbf{b})$ să scadă; numim modificările ponderilor învățare. *Algoritmul backpropagation calculează valorile gradienților* $\partial J/\partial w_{ij}^{[l]}$, $\partial J/\partial b_i^{[l]}$. Se va folosi algoritmul de căutare după direcția gradientului (gradient descent)[13], în care modificarea unei ponderi $w_{ij}^{[l]}$ se efectuează astfel:

$$w_{ij}^{[l]} = w_{ij}^{[l]} - \alpha \frac{\partial J}{\partial w_{ij}^{[l]}}(\mathbf{W}, \mathbf{b}) \tag{6.70}$$

Ponderile de bias $b_i^{[l]}$ se modifică similar:

$$b_i^{[l]} = b_i^{[l]} - \alpha \frac{\partial J}{\partial b_i^{[l]}}(\mathbf{W}, \mathbf{b}) \tag{6.71}$$

[13]Sau orice algoritm care se poate folosi de gradienții ponderilor.

deci este esențială calcularea gradienților $\frac{\partial J}{\partial w_{ij}^{[l]}}(\mathbf{W}, \mathbf{b})$, $\frac{\partial J}{\partial b_i^{[l]}}(\mathbf{W}, \mathbf{b})$. Ca și până acum, $\alpha > 0$ este rata de învățare.

6.9.2 Variante de calcul

Avem trei variante de calcul pentru modificarea ponderilor, folosind algoritm de tip gradient descent (căutare după direcția gradienților):

1. **stochastic gradient descent:** pentru o singură pereche din setul de antrenare $\left(\mathbf{x}^{(k)}, \mathbf{y}^{(k)}\right)$ se calculează valoarea erorii $J\left(\mathbf{W}, \mathbf{b}; \mathbf{x}^{(k)}, \mathbf{y}^{(k)}\right)$, se calculează gradienții și se aplică modificările pentru toate ponderile $w_{ij}^{[l]}$ și $b_i^{[l]}$; următoarea pereche de instruire folosește valorile de ponderi modificate la acest pas;

2. **off–line** sau **batch gradient descent**: se calculează gradienții pentru ponderile $w_{ij}^{[l]}$ și bias–urile $b_i^{[l]}$ pentru fiecare pereche de vectori din tot setul de antrenare; apo se calculează media lor și se actualizează fiecare pondere $w_{ij}^{[l]}$ și $b_i^{[l]}$, scăzându–se din ea media înmulțită cu rata de învățare α.

3. **minibatch gradient descent:** se împarte setul de antrenare $\mathscr{S}$ în subseturi disjuncte (mini–batches), de exemplu câte 100 de perechi $\left(\mathbf{x}^{(k)}, \mathbf{y}^{(k)}\right)$. Pentru fiecare minibatch se calculează media gradienților; se modifică toate ponderile $w_{ij}^{[l]}$ și $b_i^{[l]}$ folosind media înmulțită cu rata de învățare, apoi se trece la următorul minibatch. În practică este cea mai folosită strategie. Este o variantă intermediară între stochastic gradient descent – unde modificarea se face imediat după fiecare pereche din $\mathscr{S}$ – și cea batch – în care modificarea ponderilor se face doar după ce se face propagarea înainte pe tot setul $\mathscr{S}$.

În toate cazurile de mai sus: o trecere completă peste setul de antrenare se numește epocă, iar trecerea peste un subset – fie el și dintr-o singură pereche de instruire – se numește iterație. Se execută mai multe epoci de instruire. Condiția de oprire a învățării poate fi:

- numărul de epoci parcurse este egal cu un număr dat a priori, de exemplu 200;

- se urmărește valoarea funcției de eroare peste un set de validare, un set disjunct față de setul de antrenare $\mathscr{S}$ (secțiunea 4.5). Dacă se constată că eroarea pe setul de validare începe să crească în timp ce eroarea pe setul de antrenare continuă să scadă, atunci se oprește instruirea – figura 6.15;

- se urmărește evoluția normelor gradienților: dacă suma acestor norme e aproape de zero, înseamnă că s–a ajuns într-un punct de minim (local

sau global) sau într–un punct de inflexiune și ponderile nu vor mai fi modificate semnificativ;

- valoarea funcției de eroare scade sub un anumit prag; etc.

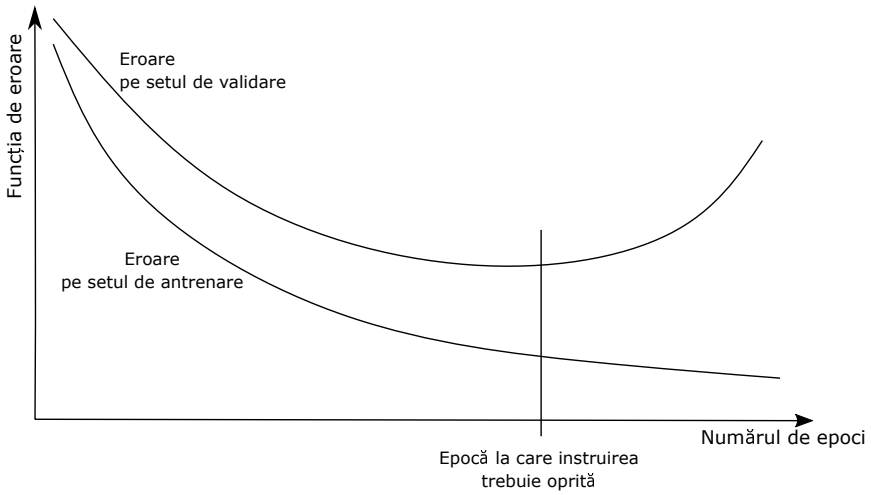


Figura 6.15: Exemplu de evoluție a valorilor funcției de eroare pentru setul de antrenare, respectiv cel de validare. Dacă se continuă antrenarea, eroare pe setul de antrenare scade, dar pentru setul de testare începe să crească de la o anumită epocă. Se recomandă oprirea instruirii dacă eroarea pe setul de validare începe să crească.

6.9.3 Calculul gradienților – justificare matematică

Vom prezenta varianta de instruire *batch*, întrucât poate fi ușor adaptată la minibatch sau stochastic gradient descent. Trecerea la cazul în care se face antrenarea pe minibatch-uri este imediată: media gradienților pentru toți cei p termeni din setul de antrenare se substituie cu media gradienților calculați pe datele din acel minibatch. Evident, pentru stochastic gradient descent, media este chiar gradientul calculat pentru exemplarul de instruire curent.

Reamintim forma funcției de eroare pentru un set de date $\mathbf{X}$ de p perechi

$\left(\mathbf{x}^{(k)}, \mathbf{y}^{(k)}\right)$:

$$J(\mathbf{W}, \mathbf{b}) = \underbrace{\left[\frac{1}{p}\sum_{k=1}^{p} \overbrace{J\left(\mathbf{W}, \mathbf{b}; \mathbf{x}^{(k)}, \mathbf{y}^{(k)}\right)}^{\text{eroare empirică pentru}\left(\mathbf{x}^{(k)}, \mathbf{y}^{(k)}\right)}\right]}_{\text{Eroarea empirică medie pe setul } \mathbf{X}} + \underbrace{\frac{\lambda}{2}\sum_{l=1}^{L-1}\sum_{i=1}^{n_l}\sum_{j=1}^{n_{l-1}}\left(w_{ij}^{[l]}\right)^2}_{\text{Termen de regularizare}} \tag{6.72}$$

Ne folosim de faptul că funcția de eroare este o sumă de termeni și derivata unei sume de funcții este suma derivatelor funcțiilor[14] și rezultă:

$$\frac{\partial J}{\partial w_{ij}^{[l]}}(\mathbf{W}, \mathbf{b}) = \frac{1}{p}\sum_{k=1}^{p}\frac{\partial J}{\partial w_{ij}^{[l]}}\left(\mathbf{W}, \mathbf{b}; \mathbf{x}^{(k)}, \mathbf{y}^{(k)}\right) + \lambda w_{ij}^{[l]} \tag{6.73}$$

respectiv pentru ponderile de bias:

$$\frac{\partial J}{\partial b_{i}^{[l]}}(\mathbf{W}, \mathbf{b}) = \frac{1}{p}\sum_{k=1}^{p}\frac{\partial J}{\partial b_{i}^{[l]}}\left(\mathbf{W}, \mathbf{b}; \mathbf{x}^{(k)}, \mathbf{y}^{(k)}\right) \tag{6.74}$$

În (6.73) am inclus derivatele parțiale pentru termenul de regularizare (6.39) în raport cu $w_{ij}^{[l]}$, al căror calcul e imediat:

$$\frac{\partial}{\partial w_{ij}^{[l]}}\left(\frac{\lambda}{2}\cdot\sum_{l=1}^{L-1}\left\|\mathbf{W}^{[l]}\right\|_F^2\right) = \frac{\lambda}{2}\frac{\partial}{\partial w_{ij}^{[l]}}\sum_{k=1}^{L-1}\sum_{a=1}^{n_l}\sum_{b=1}^{n_{l-1}}\left(w_{ab}^{[k]}\right)^2 = \lambda\cdot w_{ij}^{[l]} \tag{6.75}$$

iar pentru derivata parțială a termenului de regularizare în raport cu $b_i^{[l]}$ reamintim că termenul de regularizare nu conține ponderile de bias, deci gradientul său în raport cu $b_i^{[2]}$ este 0.

Rămâne deci să calculăm derivatele parțiale pentru o singură pereche de instruire $(\mathbf{x}, \mathbf{y})$, adică $\frac{\partial J}{\partial w_{ij}^{[l]}}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \mathbf{y})$ și $\frac{\partial J}{\partial b_{i}^{[l]}}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \mathbf{y})$. Algoritmul backpropagation arată care e ordinea de calcul a derivatelor parțiale, bazat pe teoria din secțiunea 6.8. Odată derivatele parțiale calculate, ele pot fi folosite pentru modificarea ponderilor, adică pentru instruirea rețelei.

Algoritmul a fost popularizat de articolul "Learning representations by back-propagating errors"[15].

[14] Sau pe scurt: derivarea este operator liniar.

[15] Rumelhart, D., Hinton, G. & Williams, R., *Learning representations by back-propagating errors*, Nature 323, 533–536 (1986). https://doi.org/10.1038/323533a0

Arătăm modul de calcul pentru derivatele parțiale $\partial J/\partial w_{ij}^{[k]}$ și $\partial J/\partial b_i^{[k]}$, cu exemplificare pe o rețea neurală pentru regresie și cu un singur strat ascuns, a se vedea figura 6.16. Fiind problemă de regresie, funcția de eroare empirică (de calitate) e jumătate din mean squared error, ecuația (6.40) pagina 125. Pentru problemă de clasificare se adaptează funcția de activare $f^{[2]}$ a ultimului strat. Vom explicita doar gradienții termenului de eroare de calitate, deoarece am văzut că gradienții termenului de regularizare L_2 se calculează simplu. Prezentăm calculul pentru o singură pereche de instruire $(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^n \times \mathbb{R}^m$.

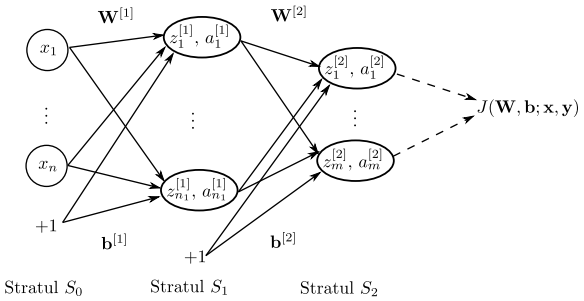


Figura 6.16: Rețea neurală cu un strat ascuns, folosită ca schemă pentru determinarea fomulelor gradienților.

Gradienții pentru stratul de ieșire

Pentru ponderile instruibile ale ultimului strat, $w_{ij}^{[2]}$ și $b_i^{[2]}$ unde $1 \leq i \leq m$, avem graful computațional din figura 6.17.

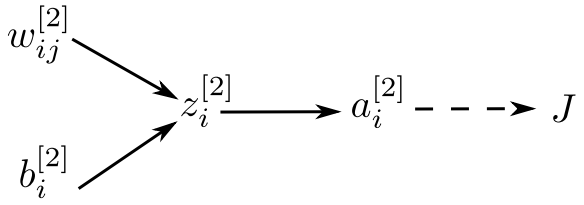


Figura 6.17: Graf computațional pentru ponderile și valorile de bias din ultimul strat.

Deoarece J nu depinde direct de $w_{ij}^{[2]}$, ci via $z_i^{[2]}$, folosim formula de derivare de funcții compuse (6.50) și obținem:

$$\frac{\partial J}{\partial w_{ij}^{[2]}} = \frac{\partial J}{\partial z_i^{[2]}} \cdot \frac{\partial z_i^{[2]}}{\partial w_{ij}^{[2]}} \tag{6.76}$$

Pentru a doua derivată parțială ținem cont că:

$$z_i^{[2]} = b_i^{[2]} + \sum_{k=1}^{n_1} w_{ik}^{[2]} a_k^{[1]} \tag{6.77}$$

de unde

$$\frac{\partial z_i^{[2]}}{\partial w_{ij}^{[2]}} = a_j^{[1]} \tag{6.78}$$

și deci

$$\frac{\partial J}{\partial w_{ij}^{[2]}} = \frac{\partial J}{\partial z_i^{[2]}} \cdot a_j^{[1]} \tag{6.79}$$

Similar, pentru derivata parțială a lui J în raport cu $b_i^{[2]}$:

$$\frac{\partial J}{\partial b_i^{[2]}} = \frac{\partial J}{\partial z_i^{[2]}} \cdot \frac{\partial z_i^{[2]}}{\partial b_i^{[2]}} = \frac{\partial J}{\partial z_i^{[2]}} \cdot \frac{\partial \left(b_i^{[2]} + \sum\limits_{k=1}^{n} w_{ik}^{[2]} a_{ik}^{[1]} \right)}{\partial b_i^{[2]}} = \frac{\partial J}{\partial z_i^{[2]}} \tag{6.80}$$

Observăm că avem termenul $\partial J / \partial z_i^{[2]}$ în ambele derivate parțiale; îl notăm cu $\delta_i^{[2]}$ și îl calculăm. Ținem cont că J depinde de $z_i^{[2]}$ doar prin intermediul lui $a_i^{[2]} = f^{[2]}\left(z_i^{[2]}\right)$ și folosim din nou formula de derivare de funcții compuse:

$$\begin{aligned}
\delta_i^{[2]} &= \frac{\partial J}{\partial z_i^{[2]}} = \frac{\partial J}{\partial a_i^{[2]}} \cdot \frac{\partial a_i^{[2]}}{\partial z_i^{[2]}} && (6.81)\\
&= \left(\frac{\partial}{\partial a_i^{[2]}} \frac{1}{2} \sum_{k=1}^{m} (a_k^{[2]} - y_k)^2 \right) \cdot \frac{\partial f^{[2]}}{\partial z_i^{[2]}} \left(z_i^{[2]} \right) && (6.82)\\
&= \left(a_i^{[2]} - y_i \right) \cdot f^{[2]\prime} \left(z_i^{[2]} \right) && (6.83)
\end{aligned}$$

de unde, pentru tot vectorul $\boldsymbol{\delta}^{[2]}$:

$$\boldsymbol{\delta}^{[2]} = \left(\mathbf{a}^{[2]} - \mathbf{y} \right) \odot f^{[2]\prime} \left(\mathbf{z}^{[2]} \right) \tag{6.84}$$

cu $\odot$ reprezentând produsul Hadamard a doi vectori cu același număr de elemente:

$$(a_1, \dots a_p)^t \odot (b_1, \dots b_p)^t = (a_1 \cdot b_1, \dots a_p \cdot b_p)^t$$

Componentele vectorului $\boldsymbol{\delta}^{[2]}$ sunt folosite pentru calculul derivatelor parțiale de ponderi instruibile, conform (6.79) și (6.80).

Gradienții pentru stratul ascuns

Pentru calcul de derivate parțiale de ponderi și valori de bias ale stratului ascuns, $\partial J/\partial w_{ij}^{[1]}$ și respectiv $\partial J/\partial b_i^{[1]}$ ținem cont că $w_{ij}^{[1]}$ și $b_i^{[1]}$ contribuie la funcția de eroare prin mai multe căi, conform grafului computațional din figura 6.18; indicii sunt $1 \leq i \leq n_1$, $1 \leq j \leq n$.

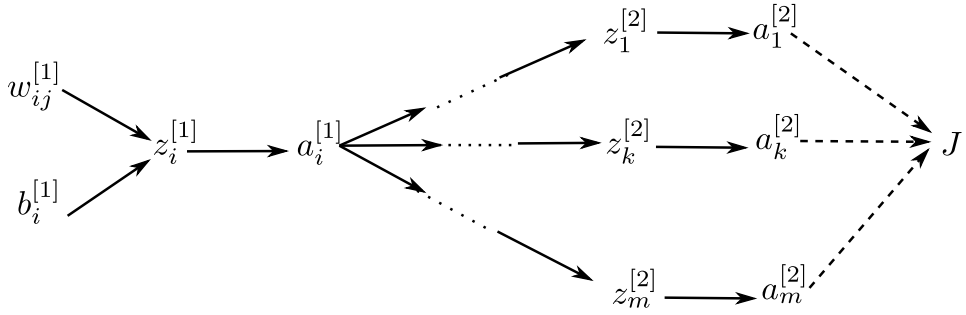


Figura 6.18: Graf computațional pentru neuronul i din stratul ascuns. Ponderile instruibile $w_{ij}^{[1]}$ și $b_i^{[1]}$ contribuie la valoarea funcției de eroare prin mai multe căi, via $z_1^{[2]}, \ldots, z_m^{[2]}$. Situația e similară cu cea din figura 6.10.

Ca atare, pentru $\partial J/\partial w_{ij}^{[1]}$ vom utiliza formula generală de derivare a funcțiilor compuse (6.52).

Avem:

$$\frac{\partial J}{\partial w_{ij}^{[1]}} = \frac{\partial J}{\partial z_i^{[1]}} \cdot \frac{\partial z_i^{[1]}}{\partial w_{ij}^{[1]}} \tag{6.85}$$

Explicităm cele două derivate parțiale de sub suma de mai sus. Deoarece

$$z_i^{[1]} = b_i^{[1]} + \sum_{k=1}^{n} w_{ik}^{[1]} a_k^{[0]} \tag{6.86}$$

obținem imediat

$$\frac{\partial z_i^{[1]}}{\partial w_{ij}^{[1]}} = a_j^{[0]} \tag{6.87}$$

și înlocuind înapoi în (6.85):

$$\frac{\partial J}{\partial w_{ij}^{[1]}} = \frac{\partial J}{\partial z_i^{[1]}} \cdot a_j^{[0]} \tag{6.88}$$

Similar, pentru derivata parțială în raport cu termenul de bias:

$$\frac{\partial J}{\partial b_i^{[1]}} = \frac{\partial J}{\partial z_i^{[1]}} \cdot \frac{\partial \left(b_i^{[1]} + \sum\limits_{k=1}^{n} w_{ik}^{[1]} a_k^{[0]} \right)}{\partial b_i^{[1]}} = \frac{\partial J}{\partial z_i^{[1]}} \tag{6.89}$$

Cantitatea $\frac{\partial J}{\partial z_i^{[1]}}$ apare atât în (6.89) cât și în (6.88); o vom nota cu $\delta_i^{[1]}$ și cele două formule devin:

$$\frac{\partial J}{\partial w_{ij}^{[1]}} = \delta_i^{[1]} \cdot a_j^{[0]} \tag{6.90}$$

$$\frac{\partial J}{\partial b_i^{[1]}} = \delta_i^{[1]} \tag{6.91}$$

Rămâne deci să calculăm $\delta_i^{[1]} = \frac{\partial J}{\partial z_i^{[1]}}$. Deoarece $z_i^{[1]}$ influențează funcția J prin intermediul variabilelor $z_1^{[2]}, \ldots, z_m^{[2]}$ – vezi figura 6.18 – folosim formula (6.52) și obținem:

$$\delta_i^{[1]} = \frac{\partial J}{\partial z_i^{[1]}} = \sum_{j=1}^{m} \frac{\partial J}{\partial z_j^{[2]}} \cdot \frac{\partial z_j^{[2]}}{\partial z_i^{[1]}} \tag{6.92}$$

Dar $\frac{\partial J}{\partial z_j^{[2]}}$ este chiar $\delta_j^{[2]}$, calculat anterior în (6.84). Explicităm $\frac{\partial z_j^{[2]}}{\partial z_i^{[1]}}$ pentru $1 \leq i \leq n_1$:

$$\begin{aligned} \frac{\partial z_j^{[2]}}{\partial z_i^{[1]}} &= \frac{\partial}{\partial z_i^{[1]}} \left(\sum_{l=1}^{n_1} w_{jl}^{[2]} \cdot f^{[1]} \left(z_l^{[1]} \right) + b_j^{[2]} \right) \\ &= w_{ji}^{[2]} \cdot f^{[1]\prime} \left(z_i^{[1]} \right) \end{aligned} \tag{6.93}$$

Înlocuind în (6.92) obținem:

$$\delta_i^{[1]} = \sum_{j=1}^{m} \left(\delta_j^{[2]} \cdot w_{ji}^{[2]} \cdot f^{[1]\prime} \left(z_i^{[1]} \right) \right) = \left(\sum_{j=1}^{m} \delta_j^{[2]} \cdot w_{ji}^{[2]} \right) \cdot f^{[1]\prime} \left(z_i^{[1]} \right) \tag{6.94}$$

sau pentru tot vectorul $\boldsymbol{\delta}^{[1]}$:

$$\boldsymbol{\delta}^{[1]} = \left(\mathbf{W}^{[2]t} \cdot \boldsymbol{\delta}^{[2]} \right) \odot f^{[1]\prime} \left(z^{[1]} \right) \tag{6.95}$$

Valorile din vectorul $\boldsymbol{\delta}^{[1]}$ sunt folosite pentru calculul de derivatelor parțiale din ecuațiile (6.90) și (6.91). Avem deci valorile derivatelor parțiale ale funcției de eroare în raport cu ponderile și valorile de bias intre stratul de intrare și stratul ascuns.

Extinderea la o rețea MLP cu mai multe straturi ascunse este ușor de făcut, fiind similară cu ceea ce s–a arătat mai sus.

6.9.4 Algoritmul de instruire a rețelei MLP

Algoritmul de instruire a rețelei, folosind gradienții determinați prin backropagation, lucrând pe un lot de p valori este:

1. Inițializează $\Delta\mathbf{W}^{[l]}$, $\Delta\mathbf{b}^{[l]}$ cu 0, pentru $l = 1, \ldots, L-1$:

$$\Delta\mathbf{W}^{[l]} = \mathbf{0}_{n_l \times n_{l-1}} \tag{6.96}$$

$$\Delta\mathbf{b}^{[l]} = \mathbf{0}_{n_l}, \text{vector coloană} \tag{6.97}$$

2. Pentru fiecare din cele p perechi $(\mathbf{x}, \mathbf{y})$ din batch calculează corecția pentru ponderi și ponderile de bias[16]:

   2.1. Efectuează pasul de propagare înainte, conform secțiunii 6.5, și obține ieșirea estimată $\hat{\mathbf{y}}$;

   2.2. Pentru fiecare strat $l = L-1, \ldots, 1$ se calculează semnalul de eroare: la ultimul strat semnalul de eroare este

$$\boldsymbol{\delta}^{[L-1]} = (\mathbf{a}^{[L-1]} - \mathbf{y}) \odot f^{[L-1]'}\left(\mathbf{z}^{[L-1]}\right) \tag{6.98}$$

   iar la restul de straturi

$$\boldsymbol{\delta}^{[l]} = \left[\left(\mathbf{W}^{[l+1]}\right)^t \cdot \boldsymbol{\delta}^{[l+1]}\right] \odot f^{[l]'}\left(\mathbf{z}^{[l]}\right) \tag{6.99}$$

   Am presupus mai sus că derivatele funcțiilor de activare se calculează punctual peste vectorii de stări.
   Calculăm derivatele parțiale pentru ponderi și pentru bias:

$$\frac{\partial J}{\partial \mathbf{W}^{[l]}}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \mathbf{y}) = \boldsymbol{\delta}^{[l]} \cdot \left(\mathbf{a}^{[l-1]}\right)^t \tag{6.100}$$

$$\frac{\partial J}{\partial \mathbf{b}^{[l]}}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \mathbf{y}) = \boldsymbol{\delta}^{[l]} \tag{6.101}$$

   2.3. Acumulează modificările de ponderi[17], pentru $l = L-1, \ldots, 1$:

$$\Delta\mathbf{W}^{[l]} = \Delta\mathbf{W}^{[l]} + \frac{\partial J}{\partial \mathbf{W}^{[l]}}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \mathbf{y}) \tag{6.102}$$

$$= \Delta\mathbf{W}^{[l]} + \boldsymbol{\delta}^{[l]} \cdot \left(\mathbf{a}^{[l-1]}\right)^t \tag{6.103}$$

$$\Delta\mathbf{b}^{[l]} = \Delta\mathbf{b}^{[l]} + \frac{\partial J}{\partial \mathbf{b}^{[l]}}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \mathbf{y}) \tag{6.104}$$

$$= \Delta\mathbf{b}^{[l]} + \boldsymbol{\delta}^{[l]} \tag{6.105}$$

[16] Respectiv: se iterează peste datele din minibatch. Alternativ, se poate face calcul în paralel peste datele din minibatchul curent.

[17] Nu am inclus aici partea datorată termenului de regularizare. Evident, acumularea se poate face imediat după calculul derivatelor parțiale de pe stratul curent.

3. După ce toate perechile de instruire din batch au fost considerate, modifică valorile ponderilor și coeficienții de bias, pentru $l = 1, \ldots L-1$:

$$\mathbf{W}^{[l]} = \mathbf{W}^{[l]} - \alpha \left[\left(\frac{1}{p} \Delta \mathbf{W}^{[l]} \right) + \lambda \mathbf{W}^{[l]} \right] \tag{6.106}$$

$$\mathbf{b}^{[l]} = \mathbf{b}^{[l]} - \alpha \left(\frac{1}{p} \Delta \mathbf{b}^{[l]} \right) \tag{6.107}$$

4. Ciclare: se repetă de la pasul 1 până când se îndeplinește una din condițiile de oprire de la pagina 135.

Pentru cazul în care se folosește clasificare și funcția de activare din ultimul strat este softmax, iar funcția de eroare este cross-entropy, se arată că pentru perechea $(\mathbf{x}, \mathbf{y})$:

$$\boldsymbol{\delta}^{[L-1]} = \mathbf{a}^{[L-1]} - \mathbf{y} \tag{6.108}$$

substituind formula de calcul (6.98). Utilizarea funcției de eroare cross entropy duce la o viteză mai mare de învățare pentru probleme de clasificare decât dacă se folosește eroarea pătratică [10].

6.10 Utilizarea rețelei pentru inferență

După ce se face antrenarea rețelei, ea se poate folosi pentru a face predicții (inferențe) pentru date din setul de testare $\mathscr{T} = \left\{ \mathbf{x}^{(j)} | 1 \leq j \leq q \right\}$. Fiecare vector din $\mathscr{T}$ este trecut prin rețea, conform pasului de propagare înainte și se obțin valori de ieșire estimate (predicții) $\hat{\mathbf{y}}^{(1)}, \ldots, \hat{\mathbf{y}}^{(q)}$, toate din $\mathbb{R}^m$.

Dacă valorile de ieșire sunt văzute ca probabilități condiționate, adică:

$$\hat{y}_i = P\left(\text{clasa } i | \mathbf{x}\right), 1 \leq i \leq m \tag{6.109}$$

atunci clasificarea se face găsind acel indice i pentru care $\hat{y}_i$ e maxim și acesta este indicele clasei prezise de rețeaua MLP.

6.11 Discuții

- problema minimelor locale și a punctelor de inflexiune
- arhitectura rețelei: număr de straturi ascunse, număr de neuroni pe strat
- dependența de ordinea de prezentare a datelor în etapa de învățare
- alți algoritmi de optimizare decât vanilla gradient descent
- learning rate scheduling
- capacitatea de aproximare universală

Capitolul 7

Rețele neurale cu funcții de activare radială

Rețelele neurale cu funcții de activare radială (eng: radial basis function networks, RBF networks) se folosesc pentru probleme de regresie, estimare de probabilitate condiționată și regresie și clasificare. Instruirea este mixtă: nesupervizată – grupare, eng. clustering – pentru determinarea prototipurilor din stratul ascuns și supervizată pentru învățarea ponderilor dintre stratul ascuns și cel de ieșire.

7.1 Motivația rețelei: teorema lui Cover

Pentru cazul vectorilor ce nu pot fi separați liniar, perceptronul multistrat poate determina o suprafață de separare. Există și o altă variantă de rezolva problema discernerii între clase ce nu se pot separa liniar, *folosind un separator liniar*, ce lucrează în doi pași:

1. mulțimea de antrenare dată în spațiul originar este transformată într–un alt spațiu, în care, în anumite condiții, liniar separabilitatea poate apărea cu probabilitate mare; fundamentul matematic este dat de teorema lui Cover (a se vedea mai jos);

2. prin utilizarea unui model de separare liniară (perceptron liniar, SVM liniar, regresie logistică etc.) se separă clasele în cel de-al doilea spațiu.

Intuitiv, situația este prezentată în figura 7.1. În partea stângă avem două clase care nu sunt separabile liniar; punctele sunt cu coordonate în două dimensiuni. Printr–o funcție convenabil aleasă se trece într–un spațiu cu trei dimensiuni în care un plan poate separa clasa punctelor roșii de cele albastre.

Rezultatul se poate materializa printr–o rețea cu funcții de activare radială, formată din 3 straturi:

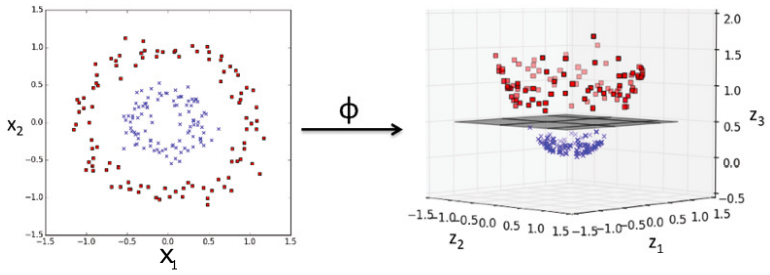


Figura 7.1: Transformarea unui set de date cu clase neseparabile liniar, într-un spațiu cu două dimensiuni, într-o mulțime cu două clase liniar separabile în spațiu cu 3 dimensiuni.

- strat de intrare care preia valorile pentru care se face inferență;
- strat de neuroni ascunși ce aplică transformări neliniare pe datele din spațiul de intrare. Neuronii din acest strat sunt antrenați prin instruire nesupervizată;
- strat de ieșire ce produce o transformare liniară, iar ponderile dintre stratul ascuns și stratul de ieșire sunt obținute prin instruire supervizată. Acest ultim strat furnizează valoarea inferată pentru vectorul de intrare curent.

Următoarea teoremă arată motivul pentru care se face o transformare a datelor originare în alte date dintr-un spațiu cu mai multe dimensiuni decât cel originar:

Teorema 2 (Cover[1], 1965)**.** *Printr–o transformare neliniară a unui set de date de intrare dintr-un spațiu A într-un spațiu B cu dimensiune mai mare, crește probabilitatea de a obține clase liniar separabile, dacă A nu este dens populat.*

Rezultatul este util, deoarece pentru cazuri liniar separabile, un perceptron discret poate să obțină un hiperplan de separare în timp finit; se pot folosi si alte modele de clasificatori liniari - regresie logistică, Support Vector Machines etc. Pentru a se obține o asemenea transformare, se pleacă de la spațiul A cu n dimensiuni în care se află vectorii $\mathbf{x}^{(1)}$, ..., $\mathbf{x}^{(p)}$ și se ajunge la un spațiu m dimensional ($m \geq n$) prin funcția:

$$\mathbf{x} \xrightarrow{\phi} \boldsymbol{\phi}(\mathbf{x}) = (\varphi_1(\mathbf{x}), \ldots, \varphi_m(\mathbf{x}))^t \in \mathbb{R}^m \tag{7.1}$$

unde $\varphi_i : \mathbb{R}^n \to \mathbb{R}, i = \overline{1,m}$ sunt funcții neliniare; în rețeaua neurală RBF funcția φ_i e calculată de neuronul i din stratul ascuns.

[1]Thomas M. Cover, "Geometrical and Statistical Properties of Systems of Linear Inequalities with Applications in Pattern Recognition, IEEE Transactions on Electronic Computers, vol. EC-14, no. 3, pp. 326-334, June 1965, doi: 10.1109/PGEC.1965.264137.

Vector de intrare	$\varphi_1(\mathbf{x}^{(i)})$	$\varphi_2(\mathbf{x}^{(i)})$
$\mathbf{x}^{(1)} = (1,1)^t$	1	0.1353
$\mathbf{x}^{(2)} = (0,0)^t$	0.1353	1
$\mathbf{x}^{(3)} = (0,1)^t$	0.3678	0.3678
$\mathbf{x}^{(4)} = (1,0)^t$	0.3678	0.3678

Tabelul 7.1: Valorile aproximative ale funcțiilor φ_i pentru punctele problemei XOR.

Exemplu: considerăm problema XOR, în care 4 puncte sunt asignate la două clase, astfel: vectorii $\mathbf{x}^{(1)} = (1,1)^t$ și $\mathbf{x}^{(2)} = (0,0)^t$ aparțin unei clase, iar $\mathbf{x}^{(3)} = (0,1)^t$ și $\mathbf{x}^{(4)} = (1,0)^t$ aparțin celeilalte clase. S–a arătat în secțiunea 5.9 că nu există o dreaptă în plan care să separe cele două clase de puncte. Considerăm acum funcțiile $\varphi_1, \varphi_2 : \mathbb{R}^2 \to \mathbb{R}$:

$$\varphi_1(\mathbf{x}) = \exp\left(-\left\|\mathbf{x} - \mathbf{x}^{(1)}\right\|\right)$$

$$\varphi_2(\mathbf{x}) = \exp\left(-\left\|\mathbf{x} - \mathbf{x}^{(2)}\right\|\right)$$

unde $\mathbf{x} \in \mathbb{R}^2$ iar $\|\cdot\|$ este norma Euclidiană în $\mathbb{R}^2$. Pornind de la un vector de intrare $\mathbf{x} \in \mathbb{R}^2$ se ajunge la un vector tot din $\mathbb{R}^2$ dat de $(\varphi_1(\mathbf{x}), \varphi_2(\mathbf{x}))$. Valorile rezultate pentru funcțiile $\varphi_{1,2}$ calculate în cele 4 puncte ale problemei XOR sunt date în tabelul 7.1. Figura 7.2 dă reprezentarea punctelor tranformate prin aplicarea celor două funcții. Se observă că problema devine una liniar separabilă[2], folosind modificări neliniare ale datelor inițiale; mai mult, nu a fost nevoie în acest caz să se mărească dimensiunea spațiului de ieșire.

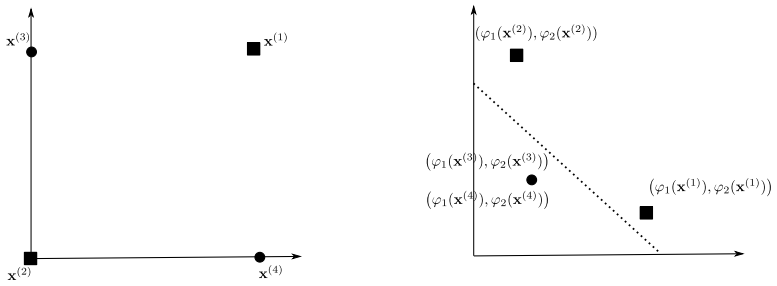


Figura 7.2: Transformarea problemei XOR, ce nu permite separare liniară (partea stângă) în problemă liniar separabilă

[2]Invităm cititorul să determine niște valori adecvate pentru coeficienții dreptei punctate din 7.2.

7.2 Funcții cu activare radială

Teorema lui Cover afirmă că pentru o problemă ce nu e liniar separabilă, prin transformare adecvată cresc șansele de a se transforma într–una care e liniar separabilă. Să considerăm o rețea neurală de tip feedforward cu un strat de intrare cu n noduri, un singur strat ascuns și un strat de ieșire cu un singur nod[3]. Această rețea produce o funcție de la un spațiu n–dimensional la unul unidimensional:

$$F : \mathbb{R}^n \to \mathbb{R} \tag{7.2}$$

Funcția F poate fi văzută ca o hipersuprafață $\Gamma \subset \mathbb{R}^{n+1}$; hipersuprafața Γ este necunoscută și se determină pe baza setului de antrenare.

În etapa de instruire se folosește o procedură oarecare prin care se determină hipersuprafața Γ, plecând de la setul de date de antrenare, adică se obține funcția F. În etapa de inferență se folosește un procedeu de interpolare pentru a determina valori de ieșire corespunzătoare unor vectori din spațiul de intrare $\mathbb{R}^n$.

În cele ce urmează discutăm despre funcții cu proprietatea de interpolare. *Dându–se un set de antrenare de p perechi*

$$\mathscr{S} = \left\{ \left(\mathbf{x}^{(i)}, y^{(i)} \right) | \mathbf{x}^{(i)} \in \mathbb{R}^n, y^{(i)} \in \mathbb{R}, i = \overline{1,p} \right\}$$

spunem că o funcție $F : \mathbb{R}^n \to \mathbb{R}$ satisface proprietatea de interpolare dacă

$$F\left(\mathbf{x}^{(i)}\right) = y^{(i)}, \quad i = \overline{1,p} \tag{7.3}$$

Tehnica funcțiilor cu activare radială (Radial Basis Functions, RBF) consideră că F are forma:

$$F(\mathbf{x}) = \sum_{i=1}^{p} w_i \varphi_i \left(\left\| \mathbf{x} - \mathbf{x}^{(i)} \right\| \right) \tag{7.4}$$

unde φ_i sunt funcții neliniare, cunoscute ca funcții cu activare radială. Punctele $\mathbf{x}^{(i)}$ sunt "centrele" (parametri ai) funcțiilor RBF. Pentru un vector $\mathbf{x} \in \mathbb{R}^n$, valoarea returnată de funcția φ_i se bazează pe distanța dintre $\mathbf{x}$ și $\mathbf{x}^{(i)}$, adică pe $\|\mathbf{x} - \mathbf{x}^{(i)}\|$.

Impunând condiția (7.3) asupra formei (7.4), avem următorul sistem liniar în care necunoscutele sunt $w_i, i = \overline{1,p}$:

$$\begin{bmatrix} \varphi_{11} & \varphi_{12} & \cdots & \varphi_{1p} \\ \varphi_{21} & \varphi_{22} & \cdots & \varphi_{2p} \\ \vdots & \vdots & \vdots & \vdots \\ \varphi_{p1} & \varphi_{p2} & \cdots & \varphi_{pp} \end{bmatrix} \cdot \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_p \end{bmatrix} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(p)} \end{bmatrix} \tag{7.5}$$

[3] În acest capitol, ieșirea unică este pentru simplificarea prezentării. Pentru cazul în care ieșirea este din spațiul $\mathbb{R}^m$ sau dacă avem o problemă de clasificare cu m clase, stratul de ieșire va avea m neuroni.

unde

$$\varphi_{ij} = \varphi_i \left(\left\| \mathbf{x}^{(j)} - \mathbf{x}^{(i)} \right\| \right), \ i, j = \overline{1, p} \tag{7.6}$$

Notăm $\mathbf{y} = \left(y^{(1)}, y^{(2)}, \ldots, y^{(p)}\right)^t$, $\mathbf{w} = (w_1, w_2, \ldots, w_p)^t$, $\mathbf{\Phi} = (\varphi_{ij})_{i,j=\overline{1,p}}$. Numim $\mathbf{\Phi}$ matricea de interpolare. Se poate rescrie (7.5) sub forma:

$$\mathbf{\Phi} \cdot \mathbf{w} = \mathbf{y} \tag{7.7}$$

Dacă matricea $\mathbf{\Phi}$ este nesingulară, atunci ponderile sunt $\mathbf{w} = \mathbf{\Phi}^{-1}\mathbf{y}$. Pentru discuția asupra caracterului nesingular al matricei $\mathbf{\Phi}$, considerăm teorema lui Michelli:

Teorema 3 (Michelli, 1986)**.** *Fie $\{\mathbf{x}_i\}_{i=\overline{1,p}}$ un set de puncte distincte din $\mathbb{R}^n$. Atunci matricea de interpolare $\mathbf{\Phi}$ este inversabilă dacă funcțiile φ_i au una din formele:*

1. *funcție multipătratică:*

$$\varphi_i(r_i) = \sqrt{r_i^2 + c^2}, \ c > 0 \tag{7.8}$$

2. *funcție inversă de multipătratică:*

$$\varphi_i(r_i) = \frac{1}{\sqrt{r_i^2 + c^2}}, \ c > 0 \tag{7.9}$$

3. *funcție Gaussiană:*

$$\varphi_i(r_i) = \exp\left(-\frac{r_i^2}{2\sigma^2}\right), \ \sigma > 0 \tag{7.10}$$

unde în toate cele trei cazuri r_i este distanța Euclidiană dintre un vector oarecare $\mathbf{x}$ și vectorul prototip $\mathbf{x}^{(i)}$ — echivalent: norma diferenței dintre $\mathbf{x}$ și $\mathbf{x}^{(i)}$, $\left\|\mathbf{x} - \mathbf{x}^{(i)}\right\|$.

7.3 Rețele cu funcții cu activare radială

O rețea cu funcții cu activare radială este ilustrată în figura 7.3; ea constă din trei straturi:

1. *stratul de intrare*, care constă din n noduri, unde n este dimensiunea spațiului de intrare.

2. *stratul ascuns*, care e format din același număr de neuroni ca numărul de date din setul de antrenare, p; fiecare neuron i, $i = \overline{1, p}$ are funcție cu activare radială $\varphi_i \left(\left\|\mathbf{x} - \mathbf{x}^{(i)}\right\|\right)$, unde φ_i e funcție cu una din formele enunțate în Teorema 3;

3. *stratul de ieșire*, care în cazul exemplificat este format dintr-un singur neuron. Stratul de ieșire poate avea m neuroni, pentru a trata problemele de clasificare sau de estimare de probabilitate condiționată pentru m clase, sau pentru probleme de regresie cu m ieșiri.

Pentru funcțiile φ_i vom considera în continuare funcțiile Gaussiene:

$$\varphi_i\left(\left\|\mathbf{x}-\mathbf{x}^{(i)}\right\|\right)=\exp\left(-\frac{1}{2\sigma_i^2}\left\|\mathbf{x}-\mathbf{x}^{(i)}\right\|^2\right),\quad i=\overline{1,p} \tag{7.11}$$

cu $\sigma_i>0$ este parametru al funcție Gaussiene φ_i centrate în $\mathbf{x}^{(i)}$. De regulă tuturor Gaussienelor li se asigneaza aceeași lățime σ; În acest caz, diferența dintre funcții este dată de centrele $\mathbf{x}^{(i)}$.

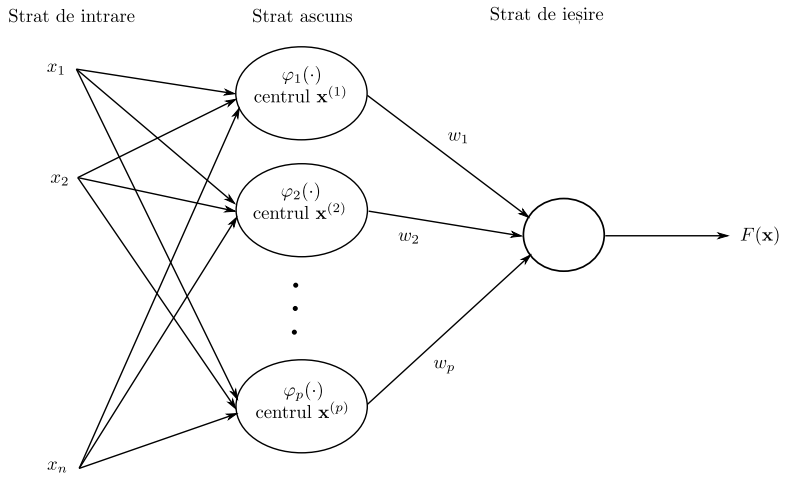


Figura 7.3: Structura unei rețele RBF, plecând de la funcția de interpolare din ecuația (7.4).

Rețeaua din figura 7.3 este un gtaf de calcul pentru funcția din ecuația (7.4).

În practică se evită folosirea tuturor datelor din setul de antrenare pentru crearea de funcții de activare de tip radial. Un motiv ar fi că setul $\left\{\left(\mathbf{x}^{(i)},y^{(i)}\right)\,|i=\overline{1,p}\right\}$ poate prezenta zgomot, de exemplu datorită erorilor de măsurare. Folosirea unui procedeu de aproximare plecând de la un set de date cu zgomot duce la model de predicție slab. Alt motiv e că numărul de neuroni din stratul ascuns ar putea să fie prohibitiv. Ca atare, în practică numărul de noduri din stratul ascuns este mult mai mic decât numărul de date din setul de antrenare. Funcția F capătă forma:

$$F(\mathbf{x})=\sum_{i=1}^{k}w_i\varphi_i\left(\|\mathbf{x}-\hat{\boldsymbol{\mu}}_i\|\right) \tag{7.12}$$

unde dimensiunea vectorului de intrare $\mathbf{x}$ este aceeași ca și cea de până acum, $k < p$ iar punctele $\hat{\boldsymbol{\mu}}_i$ pot proveni dintr–un proces de grupare automată (clustering). Reprezentarea lui F ca rețea neurală artificială este dată în figura 7.4. Diferențele față de figura 7.3 sunt că stratul ascuns are k neuroni în loc de p, iar funcțiile de activare radială sunt centrate în vectori $\hat{\boldsymbol{\mu}}_k$ în locul datelor din setul de antrenare, $\mathbf{x}^{(i)}$.

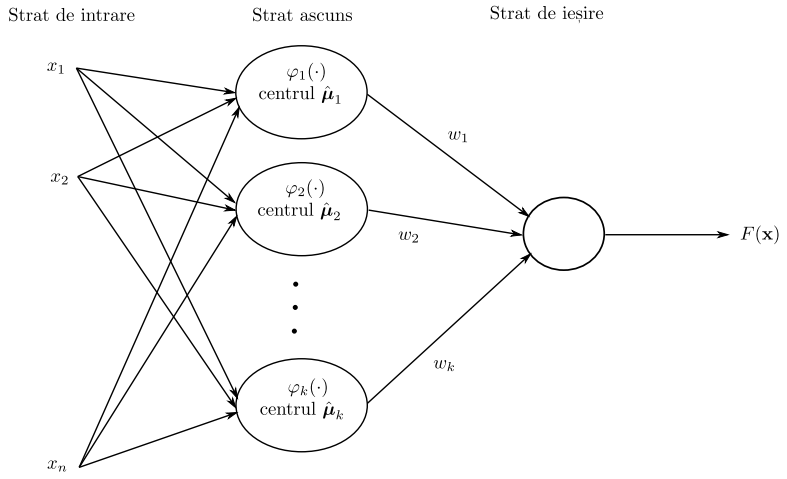


Figura 7.4: Structura unei rețele RBF, folosind mai puține noduri decât în figura 7.3. Centrii $\hat{\boldsymbol{\mu}}_i$, $i = \overline{1,k}$ se obțin printr-un procedeu de clustering.

Pentru determinarea celor k centri $\hat{\boldsymbol{\mu}}_i$, $i = \overline{1,k}$ ale centrilor din stratul ascuns se poate utiliza o metodă oarecare de grupare automată pe baza similarităților (clustering), dar care să producă centroizi[4]. Vom prezenta în cele ce urmează algoritmul K-means clustering.

7.4 Clustering folosind algoritmul K-means

Clustering-ul este o formă de învățare nesupervizată în care un set de vectori este partiționat în grupuri. Clusterele ar trebui obținute de așa manieră încât vectorii similari să fie grupați în același cluster, iar doi vectori nesimilari să fie dispuși în clustere diferite. Noțiunea de similaritate se consideră în mod convenabil.

Considerăm un set de p puncte, $\left\{\mathbf{x}^{(i)}\right\}_{i=\overline{1,p}}$ ce urmează să fie partiționat în k clustere[5]; de regulă, $k \ll p$. Fie $C(i)$ indicele de cluster de care aparține vectorul $\mathbf{x}^{(i)}$, $i = \overline{1,p}$. Evident, $1 \leq C(i) \leq k$. Considerăm $d\left(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}\right)$

[4]Excludem deci algoritmi de clustering precum DBSCAN sau OPTICS.
[5]În forma clasică a algoritmului valoarea lui k este furnizată de utilizator.

o măsură a similarității[6] – dintre perechile de vectori $\mathbf{x}^{(i)}$ și $\mathbf{x}^{(j)}$. Pentru clustering se cere minimizarea funcției:

$$J(C) = \frac{1}{2}\sum_{l=1}^{k}\sum_{i:C(i)=l}\sum_{j:C(j)=l} d\left(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}\right) \tag{7.13}$$

unde C este partiționarea dată de cele k clustere:

$$C = \left\{\{i|1 \leq i \leq p, C(i) = l\}|l = \overline{1,k}\right\}$$

În algoritmul K-means drept măsură de similaritate se folosește, de regulă, pătratul distanței Euclidiene:

$$d\left(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}\right) = \left\|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\right\|^2 \tag{7.14}$$

În urma procesului de clustering vor rezulta k centroizi – centri de clustere – notați $\hat{\boldsymbol{\mu}}_l \in \mathbb{R}^n$, $l = \overline{1,k}$.

Modificăm forma funcției de eroare J astfel încât să se ia în considerare pătratul distanțelor dintre vectorii $\mathbf{x}^{(i)}$ și centroizii $\hat{\boldsymbol{\mu}}_l$ ai clusterelor de care aparțin:

$$J(C) = \frac{1}{2}\sum_{l=1}^{k}\sum_{i:C(i)=l}\left\|\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}_l\right\|^2 \tag{7.15}$$

Cum se poziționează centroizii? Algoritmul K-means determină niște valori pentru $\hat{\boldsymbol{\mu}}_l$ printr-un proces iterativ, astfel încât $J(C)$ să scadă. Este un algoritm euristic, nu garantează faptul că se ajunge la minimul global al lui $J(C)$.

Algoritmul alege aleator k centroizi inițiali $\hat{\boldsymbol{\mu}}_l^{(1)}$, $l = \overline{1,k}$, inițializându–i cu valori fie din setul de antrenare, sau setate la întâmplare cu valori din spațiul de intrare, sau conform algoritmului K–means++, a se vedea secțiunea 7.5. Avem apoi o succesiune de iterații cu pașii:

- *Pasul de asignare:* Calculează pentru orice l, $l = 1 \ldots k$:

$$S_l^{(t)} = \left\{\mathbf{x}^{(i)} : \left\|\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}_l^{(t)}\right\|^2 \leq \left\|\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}_j^{(t)}\right\|^2, j = \overline{1,k}, j \neq l, i = \overline{1,p}\right\} \tag{7.16}$$

  adică pentru fiecare punct $\mathbf{x}^{(i)}$ se determină care este cel mai apropiat centroid de care aparține; $S_l^{(t)}$ este mulțimea vectorilor din setul de antrenare ce sunt cel mai apropiate de centroidul $\hat{\boldsymbol{\mu}}_l^{(t)}$, la iterația t.

[6] Ne putem folosi aici de orice funcție care măsoară distanța dintre doi vectori; dacă distanța este mică, atunci similaritatea dintre vectori este mare – și invers.

- *Modificarea centroizilor:*

$$\hat{\boldsymbol{\mu}}_l^{(t+1)} = \frac{1}{\left|S_l^{(t)}\right|} \cdot \sum_{\mathbf{x}^{(i)} \in S_l^{(t)}} \mathbf{x}^{(i)}, \text{ pentru } l = \overline{1,k} \tag{7.17}$$

unde $\left|S_l^{(t)}\right|$ este numărul de elemente ale mulțimii $S_l^{(t)}$.

Algoritmul K-means se oprește atunci când pasul de asignare nu mai modifică mulțimile $S_l^{(t)}$.

În general, algoritmul nu converge către minimul global al funcției J; fiind însă rapid în practică – adică necesitând puțini pași până la oprire – se poate reporni cu alte valori ale centroizilor inițiali $\hat{\boldsymbol{\mu}}_l^{(1)}$, $l = \overline{1,k}$. Situația (centroizii) pentru care $J(C)$ are valoarea cea mai mică în aceste încercări este reținută.

7.5 Algoritmul de inițializare K-means++

Se consideră că inițializarea centroizilor inițiali $\hat{\boldsymbol{\mu}}_l^{(1)}$, $l = \overline{1,k}$ nu ar trebui lăsată la voia întâmplării și că se poate îmbunătăți considerabil performanța algoritmului printr–o alegere îngrijită a lor. Un caz nefavorabil este dat în figura 7.5. Să considerăm un dreptunghi cu laturile de lungime $L \gg l$, având în cele patru vârfuri câte un punct $\mathbf{x}^{(i)} \in \mathbb{R}^2$, $i = \overline{1,4}$. Dacă considerăm $k = 2$ și centroizii sunt aleși inițial la jumătatea laturilor de lungime mai mare, atunci algoritmul se oprește după o iterație cu $J(C) = \frac{1}{2} \cdot 4 \left(\frac{L}{2}\right)^2 = \frac{L^2}{2}$ (punctele $\mathbf{x}^{(1)}$ și $\mathbf{x}^{(3)}$ aparțin clusterului de centroid $\hat{\boldsymbol{\mu}}_1$, iar celelalte două celui de al doilea cluster). Dacă alegerea centroizilor inițiali se face ca în figura 7.6, atunci se obține valoarea $J(C) = \frac{l^2}{2}$ – și se poate arăta că aceasta este și valoarea minimă a lui J. Având în vedere că L poate fi luat oricât de mare față de l, rezultă că o alegere neinspirată a centroizilor poate să ducă la o valoare oricât de depărtată față de optim pentru funcția J.

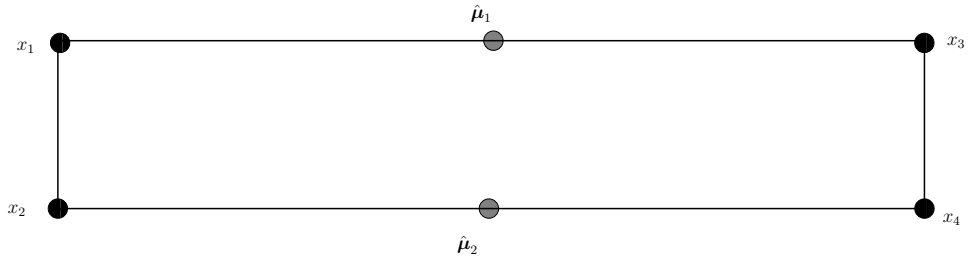


Figura 7.5: Caz nefavorabil pentru K-means la alegerea centroizilor inițiali



Figura 7.6: Alegerea optimă a centroizilor inițiali

Ca atare, s–a dezvoltat algoritmul K-means++ care are ca scop determinarea unor centroizi inițiali aleși mai potrivit. Alegerea celor K centroizi se face după următorii pași[7]:

1. Alege primul centroid aleator din setul de antrenare;
2. Pentru fiecare punct ce nu a fost încă ales drept centroid $\mathbf{x}^{(i)}$ calculează $D\left(\mathbf{x}^{(i)}\right)$ $(1 \leq i \leq p)$, distanța de la el până la cel mai apropiat din centroizii determinați până la pasul curent;
3. Alege aleator un nou punct din setul de antrenare, folosind o probabilitate de alegere o funcție crescătoare cu distanța dată de vectorul de distanțe $\mathbf{D}$;
4. Repetă pașii 2 și 3 până când s–au ales toți cei k centroizi;

Se aplică apoi algoritmul K-means pentru centroizii astfel determinați.

Costul suplimentar indus de determinarea celor k centroizi ca mai sus este neglijabil față de efectele benefice asupra rezultatului final. Motivațiile teoretice pentru K-means++ se găsesc în lucrarea lui Arthur Vassilvitskii.

7.6 Determinarea ponderilor dintre stratul ascuns și cel de ieșire

Determinarea ponderilor legăturilor dintre stratul ascuns și cel de ieșire este ultimul pas. Problema este una de determinare a ponderilor pentru o problemă de regresie liniară și se tratează cu tehnicile de învățare supervizată din secțiunile 2.3 și 2.5: metoda gradient descent sau metoda ecuațiilor normale.

Pentru cazul în care problema este una de regresie în care vectorii de ieșire sunt din $\mathbb{R}^m$, fiecare neuron de ieșire își poate ajusta setul de ponderi

[7]David Arthur, Sergei Vassilvitskii, (2007), *"k-means++: the advantages of careful seeding"*, Proceedings of the eighteenth annual ACM-SIAM Symposium on Discrete algorithms. Society for Industrial and Applied Mathematics Philadelphia, PA, USA. pp. 1027–1035.

independent de ponderile celorlalte ieșiri; se aplică una din cele două metode de mai sus.

7.7 Algoritmul de instruire a rețelei RBF

Sintetizăm pe baza expunerii de până acum procedura de instruire a unei rețele RBF. Stratul de intrare este fix, având numărul de noduri dat de dimensiunea intrării. Stratul ascuns se obține rulând un algoritm de clustering (e.g. K-means precedat de K-means++) peste setul de antrenare și rezultând k centroizi $\hat{\boldsymbol{\mu}}_j, j = \overline{1,k}$. Acești centri de clustere devin centrii unor funcții Gaussiene asignate nodurilor ascunse. Pentru fiecare astfel de Gaussiană este asignată o aceeași lățime σ, calculată ca:

$$\sigma = \frac{d_{max}}{\sqrt{2 \cdot k}} \tag{7.18}$$

unde d_{max} este distanța maximă dintre perechile de centroizi. În stratul de ieșire sunt tot atâtea noduri cât este dimensiunea spațiului de ieșire. Ponderile legăturilor dintre stratul ascuns și stratul de ieșire se calculează ca pentru o problemă de regresie sau clasificare, definind o funcție de eroare și minimizând–o printr-un procedeu adecvat – gradient descent, metoda pseudoinversei etc.

Capitolul 8

Fuzzy ARTMAP

Rețelele Fuzzy ARTMAP folosesc instruirea supervizată pentru a crea clasificatoare, estimatoare de probabilitate condiționată și modele de regresie.

Rețelele Fuzzy ARTMAP (FAM) au capacitatea de învățare incrementală, posedă o mare parte din proprietățile dorite pentru sisteme instruibile și rezolvă dilema stabilitate–plasticitate.

8.1 Învățarea incrementală

Învățarea incrementală este o caracteristică asociată unor sisteme adaptive care:

1. agregă cunoștințe noi din date noi;
2. nu cer acces la datele utilizate pentru a antrena sistemul până la momentul curent;
3. păstrează cunoștințele deprinse anterior;
4. se pot acomoda cu noi categorii care pot fi introduse de noi date de instruire.

8.2 Proprietăți dezirabile ale sistemelor instruibile

Pentru un sistem instruibil următoarele proprietăți sunt văzute ca fiind esențiale:

1. învățare rapidă;
2. învățare din noi date fără a fi nevoie să se reantreneze cu datele parcurse anterior – regăsită în învățarea incrementală;
3. rezolvarea de probleme neseparabile liniar – o varietate liniară nu este întotdeauna o suprafață de separare bună;

4. în cazul unui clasificator: abilitate de a da nu doar clasa de apartenență a unui vector de intrare, ci și plauzibilitatea acestei apartenențe; sunt favorizați aici estimatorii de probabilitate condiționată de forma $P(clasa|intrare)$; de exemplu, $P(email = spam|continut\ email)$;

5. pentru clasificatori: oferire de explicații asupra modului în care datele sunt clasificate, de ce sunt clasificate într–un anume mod; prin această trăsătură se evită tratarea clasificatorului ca o cutie neagră ce nu poate să explice modul de producere a deciziilor;

6. posibilitate de reglare automată a hiperparametrilor modelului; de exemplu, pentru perceptronul multistrat hiperparametrii de interes sunt rata de învățare, numărul de straturi ascunse, numărul neuronilor din fiecare strat ascuns etc.;

7. aproximarea de funcții fără a cunoaște distribuția inițială a datelor; rareori datele se supun unei distribuții clasice;

8. pentru clase care prezintă suprapuneri, să se creeze regiuni în spațiul de intrare care să realizeze cea mai mică suprapunere; problema asocierilor de tip un vector de intrare–la–mai multe clase trebuie tratată explicit.

8.3 Dilema stabilitate–plasticitate

Un sistem instruibil ar trebui să aibă două proprietăți:

1. *plasticitate* - înseamnă adaptarea la mediul din care provin datele de instruire. Altfel zis, plasticitatea este capacitatea de învățare.

2. *stabilitate* - se referă la păstrarea cunoștințelor învățate anterior.

Atunci când se prezintă noi intrări unei rețele neurale, cele vechi pot fi uitate. Ponderile rețelei trebuie să fie suficient de flexibile pentru a învăța noi cunoștințe (trăsătura de plasticitate), dar nu atât de mult încât să uite ceea ce s–a învățat anterior (trăsătura de stabilitate). Acest conflict dintre stabilitate și plasticitate se numește *dilema stabilitate–plasticitate*. Cele mai multe dintre rețelele neurale existente sunt fie stabile dar incapabile de a învăța rapid noi vectori, fie plastice dar instabile; de aceea, dilema menționată este una din problemele de interes în domeniul modelelor instruibile. S-a formulat întrebarea: cum poate un sistem de învățare să fie atât stabil cât și plastic?

Dilema a fost abordată de Carpenter și Grossberg în articolul "The ART of adaptive pattern recognition by a self-organizing neural network"[1]. Teoria rezonanței adaptive (Adaptive Resonance Theory, ART) dezvoltată de cei doi

[1] G. A. Carpenter and S. Grossberg, "The art of adaptive pattern recognition by a self-organizing neural network", Computer, vol. 21, no. 3, pp. 77–88, 1988.

autori este unul din răspunsurile concrete date dilemei. De asemenea, sistemul prezintă abilitate de învățare incrementală și mare parte din proprietățile dezirabile ale sistemelor instruibile.

8.4 Fuzzy ARTMAP

Familia Fuzzy ARTMAP de rețele neurale (FAM) este cunoscută ca una din puținele care posedă capacitate de învățare incrementală, rezolvă dilema stabilitate-plasticitate și are multe din proprietățile dorite pentru un sistem instruibil.

Carpenter si Grossberg au fost interesați de obținerea de sisteme care se pot organiza singure. Paradigma ART poate fi descrisă ca un tip de grupare (eng: clustering) incrementală a datelor, având posibilitatea de a învăța fără antrenare supervizată și este de asemenea în acord cu modelele cognitive și de comportament. Folosește învățare nesupervizată; rețeaua este capabilă să găsească automat categoria asociată intrării curente sau să creeze una nouă atunci când este nevoie: numărul de neuroni din rețea nu este fixat aprioric.

Rețelele neurale Fuzzy ART sunt capabile să producă rapid o învățare stabilă a unor categorii de semnale ca răspuns la secvențe arbitrare de intrări binare sau continue. Fuzzy ART încorporează operatori din teoria mulțimilor fuzzy.

Sistemele de tip Fuzzy ARTMAP învață în mod autonom arbitrar de mulți vectori, formând categorii de recunoaștere în funcție de succesul de predicție. Acest sistem de învățare supervizată este construit dintr-o pereche de module ART capabile de auto-organizare și obținere de categorii de recunoaștere stabile.

Succesul rețelelor bazate pe teoria rezonanței adaptive este dat de avantajele pe care le au față de alte rețele multistrat dezvoltate anterior:

- permit crearea dinamică a nodurilor (neuronilor) fără distrugerea celor existente;
- necesită mai puține cicluri de antrenare cerute, se pot folosi chiar cu învățare incrementală, adică să treacă o singură dată peste setul de instruire;
- rețeaua Fuzzy ARTMAP are convergență garantată datorită utilizării unor ponderi mărginite și monotone.

Fuzzy ARTMAP este utilizabil pentru probleme de clasificare, estimare de probabilitate și regresie. S–a demonstrat că FAM este aproximator universal, adică poate aproxima oricât de bine orice funcție continuă. Deoarece atât clasificarea cât și estimarea de probabilitate sunt cazuri particulare ale regresiei, rezultă că FAM poate fi utilizat în orice problemă ce presupune stabilirea de legături dintre două submulțimi din $\mathbb{R}^n$ și respectiv din $\mathbb{R}^m$.

În plus, rețeaua FAM mai are o calitate: reprezentarea vectorilor învățați prin categorii facilitează extragerea de reguli, aspect esențial în domeniul extragerii de cunoștințe din date.

8.4.1 Arhitectura rețelei FAM

O rețea FAM constă într–o pereche de module ART notate ART_a și ART_b, conectate printr-un modul numit Mapfield, notat F^{ab}. ART_a și ART_b sunt folosite pentru codificarea vectorilor de intrare și respectiv de ieșire, iar Mapfield permite asocierea între intrări și ieșiri. Figura 8.1 conține componentele unei arhitecturi FAM.

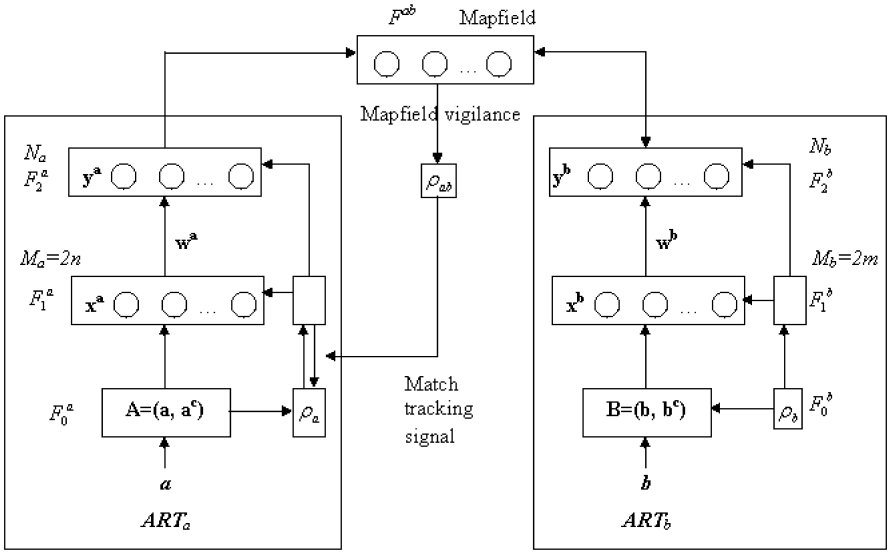


Figura 8.1: Arhitectura Fuzzy ARTMAP

Modulul Fuzzy ART_a conține stratul de intrare F_1^a și stratul competitiv F_2^a. Se adaugă de asemenea un strat de preprocesare F_0^a înaintea lui F_1^a. Straturi echivalente apar în ART_b.

Vectorii de intrare inițiali[2] sunt dați sub forma:

$$\mathbf{a}^{(i)} = \left(a_1^{(i)}, \ldots, a_n^{(i)}\right),\ a_j^{(i)} \in [0,1],\quad j = \overline{1,n}, i = \overline{1,p} \tag{8.1}$$

p fiind numărul de vectori din setul de instruire.

În cazul în care datele inițiale nu sunt din intervalul $[0,1]$, se poate aplica

[2]În acest capitol vectorii sunt văzuți ca matrice cu o linie.

o scalare globală:

$$a_j^{(i)} \to \frac{a_j^{(i)} - MIN}{MAX - MIN}, j = \overline{1,n}, i = \overline{1,p}$$

pentru fiecare vector de intrare $\mathbf{a}^{(i)} = \left(a_1^{(i)}, \ldots, a_n^{(i)}\right)$, $1 \leq i \leq p$, unde MIN și MAX sunt valoarea minimă și respectiv maximă din vectorii de intrare:

$$MIN(MAX) = \min(\max) \left\{a_j^{(i)}\right\}, \quad j = \overline{1,n}, i = \overline{1,p}$$

sau se poate lua un minorant (respectiv majorant) al tuturor valorilor de intrare[3]. Alternativ, fiecare din trăsături (coloane) poate fi scalată independent de celelalte. În urma aplicării oricărei din aceste scalări, vectorii din setul de instruire vor avea valori în intervalul $[0, 1]$.

O tehnică de preprocesare numită *codificare complementară* este efectuată în cele două module fuzzy ART de către stratul F_0^a, respectiv F_0^b pentru a evita proliferarea nodurilor. S–a dovedit că fără codificarea complementară se vor produce numeroase categorii grupate lângă origine, fără a crea altele care să le înlocuiască. Codificarea complementară este utilizată pentru a obține vectori normalizați, adică vectori cu normă constantă:

$$|\mathbf{A}| = const \tag{8.2}$$

unde $|\cdot|$ este o funcție normă. În cazul de față $|\cdot|$ reprezintă norma L_1: pentru un vector $\mathbf{z} = (z_1, \ldots, z_k)$

$$|\mathbf{z}| = L_1(\mathbf{z}) = \sum_{i=1}^{k} |z_i| \tag{8.3}$$

Fiecare vector de intrare $\mathbf{a} = (a_1, \ldots, a_n)$ produce vectorul:

$$\mathbf{A} = (\mathbf{a}, \mathbf{a}^c) = (\mathbf{a}, \mathbf{1} - \mathbf{a}) = (a_1, \ldots, a_n, 1 - a_1, \ldots, 1 - a_n) \tag{8.4}$$

a cărui normă este:

$$|\mathbf{A}| = \sum_{i=1}^{n} a_i + \sum_{i=1}^{n} (1 - a_i) = n = constant \tag{8.5}$$

motiv pentru care spunem că vectorul $\mathbf{A}$ este normalizat.

Pentru ART_a folosim următoarele notații: M_a este numărul de noduri în F_1^a și N_a este numărul de noduri din F_2^a. Datorită pasului de preprocesare, $M_a = 2n$. Fiecare nod F_2^a reprezintă un grup de intrări similare (numit în alte contexte cluster); vom folosi termenul "categorie" pentru a ne referi la

[3] De exemplu, pentru pixeli, valorile sunt de regulă între 0 și 255.

un nod F_2^a. Fiecare categorie F_2^a are propriul set de ponderi adaptive stocate sub forma unui vector:

$$\mathbf{w}_j^a = \left(w_{j,1}^a, \ldots, w_{j,M_a}^a\right),\ j = \overline{1, N_a}. \tag{8.6}$$

Aceste ponderi formează memoria pe termen lung a sistemului. Inițial, toți vectorii au valorile:

$$w_{ji}^a = 1,\ j = \overline{1, N_a},\ i = \overline{1, M_a} \tag{8.7}$$

Spunem că un nod din F_2^a este *necomis* dacă nu a învățat încă nici un vector de intrare, *comis* în caz contrar. Modulul ART_a este responabil cu crearea grupărilor de vectori de intrare. În timpul etapei de învățare, N_a este numărul de noduri (categorii) comise. Notații și afirmații similare se folosesc pentru ART_b, care primește vectori m-dimensionali. Pentru o problemă de clasificare, adică o problemă pentru care numărul – total sau inițial – de clase de ieșire este aprioric cunoscut, indexul de clasă este același cu indexul de categorie din F_2^b și astfel ART_b poate fi substituit de un vector N_b-dimensional. Într-o astfel de codificare, valoarea lui N_b poate să crească – dacă noi clase de ieșire sunt adăugate la setul de instruire.

Modulul Mapfield permite FAM să creeze legături între cele două module ART, stabilind legături de tip mulți–la–unu între categorii din ART_a și ART_b. Numărul de noduri din F^{ab} este egal cu numărul de noduri din F_2^b. Fiecare nod j din F_2^a este legat cu fiecare nod din F_2^b via un vector de ponderi $\mathbf{w}_j^{ab}$, unde $\mathbf{w}_j^{ab}$ este a j–a linie dintr-o matrice $\mathbf{w}^{ab}$ ($j = \overline{1, N_a}$). Toate ponderile din $\mathbf{w}^{ab}$ sunt inițializate cu 1:

$$w_{jk}^{ab} = 1,\ j = \overline{1, N_a},\ k = \overline{1, N_b} \tag{8.8}$$

8.4.2 Algoritmul de învățare pentru FAM

În următorul algoritm, operatorul $\wedge$ este așa–numitul operator fuzzy AND, care pentru doi vectori

$$\mathbf{p} = (p_1, \ldots, p_k), \mathbf{q} = (q_1, \ldots, q_k) \tag{8.9}$$

cu $0 \leq p_i,\ q_i \leq 1$, $i = \overline{1, k}$ este definit ca:

$$(\mathbf{p} \wedge \mathbf{q})_i = \min(p_i, q_i), i = \overline{1, k} \tag{8.10}$$

1. Se setează parametrul de factor de vigilență ρ_a la o valoare egală cu o valoare de bază prestabilită: $\rho_a = \overline{\rho}_a \in [0, 1)$ și se consideră că toate categoriile din F_2^a sunt neinhibate – adică fiecare nod participă la căutarea unei categorii adecvate pentru vectorul de intrare curent;

2. Pentru fiecare vector de intrare preprocesat $\mathbf{A}$, o funcție fuzzy este folosită pentru a obține un răspuns de la fiecare categorie F_2^a:

$$T_j(\mathbf{A}) = \frac{|\mathbf{A} \wedge \mathbf{w}_j^a|}{\alpha_a + |\mathbf{w}_j^a|}, \quad j = \overline{1, N_a} \tag{8.11}$$

3. Fie J indicele de nod neinhibat care dă cea mai mare valoare calculată precum în (8.11):

$$J = \arg\max \{T_j | 1 \leq j \leq N_a \text{ și nodul } j \text{ nu este inhibat}\} \tag{8.12}$$

4. Verifică condiția de rezonanță, i.e. dacă intrarea este suficient de similară cu prototipul câștigătorului:

$$\frac{|\mathbf{A} \wedge \mathbf{w}_J^a|}{|\mathbf{A}|} \geq \rho_a \tag{8.13}$$

   Dacă condiția este îndeplinită, atunci mergi la pasul 5, altfel inhibă nodul J astfel încât el nu va mai participa la competiția pentru vectorul curent. Dacă există noduri neinhibate, atunci mergi la pasul 3, altfel recrutează o nouă categorie (creează un nou nod în F_2^a) pentru a reprezenta vectorul de intrare și fie J indicele acestui nou nod.

5. Un proces similar se desfășoară și în ART_b. Fie K indicele nodului câștigător din ART_b. Vectorul de ieșire N_b-dimensional F_2^b este setat la:

$$y_k^b = I(k = K), \quad k = \overline{1, N_b} \tag{8.14}$$

   unde I este funcția indicator (3.39) de la pagina 62.

   În Mapfield se formează vector de ieșire $\mathbf{x}^{ab}$:

$$\mathbf{x}^{ab} = \mathbf{y}^b \wedge \mathbf{w}_J^{ab} \tag{8.15}$$

6. Un test de verificare în Mapfield controlează potrivirea dintre valoarea prezisă $\mathbf{x}^{ab}$ și vectorul de ieșire atașat vectorului de instruire curent $\mathbf{y}^b$:

$$\frac{|\mathbf{x}^{ab}|}{|\mathbf{y}^b|} \geq \rho_{ab} \tag{8.16}$$

   unde $\rho_{ab} \in [0, 1]$ este un parametru de vigilență Mapfield. Dacă testul din inecuația (8.16) este trecut, atunci se face învățare ART_a, ART_b și Mapfield (pasul 7). Altfel, se inițiază o secvență de pași numită *match tracking* (pasul 8).

7. În modulele fuzzy ART și în Mapfield se efectuează învățare:

$$\mathbf{w}_J^{a(new)} = \beta_a \left(\mathbf{A} \wedge \mathbf{w}_J^{a(old)} \right) + (1 - \beta_a) \mathbf{w}_J^{a(old)} \tag{8.17}$$

(și analog în ART_b) și

$$w_{Jk}^{ab} = I(k = K) \tag{8.18}$$

Se merge la pasul 9.

8. Faza de match tracking, în care se intră doar dacă inecuația (8.16) nu e în deplinită: crește valoarea lui ρ_a:

$$\rho_a = \frac{|\mathbf{A} \wedge \mathbf{w}_J^a|}{|\mathbf{A}|} + \delta \tag{8.19}$$

unde $0 < \delta < 1$. Dacă $\rho_a > 1$ atunci vectorul curent de instruire este rejectat, altfel mergi la pasul 3.

9. Dacă mai sunt vectori în setul de învățare, mergi la pasul 1, altfel STOP.

Urmează câteva comentarii privind pașii de mai sus:

1. La pasul 2, fiecare vector este preprocesat datorită straturilor F_0^a și respectiv F_0^b. $\alpha_a > 0$ este un parametru de alegere. Pentru doi vectori $\mathbf{p}$ și $\mathbf{q}$, raportul:

$$r = \frac{|\mathbf{p} \wedge \mathbf{q}|}{|\mathbf{q}|} \tag{8.20}$$

cu $0 \leq r \leq 1$ dă gradul în care $\mathbf{p}$ este subset fuzzy al lui (adică "se potrivește" cu) $\mathbf{q}$ și deci pentru $0 < \alpha_a \ll 1$, $T_j(\mathbf{A})$ din ecuația (8.11) măsoară gradul în care $\mathbf{A}$ este o submulțime fuzzy a categoriei w_j^a. Dacă se crește valoarea lui α_a atunci se va mări numărul de categorii, lucru nu întotdeauna benefic. Este deci sugerat ca să se mențină α_a la o valoare mică, de exemplu $\alpha_a = 0.001$; valori mai mici ale lui α_a nu duc la o diferență semnificativă.

2. La pasul 3, dacă există mai multe categorii ale căror funcție de alegere atinge maximul, vom considera pe acea categorie care are indicele minim.

3. Parametrul ρ_a calibrează încrederea minimă pe care ART_a trebuie să o aibă vizavi de o categorie activată de o intrare pentru ca ART_a să accepte această categorie, în loc de a căuta o categorie mai bună. Valori mici ρ_a duc la un grad mare de generalizare și un număr mai mic de categorii ART_a.

4. Dacă inecuația (8.13) este îndeplinită, spunem că avem rezonanță în ART_a pentru vectorul de intrare curent. Datorită pasului de preprocesare, conform (8.5), numitorul din (8.13) este exact dimensiunea originară a vectorilor de intrare, n.

5. Aceiași pași ca în 1 – 4 sunt efectuați în paralel pentru modulul ART_b, dacă nu cumva acesta este substituit cu un vector N_b-dimensional; în acest din urmă caz indicele nodului câștigător K este indicele de clasă corespunzător intrării curente;

6. Când se intră în pasul 5, avem rezonanță atât în ART_a cât și în ART_b. Vectorul $\mathbf{x}^{ab}$ dă activarea Mapfield și se folosește atât când ambele module F_2^a și F_2^b sunt active, *i.e.* la faza de învățare, cât și când F_2^a este activ și F_2^b e inactiv (faza de predicție). În faza de învățare $\mathbf{x}^{ab}$ are forma din ecuația (8.15); în faza de predicție acest vector este calculat ca:
$$\mathbf{x}^{ab} = \mathbf{w}_J^{ab} \tag{8.21}$$

7. Atunci când ambele module F_2^a și F_2^b sunt active – lucru valabil la instruirea rețelei, când se furnizează atât vectorul de intrare cât și cel de ieșire – a J-a categorie câștigătoare din ART_a va corespunde unui vector de ponderi $\mathbf{w}_J^{ab}$ din Mapfield care leagă nodul F_2^a cu categoria F_2^b prezisă. În paralel, pentru modulul ART_b am obținut vectorul de ieșire $\mathbf{y}^b$ ca în ecuația (8.14). Operația fuzzy AND ne asigură că $\mathbf{x}^{ab}$ nu e plin cu valoarea zero dacă și numai dacă valoarea de ieșire prezisă și cea actuală coincid. Atunci când categoria J este necomisă avem:
$$\mathbf{w}_J^{ab} = (1, 1, \ldots, 1) \tag{8.22}$$
și deci $\mathbf{x}^{ab} = \mathbf{y}^b$. Când doar modulul F_2^a este activ, matricea $\mathbf{w}^{ab}$ dă valoarea prezisă: indicele categoriei din ART_b asociată cu intrarea curentă este unica poziție k din linia j a matricei $\mathbf{w}^{ab}$ pentru care $w_{jk}^{ab} = 1$.

   Ecuația (8.18) indică faptul că al J-lea nod din ART_a este legat cu categoria de ieșire K, iar legătura, odată făcută, nu se mai schimbă.

8. Pentru β_a (Pasul 7), există două moduri de învățare:

   (a) *fast learning* corespunde la a seta $\beta_a = 1$ atât pentru nodurile comise cât și pentru cele necomise;

   (b) *fast-commit and slow-recode learning* corespunde la a seta $\beta_a = 1$ pentru nod necomis și $\beta_a < 1$ pentru cele comise.

9. La faza de match tracking, datorită creșterii valorii lui ρ_a conform ecuației (8.19), nodul J nu va mai fi în stare să câștige în competițiile următoare pentru vectorul de intrare curent. Match tracking–ul va

declanșa o nouă căutare în ART_a pentru a găsi un alt nod câștigător. Se poate ca asta să ducă la crearea unui nou nod în ART_a. Căutarea se repetă până când $\rho_a > 1$, sau până când se găsește o categorie adecvată în ART_a.

10. O valoare mare a lui δ în pasul 8 va crește numărul de categorii. Se folosește de regulă o valoare mică $\delta = 0.001$

11. Ordinea de prezentare a vectorilor din setul de instruire influențează comportamentul rețelei, adică numărul și pozițiile categoriilor va diferi. Putem face antrenarea în paralel cu diferite permutări ale setului de intrare, apoi se contorizează voturile pentru fiecare vector care trebuie clasificat.

Pasul de învățare este repetat până când nu mai este nicio eroare pentru setul de învățare, sau până se atinge o eroare acceptabilă; dacă se vrea învățare incrementală atunci se face o singură trecere pe setul de antrenare. După învățare, FAM poate fi utilizat pentru predicție. În această fază, stratul F_2^b este inactiv și F_2^a este activ. Conform ecuației (8.21), predicția este făcută doar pe baza lui $\mathbf{w}_J^{ab}$.

Există o interpretare geometrică interesantă a categoriilor ART_a: fiecare categorie de intrare $\mathbf{w}_j^a$ se reprezintă ca un hiper–dreptunghi R_j, deoarece vectorul de ponderi poate fi scris ca:

$$\mathbf{w}_j^a = \left(\mathbf{u}_j, \mathbf{v}_j^c\right) \tag{8.23}$$

unde

$$\mathbf{v}_j^c = \left(v_{j1}^c, \ldots, v_{jn}^c\right) \tag{8.24}$$

(atât $\mathbf{u}$ cât și $\mathbf{v}$ au n elemente). Vectorul $\mathbf{u}_j$ definește un colț al hiper–dreptunghiului R_j iar $\mathbf{v}_j$ este colț diagonal opus lui. Pentru $n = 2$, reprezentarea grafică este dată în figura 8.2. Dimensiunea lui R_j este definită ca:

$$|R_j| = |\mathbf{v}_j - \mathbf{u}_j| \tag{8.25}$$

O interpretare similară este valabilă și pentru modulul ART_b.

În modulul ART_a, atunci când se creează un nou nod în pasul 4, acesta va fi de fapt vectorul de intrare preprocesat curent: $\mathbf{w}_J^{(new)} = \mathbf{A} = (\mathbf{a}, \mathbf{a}^c)$; cu alte cuvinte, $R_j^{(new)}$ este de fapt un punct reprezentând vectorul de intrare preprocesat A. În timpul fiecărui pas de învățare fast-learning ($\beta_a = 1$), R_j se expandează la $R_j \oplus \mathbf{a}$, hiper–dreptunghiul minim care conține R_j și $\mathbf{a}$ – a se vedea figura 8.3; dacă punctul $\mathbf{A}$ este chiar în interiorul lui R_j, atunci R_j nu se modifică. Colțurile lui $R_j \oplus \mathbf{a}$ sunt definite de $\mathbf{a} \wedge \mathbf{u}_J$ și $\mathbf{a} \vee \mathbf{v}_J$, unde operatorul $\vee$ este definit ca operatorul fuzzy OR:

$$(\mathbf{p} \vee \mathbf{q})_i = \max\left(p_i, q_i\right), \; i = 1 \ldots, n \tag{8.26}$$

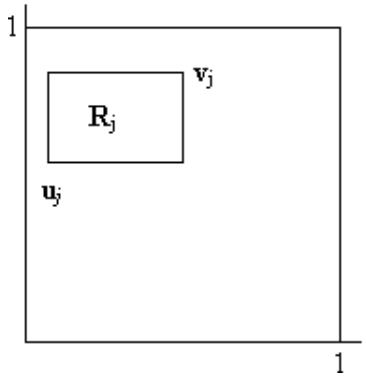


Figura 8.2: Fiecare vector pondere $\mathbf{w}_j^a$ are o interpretare geometrică precum un hiper–dreptunghi R_j cu colțurile definite de $\mathbf{u}_j$ și $\mathbf{v}_j$

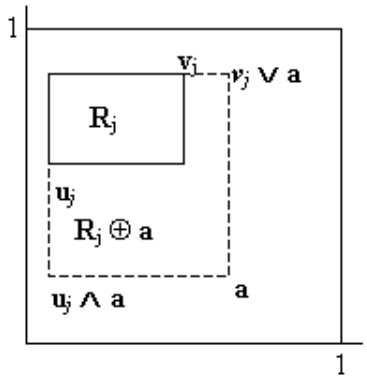


Figura 8.3: Expandarea de categorie în timpul fast learning: de la R_j la hiper–dreptunghiul mai mare conținând R_j și $\mathbf{a}$.

pentru **p** și **q** vectori precum în (8.9).

Se poate arăta că:

$$|R_j| = n - |\mathbf{w}_j| \tag{8.27}$$

iar hiper–dreptunghiul corespunzător unei categorii nu poate crește oricât de mult:

$$|R_j| \leq (1 - \rho_a)n \tag{8.28}$$

Proprietate similară este valabilă și pentru ART_b.

Aceste proprietăți sunt sumarizate de teorema:

Teorema 4. [4] *Un sistem Fuzzy ART cu codificarea complementară, fast learning și termen de vigilență constant formează categorii hiper–dreptunghiuri care converg în limită la o secvență arbitrară de vectori analogici sau binari. Hiper–dreptunghiurile cresc monoton în toate dimensiunile. Dimensiunea $|R_j|$ a unui hiper–dreptunghi este $n - |\mathbf{w}_j|$, unde $\mathbf{w}_j$ este vectorul pondere corespunzător. Dimensiunea $|R_j|$ este mărginită superior de $n(1 - \rho)$. Dacă $0 \leq \rho < 1$, numărul de categorii este mărginit, chiar dacă numărul de exemplare din setul de antrenare este infinit. Proprietăți similare au loc pentru fast-learn, slow-recode, exceptând cazul în care este nevoie de prezentări repetate ale fiecărei intrări înainte de stabilizarea sistemului.*

Sumarizând: FAM aplică o învățare bazată pe potrivire, conform căreia vectorii sunt grupați în categorii pe baza unor măsuri de similaritate. Dacă un vector din setul de instruire nu se potrivește suficient de bine cu o categorie existentă, atunci se va crea una nouă pentru a o reprezenta. Datorită acestui comportament, FAM nu încearcă să minimizeze o funcție de cost, evitând problemele întâlnite în optimizarea funcțiilor. Strategia de învățare prezentată este deci diferită de cea bazată pe minimizarea erorii, care cere continuarea antrenării prin epoci suplimentare, dacă valoarea erorii este inacceptabilă sau ponderile nu se stabilizează.

[4] G. A. Carpenter, S. Grossberg, N. Markuzon, J. H. Reynolds, and D. B. Rosen, "Fuzzy ARTMAP: A neural network architecture for incremental supervised learning of analog multidimensional maps", IEEE Transactions on Neural Networks, vol. 3 5, pp. 698–713, 1992.

Capitolul 9

Rețele neurale convoluționale

Rețelele neurale convoluționale (eng. convolutional neural networks, CNNs) sunt modele instruibile care s–au dovedit eficiente în învățarea de date perceptuale — imagini, secvențe de film, sunete, serii de timp cu date perceptuale etc. Domeniile de utilizare ale rețelelor neurale de convoluție includ clasificare, detectare de obiecte, segmentare de imagini – figurile 9.1–9.3.

În problemele tratate de modelel de până acum, intrarea era dată de trăsături selectate sau create de un expert în domeniu. De exemplu, pentru predicția prețului de vânzare al unui apartament se consideră relevante anumite trăsături – suprafața, numărul de camere, numărul de băi, etajul, locația etc.; pentru alte probleme se pot folosi metode de feature engineering care să producă acele trăsături care sunt folosite mai departe într-un proces de învățare (de exemplu, pentru un medic e mai important indicele de masă corporală decât greutatea și înălțimea separate; la procesarea de sunet se face trecerea în spectrul de frecvețe etc.). *În deep learning formarea de trăsături este efectuată direct de rețeaua neurală* care face și restul muncii (clasificare, segmentare etc.). Efectivitatea acestei abordări a fost clar demonstrată în 2012, când o rețea convoluțională a câștigat competiția ImageNet Large Scale Visual Recognition Challenge cu o diferență mai mare de 10% față de locul următor[1]. Succesul a fost dat și de utilizarea de hardware eficient (GPU), precum și folosirea unui set de date mare pentru instruirea modelului.

Datele abordate de rețelele neurale convoluționale sunt organizate ca tablouri uni- sau multi-dimensionale; spre exemplu, o imagine în tonuri de gri este o matrice de pixeli (0 și 255 codifică negru, respectiv alb) – figura 9.4 – iar o imagine color este un tablou tridimensional de date – figura 9.5. De asemenea, ponderile instruibile sunt structurate ca tablouri. Tablourile se mai numesc și tensori; tensorii de date sunt populați cu valori provenite din

[1] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. In F. Pereira, C. J. C. Burges, L. Bottou, & K. Q. Weinberger (Eds.), Advances in neural information processing systems 25 (pp. 1097–1105), http://proceedings.neurips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.

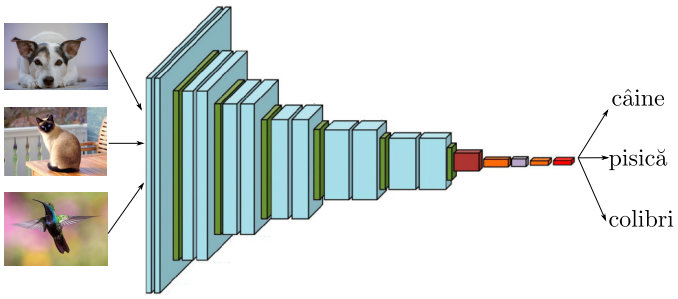


Figura 9.1: Clasificare de imagini cu rețea neurală de convoluție

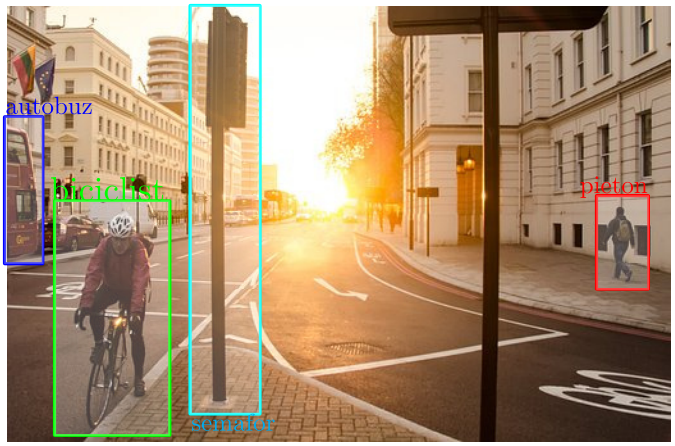
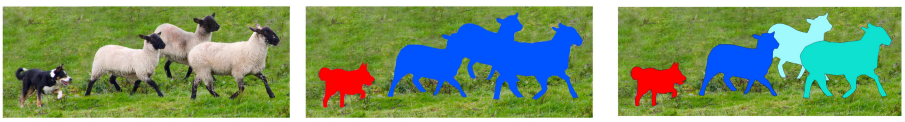


Figura 9.2: Detectare de obiecte cu rețea neurală de convoluție



Imagine de intrare | Segmentare semantică | Segmentare de instanțe

Figura 9.3: Segmentare semantică și segmentare de instanțe

măsurătorile obiectelor sau fenomenelor (de exemplu, intensitatea luminoasă a unui pixel). Un scalar este un tensor de dimensiune 0, un vector este un tensor de dimensiune 1, o matrice este un tensor de dimensiune 2, o imagine RGB ca în figura 9.5 este un tensor de dimensiune 3, o secvență video color e un tensor de dimensiune 4 etc.

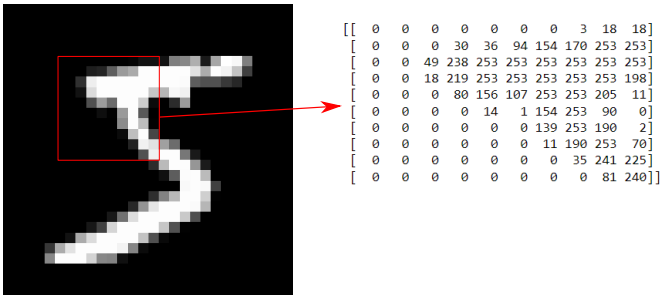


Figura 9.4: Exemplu de cifră 5 din setul de date MNIST. Porțiunea de imagine încadrată este matrice de de valori întregi



Figura 9.5: Imagine color separată pe canale roșu, verde, albastru; fiecare pixel este o colecție de 3 valori, pentru canalele de culoare roșu, verde, respectiv albastru

Pe lângă înmulțirile matriciale prezente în rețelele de tip perceptron multistrat, aceste rețele folosesc și operații de convoluție. Mai sunt utilizate straturi de pooling și cu conexiuni complete (eng. fully connected), acestea din urmă întâlnite deja în modelele de regresie liniară, regresie logistică și în rețelele MLP.

9.1 Convoluție

Operația de convoluție este des întâlnită în procesarea de semnale audio sau de imagini. În cazul procesării de imagini, unde intrarea este văzută ca o

(stivă de) matrice de valori, prin convoluție discretă se generează pixeli noi: pentru fiecare pixel de intrare se consideră o vecinătate convenabil aleasă, peste care se calculează o medie ponderată; ponderile (parametrii, coeficienții; eng: weights) instruibile sunt preluate dintr–o matrice de valori, ce formează nucleul (eng. kernel) de convoluție. Mai clar, o valoare $y[m, n]$ rezultată prin aplicarea convoluției cu o matrice kernel k peste o matrice de intrare x se calculează ca:

$$y[m, n] = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} x[i, j] \cdot k[m - i, n - j] \tag{9.1}$$

În formula de mai sus, centrul kernelului k are coordonatele $(0, 0)$, deci indicii pentru matricea k pot fi pozitivi și negativi. În practică, indicii de sumare din (9.1) au valori finite: kernelul este o matrice cu dimensiuni finite, deci valorile indicilor i și j sunt mărginite.

9.1.1 Exemplificare de convoluție

Un kernel bidimensional este o matrice de valori fixe sau determinate prin instruire; este deci caracterizat de numărul de linii și coloane (forma sa), se deplasează cu un anumit pas (eng. stride); intrarea peste care se aplică poate să fie sau nu bordată (eng. padding). Pasul dă numărul de pixeli cu care kernelul se mută față de aplicarea precedentă. Bordarea arată câte rânduri de pixeli sunt adăugate la marginile matricei de intrare, pentru a produce o dimensiune convenabilă la ieșire. În figurile 9.8—9.11 sunt date formulele de calcul pentru convoluția unei matrice de intrare de 4×4 cu un kernel de dimensiune 3×3, kernelul este deplasat la fiecare pas cu o poziție (stride de 1), intrarea este fără bordare (padding 0). Dimensiunea ieșirii este 2×2, fiind dată în funcție de dimensiunea intrării, mărimea kernelului, padding și stride.

Tehnic, formulele din figurile 9.8—9.11 nu calculează convoluția precum în ecuația (9.1), ci corelarea încrucișată (eng. cross–correlation), sau convoluție cu kernel răsturnat (eng. flipped kernel). Umărind formula (9.1), valoarea de convoluție calculată în figura 9.8 ar fi de fapt $(a_{11}w_{33} + a_{12}w_{32} + a_{13}w_{31}) + (a_{21}w_{23} + a_{22}w_{22} + a_{23}w_{21}) + (a_{31}w_{13} + a_{32}w_{12} + a_{33}w_{11})$ în loc de $(a_{11}w_{11} + a_{12}w_{12} + a_{13}w_{13}) + (a_{21}w_{21} + a_{22}w_{22} + a_{23}w_{23}) + (a_{31}w_{31} + a_{32}w_{32} + a_{33}w_{33})$. Se ajunge de la o variantă la alta făcând inversarea ordinii liniilor și apoi a ordinii coloanelor din matricea kernel. În cele ce urmează vom folosi termenul de convoluție, chiar dacă formula de calcul este de corelare încrucișată.

Un exemplu numeric, preluat din excelenta sinteză a lui Dumoulin și Visin [1] este dat în figura 9.12.

În funcție de configurația stratului de convoluție, la sumă se mai poate adăuga un termen liber – coeficient de bias, b – deci de exemplu prima convoluție (figura 9.8) va produce valoarea: $(a_{11}w_{11} + a_{12}w_{12} + a_{13}w_{13}) +$

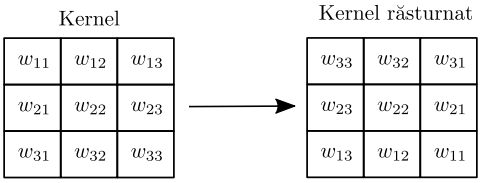


Figura 9.6: Kernel răsturnat

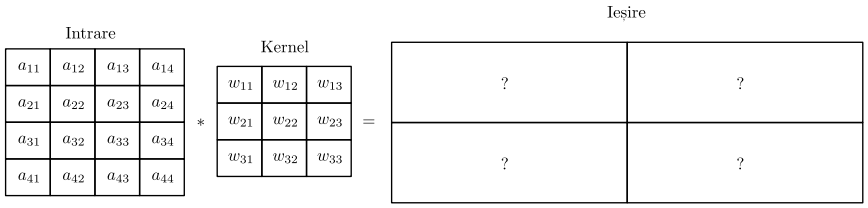


Figura 9.7: O intrare de 4×4 cu o convoluție de 3×3, stride 1 si padding 0 (fără bordare) produce o ieșire de 2×2. Cele 4 valori de ieșire se calculează prin aplicarea repetată a kernelului de convoluție pe fiecare sub–matrice de 3×3 din matricea de intrare.

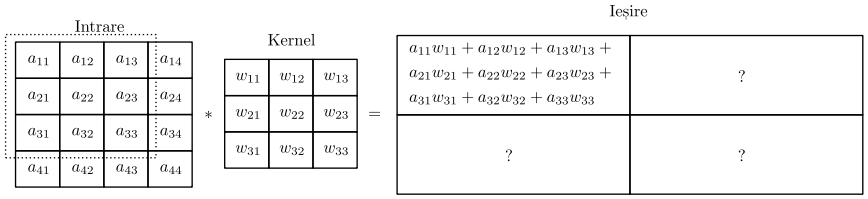


Figura 9.8: Prima convoluție (cu kernel răsturnat), aplicată peste intrările marcate cu linie punctată

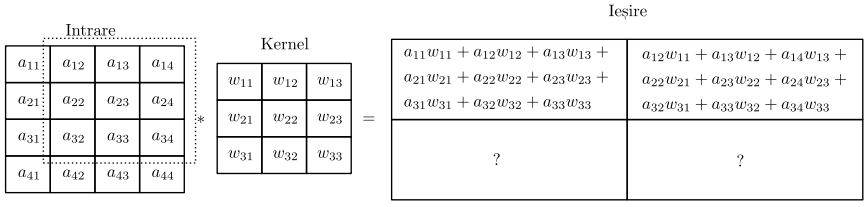


Figura 9.9: A doua convoluție

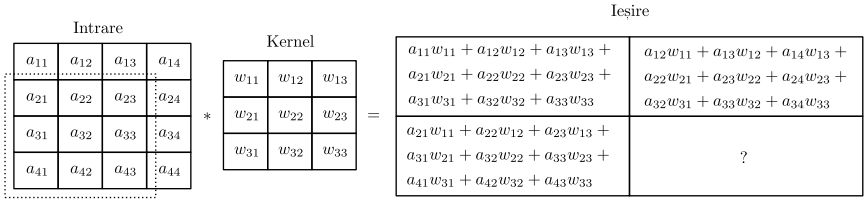


Figura 9.10: A treia convoluție

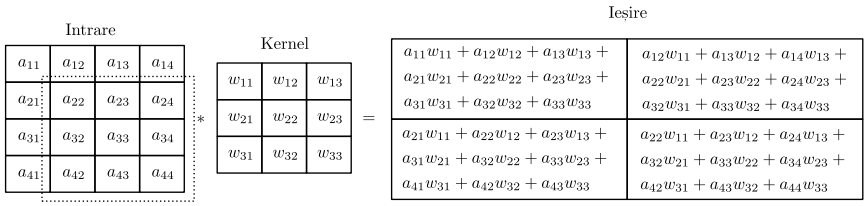


Figura 9.11: Ultima convoluție

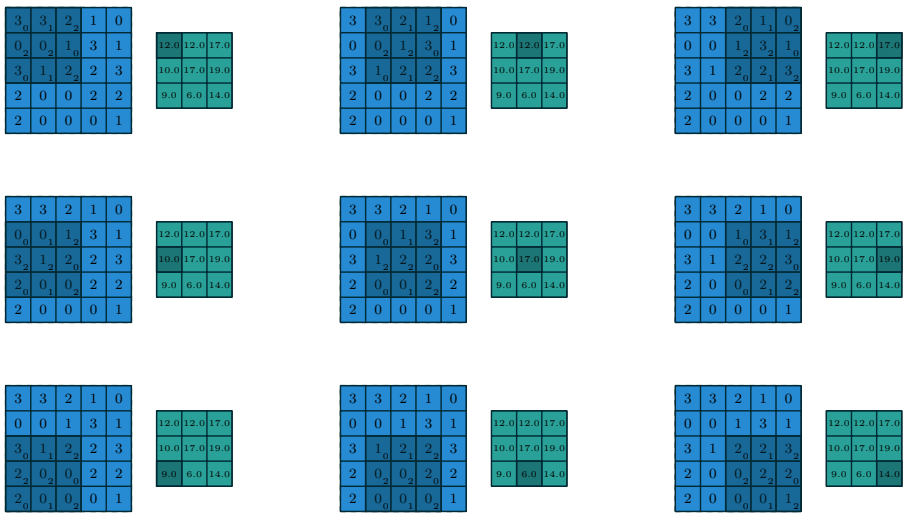


Figura 9.12: Exemplu numeric pentru convoluție de 3×3 peste o intrare de 5×5, fără bordare și cu pas 1. Preluare din [1]

$(a_{21}w_{21}+a_{22}w_{22}+a_{23}w_{23})+(a_{31}w_{31}+a_{32}w_{32}+a_{33}w_{33})+b$. În figurile 9.8—9.12 nu s–a folosit coeficient de bias.

Se observă că același kernel se "plimbă" peste matricea de intrare. Kernelul se comportă ca o lupă care scanează toată imaginea, încercând să găsească o trăsătură undeva în intrare, oriunde ar fi ea situată. Trăsătura căutată este dependentă de coeficienții numerici din kernel. Plimbarea kernelului se face peste toate datele din tensorul (mai sus: matricea de intrare), eventual considerând și valorile date de bordare (padding).

Un exemplu numeric concret este arătat mai jos pentru detectarea de muchii verticale. Matricea de kernel este anume aleasă pentru a depista tranziția de la o zonă mai luminată la una mai întunecată. Dacă valorile din matricea de ieșire se compară cu o valoare de prag, atunci va rezulta o muchie verticală.

$$\underbrace{\begin{pmatrix} 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \end{pmatrix}}_{\text{Intrare}} * \underbrace{\begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}}_{\text{Kernel}} = \underbrace{\begin{pmatrix} 0 & 30 & 30 & 0 \\ 0 & 30 & 30 & 0 \\ 0 & 30 & 30 & 0 \\ 0 & 30 & 30 & 0 \end{pmatrix}}_{\text{Ieșire}} \tag{9.2}$$

O diferență între kernelele populare în procesarea de imagini și cele din rețelele de convoluție este că acestea din urmă își ajustează ponderile în etapa de învățare; în particular, prin învățare se poate ajunge la coeficienții din kernelele din procesarea de imagini. Scopul este ca învățarea automată să determine kernelele potrivite pentru sarcina vizată, de exemplu pentru clasificarea de imagini.

Comentăm efectul folosirii aceluiași kernel peste toată intrarea considerând numărul de parametri: pentru intrarea de dimensiune 4×4 și kernelul de 3×3 exemplificate în figurile 9.7–9.11, avem $3 \times 3 = 9$ coeficienți instruibili – cei din kernel – la care se mai adaugă eventual ponderea de bias, deci 10 ponderi instruibile. Dacă am folosi precum în rețelele de tip MLP, câte o legătură (și deci câte o pondere) de la fiecare din cele 16 valori din matricea de intrare la fiecare valoare din matricea de ieșire (4 valori), la care adăugăm și ponderile de bias, obținem $16 \times 4 + 4 = 68$ de ponderi instruibile, mai multe decât cele de la convoluție. Pe lângă reducerea numărului de ponderi – și deci: a memoriei necesare modelului, a numărului de calcule necesare – avem și o scădere a șansei de overfitting, deoarece sunt mai puțini parametri. Un kernel produce deci *interacțiuni rare* (eng. sparse interactions): el angrenează un număr limitat de intrări în calcul, față de straturile complet conectate din MLP, unde valoarea de intrare într–un neuron e calculată cu activările tuturor nodurilor din stratul anterior.

De asemenea, un kernel promovează *partajarea ponderilor* (eng. parameter sharing): aceleași ponderi sunt folosite când kernelul este deplasat peste intrare. Prin contrast, ponderea unei legături dintre doi neuroni dintr-o rețea MLP e specifică acelei perechi, nefiind reutilizată în vreun fel.

În exemplele de mai sus s–a considerat că nu se face bordarea matricei de intrare, adică valoarea de padding este $p = 0$; aceasta se mai numește și convoluție validă: kernelul se plimbă, pas cu pas, peste matricea de intrare, fără a depăși marginile ei. Pentru o intrare de dimensiuni $w \times h$ și un kernel de dimensiune $k \times l$ și pas (stride) 1, ieșirea rezultată pentru convoluție validă va avea dimensiunea:

$$w' \times h' = (w - k + 1) \times (h - l + 1) \tag{9.3}$$

Se observă că aplicarea unui kernel, în aceste condiții, duce la reducerea dimensiunilor matricei de ieșire. Aplicarea în mod repetat duce la micșorarea succesivă a intrării. Uneori, acest lucru nu e de dorit.

9.1.2 Bordarea intrării

Dacă se ia coeficientul de padding $p > 0$, atunci matricea de intrare se va borda cu p linii și coloane setate cu o valoare constantă, de regulă 0, de fiecare parte a intrării[2]; peste această matrice de dimensiune $(w + 2 \cdot p) \times (h + 2 \cdot p)$ se va aplica kernelul de convoluție de $k \times l$ cu stride 1, rezultând o ieșire de dimensiune

$$w' \times h' = (w + 2 \cdot p - k + 1) \times (h + 2 \cdot p - l + 1) \tag{9.4}$$

Putem justifica bordarea astfel:

1. În lipsa bordării, valorile din colțurile matricelor sunt folosite fiecare o singură dată în producerea unei valori rezultat; valorile ce nu sunt la marginea matricei sunt folosite de mai multe ori, deoarece kernelul se plimbă peste matricea de intrare; pentru pasul de deplasare 1, o valoare de intrare care nu e pe laturile matricei de intrare este acoperită de mai multe ori de kernel. Acest efect se arată în figura 9.13, pentru un kernel de dimensiune 3×3: în timp ce valoarea a_{22} este folosită de patru ori în calcularea diverselor valori din matricea de ieșire, valoarea a_{11} este folosită o singură dată, doar în prima convoluție figurată. Similar, valorile de pe marginile matricei sunt de mai puține ori folosite în aplicarea de kernel decât valorile care nu sunt pe margini.

2. Vrem să evităm micșorarea dimensiunilor matricei de ieșire după convoluție; putem cere ca după aplicarea convoluției, dimensiunile ma-

[2]Se poate face o bordare cu număr diferit de coloane la stânga și la dreapta, respectiv număr aparte de linii deasupra și dedesubtul intrării.

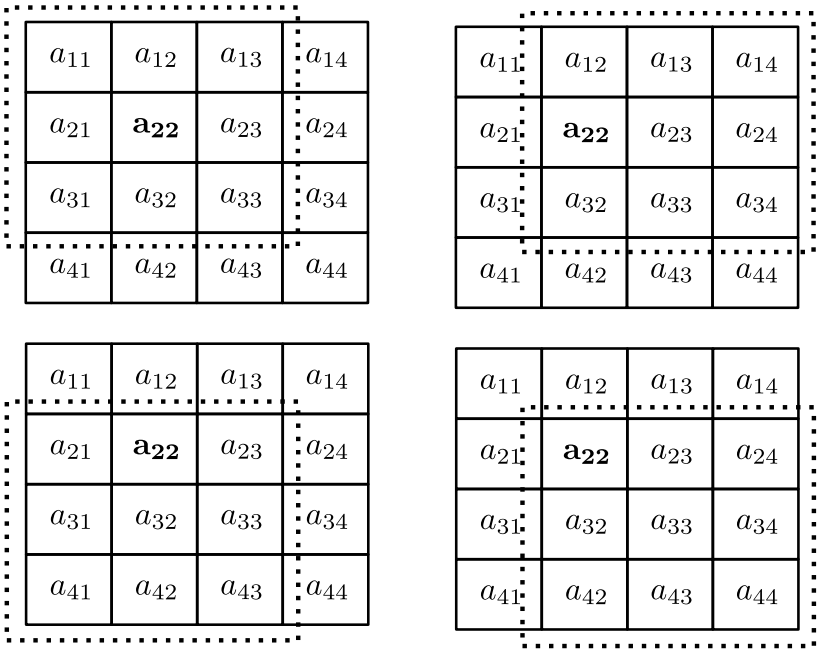


Figura 9.13: O valoare centrală a intrării va fi folosită de mai multe ori în aplicarea kernelului. În exemplul arătat, valoarea a_{22} este folosită de 4 ori la aplicarea kernelului. În schimb, o valoare din colț este folosită doar o dată.

tricei de ieșire să fie aceleași ca la matricea de intrare. Spre exemplu, pentru un kernel de $k \times l = 3 \times 3$, aplicarea lui peste o matrice de dimensiune $w \times h$ ar duce la obținerea unei ieșiri de forma $(w - k + 1) \times (h - l + 1) = (w - 2) \times (h - 2)$. Dacă bordăm pe fiecare latură cu un rând de valori 0 ($p = 1$), atunci obținem dimensiunile $(w + 2 \cdot p - k + 1) \times (h + 2 \cdot p - l + 1) = w \times h$.

Bordarea de așa manieră ca dimensiunea intrării să coincidă cu cea a ieșirii rezultate se numește *convoluție asemenea* (eng. same convolution). Exemplu: plecăm de la matricea de intrare

$$I = \begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \\ 12 & 13 & 14 & 15 \end{bmatrix} \tag{9.5}$$

și kernelul de 3×3

$$K = \begin{bmatrix} -4 & -3 & -2 \\ -1 & 0 & 1 \\ 2 & 3 & 4 \end{bmatrix} \tag{9.6}$$

Convoluția validă dintre I și K produce ieșirea

$$I * K = \begin{bmatrix} 78 & 78 \\ 78 & 78 \end{bmatrix} \tag{9.7}$$

Pentru convoluție asemenea facem bordarea cu câte o linie și coloană pline cu 0, adică $p = 1$, și obținem intrarea

$$I' = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 2 & 3 & 0 \\ 0 & 4 & 5 & 6 & 7 & 0 \\ 0 & 8 & 9 & 10 & 11 & 0 \\ 0 & 12 & 13 & 14 & 15 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{9.8}$$

Făcând convoluția lui I' cu K obținem:

$$I' * K = \begin{bmatrix} 33 & 49 & 58 & 31 \\ 63 & 78 & 78 & 30 \\ 75 & 78 & 78 & 18 \\ -29 & -77 & -86 & -87 \end{bmatrix} \tag{9.9}$$

și observăm că ieșirea are aceeași dimensiune ca și intrarea inițială I.

9.1.3 Pasul kernelului

În cele de mai sus s–a considerat că kernelul de convoluție se mută cu câte o poziție mai la dreapta sau mai jos. Se poate seta ca pasul de deplasare[3] să fie $s > 1$. Figura 9.14 arată o convoluție validă cu pas 2. Se observă că dimensiunea ieșirii este influențată de pasul de deplasare: deoarece se face salt peste valori și kernelul se aplică de mai puține ori decât anterior, matricea de ieșire are dimensiuni mai mici față de cazul în care pasul este 1.

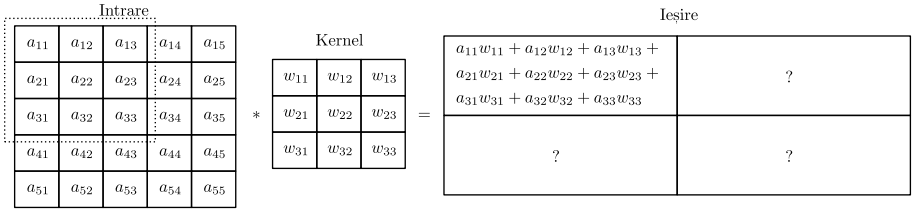
Se obține astfel un efect de subeșantionare; un proces similar are loc la folosirea straturilor de pooling (secțiunea 9.3), doar că la convoluții cu stride mai mare ca 1 subeșantionarea este învățată, și nu o strategie fixă. Subeșantionarea este utilă deoarece se reduce din redundanța semnalului – de exemplu, într–o imagine, pixelii alăturați sunt foarte similari și se referă frecvent la același obiect.

Dăm mai jos formula de calcul pentru dimensiunea ieșirii, în funcție de: dimensiunea intrării $w \times h$, dimensiunea kernelului $k \times l$, padding $p \geq 0$ și stride $s \geq 1$.

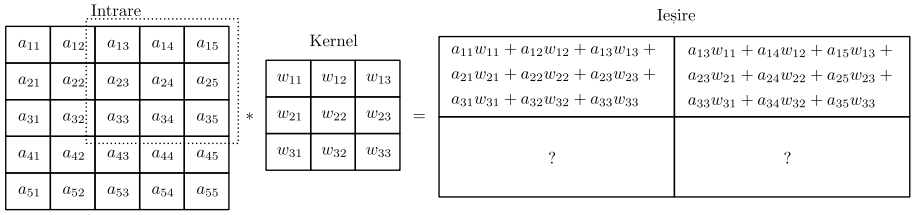
Ieșirea va fi de dimensiune:

$$w' \times h' = \left(\left[\frac{w + 2p - k}{s} \right] + 1 \right) \times \left(\left[\frac{h + 2p - l}{s} \right] + 1 \right) \tag{9.10}$$

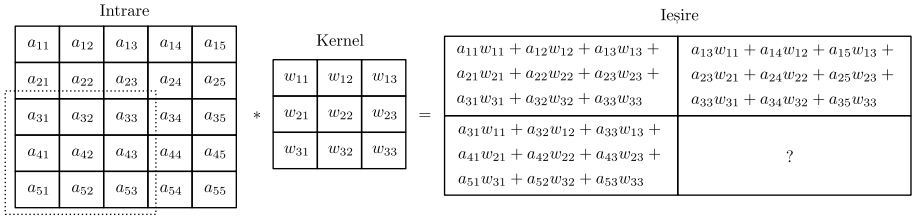
[3]Deplasarea se poate face cu pas diferit pe orizontală față de verticală. În cele ce urmează vom folosi pas identic pe cele două direcții.



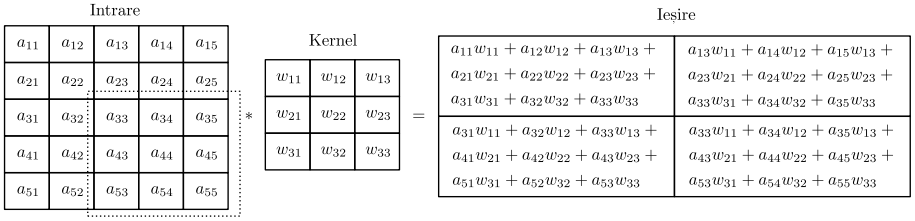
(a) Prima convoluție



(b) A doua convoluție. Se observă că s–a mutat kernelul cu două poziții mai la dreapta



(c) A treia convoluție – s–a sărit de la prima la a treia linie



(d) Ultima convoluție

Figura 9.14: Convoluție validă cu pas 2

unde $[x]$ este partea întreagă a lui x. Evident, această formulă generalizează variantele date mai sus calculate pentru $s = 1$.

Un exemplu numeric de convoluție cu pas 2 și padding 1, similar cu 9.12 este dat in figura 9.15 [1].

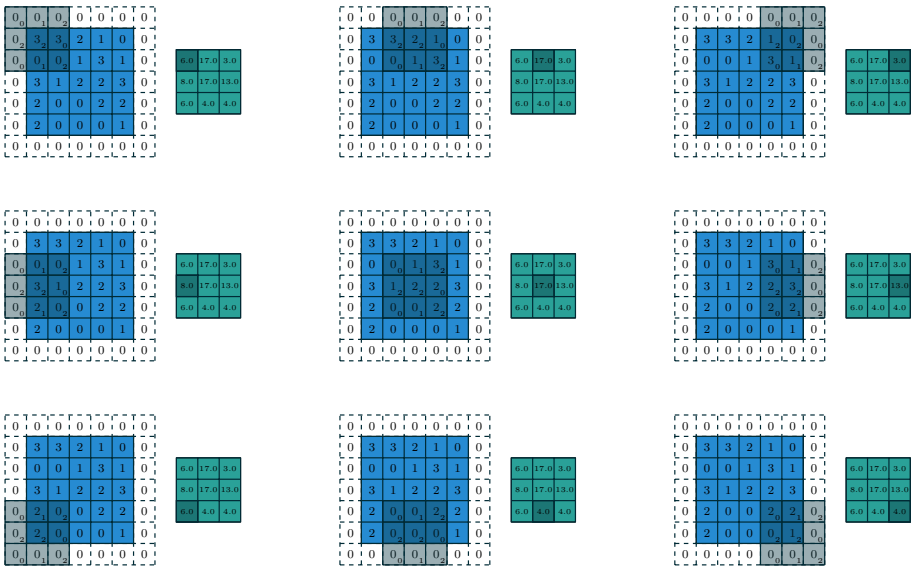


Figura 9.15: Exemplu numeric pentru convoluție de 3×3 peste o intrare de 5×5, cu pas 2 si padding 1. Preluare din [1]

9.1.4 Numărul de ponderi și de operații pentru convoluție cu intrare pe un singur canal

Este important să precizăm numărul de ponderi instruibile și de operații efectuate pentru convoluțiile discutate până acum. Să presupunem că lucrăm cu un kernel de convoluție de dimensiune $k \times l$. Pentru calculul unei singure valori din matricea de ieșire trebuie să facem $k \times l$ inmulțiri de perechi de termeni (un termen e pondere din kernel, celălalt e valoare din matricea de intrare); fiecare din cele $k \times l$ valori este acumulată la o variabilă setată inițial la 0, obținând deci $k \times l$ înmulțiri și acumulări (eng. multiply–accumulate operations, MACs). Acest lucru se repetă pentru calculul fiecărui termen din matricea de ieșire.

Sintetizând: dacă la o matrice de intrare de forma $w \times h$ aplicăm un kernel de convoluție de dimensiune $k \times l$, cu stride s și padding p, atunci:

- numărul de ponderi instruibile din kernel este:

$$n_{ponderi} = k \cdot l \tag{9.11}$$

- dimensiunea ieșirii $w' \times h'$ este cea din ecuația (9.10);
- numărul de înmulțiri–acumulări este

$$MACs = (w' \cdot h') \cdot (k \cdot l) \tag{9.12}$$

Dacă considerăm și bias, atunci va fi un coeficient de bias pentru tot kernelul, și formulele de mai sus trebuie ajustate corespunzător: $n_{ponderi} = k \cdot l + 1$, $MACs = (w' \cdot h') \cdot (k \cdot l + 1)$.

9.1.5 Convoluții pentru intrări cu mai multe canale

În exemplificările anterioare s-a considerat mereu intrarea ca matrice, adică un tensor bidimensional. În general, putem avea ca intrare pentru operația de convoluție un tensor cu mai mult de două dimensiuni, deoarece:

1. semnalul de intrare inițial poate fi cu mai multe canale, de exemplu imagine codificată RGB – a se vedea figura 9.5;
2. putem decide ca pe o intrare oarecare să se detecteze mai multe trăsături, rezultând mai multe hărți de trăsături corespunzătoare. Aceste multiple hărți vor produce următoarea intrare, care va fi deci cu mai multe canale.

Un *filtru de convoluție* este o colecție de kernele de convoluție, câte un kernel pentru fiecare canal de intrare. În figura 9.16 este reprezentat un filtru de convoluție aplicat peste o intrare cu $c = 4$ canale, fiecare canal de intrare fiind câte o matrice de 6×6; forma intrării este deci $w \times h \times c = 6 \times 6 \times 4$. Pentru exemplul dat filtrul are forma $k \times l \times c = 3 \times 3 \times 4$; se folosește aici convoluție validă cu pas 1. Rezultatul aplicării filtrului este o matrice de forma 4×4. Lungimea și lățimea ieșirii sunt calculate conform ecuației (9.10). Pentru întreg filtrul de convoluție se poate adăuga un coeficient de bias. Pentru exemplul din figura 9.16 numărul de parametri instruibil este dat de câte valori sunt în filtru – $3 \cdot 3 \cdot 4 = 36$ – la care se mai adaugă eventual un coeficient de bias.

Pentru calculul unei singure valori din matricea de ieșire, acești 36 de coeficienți se înmulțesc cu tot atâtea valori din tensorul de intrare (3×3 valori din matricea din plan apropiat; alte 3×3 valori din matricea din planul următor; la fel până la matricea de intrare din plan îndepărtat), deci 36 de înmulțiri; fiecare din acești termeni este acumulat la o variabilă setată inițial la 0 pentru a obține termenul final, obținând deci 36 de înmulțiri–acumulări (MACs). Formal, prima valoare din matricea de iesire o se calculează astfel (tensorul tridimensional a este matrice de intrare, cu $a_{1,1..6,1..6}$ matricea din plan apropiat și $a_{4,1..6,1..6}$ matricea de valori din plan indepărtat; similar pentru ponderile w din tensorul tridimensional al filtrului):

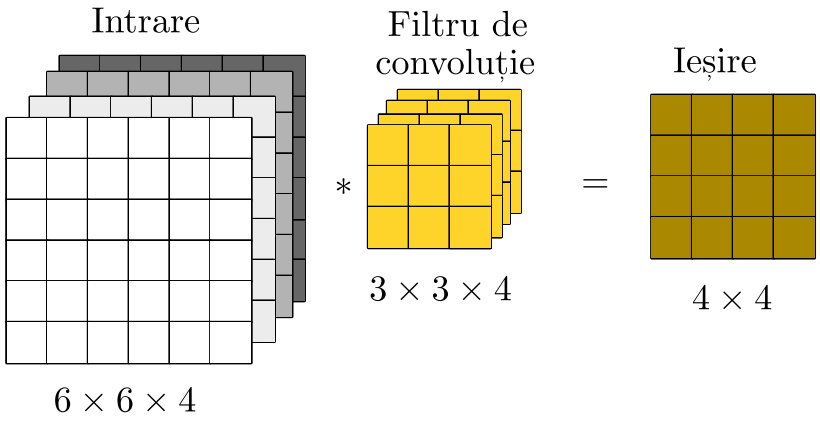


Figura 9.16: Convoluție validă pentru o intrare cu 4 canale, cu stride 1: intrarea are forma $w \times h \times c$ dată de lățimea și înălțimea intrării (6×6) și numărul de canale de intrare, adică 4; filtrul de convoluție este un tensor tridimensional, cu mărimea fiecărui kernel 3×3, iar a treia dimensiune e numărul de canale de intrare. Rezultatul este un tensor de forma 4×4.

$$
\begin{aligned}
o[1,1] = & \overbrace{a_{111}w_{111} + a_{112}w_{112} + a_{113}w_{113} + \cdots + a_{131}w_{131} + a_{132}w_{132} + a_{133}w_{133}}^{\text{convoluția cu primul kernel din filtru: 9 MACs}} + \\
& + \overbrace{a_{211}w_{211} + a_{212}w_{212} + a_{213}w_{213} + \cdots + a_{231}w_{231} + a_{232}w_{232} + a_{233}w_{233}}^{\text{convoluția cu al doilea kernel din filtru: 9 MACs}} + \\
& + \overbrace{a_{311}w_{311} + a_{312}w_{312} + a_{313}w_{313} + \cdots + a_{331}w_{331} + a_{332}w_{332} + a_{333}w_{333}}^{\text{convoluția cu al treilea kernel din filtru: 9 MACs}} + \\
& + \overbrace{a_{411}w_{411} + a_{412}w_{412} + a_{413}w_{413} + \cdots + a_{431}w_{431} + a_{432}w_{432} + a_{433}w_{433}}^{\text{convoluția cu ultimul kernel din filtru: 9 MACs}}
\end{aligned}
\tag{9.13}
$$

Similar se procedează pentru a calcula restul de valori din matricea de ieșire, obținând în total $4 \cdot 4 \cdot 36 = 576$ MACs.

Mai general, dacă pornim de la o intrare de forma $w \times h \times n_{in}$ și facem o convoluție cu un singur filtru de forma $k \times l \times n_{in}$, cu stride s și padding p, atunci:

1. numărul de ponderi din filtru[4] este

$$n_{ponderi} = k \cdot l \cdot n_{in} \tag{9.14}$$

2. ieșirea este o matrice de dimensiune $w' \times h'$ cu dimensiunile date ca în formula (9.10);

[4] În formulele imediat următoare nu considerăm ponderi de bias; aceasta se introduce mai jos.

3. numărul total de înmulțiri–acumulări pentru matricea de ieșire este

$$MACs = (w' \cdot h') \cdot n_{ponderi} = (w' \cdot h') \cdot (k \cdot l \cdot n_{in}) \tag{9.15}$$

Dacă se dorește și includere de bias, atunci va fi un unic coeficient de bias pentru tot filtrul și cele două formule de mai sus trebuie actualizate corespunzător: $n_{ponderi} = k \cdot l \cdot n_{in} + 1$, $MACs = (w' \cdot h') \cdot (k \cdot l \cdot n_{in} + 1)$.

Faptul că în exemplul 9.16 ieșirea este un tensor cu două dimensiuni (o matrice) se explică prin aceea că s–a aplicat un singur filtru. Folosirea de n_{out} filtre pentru intrarea din figura 9.16 produce un tensor de forma $4 \times 4 \times n_{out}$. Fiecare filtru trebuie să fie de forma $3 \times 3 \times 4$, ultima valoare (4) fiind numărul de canale de intrare. Cazul convoluției cu două filtre – deci și 2 canale de ieșire – este reprezentat în figura 9.17.

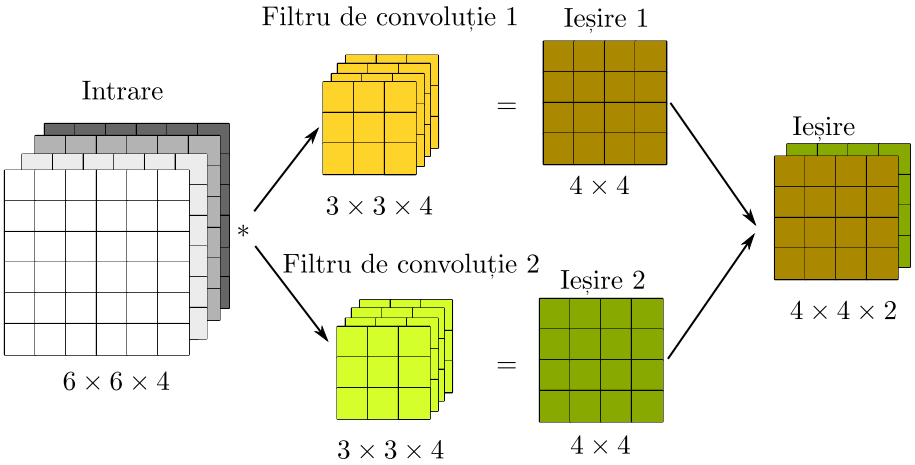


Figura 9.17: Convoluție validă pentru un strat de intrare cu 4 canale, 2 filtre, padding 0 și stride 1: intrarea are forma $w \times h \times c = 6 \times 6 \times 4$; fiecare filtru este un tensor $3 \times 3 \times 4$. Rezultatul este un tensor de forma $4 \times 4 \times 2$.

Folosirea mai multor filtre este justificată de faptul că fiecare produce o vedere proprie pornind de la aceeași intrare. De exemplu, un filtru poate fi specializat pentru detectarea de muchii orizontale, altul pentru detectarea de muchii verticale. Combinarea detecțiilor în straturile următoare poate ajuta în sarcina propusă – clasificare, segmentare, detectare de obiecte etc. În final, trăsăturile detectate prin filtrare succesivă sunt date unor straturi complet conectate, care fac clasificarea finală.

În general, dacă pe o intrare de forma $w \times h \times n_{in}$ – adică: n_{in} canale de intrare, fiecare canal fiind o matrice de dimensiune $w \times h$ – se aplica n_{out} filtre, toate cu padding p și stride s, atunci:

1. fiecare filtru trebuie să traverseze toate cele n_{in} canale de intrare, fiind deci compus din n_{in} kernele;

2. un filtru are forma $k \times l \times n_{in}$ (9.14); numărul total de ponderi instruibile din cele n_{out} filtre este

$$n_{ponderi} = k \cdot l \cdot n_{in} \cdot n_{out} \tag{9.16}$$

3. ieșirea va fi un tensor de forma

$$w' \times h' \times n_{out} = \left(\left[\frac{w + 2p - k}{s} \right] + 1 \right) \times \left(\left[\frac{h + 2p - l}{s} \right] + 1 \right) \times n_{out} \tag{9.17}$$

unde calculele pentru w' și h' sunt aceleași ca în ecuația (9.10);

4. numărul de operații de înmulțiri–acumulări este:

$$MACs = (w' \cdot h' \cdot n_{out}) \cdot (k \cdot l \cdot n_{in}) \tag{9.18}$$

formula fiind obținută pornind de la (9.15) și considerând și numărul de filtre – același cu numărul de canale de ieșire – n_{out}.

Dacă se folosește bias, atunci, la fel ca mai sus, fiecare filtru va avea propriul său coeficient de bias; se actualizează ușor numărul de ponderi și de operații. De exemplu, numărul de ponderi din (9.16) devine $n_{ponderi} = (k \cdot l \cdot n_{in} + 1) \cdot n_{out}$.

9.2 Funcții de activare

Dacă urmărim calculele din imaginile 9.8–9.11, observăm că operația de convoluție e o operație liniară; a se vedea și [1] secțiunea 4.1 pentru exemplu de convoluție aratată ca matrice (operator liniar). Ca atare, se impune utilizarea de funcții de activare neliniare, de forma celor folosite la rețelele de tip perceptron multistrat, secțiunea 6.4.2 pagina 117. În arhitecturile deep learning actuale sunt populare funcțiile de activare ReLU, PReLU, ELU. Funcțiile de activare se aplică punctual, adică peste fiecare valoare din rezultatul dat de convoluție.

Tensorii rezultați după aplicarea funcției de activare se numesc *hărți de trăsături* (eng. feature maps).

9.3 Straturi de pooling

Un strat de pooling este un filtru bidimensional care se aplică peste fiecare canal din harta de trăsături. Prin pooling se produc sumarizări locale ale valorilor din feature maps și totodată se reduce dimensiunea acestora (subeșantionare). Suplimentar, deoarece valoarea agregată prin pooling nu depinde de pozițiile valorilor peste care se aplică, se obține în mod natural invarianță la mici translații.

Un strat de pooling este caracterizat de: dimensiunea lui (lățime, înălțime), pas (stride) și forma de agregare a valorilor; nu se practică aici bordarea intrării. Filtrele se pot aplica pe regiuni adiacente sau cu suprapuneri. Tensorii rezultați au mărimea conform ecuației (9.10). Deoarece folosesc agregări de valori (maxim, medie), straturile de pooling nu au ponderi instruibile.

9.3.1 Max pooling

Filtrul max pooling calculează valoarea maximă peste fiecare regiune aleasă. Un exemplu este dat în figura 9.18, pentru un filtru de tip max pooling de dimensiune 2×2, cu pasul 2, aplicat pe o intrare cu un singur canal. Dacă sunt mai multe canale, se va aplica operația de pooling pe fiecare canal în parte, în mod independent.

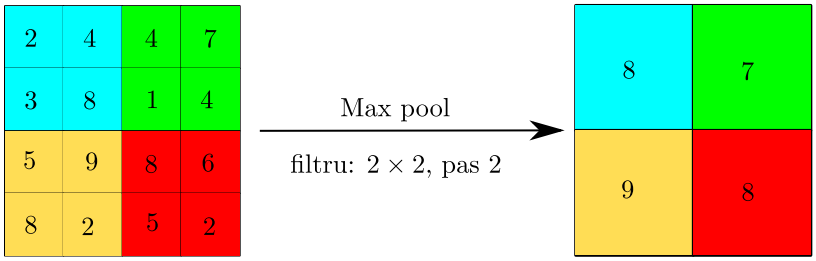


Figura 9.18: Max pooling aplicat peste un canal al unei hărți de trăsături de dimensiune 4×4

9.3.2 Average pooling

Un filtru de tip average pooling folosește calculul de medie ca formă de agregare. Restul detaliilor de la filtrul max pooling sunt valabile și aici. Un exemplu numeric este dat în figura 9.19, pentru un filtru de tip max pooling de dimensiune 2×2, cu pasul 2, un singur canal de intrare. Dacă intrarea este pe mai multe canale, atunci se face aplică filtrul de average pooling în mod independent, pe fiecare canal.

Sunt mai puțin folosite decât max pooling, pentru că se consideră că e mai util a se procesa prezența maximală a unei trăsături dintr–o hartă decât valoarea medie a trăsăturilor.

9.3.3 Global average pooling

Un filtru de global average pooling (GAP) reduce fiecare canal din harta de trăsături peste care se aplică la o singură valoare, media de pe acel canal. Este echivalent cu a aplica un filtru de average pooling de dimensiunea intrării. Astfel, se reduce o hartă de trăsături de forma $w \times h \times n_{in}$ la un tensor de forma $1 \times 1 \times n_{in}$, indiferent de valorile lui w și h.

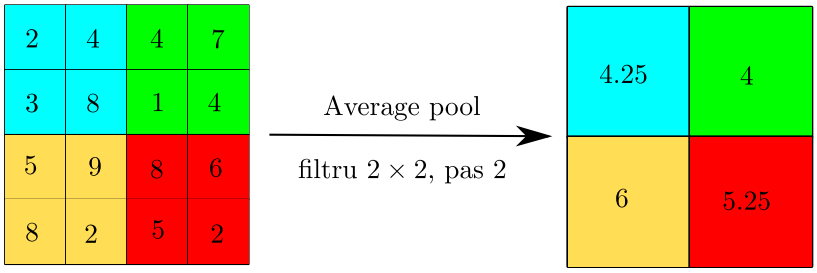
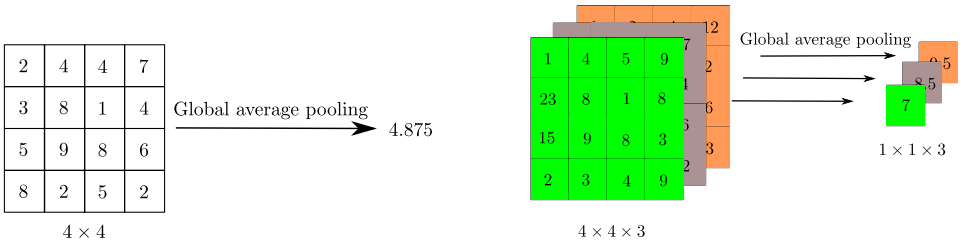


Figura 9.19: Average pooling aplicat peste un canal al unei hărți de trăsături de dimensiune 4×4



(a) Filtru de global average pooling pe o hartă de trăsături cu un canal de forma 4×4

(b) Filtru de global average pooling aplicat pe o hartă de trăsături de forma $4 \times 4 \times 3$

Figura 9.20: Global average pooling pe un canal, respectiv pe 3 canale

Două exemple sunt date în figura 9.20. Filtrul GAP se folosește cu precădere în rețelele fully convolutional – adică acelea în care nu se folosesc straturi complet conectate – a se vedea secțiunea următoare.

9.4 Straturi complet conectate

Pentru rețelele neurale de convoluție folosite pentru estimare de probabilitate condiționată, clasificare sau regresie se folosesc în finalul rețelei straturi complet conectate (eng. fully connected layers), exact ca într-o rețea neurală perceptron multistrat. Se acționează pe principiul: filtrele de convoluție și straturile de pooling fac filtrări succesive, plecând de la intrarea originară (e.g. imagine RGB) și obținând trăsături din ce în ce mai rafinate; acestea devin intrare pentru o arhitectură de tip MLP – parte a rețelei convoluționale – și aici se face efectiv clasificarea, regresia sau estimarea de probabilitate condiționată.

Înainte de primul strat complet conectat se face o aplatizare, adică se dispun toți neuronii într–un șir – a se vedea figura 9.21. De la stratul aplatizat urmează unul sau mai multe straturi complet conectate, care efectuează operație de înmulțire matricială, urmate imediat de funcții de

activare neliniare. De exemplu, pentru problemă de clasificare sau de estimare de probabilitate condiționată se folosește după ultimul strat complet conectat funcția softmax, definită în secțiunea 3.3.2 pagina 60.

Numărul de neuroni din stratul de ieșire este dat de numărul de ieșiri cerute de problemă – de exemplu, pentru o problemă de clasificare este numărul de clase, pentru fiecare producându–se o valoare de probabilitate condiționată. Numărul de neuroni din celelalte straturi complet conectate ține de decizie arhitecturală a rețelei neurale convoluționale din care fac parte straturile complet conectate.

În figura 9.21 este arătată o succesiune de straturi complet conectate, plecând de la o hartă de trăsături (sau rezultatul aplicării unui filtru de pooling peste o hartă de trăsături) ce se aplatizează.

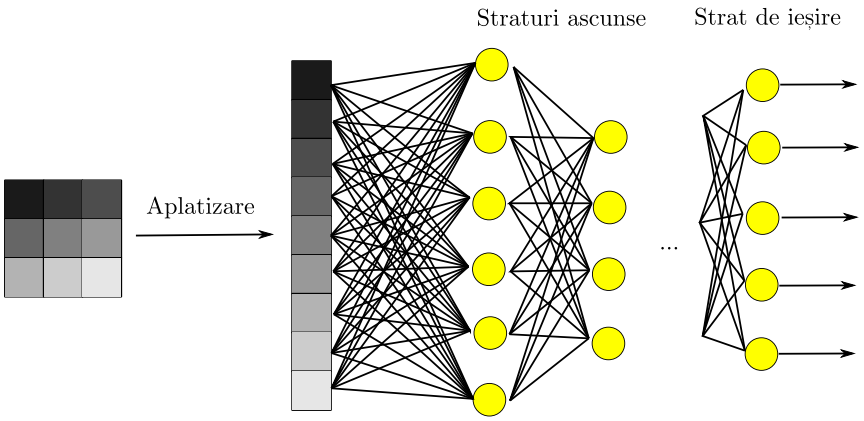


Figura 9.21: Straturi complet conectate la final de rețea neurală de convoluție

9.5 Funcții de cost, regularizare

Pentru modificare ponderilor se folosește o funcție de cost, care măsoară discrepanța între ieșire produsă de rețea și valoarea cunoscută pentru intrarea curentă. Funcția de cost trebuie să fie diferențiabilă, pentru a permite calcul de gradienți ce vor fi folosiți pentru modificarea ponderilor rețelei. Se pot folosi funcțiile de cost din secțiunea 6.6, pagina 124. În literatura de deep learning există și alte propuneri, influențate de problema ce trebuie rezolvată.

Se recomandă utilizarea unei forme de regularizare precum penalizare L_1, L_2 sau utilizarea unor forme aparte de regularizare, dezvoltate specific pentru rețele de tip deep learning, precum Dropout[5], Stochastic Depth[6].

[5]Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, Ruslan Salakhutdinov. 2014. Dropout: a simple way to prevent neural networks from overfitting. J. Mach. Learn. Res. 15, 1 (January 2014), 1929–1958.

[6]Huang, G., Sun, Y., Liu, Z., Sedra, D., Weinberger, K.Q. (2016). Deep Networks with

Se utilizează algoritmi de modificare a parametrilor rețelei folosind gradienții, de exemplu gradient descent, ADAM[7], Adagrad[8], RMSProp[9].

9.6 Rețelele neurale de convoluție: soluție end-to-end

O rețea neurală de convoluție are o arhitectură schițată în figura 9.22.

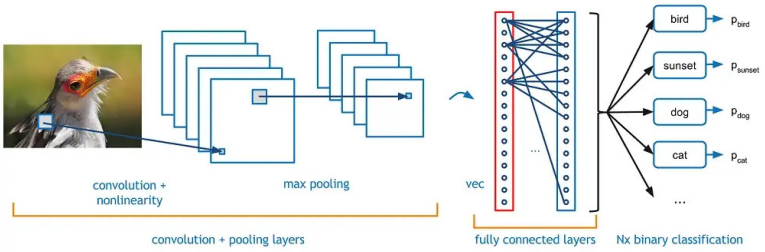


Figura 9.22: Arhitectura generică a unei rețele neurale convoluționale folosită pentru clasificare de imagini. Secvența convoluție + neliniaritate + pooling se poate repeta. La final se află straturile complet conectate. Aici, ultimul strat produce probabilități condiționate

În succesiunea convoluție - neliniaritate - pooling pleacă de la o intrare (imagine RGB, de exemplu) care este bogată în detalii, dar cu semantică puțină. Pe măsură ce se extrag succesiv trăsături, se pierd detalii (considerate redundante) și se câștigă în plan semantic. Figura 9.23[10] arată acest lucru.

9.7 Exemple de rețele neurale de convoluție

Punând cap la cap elementele arhitecturale (convoluție, funcție de activare, strat de pooling, straturi complet conectate), rezultă că, schematic, o rețea neurală de convoluție ar arăta precum în figura 9.24. Blocul format din

Stochastic Depth. In: Leibe, B., Matas, J., Sebe, N., Welling, M. (eds) Computer Vision – ECCV 2016. ECCV 2016. Lecture Notes in Computer Scienc, vol 9908. Springer, Cham. https://doi.org/10.1007/978-3-319-46493-0_39

[7]Kingma, D. P., Ba, J. (2014). Adam: A Method for Stochastic Optimization (cite arxiv:1412.6980Comment: Published as a conference paper at the 3rd International Conference for Learning Representations, San Diego, 2015)

[8]Duchi, J., Hazan, E., Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. Journal of machine learning research, 12(7).

[9]Tieleman, T. and Hinton, G. (2012). Lecture 6.5 - RMSProp, COURSERA: Neural Networks for Machine Learning. Technical report.

[10]Preluare din Honglak Lee, Roger Grosse, Rajesh Ranganath, and Andrew Y. Ng. 2011. Unsupervised learning of hierarchical representations with convolutional deep belief networks. Commun. ACM 54, 10 (October 2011), 95–103. https://doi.org/10.1145/2001269.2001295

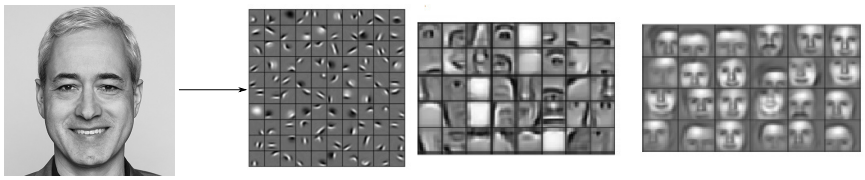


Figura 9.23: Trăsături învățate de o rețea neurală profundă. Primele trăsături de intrare sunt chiar pixelii, detaliați, dar nu de mare ajutor în clasificarea dorită. Pe măsură ce se avansează cu filtrarea, se obțin trăsături din ce în ce mai rafinate – segmente cu diferite orientări, elemente de trăsături faciale, părți mai mari din față etc. Un filtru este văzut ca un mecanism care caută în imagine o anumită trăsătura – de exemplu, un nas. Dacă trăsătura este găsită, valoarea de activare corespunzătoare este mare. În straturile următoare se combină trăsăturile detectate. Clasificarea din final este sarcină rezolvată de straturile complet conectate.

succesiunea convoluție → neliniaritate → pooling se poate repeta de mai multe ori, înainte de a trece la straturile complet conectate.

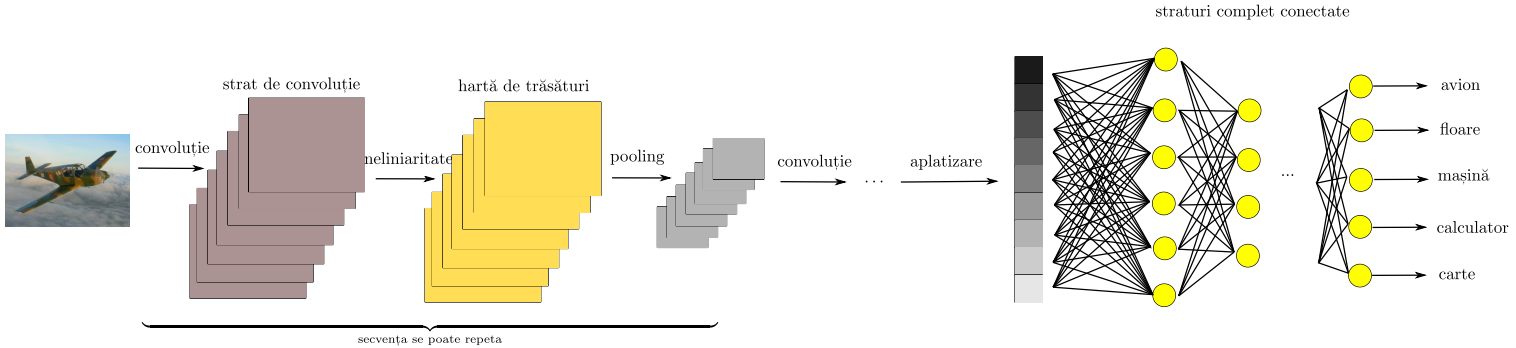


Figura 9.24: Schiță a unei rețele de convoluție folosită pentru clasificare

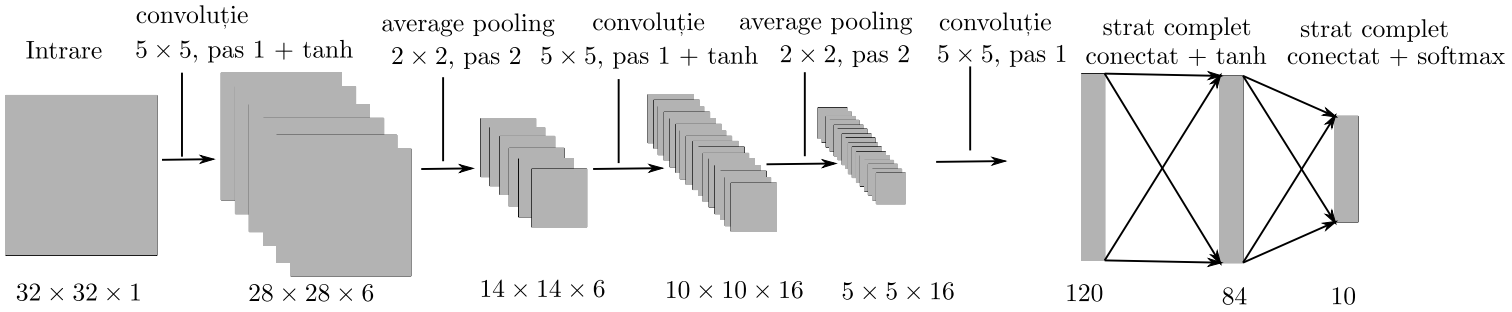


Figura 9.25: Arhitectura rețelei convoluționale LeNet-5

În cele ce urmează prezentăm două arhitecturi clasice de rețele de convoluție.

9.7.1 LeNet-5

Rețeaua LeNet-5 a fost introdusă de Yann LeCun[11] pentru recunoașterea de cifre scrise de mână. Rețeaua constă din 3 straturi convoluționale, 2 straturi de pooling și 2 straturi complet conectate. Arhitectura rețelei este desenată în figura 9.25 și detaliată în tabelul 9.1.

Strat	Nr. filtre /neuroni	Dim. kernel	Pas	Dim. hartă de trăsături	Activare
Intrare	-	-	-	$32 \times 32 \times 1$	-
Conv 1	6	5×5	1	$28 \times 28 \times 6$	tanh
Pool 2	-	2×2	2	$14 \times 14 \times 6$	-
Conv 3	16	5×5	1	$10 \times 10 \times 16$	tanh
Pool 4	-	2×2	2	$5 \times 5 \times 16$	-
Conv 5	120	5×5	1	$1 \times 1 \times 120$	tanh
Fully connected 6	-	-	-	84	tanh
Fully connected 7	-	-	-	10	softmax

Tabelul 9.1: Detalii arhitecturale pentru rețeaua LeNet-5.

9.7.2 VGG16

Rețeaua VGG16 este o rețea de tip convoluțional introdusă in articolul "Very Deep Convolutional Networks for Large-Scale Image Recognition"[12]. Modelele VGG au plecat de la idea de a folosi succesiuni de convoluții cu kernel de 3×3 în locul kernelelor de convoluție de dimensiuni mai mari – 5×5, 7×7 – intens folosite în modelele anterioare; astfel, numărul de ponderi instruibile scade. Rețelele VGG au fost folosite cu succes la clasificarea de imagini (fiind antrenate pe setul de date ImageNet, care are 1000 de clase), detectare de obiecte și calcul de reprezentări numerice (eng. embeddings). VGG16 este una din variantele propuse de autori și cu cele mai bune rezultatele empirice.

O reprezentare grafică a arhitecturii este în figura 9.26. Intrarea este dată de imagini de dimensiune 224×224 pe 3 canale (RGB); se face o preprocesare a imaginilor astfel încât fiecare canal să fie cu valoarea medie 0 și deviația standard 1 (standardizare). Imaginea este trecută printr–o

[11] Lecun, Y.; Bottou, L.; Bengio, Y.; Haffner, P. (1998). "Gradient-based learning applied to document recognition", Proceedings of the IEEE. 86 (11): 2278–2324. doi:10.1109/5.726791, link

[12] K. Simonyan, A. Zisserman, "Very deep convolutional networks for large-scale image recognition", in ICLR, 2015, link

secvență de straturi de convoluție cu kernel de dimensiune 3×3. Fiecare strat are 64 de filtre de convoluție. Convoluția e de tip asemenea (bordare cu un rând de valori 0) și pas 1. Ultima hartă de trăsături este trecută printr–un filtru de tip max pooling de dimensiune 2×2 cu pas 2. La finalul acestui bloc se ajunge la tensori de forma $112 \times 112 \times 64$.

Urmează un bloc similar, cu 128 de filtre de convoluție, urmate de max pooling cu aceiași parametri ca mai sus. Rezultă un tensor de forma $56 \times 56 \times 128$. Al treilea bloc are trei straturi convoluționale de câte 256 de filtre (convoluție asemenea, bordare și pas de 1) urmate de un max pooling ca mai sus. Rezultă un tensor de forma $28 \times 28 \times 256$. Urmează două blocuri de câte 3 straturi de convoluție, fiecare cu 512 filtre + max pooling. Se ajunge aici la tensor de forma $7 \times 7 \times 512$. În final se pun 3 straturi complet conectate, două cu 4096 de neuroni și ultimul cu 1000 de neuroni (numărul de clase).

Ultimul strat folosește funcție de activare softmax. Celelalte straturi complet conectate sau de convoluție sunt urmate de funcție de activare ReLU.

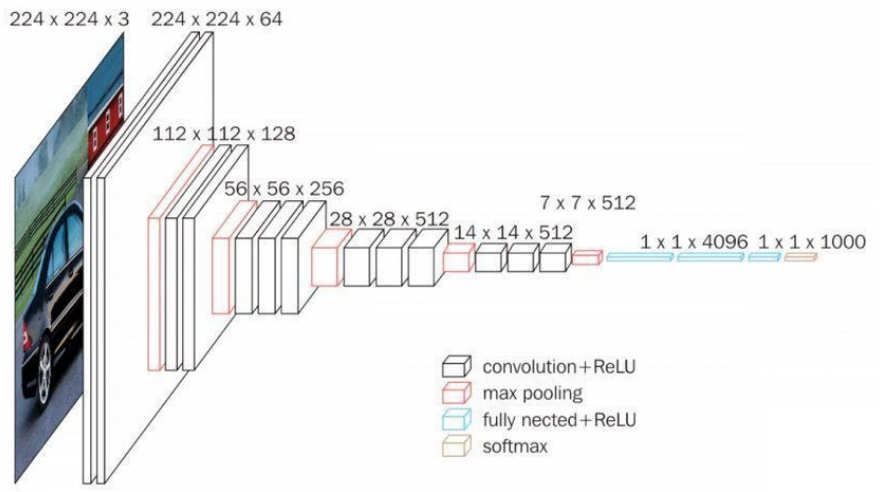


Figura 9.26: Arhitectura rețelei convoluționale VGG16

Capitolul 10

Calcul evoluționist

Calculul evoluționist este inspirat din teoria evoluției dezvoltate de către Charles Darwin și de genetică – știința eredității, având ca părinte pe Gregor Mendel. Au în comun faptul că folosesc populații de elemente care sunt folosite pentru căutarea soluției unei probleme, spre deosebire de alte abordări care încearcă îmbunătățirea printr-un proces iterativ a unei singure valori.

10.1 Taxonomie

Calculul evoluționist se împarte în:

1. algoritmi genetici;
2. programare evoluționistă;
3. strategii de evoluție;
4. programare genetică.

Domeniile enumerate au concepte comune; dintre toate, cele mai multe rezultate sunt în domeniului algoritmilor genetici, dar la ora actuală există hibridizări ale acestor 4 arii.

Cel care este creditat ca fiind pionierul domeniului algoritmilor genetici este John T. Holland de la Universitatea din Michigan. El a introdus conceptul de populație de indivizi care participă la căutarea unei soluții; de asemenea, a dat teorema schemelor. El a fost cel care a stabilit operațiile care trebuie să se aplice unei populații genetice - selecția, încrucișarea și mutația.

Programarea evoluționistă (avându–l ca pionier pe Larry J. Fogel) folosește ca operatori selecția celui mai potrivit individ și mutația, dar nu și încrucișarea. În timp ce algoritmii genetici văd procesul evolutiv ca fiind aplicat pe o *populație de indivizi din aceeași specie*, programarea evoluționistă

vede evoluția ca aplicându–se unei *populații de specii*. Fiecare element din populație este interpretat ca o specie întreagă.

Strategiile de evoluție au fost dezvoltate de Ingo Rechenberg și Hans-Paul Schwefel, care au experimentat diferite variante de mutație pentru optimizarea de suprafețe aflate în contact cu un fluid. Mutațiile reprezentau perturbări ale unor suprafețe, efectuând o căutare locală. Multiplele variante de perturbare au construit un întreg domeniu.

Programarea genetică (Richard Friedberg) a pornit cu coduri program de lungime fixă. Prin modificări efectuate în mod automat asupra acestor programe se dorea obținerea unor variante de cod optimizate. Esențiale sunt aici modul de reprezentare a acestor programe și funcțiile de măsurare a calității codului.

De cele mai multe ori, pentru o abordare dintr-unul din cele patru domenii se urmează pașii:

1. Inițializează populația
2. Calculează performanța fiecărui individ din populație
3. Aplică un pas de selecție
4. Aplică operații precum încrucișarea sau mutația
5. Reia de la pasul 2 până când se îndeplinește o anumită condiție.

Diferența între domenii constă în detaliile fiecărui pas. Pașii sunt bazați pe alegeri de valori aleatoare, ceea ce înseamnă că rulări diferite pot duce la rezultate diferite. Totodată, algoritmii nu garantează descoperirea unei valori optime. De cele mai multe ori, însă, nu este nevoie să se cunoască exact optimul, ci o valoare suficient de bună. În practică, calculul evoluționist dă rezultate bune într–un timp rezonabil.

În cele ce urmează vom prezenta algoritmii genetici.

10.2 Algoritmi genetici

Rolul mediului ca factor modelator în teoria evoluționistă este preluat în algoritmii genetici de către o funcție scop (sau funcție obiectiv). Vom detalia algoritmul pentru maximizarea unei funcții $f : [a, b] \to \mathbb{R}_+^*$.

Constrângerea ca funcția f să fie strict pozitivă poate fi asigurată prin adunarea unei cantități convenabile, suficient de mare, la funcția inițială, dacă are și porțiuni negative, sau considerarea funcției $g(x) = \max(\varepsilon, f(x))$, unde ε este o constantă strict pozitivă și mică; sau aplicăm o transformare inversabilă cu codomeniu strict pozitiv de forma $\exp(f(x))$ etc. Alegerea de a maximiza funcția obiectiv este convenabilă pentru descrierea care urmează. Dacă se dorește minimizarea unei funcții, se poate ține cont de relația:

$$\min_{x \in \mathscr{D}} f(x) = -\max_{x \in \mathscr{D}} [-f(x)] \tag{10.1}$$

unde $\mathscr{D}$ este domeniul de definiție al lui f.

Indivizii care alcătuiesc populația se numesc *cromozomi* și sunt alcătuiți din *gene*, reprezentați ca biți.

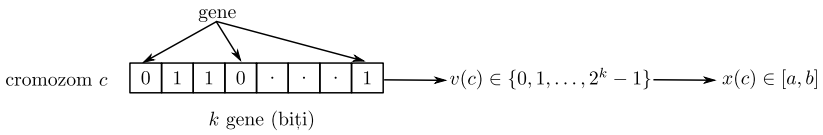


Figura 10.1: Reprezentarea unui cromozom în algoritmii genetici. Cromozomomul se transformă în final într-o valoare din intervalul $[a, b]$. Săgețile orizontale reprezintă transformări de valori.

Se pornește cu o populație inițială, care este supusă apoi unei secvențe de procese de tipul:

1. selecție: indivizii care sunt cei mai buni (considerând valoarea funcției f ce se vrea maximizată) sunt favorizați să apară de mai multe ori într-o populație nouă față de indivizii mai puțin performanți;
2. încrucișare: are loc un schimb de gene între perechi de părinți, formându-se copii; aceștia se presupune că moștenesc și combină performanțele părinților.
3. mutație: se efectuează câteva modificări pe gene.

Pas 1. Crearea unei populații inițiale de cromozomi. Se consideră mai multe valori pentru variabila $x \in [a, b]$. Numărul acestor valori – numit dimensiunea populației – este dat ca parametrul al algoritmului, n, dependent de problemă. Toate valorile sunt cuantificate prin cromozomi care sunt șiruri de k biți; un bit reprezintă în acest caz o genă a cromozomului, iar k e un alt parametru de intrare.

Generarea celor n cromozomi se face aleator, prin setarea fiecărei gene la valoarea 0 sau 1, la întâmplare. Se obține astfel o populație inițială formată din cromozomii $c_1, \ldots, c_n$.

Fiecare cromozom c – șir de k biți – va produce un număr $x(c)$ din intervalul $[a, b]$ (figura 10.1), astfel: dacă valoarea în baza 10 a cromozomului este $v(c)$ (evident, avem $v(c) \in \{0, \ldots, 2^k - 1\}$) atunci valoarea asociată din intervalul $[a, b]$ este:

$$x(c) = a + v(c) \cdot \frac{b-a}{2^k - 1} \in \left\{ a, a + \frac{b-a}{2^k - 1}, a + 2 \cdot \frac{b-a}{2^k - 1}, \ldots, b \right\} \quad (10.2)$$

Pas 2. Evoluția populației. În acest pas se obțin generații succesive plecând de la populația inițială; populația de la generația $g+1$ se obține pe baza populației de la generația g. Operatorii sunt selecția, împerecherea (crossover, încrucișarea) și mutația.

Pas 2.1. Selecția. Pentru fiecare cromozom c_i din populatie se calculează funcția obiectiv $y_i = f(x(c_i))$, $1 \leq i \leq n$. Apoi se însumează valorile funcțiilor obiectiv obținute pentru fiecare cromozom în parte:

$$S = \sum_{i=1}^{n} y_i \tag{10.3}$$

Pentru fiecare din cei n cromozomi se calculează probabilitatea de selecție:

$$p_i = \frac{y_i}{S}, 1 \leq i \leq n \tag{10.4}$$

Se observă că $0 < p_i < 1,\ 1 \leq i \leq n$ și $\sum_{i=1}^{n} p_i = 1$.

Pentru fiecare cromozom se calculează probabilitatea cumulativă de selecție:

$$q_i = \sum_{j=1}^{i} p_j, 1 \leq i \leq n \tag{10.5}$$

Remarcăm că avem $0 < q_1 < q_2 < \cdots < q_n = 1$. Cu cât cromozomul c_i determină o valoare mai mare pentru funcția f (*i.e.* cu cât valoarea $f(x(c_i))$ este mai mare), cu atât diferența dintre q_i și q_{i-1} – adică lungimea y_i/S a intervalului $(q_{i-1}, q_i]$ – este mai mare[1].

Se selectează n numere aleatoare uniform distribuite în $(0, 1]$. Pentru fiecare număr, dacă el se găsește în intervalul $(0, q_1]$ atunci cromozomul c_1 este ales și depus într-o populație nouă; dacă acest număr se află în intervalul $(q_{i-1}, q_i]$ atunci se alege cromozomul c_i. Remarcăm ca numărul de cromozomi prezenți în noua populație este tot n. Cu cât valoarea $y_i = f(x(c_i))$ asociată unui cromozom c este mai mare, cu atât cresc șansele lui spre a fi selectat și depus în noua populație. Este foarte probabil ca un astfel de cromozom valoros să apară de mai multe ori in populația nouă; de asemenea, este foarte probabil ca un cromozom cu o valoare mică pentru funcția f să nu apară deloc.

Pas 2.2. Încrucișarea (împerecherea, crossover) Pentru fiecare cromozom care a rezultat la pasul anterior se alege o valoare aleatoare, uniform distribuită în intervalul $(0, 1]$. Dacă această valoare este mai mică decât un parametru p_c (parametru al aplicației, *e.g.* 0.1), atunci cromozomul este ales pentru incrucișare. Se procedează astfel încât să se obțină un număr par de cromozomi (de exemplu se renunță la ultimul dacă numărul lor este impar).

Cromozomii aleși se încrucișează astfel: primul selectat cu al doilea selectat, al 3-lea cu al 4-lea etc. Pentru fiecare pereche, încrucișarea decurge astfel:

[1] Putem considera conveabil că $q_0 = 0$.

- se alege un număr aleator t intre 1 și $k-1$;
- se obțin 2 cromozomi copii astfel: primul va conține primele t gene ale primului părinte și ultimele $k-t$ gene ale celui de–al doilea părinte; al doilea copil conține primele t gene ale celui de–al doilea părinte și ultimele $k-t$ gene ale primului părinte;
- cei doi cromozomi copii îi vor înlocui în populație pe părinții din care provin.

Subliniem că punctul de tăiere poate diferi de la o pereche la alta.

Pas 2.3. Mutația. Populației obținute i se aplică operator de mutație, astfel: pentru fiecare genă a fiecărui cromozom se alege o valoare aleatoare, uniform distribuită în $(0, 1]$; dacă acest număr este mai mic decât o probabilitate de mutație p_m (parametru al aplicației, valoare mică, *e.g.* $p_m = 0.01$), atunci se modifică valoarea curentă a genei cu complementul său față de 1.

Populația obținută în pasul 2 reia ciclul de evoluție. După ce se obțin câteva astfel de generații (sau se epuizează un timp alocat procesului, sau se observă că media populației nu se îmbunătățește în ultimele iterații efectuate), se raportează valoarea celui mai bun cromozom din ultima generație[2].

Avantajul primar al algoritmilor genetici constă în schimbul de informație dintre indivizi realizat la etapa de încrucișare, adică schimbarea de blocuri de date care au evoluat. O utilizare eficientă a algoritmilor genetici presupune crearea unor structuri de date pentru gene și a unor operatori adecvați problemei ce trebuie rezolvată[3] – a se vedea secțiunea 10.4.

10.3 Fundamente teoretice

Studiul comportamentului algoritmilor genetici se face pe baza unor scheme (sau șabloane) care descriu colecții de cromozomi. O schemă se reprezintă ca un șir de caractere construit cu simbolurile "0", "1" și "*", unde "*" poate fi substituit cu orice bit; simbolul "*" poate apărea de oricâte ori, inclusiv niciodată. De exemplu, schema $(*0101)$ se potrivește cu doi cromozomi[4]: (00101) și (10101). Dacă o schemă are l simboluri "*", atunci ea poate să fie reprezentată de 2^l cromozomi, iar un cromozom de lungime k poate fi descris de $C_k^0 + C_k^1 + \cdots + C_k^k = 2^k$ scheme.

Ideea pentru ceea ce urmează este destul de simplă: se va arăta că dacă o schemă – adică o subpopulație de indivizi – este bună pentru funcția obiectiv,

[2] În practică se preferă strategia elitistă: se returnează cel mai bun individ al tuturor generațiilor.

[3] S-a stabilit "ecuația" Genetic Algorithms + Data Structures = Evolution Programs, [12].

[4] În acest caz spunem că schema este reprezentată de cei doi cromozomi.

atunci ea va prolifera în decursul evoluției.

Pentru o schemă S definim ordinul ei (și îl notăm cu $o(S)$) numărul de poziții pe care se află valorile 0 sau 1, adică numărul de poziții fixate. De exemplu, pentru schema $S = (*\ 0\ *\ 1\ 1\ 0)$, $o(S) = 4$. Ordinul unei scheme dă gradul de specializare a ei și este utilă mai departe în calcularea probabilității de supraviețuire a sa în cadrul mutațiilor.

Lungimea unei scheme S, notată cu $\delta(S)$, este distanța dintre prima și ultima poziție fixată. Pentru schema dată mai sus, $\delta(S) = 6 - 2 = 4$ (sau $\delta(S) = 5 - 1 = 4$, după cum indicierea începe de la 1 sau de la 0). Noțiunea de lungime a unei scheme este utilă pentru calculul probabilității de supraviețuire a unei scheme în cadrul operațiilor de încrucișare.

Pentru o populație de indivizi aflată la momentul t al evoluției, vom nota cu $n(S,t)$ numărul de cromozomi din populație care reprezintă (se potrivesc cu) schema S. De asemenea, vom considera valoarea medie a schemei din populația de la un timp t, notată cu $f(S,t)$ și definită ca suma valorilor cromozomilor din populație care reprezintă schema S împărțită la numărul acestor cromozomi, $n(S,t)$.

La pasul de selecție, un cromozom A este copiat în populația următoare cu probabilitatea:

$$P(A) = \frac{f(A)}{\sum\limits_{c} f(c)} \tag{10.6}$$

unde însumarea se face după toți cromozomii c ai populației curente. Reamintind că n este numărul de cromozomi din populație, avem că valoarea cea mai probabilă $n(S,t+1)$ este:

$$n(S,t+1) = n(S,t) \cdot \frac{f(S,t)}{\frac{1}{n}\sum\limits_{c} f(c)} \tag{10.7}$$

Cantitatea $\overline{f(t)} = \sum_c f(c)/n$ este chiar valoarea medie a populației de la momentul t, deci avem:

$$n(S,t+1) = n(S,t) \cdot \frac{f(S,t)}{\overline{f(t)}} \tag{10.8}$$

Interpretarea ecuației de mai sus este următoarea: numărul de reprezentanți ai schemei S care vor exista la momentul $t+1$ este dependent de valoarea schemei în populația de la momentul t. De exemplu, o schemă S care produce o valoare relativ mare a lui $f(S,t)$ față de $\overline{f(t)}$ va impune creșterea numărului de reprezentanți ai săi. Dacă presupunem de exemplu că $f(S,t) = \overline{f(t)} + \varepsilon \cdot \overline{f(t)} = \overline{f(t)}(1+\varepsilon)$, $\forall t > 0$ (unde $\varepsilon > 0$) atunci se arată prin inducție că:

$$n(S,t) = n(S,0)(1+\varepsilon)^t,\ \forall t \in \{1,2,\dots\} \tag{10.9}$$

adică pentru scheme care au valoare medie desupra valorii medii a populației numărul de reprezentanți va crește exponențial în timp – respectiv dacă

valoarea schemei este sub medie, numărul de reprezentanți obținuți prin selecție scade exponențial.

În ceea ce privește încrucișarea, să presupunem că cromozomul cu 7 gene $c = (1010100)$ este selectat pentru reproducere; există 2^7 scheme care îl au pe c drept reprezentant, de exemplu:

$$S_1 = (*01****) \tag{10.10}$$

și

$$S_2 = (1****0*) \tag{10.11}$$

Să presupunem că în procesul de încrucișare tăietura se face după a patra genă:

$$\begin{array}{rcccccc|ccc} c & = & (1 & 0 & 1 & 0 & | & 1 & 0 & 0) \\ S_1 & = & (* & 0 & 1 & * & | & * & * & *) \\ S_2 & = & (0 & * & * & * & | & * & 0 & *) \end{array} \tag{10.12}$$

Se observă că pentru exemplul considerat schema S_1 sigur se va regăsi într–un descendent (deci schema supraviețuiește), deoarece valorile 0 și 1 se regăsesc pe pozițiile inițiale, în timp ce S_2 are șanse de a fi distrusă[5]. Intuitiv, este clar faptul că lungimea mică a schemei S_1 mărește șansa de supraviețuire, față de S_2 care poate fi ușor "spartă" în cromozomii copii. Desigur, contează poziția tăieturii.

Tăietura poate să apară uniform aleator (echiprobabil) în $k-1$ poziții. Probabilitatea de distrugere a unei scheme este:

$$P_d(S) = \frac{\delta(S)}{k-1} \tag{10.13}$$

și evident probabilitatea evenimentului contrar, reprezentând supraviețuirea schemei este

$$P_s(S) = 1 - P_d(S) = 1 - \frac{\delta(S)}{k-1} \tag{10.14}$$

Conform strategiei de alegere a cromozomilor ce se supun împerecherii, probabilitatea ca un cromozom să participe la încrucișare este p_c, deci probabilitatea de supraviețuire a unei scheme S este:

$$P_s(S) = 1 - p_c \cdot \frac{\delta(S)}{k-1} \tag{10.15}$$

Se mai poate lua în considerare faptul că o schemă S poate totuși să supraviețuiască, dacă cromozomii care se încrucișează au pe pozițiile fixe ale schemei chiar valorile din S. Așa ceva este posibil și trebuie considerat ca mărind

[5]În exemplul dat, schema S_2 nu e distrusă dacă al doilea cromozom care participă la încrucișare vine cu aceleași gene pe pozițiile fixe din S_2.

șansele de supraviețuire a unei scheme. Ca atare, șansa de supraviețuire este de fapt dată printr–o inegalitate:

$$P_s(S) \geq 1 - p_c \cdot \frac{\delta(S)}{k-1} \tag{10.16}$$

deci schemele de lungime mică au șanse crescute de supraviețuire după mutație.

Combinând rezultatele obținute pentru partea de selecție și încrucișare, obținem:

$$n(S, t+1) \geq n(S, t) \cdot \frac{f(S,t)}{\overline{f(t)}} \cdot \left[1 - p_c \cdot \frac{\delta(S)}{k-1}\right] \tag{10.17}$$

Mutația schimbă aleator biți din cromozom cu complementul lor. Este clar că pentru ca o schemă să supraviețuiască, pozițiile sale fixe nu trebuie să fie alese pentru mutație. Probabilitatea ca un bit oarecare să nu fie modificat este $(1 - p_m)$. Alegerile biților care să sufere mutație sunt evenimente independente, deci probabilitatea ca cei $o(S)$ biți ficși ai unei scheme să se mențină (și deci ca întreaga schemă să se mențină) este:

$$P_s(S) = (1 - p_m)^{o(S)} \tag{10.18}$$

Pentru că $p_m \ll 1$, putem aproxima $(1 - p_m)^{o(S)}$ cu $1 - p_m o(S)$. Am obținut că schemele cu ordinul mic au șanse crescute de supraviețuire după mutație.

Efectul combinat al operațiilor de selecție, încrucișare, mutație este deci:

$$n(S, t+1) \geq n(S, t) \cdot \frac{f(S,t)}{\overline{f(t)}} \cdot \left[1 - p_c \cdot \frac{\delta(S)}{k-1} - p_m \cdot o(S)\right] \tag{10.19}$$

Se poate da acum enunțul teoremei schemelor, teorema fundamentală a algoritmilor genetici datorată lui Holland (1975):

Teorema 5 (Teorema schemelor)**.** *Schemele scurte, de ordin mic, cu valoare peste medie cresc ca număr de reprezentanți în decursul generațiilor.*

S–a formulat următoarea ipoteză:

Ipoteza blocurilor de construcție, [12]. Un algoritm genetic execută un proces de căutare prin suprapunerea unor scheme scurte, de ordin mic și de valoare mare, numite blocuri de construcție. Se poate arăta că pentru o populație de n cromozomi, numărul de scheme efectiv procesate este în ordinul lui n^3, ceea ce dă caracter de paralelism implicit al algoritmilor genetici: se procesează nu doar o singură schemă, ci mai multe.

10.4 Problema reprezentării datelor în algoritmii genetici

Reprezentarea indivizilor ca șiruri de biți este simplistă pentru multe probleme practice. Să considerăm, de exemplu, problema comis-voiajorului:

fiind date n orașe și distanțele dintre ele, să se determine un tur al lor, astfel încât fiecare oraș să fie vizitat exact o singură dată, să se revină la orașul de plecare iar costul total al drumului să fie minim[6]. O soluție este dată ca o permutare a mulțimii $\{1, \ldots, n\}$. Dacă considerăm primul oraș din tur fixat – de exemplu orașul n – atunci trebuie să dăm rezultatul ca permutare a mulțimii $\{1, \ldots, n-1\}$.

Pentru cazul $n = 20$, dacă folosim reprezentarea binară, putem vedea că cinci biți sunt suficienți pentru a reprezenta orice număr de la 1 la 19, deci ar trebui să folosim $19 \cdot 5 = 95$ de biți pentru reprezentarea unei soluții potențiale. Să presupunem că la un moment dat avem grupul de 5 biți 01101 reprezentând orașul cu numărul 13; prin mutație se poate ajunge la valoarea binară 11101, adică în zecimal 29, un oraș care nu există. S-ar obține deci o valoare invalidă datorată unei reprezentări neadecvate a elementelor din problemă sau a unor operatori care nu sunt adaptați corespunzător. La fel de bine, se poate ca prin mutație sau încrucișare să se obțină valori de orașe repetate, adică un ciclu prematur.

Pentru cei 95 de biți asociați problemei, spațiul de căutare realizat este $2^{95} \simeq 10^{29}$, în timp ce mulțimea tuturor ciclurilor Hamiltoniene este mulțimea permutărilor de 19 elemente (primul oraș din ciclu fiind fixat), având un număr de $19! < 10^{19}$ elemente. Pentru situația dată deducem că utilizarea unei codificări binare conduce la un spațiu de căutare mărit artificial, existând zone mari din spațiul binar care nu corespund unor soluții viabile.

Alte exemple de probleme aflate în aceeași situație pot fi încă date; se ajunge la concluzia că varianta naivă de reprezentare a valorilor și a operatorilor genetici nu se potrivește neapărat la orice problemă de căutare. Modelarea unui individ și a operatorilor asociați trebuie să se facă ținând cont de domeniu și de particularitățile problemei. Vor fi exemplificate codificări adecvate pentru câteva probleme clasice.

O problemă înrudită care trebuie tratată este: cum procedăm când există constrângeri ce trebuie onorate? De exemplu, dacă vrem să maximizăm funcția:

$$f(x, y) = x^2 - y^3 + 2 \cdot x \cdot \sin(y) \tag{10.20}$$

cu condiția ca variabilele x și y să satisfacă constrângerea:

$$1 \leq x^3 - \cos(y) + y^2 \leq 5 \tag{10.21}$$

cum încorporăm restricția în algoritmul genetic? Dacă folosim varianta clasică de codificare a unui individ, împreună cu operatorii de încrucișare și de mutație prezentați, cum asigurăm faptul că operatorii dați nu duc indivizii în zone în care constrângerea (10.21) nu este îndeplinită?

Pentru această din urmă problemă s–au dat următoarele variante:

1. impunerea de penalizări pentru indivizii care încalcă constrângerile;

[6] În termeni de grafuri: se cere determinarea unui ciclu Hamiltonian de cost minim.

2. implementarea unei metode de "reparare" a indivizilor care nu satisfac constrângerile;

3. implementarea unor tipuri de date particularizate pentru reprezentarea de indivizi și a unor operatori de încrucișare și de mutație care produce indivizi valizi.

Pe marginea fiecăreia din cele trei variante există multiple versiuni:

- pentru penalizări, valoarea acestora poate fi constantă, sau să varieze cu gradul în care se încalcă constrângerile date; această ultimă variantă poate fi codificată sub forma unei funcții logaritmice, liniare, pătratice etc. O formă extremă de penalizare este eliminarea indivizilor care încalcă restricțiile, dar trebuie dat răspuns la întrebarea: cu ce se umple locul lăsat gol prin eliminare? sau se permit populații de dimensiune variabilă? cei mai mulți autori afirmă că o astfel de eliminare este prea dură, în timp ce menținerea unor indivizi penalizați oferă variabilitate populației – se pot produce descendenți valizi, chiar și din cei care nu respectă constrângerile.

- pentru algoritmii de reparare – este posibil să se integreze cunoștințe din domeniu în metodele de corecție; trebuie zis însă că elaborarea unui algoritm de corecție poate uneori să fie o problemă la fel de grea ca și rezolvarea problemei de la care s–a plecat.

- pentru ultima variantă – este cunoscut deja că orice tip de date trebuie să vină cu un set de operatori dedicați care să permită prelucrarea tipurilor; o codificare potrivită a problemei împreună operatorii asociați trebuie să favorizeze (ideal: să garanteze) generarea de indivizi valizi. Aici se folosesc cunoștințe despre problema care trebuie rezolvată, cunoștințe care, prin implementare, favorizează obținerea de indivizi care nu încalcă (prea mult, sau deloc) restricțiile;

Pentru fiecare din abordări s–au studiat variante și comportamente; studiul s–a făcut în mare măsură empiric, pe probleme concrete; la ora actuală, un rezultat precum teorema schemei este inexistent pentru alte codificări decât cea binară. Desigur, se poate folosi și o combinație a celor trei variante de mai sus.

Pentru numeroase probleme practice s–a constatat experimental că reprezentarea adecvată a indivizilor, împreună cu definirea unor operatori particularizați dau rezultate mai bune decât aplicarea *ad literam* a algoritmului genetic peste o formă binarizată a problemei.

Vom exemplifica pentru problema discretă a rucsacului: se dă un rucsac de capacitate C, un set de m obiecte având greutățile $G_i > 0$ și valorile asociate $V_i > 0$, $1 \leq i \leq m$. Un obiect poate fi luat doar în întregime în

rucsac; problema este: care sunt obiectele care trebuie încărcate, astfel încât greutatea totală să nu depășească C iar valoarea cumulată să fie maximă?

Problema este NP-completă, deci la ora actuală nu cunoaștem un algoritm de complexitate polinomială care să o rezolve. Multe probleme pot fi reduse la aceasta, de aici interesul acordat.

O reprezentare naturală a unui individ – respectiv încărcare de rucsac – este un vector $\mathbf{x}$ cu elementele x_i, $1 \leq i \leq m$, $x_i \in \{0, 1\}$, valoarea 0 însemnând că obiectul nu este luat, iar 1 - că e adăugat în rucsac. Se impune, evident, condiția de viabilitate a unui vector $\mathbf{x}$:

$$\sum_{i=1}^{m} x_i \cdot G_i \leq C$$

iar funcția de maximizat – numită și profit în acest caz – este:

$$P(\mathbf{x}) = \sum_{i=1}^{m} x_i \cdot V_i$$

10.4.1 Varianta cu penalizare

Pentru fiecare individ $\mathbf{x}$ se va considera valoarea sa $f(\mathbf{x})$:

$$f(\mathbf{x}) = \sum_{i=1}^{m} x_i \cdot V_i - Pen(\mathbf{x})$$

unde $Pen(\cdot)$ este funcția de penalizare:

$$Pen(\mathbf{x}) \begin{cases} = 0, & \text{dacă } \mathbf{x} \text{ este viabil} \\ > 0, & \text{dacă } \mathbf{x} \text{ nu este viabil} \end{cases}$$

Dacă valoarea funcției de penalizare depinde de gradul în care se face încălcarea restricțiilor – gradul de încălcare poate fi de exemplu de diferența dintre $\sum_{i=1}^{m} x_i \cdot G_i$ și C – atunci se poate folosi o funcție de tip logaritmic, liniar, pătratic, exponențial etc. Efectele alegerii unei asemenea funcții au fost analizate pe diferite situații; a se vedea [12] pentru rezultate experimentale și interpretarea lor.

10.4.2 Varianta cu reparare

Putem folosi aici tot codificarea binară. Algoritmul de corectare este simplu: dacă setul de obiecte ales depășește ca greutate totală capacitatea C, atunci se scot obiecte până când greutatea celor rămase devine acceptabilă (cel mult G). Vom transforma deci vectorul $\mathbf{x}$ în $\mathbf{x}' = (x'_1, \ldots, x'_m)$ astfel încât $\sum_{i=1}^{m} x'_i G_i \leq C$.

Algoritmul de reparare a unui individ este:

Listing 10.1: Repararea unui vector invalid

```
function reparare(x, G, C) returns a vector
begin
 rucsac-supraincarcat := false
 x' := x
 if sum_{i=1}^{m} x'_i * G_i > C
  then rucsac-supraincarcat := true
 end if
 while rucsac-supraincarcat = true
  i:= selecteaza un obiect din rucsac (#)
  scoate obiectul i din rucsac: x'_i := 0
  if sum_{i=1}^{m} x'_i * G_i <= C
   then rucsac-supraincarcat := false
  end if
 end while
 return x'
end
```

În ce privește metoda de selectare a lui i din linia marcată cu (#), avem variantele evidente, dar nu și singurele posibile:

- (reparare aleatoare) valoarea i se alege aleator din setul indicilor obiectelor care se găsesc în rucsac;
- (reparare greedy) se alege obiectul cel mai ușor, sau cel care are raportul P_i/G_i cel mai mic.

10.4.3 Codificarea adecvată a indivizilor

Vom prezenta o strategie de codificare a indivizilor, diferită de cea binară utilizată până acum, numită reprezentarea ordinală. Codificarea este larg utilizată și în alte probleme care presupun manipularea unor secvențe de valori, de exemplu în problema comis-voiajorului. Vectorul $\mathbf{x}$ este cu cele m componente în baza 10, fiecare element x_i având proprietatea că $1 \leq x_i \leq m - i + 1$, $\forall i \in \{1, \ldots, m\}$. De exemplu, pentru vectorul $\mathbf{x} = (3, 3, 4, 1, 1, 1)$ asociat listei de obiecte $L = (1, 2, 3, 4, 5, 6)$, decodificarea se obține astfel: se scoate elementul de indice $x_1 = 3$ din lista L, adică obiectul 3 și îl adăugăm în rucsac; L devine $(1, 2, 4, 5, 6)$; apoi se scoate elementul de indice $x_2 = 3$ (adică obiectul 4) din L și îl adăugăm în rucsac, L devine $(1, 2, 5, 6)$; se scoate elementul de indice $x_3 = 4$ din L, adică obiectul 6, L devine $(1, 2, 5)$ etc. Obținem astfel ordinea de depunere în rucsac: 3, 4, 6, 1, 2, 5. Fiecare cromozom codifică astfel o ordine de adăugare a obiectelor în rucsac. Adăugarea obiectului în rucsac se face numai dacă nu duce la depășirea capacității C.

Se poate vedea că operație de încrucișare va duce întotdeauna la copii valizi, adică pentru un copil $\mathbf{z} = (z_1, \ldots, z_m)$ avem că $1 \leq z_i \leq m - i + 1$

dacă și părinții au aceeasi proprietate. Mutația este similară cu cea de la cazul binar: o componentă aleasă pentru mutație, fie ea x_i este modificată cu o valoare aleatoare uniform distribuită în mulțimea $\{1, \ldots, m - i + 1\}$ diferită de x_i.

Listing 10.2: Utilizarea codificării ordinale

```
procedure decodificare(x)
begin
 construieste o lista L de obiecte
 greutateTotala := 0
 profitTotal := 0
 for i = 1, m
  j := x_i
  o := L_j
  sterge elementul al j-lea din lista L
  if greutateTotala + G_o ≤ C
   then begin
    greutateTotala := greutateTotala + G_o
    profitTotal := profitTotal + P_o
   end
  end if
 end for
end
```

Lista L se poate crea într–o ordine aleatoare, ca o permutare a mulțimii $\{1, \ldots, n\}$. Indivizii rezultați — adică cei care realizează populația inițială — vor fi deci generați aleatori.

Rezultate experimentale pentru cele trei variante de rezolvare sunt date în [12]. Concluziile experimentelor însă nu pot fi generalizate la orice problemă care vine cu impunere de restricții. Se exemplifică însă că diferențele de performanță pot fi mari pentru aceste abordări.

10.5 Exemplu: problema orarului

Problema orarului este o altă situația practică care se abordează prin intermediul algoritmilor genetici. Problema este NP-completă. Are diferite enunțuri, vom prezenta varianta dată în [12].

Se dau următoarele:

- o mulțime de profesori $\{T_1, \ldots, T_m\}$
- o listă de intervale de timp (ore) $\{H_1, \ldots, H_n\}$
- o listă de săli de clasă $\{C_1, \ldots, C_k\}$

Orarul *trebuie* să respecte următoarele cerințe:

- există un număr predefinit de ore pentru fiecare profesor și clasă;
- la un moment dat, la o clasă predă un singur profesor;
- un profesor nu poate preda la mai multe clase simultan;
- la fiecare clasă programată la o anumită oră trebuie să existe exact un profesor

Mai avem și constrângeri care *ar trebui* respectate; acestea sunt legate de:

- nicio "fereastră" în orarul elevilor, cât mai puține în cel al profesorilor;
- preferințe exprimate de profesori sau elevi: ore doar într–o anumită parte a zilei sau săptămânii;
- împărțire cât mai echilibrată a orelor;
- număr maxim de ore pe zi pentru elevi/profesori

Codificarea binară pentru această problemă, cu toate că este posibilă, poate apărea drept nenaturală; mai mult decât atât, există riscul ca spațiul de căutare să se mărească artificial, precum la problema comis voiajorului. Putem să codificăm un orar (un individ) ca fiind o matrice $\mathbf{O}$ cu m linii și n coloane, unde liniile corespund profesorilor iar coloanele – orelor disponibile. Fiecare celulă este fie liberă, fie conține o clasă C_i, $1 \leq i \leq k$.

Pentru reprezentarea dată, operatorii genetici ar putea fi[7]:

- mutația de ordin k: se iau 2 secvențe adiacente formate din p elemente și se interschimbă
- mutația de zile: se iau două zile și se interschimbă între ele
- încrucișare: se pornește de la două orare părinte O_1 și O_2, se efectuează tăietură pe orizontală sau pe verticală și se face interschimbarea de porțiuni, întocmai ca la cromozomii binari

Este posibil ca să fie nevoie să se intervină cu algoritmi de corecție după aplicarea unor astfel de operatori. Din punct de vedere practic, abordarea prin algoritmi genetici este confirmată ca o metodă funcțională de către mai mulți autori.

[7] Cititorul e invitat să imagineze și alți operatori.

Bibliografie

[1] V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning." arXiv:1603.07285, 2016.

[2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Harlow: Pearson Education Limited, fourth, global ed., 2021.

[3] S. Haykin, *Neural Networks and Learning Machines*. Upper Saddle River, NJ: Pearson Education, third ed., 2009.

[4] R. Andonie and A. Cațaron, *Inteligență computațională*. Brașov: Editura Universității Transilvania din Brașov, 2002.

[5] A. P. Engelbrecht, *Computational Intelligence: An Introduction*. Wiley, 2002.

[6] R. C. Eberhart and Y. Shi, *Computational Intelligence: Concepts to Implementations*. Morgan Kaufmann, 2007.

[7] A. Ng, "Machine Learning Specialization, curs online Coursera." DeepLearning.AI și Stanford Online, https://www.coursera.org/specializations/machine-learning-introduction, 2022. Accesat: 2025-03-02.

[8] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

[9] K. B. Petersen and M. S. Pedersen, "The Matrix Cookbook." Technical University of Denmark, Nov. 2012. Versiunea 20121115, https://www2.imm.dtu.dk/pubdb/pubs/3274-full.html.

[10] M. A. Nielsen, *Neural Networks and Deep Learning*. Determination Press, 2015. http://neuralnetworksanddeeplearning.com.

[11] S. J. D. Prince, *Understanding Deep Learning*. The MIT Press, 2023. http://udlbook.com.

[12] Z. Michalewicz, *Genetic Algorithms + Data Structures = Evolution Programs*. Berlin, Heidelberg: Springer-Verlag, third ed., 1996.